ALGOVIZ 2.0

COMPLETE TECHNICAL INTERVIEW HANDBOOK

A source-grounded defence manual for the algorithm visualisation platform: architecture, recorded-frame playback, the reducer, comparison mode, persistence, the 150 & 768 verification cases, CI, weaknesses and resume defence.

Every claim in this book is classified:
[VERIFIED] from the repository · [INFERENCE] reasoned from the code
[GENERAL KNOWLEDGE] required background · [NOT VERIFIED] the source does not establish it
[PROPOSED] a future idea, not an existing feature

Built by reading the repository at commit `80f9862` — 61 JavaScript/JSX modules under src/, 14 automated test groups, 69 chapters, 42 handbook parts.

Prepared for: Siva Charan K.G. · Project: AlgoViz 2.0 — live URL per README: algoviz-2-0.vercel.app

Table of contents

Chapters, sections and the page where each one starts.

Table of contents 2
01 · How to use this handbook 17
1.1 The five claim labels used in this book 17
1.2 How to read this book 17
1.3 What AlgoViz is, in one paragraph you can memorise 18
02 · Resume reality check: every claim, audited 19
2.1 The audit table 19
2.2 The five nuances that will separate you from a bluffing candidate 20
2.3 The honesty script (memorise this shape) 21
2.4 The deployment claim, handled honestly 21
Project identity 22
03 · What AlgoViz is and why it exists 23
3.1 The problem the project actually solves 23
3.2 The five capabilities that define the product 23
3.3 Who it is for 24
3.4 What AlgoViz is NOT — the boundary list 24
3.5 Feature inventory, verified against the source 25
04 · The stack, and the four pitches 26
4.1 The stack, verified from package.json and the lockfile 26
4.2 Why each choice is defensible (and what the alternative would cost) 26
4.3 The four pitches 27
Complete project architecture 30
05 · The repository, mapped 31
5.1 The honest high-level map 31
5.2 The RED / YELLOW / GREEN file classification 31
5.3 What happens to dead code in the build 33
5.4 The dead-code inventory, verified 33
06 · Architecture and data flow 34
6.1 The pipeline as implemented 34
6.2 The nine stages, each with its guarantee 36
6.3 Why this architecture, and what the alternative would have been 36
6.4 The dependency graph (who imports whom) 37
6.5 The two execution models side by side 38
07 · Important files you must know 39
7.1 src/lab/engine.js — 169 lines — RED 39
7.2 src/lab/SortingLab.jsx — 596 lines — RED 39

7.3 The six generators — RED . . . 39
7.4 The support files — YELLOW . . . 40
7.5 The files you must be able to name but not explain — GREEN . . . 40
Application startup and routing . . . 42
08 · Startup: index.html → main.jsx → App → Home → routes . . . 43
8.1 Stage 1 — index.html (the only HTML file) . . . 43
8.2 Stage 2 — src/main.jsx (9 lines) . . . 43
8.3 Stage 3 — src/App.jsx (12 lines) . . . 44
8.4 Stage 4 — src/pages/Home.jsx (104 lines): routing . . . 44
8.5 Stage 5 — the Suspense boundary . . . 46
8.6 StrictMode and the timer: the question that catches candidates . . . 47
The Sorting Lab, end to end . . . 48
09 · The Sorting Lab state register . . . 49
9.1 Why a state register matters . . . 49
9.2 The register (all [VERIFIED] from src/lab/SortingLab.jsx) . . . 49
9.3 The two properties that make this register safe . . . 51
9.4 The one place the register costs something . . . 51
10 · The input pipeline: draft → validation → committed dataset . . . 53
10.1 The five validation rules in parseInput . . . 53
10.2 What actually happens on submit — the full trace . . . 54
10.3 The other three ways a dataset enters the lab . . . 54
10.4 makeDataset: a hand-rolled deterministic generator . . . 55
10.5 The input panel's defensive details . . . 56
Recorded-frame architecture . . . 57
11 · Frame anatomy . . . 58
11.1 The definition . . . 58
11.2 The complete field table . . . 58
11.3 The codeLine vocabulary — the complete list . . . 59
11.4 The ID scheme, and why it is not an index . . . 59
11.5 The currentStats clone . . . 62
12 · How a generator builds a trace . . . 63
12.1 The general shape (all six generators) . . . 63
12.2 Annotated source: the first twelve frames of bubble sort . . . 63
12.3 The frame-count arithmetic (measured, not guessed) . . . 65
12.4 The same pattern, seen in the other five . . . 66
13 · Execution vs playback — the separation that defines the project . . . 67
13.1 The two phases . . . 67
13.2 "Why does the algorithm not need to run backwards when you seek backwards?" . . . 67
13.3 What you give up — the honest trade-off . . . 67
13.4 The alternative designs, and why they lose (for this use case) . . . 68
14 · The memory trade-off, quantified . . . 69

14.1 Measured trace sizes . . . 69
14.2 The asymptotics, stated exactly . . . 69
14.3 ”What happens with a huge trace?” — the failure sequence . . . 70
14.4 The delta-frame design, described properly [PROPOSED] . . . 70
React state management in AlgoViz . . . 72
15 · useState — ten independent pieces of state . . . 73
15.1 Where AlgoViz uses it . . . 73
15.2 The deep dive: initialising saved lazily . . . 73
15.3 Alternative · trade-off · question . . . 74
16 · useReducer — the playback state machine . . . 76
16.1 The exact call site . . . 76
16.2 Alternative · trade-off · question . . . 76
17 · useEffect — two effects, both with cleanup . . . 78
17.1 Effect #1 — the playback timer . . . 78
17.2 Effect #2 — the keyboard listener . . . 78
17.3 What is not in an effect, and why that matters . . . 79
18 · useMemo — two memoised traces . . . 81
18.1 Where it is used . . . 81
18.2 Alternative · trade-off . . . 81
18.3 useCallback — used exactly once, and not here . . . 82
19 · useRef — two refs, two jobs . . . 83
20 · Props, lifting, and derived state . . . 84
20.1 Every significant prop in the project . . . 84
20.2 Derived state, done right . . . 84
20.3 The cleanup checklist (everything that must be cleaned up) . . . 85
The playback reducer . . . 86
21 · The reducer, line by line . . . 87
21.1 The complete source . . . 87
21.2 Every line, annotated . . . 87
21.3 The three invariants the reducer maintains . . . 88
21.4 The state machine . . . 88
21.5 The transition table — state × action . . . 89
21.6 What the reducer is not allowed to know . . . 89
22 · Twenty-two worked simulations . . . 90
23 · Twenty-eight reducer questions, with answers . . . 92
Play, pause and the timer system . . . 96
24 · The complete Play sequence, in nine steps . . . 97
24.1 The code involved . . . 97
24.2 The nine steps . . . 97
24.3 The pause sequence, and why it is clean . . . 98
25 · The timer system: ownership, cleanup, and duplicates . . . 99

25.1 The ownership model . . . 99
25.2 The five timer bugs this pattern prevents . . . 99
25.3 Why setTimeout and not setInterval . . . 99
25.4 Stale closures — the concept, and where it would bite here . . . 100
25.5 Race conditions and unmount . . . 100
26 · Speed control, and what it does not change . . . 102
26.1 The control . . . 102
27 · Keyboard control, second by second . . . 104
Step, seek and the timeline . . . 106
28 · Timeline navigation . . . 107
28.1 The five navigation affordances . . . 107
28.2 The two disabled states, and why they are exactly right . . . 108
28.3 frames[cursor] — the single most important expression . . . 108
28.4 Completion, and why the final state stays visible . . . 109
29 · Debugging navigation . . . 110
29.1 The debugging method for anything playback-related . . . 110
29.2 Scenario A — the cursor goes past the end of the trace . . . 110
29.3 Scenario B — playback advances two frames per step . . . 110
29.4 Scenario C — pause appears to stutter (one extra frame) . . . 111
29.5 Scenario D — arrow keys stop working after the first press . . . 111
29.6 Scenario E — the timeline slider jumps back while dragging . . . 111
29.7 Scenario F — the last frame never renders . . . 112
29.8 Scenario G — the frame shown does not match the step badge . . . 112
29.9 Scenario H — the timeline shows 0 / 0 . . . 112
The six sorting algorithms . . . 114
30 · The algorithm set, and what is not in it . . . 115
30.1 What ”six algorithms” means, precisely . . . 115
30.2 The colour code for study priority . . . 115
31 · Bubble sort . . . 117
31.1 Problem and intuition . . . 117
31.2 The invariant . . . 117
31.3 The actual implementation . . . 117
31.4 The real trace: [3, 1, 2] → 19 frames . . . 118
31.5 Complexity . . . 119
31.6 Stability, in-place, edge cases . . . 120
31.7 Bugs to watch for . . . 120
31.8 Interview questions . . . 120
31.9 Whiteboard . . . 121
32 · Selection sort . . . 122
32.1 Problem and intuition . . . 122
32.2 The invariant . . . 122

32.3 The actual implementation . . . 122
32.4 The real trace: [3, 1, 2] → 15 frames . . . 123
32.5 Complexity . . . 123
32.6 Stability, in-place, edge cases . . . 123
32.7 Bugs to watch for, and interview questions . . . 124
32.8 Whiteboard . . . 124
33 · Insertion sort . . . 125
33.1 Problem and intuition . . . 125
33.2 The invariant . . . 125
33.3 The actual implementation . . . 125
33.4 The real trace: [3, 1, 2] → 14 frames, including the break . . . 126
33.5 Complexity . . . 126
33.6 Stability, in-place, edge cases . . . 127
33.7 Interview questions . . . 127
33.8 Whiteboard . . . 128
34 · Merge sort . . . 129
34.1 Problem and intuition . . . 129
34.2 The invariant . . . 129
34.3 The actual implementation . . . 129
34.4 The real trace: [3, 1, 2] → 30 frames . . . 130
34.5 Three things this trace exposes . . . 131
34.6 Complexity . . . 132
34.7 Stability, edge cases, bugs . . . 132
34.8 Whiteboard . . . 133
35 · Quick sort . . . 134
35.1 Problem and intuition . . . 134
35.2 The invariant . . . 134
35.3 The actual implementation . . . 134
35.4 The real trace: [3, 1, 2] → 20 frames . . . 135
35.5 The complexity, and the worst case you can demonstrate live . . . 136
35.6 Recursion depth and space . . . 136
35.7 Stability, edge cases, bugs . . . 137
35.8 Whiteboard . . . 137
36 · Heap sort . . . 139
36.1 Problem and intuition . . . 139
36.2 The invariant . . . 139
36.3 The actual implementation . . . 139
36.4 The real trace: [3, 1, 2] → 14 frames . . . 140
36.5 The two phases, and the sorting-region behaviour . . . 141
36.6 Complexity . . . 141
36.7 Stability, edge cases, bugs . . . 142

36.8 Whiteboard . . . 142

Algorithm visualization design . . . 144

37 · From result to recorded explanation . . . 145

37.1 The mismatch that defines the project . . . 145

37.2 Instrumentation: how execution becomes data . . . 145

37.3 Why ”just animate the CSS” is a different (and weaker) design . . . 146

37.4 The visual encoding, mapped to the data . . . 146

37.5 What makes an explanation explainable . . . 147

37.6 Comparison with the alternative designs . . . 148

Comparison, alignment and operation counts . . . 149

38 · Same-input comparison . . . 150

38.1 The code . . . 150

38.2 What ”same input” is guaranteed to mean . . . 150

38.3 The four pieces of disclosure in the UI . . . 150

38.4 What the feature compares, and what it refuses to . . . 151

38.5 The five comparison questions, answered . . . 151

39 · Percentage-aligned replay, explained precisely . . . 153

39.1 The formula . . . 153

39.2 Worked examples . . . 153

39.3 What percentage alignment is actually for . . . 154

39.4 Why direct frame-index alignment is worse . . . 154

39.5 The limitations, stated without hedging . . . 154

39.6 Ten interviewer attacks on alignment, with answers . . . 155

40 · Operation counts: what is recorded, and what it is not . . . 157

40.1 The counter contract . . . 157

40.2 The exact increment sites, verified across all six generators . . . 157

40.3 What the numbers do not mean . . . 158

40.4 Worked example — why ”merge has fewer comparisons” is a statement about this input . . . 158

40.5 The counter questions . . . 159

Persistence and portability . . . 160

41 · Browser-local experiment saves . . . 161

41.1 The storage contract . . . 161

41.2 The read path, step by step . . . 162

41.3 The write path, and the two failure modes it handles . . . 162

41.4 Reload behaviour, precisely . . . 163

41.5 The persistence questions . . . 164

42 · JSON import and export . . . 166

42.1 The export path . . . 166

42.2 The import path, gate by gate . . . 166

42.3 What is NOT validated — the honest gap list . . . 168

42.4 Fifteen import/export questions . . . 168

Testing and verification . . . 171
43 · The testing architecture, as it actually is . . . 172
43.1 The mechanism, verified from package.json . . . 172
43.2 What the test file covers, group by group . . . 172
43.3 Unit vs integration vs end-to-end — what exists here . . . 173
43.4 What the tests deliberately do not cover, and why . . . 174
44 · The 150 sorting cases . . . 175
44.1 The generator of the 150 . . . 175
44.2 The 25 inputs, broken down by category . . . 175
44.3 The four assertions, precisely . . . 176
44.4 Attack: ”Does 150 prove correctness?” . . . 176
45 · Trace reproducibility . . . 178
45.1 The concept, precisely . . . 178
45.2 Why this matters more here than in a sorting utility . . . 178
45.3 What could break reproducibility — the checklist . . . 178
45.4 The question in this area you should expect . . . 179
46 · The 768 sliding-window cases and the brute-force oracle . . . 180
46.1 The problem under test . . . 180
46.2 The 768 cases . . . 180
46.3 What an oracle is . . . 181
46.4 The oracle, line by line . . . 181
46.5 Why brute force, specifically . . . 182
46.6 The assumptions the sliding window requires . . . 182
46.7 The 768 cases: what they prove and what they do not . . . 183
46.8 The bounded differential search — the most interesting test in the project . . . 184
The toolchain . . . 186
47 · GitHub Actions, line by line . . . 187
47.1 The workflow . . . 187
47.2 The pipeline as a gate . . . 188
47.3 What CI catches, and what it cannot . . . 188
47.4 The CI questions . . . 189
48 · Vite, in the amount this project needs . . . 190
48.1 The scripts . . . 190
48.2 The measured production build . . . 190
48.3 The Rolldown alias — the detail that shows you read your own config . . . 191
48.4 Vite questions . . . 191
49 · React Router, as this project uses it . . . 193
49.1 The pattern . . . 193
49.2 The URL parameters, precisely . . . 193
49.3 Client-side routing questions . . . 194
50 · Tailwind, in three pages instead of thirty . . . 196

50.1 The configuration . . . 196
50.2 Where Tailwind is actually used — the honest split . . . 196
50.3 The three Tailwind facts that matter here . . . 197
51 · JavaScript, only where it matters to AlgoViz . . . 198
51.1 Three language questions AlgoViz makes concrete . . . 199
52 · Performance: the real limits and the real fixes . . . 200
52.1 The measured baseline . . . 200
52.2 Where the time actually goes (in order) . . . 200
52.3 What would break first, in order . . . 201
52.4 The optimizations already present . . . 201
53 · Security and trust boundaries . . . 203
53.1 The trust boundary diagram . . . 203
53.2 What each control defends against . . . 203
53.3 What the project is safe from, by construction . . . 204
53.4 What is genuinely exposed . . . 204
Debugging master section . . . 206
54 · Forty debugging scenarios . . . 207
54.0 The universal method . . . 207
Playback and transport . . . 207
1 — Playback advances two frames per step . . . 207
2 — Pause does not stop the animation . . . 207
3 — The cursor exceeds the frame count . . . 208
4 — The last frame never renders . . . 208
5 — Seek jumps to the wrong frame . . . 208
6 — Step backward does nothing at frame 5 . . . 208
7 — The keyboard arrows stop working after the first press . . . 209
8 — Multiple keyboard listeners accumulate . . . 209
9 — Space toggles playback while typing in the dataset field . . . 209
10 — Speed changes have no effect until playback is toggled . . . 209
11 — The animation continues after navigating away . . . 210
12 — Play does nothing at the very start (frame 0 of a 2-frame trace) . . . 210
Engine and frames . . . 210
13 — The final frame is not sorted . . . 210
14 — The algorithm returns correct results but the frames are wrong . . . 210
15 — Duplicate element IDs in a frame . . . 211
16 — A merge-sort value appears to vanish mid-animation . . . 211
17 — Trace reproducibility fails intermittently . . . 211
18 — The generator mutates the caller’s array . . . 211
19 — Empty input produces a frame claiming a non-existent index is sorted . . . 212
20 — Frame count is unexpectedly large . . . 212
Validation and input . . . 212

21 — Invalid input is accepted . . . 212
22 — A valid input is rejected . . . 212
23 — Applying the same input resets the animation unexpectedly . . . 213
24 — The commit is invisible because the animation is still running . . . 213
Comparison . . . 213
25 — Comparison starts from different data . . . 213
26 — Operation counts look inconsistent between panels . . . 213
27 — The comparison preview goes blank . . . 214
28 — Selecting the same algorithm twice . . . 214
29 — The meters look wrong for very small counts . . . 214
30 — Swapping which algorithm is primary changes the numbers . . . 214
Persistence and import . . . 215
31 — A saved experiment disappears after reload . . . 215
32 — localStorage contains corrupted data . . . 215
33 — JSON import crashes the app . . . 215
34 — Exported file cannot be re-imported . . . 215
35 — Save reports failure even though storage works . . . 216
Rendering and React . . . 216
36 — Excessive re-rendering . . . 216
37 — The wrong frame renders for one paint . . . 216
38 — An error in one module blanks the whole app . . . 216
39 — A lazy route shows the fallback forever . . . 217
40 — CI passes but the feature is broken . . . 217
Code walkthrough master section . . . 218
55 · Thirty walkthroughs . . . 219
Block 01 · The algorithm registry . . . 219
Block 02 · parseInput in full . . . 219
Block 03 · The reducer . . . 220
Block 04 · The seeded dataset generator . . . 220
Block 05 · The timer effect . . . 221
Block 06 · The keyboard effect . . . 221
Block 07 · The load function . . . 222
Block 08 · apply and the error path . . . 223
Block 09 · Import, gate by gate . . . 223
Block 10 · Save . . . 224
Block 11 · The derivation block . . . 224
Block 12 · ArrayView . . . 224
Block 13 · The status tag ternary chain . . . 225
Block 14 · The timeline range input . . . 226
Block 15 · The transport buttons . . . 226
Block 16 · Speed select . . . 227

Block 17 · The state inspector . . . 227
Block 18 · The comparison panel . . . 228
Block 19 · The page footnote . . . 228
Block 20 · Bubble sort, the whole generator . . . 228
Block 21 · Merge sort’s buffer write-back . . . 230
Block 22 · Quick sort’s partition . . . 231
Block 23 · Heap sort’s heapify . . . 232
Block 24 · The windowTrace bug flag . . . 233
Block 25 · The bounded search . . . 234
Block 26 · The test harness for sorting . . . 234
Block 27 · The brute-force oracle . . . 235
Block 28 · The routing table . . . 235
Block 29 · The ESLint configuration . . . 236
Block 30 · package.json . . . 238

Whiteboard coding . . . 239

56 · Whiteboard exercises . . . 240

56.0 How to perform on a whiteboard . . . 240
Exercise 1 · Bubble sort — the interview version . . . 240
Exercise 2 · Selection sort . . . 241
Exercise 3 · Insertion sort . . . 241
Exercise 4 · Merge sort . . . 242
Exercise 5 · Quick sort . . . 242
Exercise 6 · Heap sort — the sixth algorithm . . . 243
Exercise 7 · The playback reducer . . . 243
Exercise 8 · A frame generator . . . 244
Exercise 9 · The input parser . . . 245
Exercise 10 · JSON validation . . . 246
Exercise 11 · The sliding window, and its precondition . . . 247

Resume defence . . . 249

57 · Defending the resume entry, claim by claim . . . 250

57.0 The entry, split into atomic claims . . . 250
57.1 Claim 1–3 — ”Engineered recorded-frame playback for 6 sorting algorithms using React reducers” . . . 251
57.2 Claims 4–7 — ”enabling pause/resume, single-step navigation, speed control, and timeline seeking” . . . 251
57.3 Claim 8–9 — ”same-input algorithm comparison with percentage-aligned replay” . . . 252
57.4 Claim 10 — ”recorded operation counts” . . . 253
57.5 Claim 11 — ”browser-local experiment saves and validated JSON import/export” . . . 254
57.6 Claims 12–14 — the verification bundle . . . 255
57.7 The three sentences that must never leave your mouth . . . 256

The hostile interviewer . . . 258

58 · ”You’re bluffing” — the hostile round . . . 259

58.0 The rules for this round . . . 259

Group A · ”You didn’t build this” (Q1–Q6) . . . 259
Group B · ”The architecture is wrong” (Q7–Q18) . . . 260
Group C · ”Your tests prove nothing” (Q19–Q28) . . . 261
Group D · ”The product is trivial” (Q29–Q38) . . . 262
Group E · ”Your numbers are made up” (Q39–Q46) . . . 264
Group F · ”You’re hiding something” (Q47–Q55) . . . 264
Honest weakness defence . . . 267
59 · Naming the weaknesses first . . . 268
59.0 The severity model . . . 268
59.1 S1 — Defects you should volunteer . . . 268
W1 · Merge sort emits frames with duplicate element ids . . . 268
W2 · The reducer accepts non-integer and NaN cursors . . . 268
W3 · tick starts playback even when it was paused . . . 269
W4 · windowTrace has no positivity guard . . . 269
W5 · Merge’s buffer state is invisible, so a value appears to vanish . . . 269
59.2 S2 — Verification gaps (these matter more than the bugs) . . . 269
W6 · The id assertion is on the final frame only . . . 269
W7 · Nothing renders a component, so the timer is untested . . . 269
W8 · No error boundary anywhere . . . 269
W9 · Nothing verifies the alignment formula beyond its arithmetic . . . 270
W10 · Import has nine defined behaviour cases and zero automated ones . . . 270
W11 · No test would catch a regression in the derived clamp . . . 270
W12 · catch {} on the storage write discards the reason . . . 270
59.3 S3 — Deliberate scope boundaries (disclosed, therefore defensible) . . . 270
W13 · A 24-element cap . . . 270
W14 · Recording frames is memory-hungry by construction . . . 270
W15 · length duplicated in the reducer state . . . 270
W16 · The trace is computed twice per commit . . . 270
W17 · Counter units differ per algorithm . . . 271
W18 · Heap sort’s comparison counter increments unconditionally . . . 271
W19 · Bubble sort has no early exit . . . 271
W20 · No backend, no accounts, no sync . . . 271
W21 · localStorage is not secure storage . . . 271
W22 · No URL-encoded full experiment . . . 271
59.4 S4 — Debt and polish . . . 271
W23 · Nine dead components plus two dead modules . . . 271
W24 · A fetch to a third-party counter sits in unimported code . . . 271
W25 · A lint rule is disabled for a directory . . . 271
W26 · SortingLab.jsx is 596 lines . . . 271
W27 · The keyboard effect re-registers on every cursor change . . . 272
W28 · The bar height has a 25% floor . . . 272

W29 · No dark mode, no shortcut discovery, no onboarding . . . 272
W30 · Shuffle's first click reloads seed 42 . . . 272
59.5 The three-sentence version . . . 272
Cross-domain questioning . . . 274
60 · Fifty-four cross-domain questions . . . 275
60.1 Algorithms × React rendering . . . 275
60.2 Algorithms × the browser event loop . . . 275
60.3 State management × testing . . . 276
60.4 Persistence × security . . . 277
60.5 Build, bundling and module systems . . . 278
60.6 URL, routing and HTTP semantics . . . 278
60.7 CSS, layout and rendering . . . 279
60.8 Numerics and JavaScript semantics . . . 279
60.9 Complexity, scale and cost . . . 280
60.10 Process, tooling and the human layer . . . 281
Two hundred rapid-fire questions . . . 282
61 · 200 rapid-fire questions . . . 283
61.1 Engine and frames (1–18) . . . 283
61.2 The reducer and transport (19–34) . . . 284
61.3 Playback, timers and input (35–52) . . . 284
61.4 React state and effects (53–70) . . . 285
61.5 The six algorithms (71–90) . . . 286
61.6 Comparison and counts (91–106) . . . 286
61.7 Testing (107–124) . . . 287
61.8 Persistence and import (125–142) . . . 288
61.9 Build, CI and tooling (143–160) . . . 289
61.10 JavaScript and React fundamentals (161–178) . . . 289
61.11 Performance and security (179–200) . . . 290
One hundred flashcards . . . 293
62 · 100 flashcards . . . 294
62.1 Engine, frames and algorithms (1–30) . . . 294
62.2 Reducer and transport (31–52) . . . 295
62.3 State, input and comparison (53–72) . . . 295
62.4 Persistence, import, build and CI (73–90) . . . 296
62.5 Testing (91–100) . . . 297
Ten mock interviews, first half . . . 298
63 · Mock interviews 1–5 . . . 299
Interview 1 · Dr. Arundhati Rao — the algorithms professor . . . 299
Questions . . . 299
Model answers . . . 299
Interview 2 · Kevin Osei — the senior frontend engineer . . . 301

Questions . . . 301
Model answers . . . 301
Interview 3 · Marisol Vega — the staff engineer . . . 303
Questions . . . 303
Model answers . . . 304
Interview 4 · Tim Brennan — the skeptical hiring manager . . . 305
Questions . . . 305
Model answers . . . 306
Interview 5 · Priya Nair — the QA lead . . . 307
Questions . . . 308
Model answers . . . 308
Ten mock interviews, second half . . . 310
64 · Mock interviews 6–10 . . . 311
Interview 6 · Dmytro Kovalenko — the platform engineer . . . 311
Questions . . . 311
Model answers . . . 311
Interview 7 · Sofia Lindqvist — the security engineer . . . 313
Questions . . . 313
Model answers . . . 313
Interview 8 · Jake Torres — the startup CTO . . . 315
Questions . . . 315
Model answers . . . 316
Interview 9 · Hana Sato — the performance engineer . . . 317
Questions . . . 317
Model answers . . . 318
Interview 10 · Youssef Al-Amin — the director’s round . . . 320
Questions . . . 320
Model answers . . . 320
Five-day plan and priority tiers . . . 323
65 · The five-day master plan . . . 324
65.1 How the five days are shaped . . . 324
65.2 Day 1 — READ the source . . . 324
65.3 Day 2 — UNDERSTAND the engine . . . 325
65.4 Day 3 — SPEAK it . . . 325
65.5 Day 4 — CODE it . . . 326
65.6 Day 5 — PRACTICE under pressure . . . 326
65.7 If you only have one day . . . 327
66 · Priority tiers: what to know versus what to recognise . . . 328
66.1 Files . . . 328
66.2 Hooks and React concepts . . . 328
66.3 Algorithms . . . 328

66.4 Tests and verification . . . 329
66.5 Config and tooling . . . 329
66.6 Concepts, in one ranked list . . . 329
Final checklist, night before, and the cheat sheet . . . 331
67 · The final checklist . . . 332
67.1 Project identity (1–12) . . . 332
67.2 Engine and frames (13–32) . . . 332
67.3 Reducer and playback (33–50) . . . 333
67.4 State, input and comparison (51–68) . . . 333
67.5 Persistence, import, testing, tooling (69–90) . . . 333
67.6 Weaknesses, defence and meta (91–110) . . . 334
68 · Night-before revision sheet . . . 335
68.1 The numbers . . . 335
68.2 The five expressions . . . 335
68.3 The five sentences . . . 335
68.4 The three never-says . . . 336
68.5 The ten attacks and their first sentence . . . 336
68.6 Tonight’s routine . . . 336
69 · The fifteen-minute cheat sheet . . . 338
69.1 The pitch — 20 seconds . . . 338
69.2 The three files . . . 338
69.3 The architecture in four lines . . . 338
69.4 The reducer, complete . . . 338
69.5 Numbers to have ready . . . 338
69.6 The four disclosures around comparison . . . 338
69.7 Top weaknesses, ranked . . . 339
69.8 Answers for the six most likely questions . . . 339
69.9 Your three questions for them . . . 339
69.10 The last thing . . . 339

CHAPTER 01

01 · How to use this handbook

Read this page first. It tells you what you are holding, how claims are labelled, and how to study it.

This handbook was written by reading your repository, not by reading your resume. Every technical statement about AlgoViz is grounded in a file that exists in the project at commit `80f9862` ("Update Shell.jsx", 15 Sep 2026), in the 61 JavaScript/JSX modules under `src/`, the 140-line test file, the CI workflow, and the build configuration. Where a statement goes beyond what the code shows, it is labelled.

Project: AlgoViz 2.0 — an in-browser algorithm laboratory. **Stack as installed:** React 19.2.1, react-dom 19.2.1, react-router-dom 7.18.3, Tailwind CSS 3.4.17, Vite (aliased to `rolldown-vite@7.2.5`), Lucide React icons, ESLint 9, Node's built-in test runner. **No backend, no database, no authentication, no AI service, no API keys.**

1.1 The five claim labels used in this book

Every important claim carries one of these tags. They are not decoration: in a real interview, the difference between "the code does this" and "I believe this is why the code does this" is the difference between a hire and a rejection. Say the label out loud mentally before you answer.

Label	Meaning	How to say it
[VERIFIED]	Directly supported by source code you can open right now	"If you open `src/lab/engine.js` line 80, you can see..."
[INFERENCE]	A reasonable engineering explanation inferred from the implementation, not written down anywhere	"My reading of the code is that... I did not document that at the time."
[GENERAL KNOWLEDGE]	Standard React / JavaScript / algorithm / testing background needed to understand the implementation	"That is the general rule; here is how it shows up in my code."
[NOT VERIFIED]	The available source does not establish the claim	"I cannot prove that from the repository."
[PROPOSED]	A future improvement or design idea, **not** an existing feature	"That is not implemented. It is the next thing I would do, and here is how."

DANGER ZONE — WHERE CANDIDATES DIE

THE SINGLE MOST DANGEROUS SENTENCE IN AN INTERVIEW

"The code does X" — when the code does not do X. A senior interviewer will ask you to open the file. Everything in this handbook is written so that you never have to walk a claim back. When the source is silent, this book says [NOT VERIFIED] and gives you a defensible sentence that still sounds strong: "The repository doesn't establish that; here is what I can prove."

1.2 How to read this book

- **Days before the interview:** use Chapter 65 (the 5-day plan). It tells you exactly which chapters to read on which day, and what to say out loud at the end of each day.
- **Two days before:** use Chapter 66 (RED / ORANGE / YELLOW / GREEN). It filters this entire book down to what is actually worth memorising.
- **The night before:** read Chapter 68 only. Five pages, everything that matters.
- **Fifteen minutes before:** read Chapter 69. Two pages, written to be read in fifteen minutes.

- **When you are stuck on a file:** Chapter 55 is a walkthrough of the ~30 most important code blocks in the project, with line-level explanation and the question an interviewer would ask about each one.
- **When you need to rehearse:** Chapters 63–64 are ten full mock interviews with model answers. Chapters 58 and 60 are the aggressive and cross-domain question banks. Chapter 61 is the 200-question rapid-fire drill, Chapter 62 the flashcards, and Chapter 57 the claim-by-claim defence of your resume entry.

Boxes are colour-coded by purpose: **green** is a must-know or a model answer, **navy** is an interview question, **red** is a danger zone or trap, **amber** is a weakness or an unverified claim, **purple** is a proposed (not implemented) improvement, and **teal** is a line you can say out loud.

1.3 What AlgoViz is, in one paragraph you can memorise

[VERIFIED] AlgoViz 2.0 is a client-side React application that makes algorithm execution inspectable. Its central idea is stated in the product copy itself: an animation can show **what happened**, but AlgoViz shows **why**. The mechanism is a **recorded-frame architecture**: instead of animating while the algorithm runs, each algorithm generator executes to completion and returns a complete array of immutable frames — one frame per meaningful state. The UI then simply renders whichever frame a playback cursor points at. Playback is therefore not execution: pausing, stepping, seeking and rewinding are all array indexing into a recording. Rewind is recorded-state replay, not JavaScript running backwards.

CHAPTER 02

02 · Resume reality check: every claim, audited

Before you defend the project, know exactly which of your public claims the repository proves.

Your resume entry makes eight assertions. Each one was checked against the source. This is the audit an aggressive interviewer performs mentally in the first two minutes.

2.1 The audit table

#	Resume claim	Verdict	Where it is proven
1	"recorded-frame playback for 6 sorting algorithms using React reducers"	[VERIFIED]	`src/lab/engine.js` lines 8–62 register exactly six algorithms (`bubble, selection, insertion, merge, quick, heap`); each `run` is a frame generator; `SortingLab.jsx` line 98 uses `useReducer(playbackReducer, …)`
2	"pause/resume, single-step navigation, speed control, timeline seeking"	[VERIFIED]	`toggle` action = play/pause (`engine.js` 84–89); step buttons = `seek(cursor ± 1)` (`SortingLab.jsx` 378–408); speed `<select>` with four values 600/300/160/50 (`SortingLab.jsx` 410–421); `<input type="range">` timeline (`SortingLab.jsx` 356–367)
3	"same-input algorithm comparison with percentage-aligned replay"	[VERIFIED]	`otherFrames` runs the second algorithm on the **same** `input` (`SortingLab.jsx` 94–97); `compareCursor` is a proportional mapping (`SortingLab.jsx` 201–203); the UI's own caption reads "Preview aligned by percentage of trace—not equivalent operations."
4	"recorded operation counts"	[VERIFIED], with a scope limit	Every frame carries `currentStats {passes, comps, swaps}`; the comparison panel reads **`comps` only** (`SortingLab.jsx` 515–521). Pass/exchange counters are labelled implementation-specific in the UI footnote (lines 587–592)
5	"browser-local experiment saves"	[VERIFIED], with limits	`localStorage` key `algoviz.experiments.v1`, latest **6** saved (`SortingLab.jsx` 181–197, 77–92)
6	"validated JSON import/export"	[VERIFIED]	Export builds `{version:1, algorithm, input, cursor}` (line 236); import enforces a 16 KB size cap, `version === 1`, a known algorithm key, and an array input, then re-validates through `parseInput` (lines 156–180)
7	"150 algorithm-input cases" for sorting correctness and trace reproducibility	[VERIFIED]	`tests/engine.test.js` lines 12–36: six algorithm groups × 25 inputs (5 fixed edge cases + 20 seeded datasets via `makeDataset("random", seed)`), asserting sorted output, input preservation, deep-equal trace reproducibility and final ID uniqueness

#	Resume claim	Verdict	Where it is proven
8	"768 sliding-window cases against brute force"	[VERIFIED]	`tests/engine.test.js` lines 82–101: 64 arrays (all 4^3 combinations of values 1–4 at length 3) × targets 1–12 = 768 assertions against an in-test brute-force oracle
9	"automated tests, lint, and build checks through GitHub Actions"	[VERIFIED]	`.github/workflows/ci.yml`: `npm ci` → `npm test` → `npm run lint` → `npm run build` on every push and pull request

Result: there is no claim in your resume entry that the repository contradicts. That is a genuinely strong position, and it is worth saying calmly in an interview: "Everything in that bullet is traceable to a file — I can open any of them."

2.2 The five nuances that will separate you from a bluffing candidate

These are not problems. They are the details that show you actually read the code you shipped. Raise them **before** the interviewer finds them.

MUST KNOW
NUANCE 1 — "150 CASES" MEANS 150 ALGORITHM-INPUT PAIRS, NOT 150 DISTINCT INPUTS

The loop is `6 algorithms × 25 inputs`. Twenty of those 25 inputs per algorithm are generated from a fixed linear-congruential sequence seeded 0…19, so all six algorithms are tested on the same 25 arrays. The 150 is a real count of executions checked; it is not 150 hand-written test vectors. [VERIFIED]

MUST KNOW
NUANCE 2 — "OPERATION COUNTS" IN THE COMPARISON PANEL MEANS COMPARISONS ONLY

The panel's own footnote says it: "Legacy generators count exchanges differently; comparison mode deliberately uses comparisons only." Merge sort's generator increments `swaps` on every write out of the temporary buffer, not on a physical swap — so its `swaps` number is not the same kind of event as bubble sort's `swaps`. [VERIFIED] This is the single best answer to "are your counters comparable?" and it is already written into your own UI.

MUST KNOW
NUANCE 3 — PERCENTAGE-ALIGNED REPLAY IS A VIEWING CONVENTION, NOT A RESULT

It positions a preview frame proportional to the primary cursor's progress through its own trace. A 100-frame trace and a 40-frame trace are shown at 10% / 10%, 50% / 50%, 100% / 100% — so at the same percentage you are looking at different numbers of executed operations. [VERIFIED]

WEAKNESS / LIMITATION
NUANCE 4 — THE UI ALLOWS COMPARING AN ALGORITHM WITH ITSELF

The comparison `<select>` lists all six algorithms and is not filtered against the primary selection, so a user can select Bubble vs Bubble and see two identical traces. There is no guard. It is harmless (the counts match) but it is a real gap. [VERIFIED]

WEAKNESS / LIMITATION
NUANCE 5 — TAILWIND IS IN THE STACK BUT NOT IN THE NEW LAB UI

`index.css` contains the three `@tailwind` directives and the legacy module pages under `src/modules/**` are styled almost entirely with Tailwind utility classes. The current Sorting Lab uses hand-written CSS (`src/lab/lab.css`, 1,756 lines) with an `av-` prefix. Tailwind is still compiled and shipped, and it is still used by ten of the twelve module routes. [VERIFIED] Do not say "AlgoViz is a Tailwind app"; say "Tailwind ships and styles the legacy modules; the lab I rebuilt uses scoped CSS."

2.3 The honesty script (memorise this shape)

> **SAY THIS OUT LOUD**
> **HOW TO OPEN THE DISCUSSION**
> "Quick framing before we go deep, because I think it matters: AlgoViz is a client-side React app with no backend. The interesting engineering is in three places — recording execution as frames instead of animating it live, a reducer that owns playback, and a verification story built on Node's test runner rather than a test framework. I'm happy to open any file. And if you find a claim I can't back with code, I'd rather you tell me — I'd like to know."

That paragraph does three things at once: it scopes the project before the interviewer can scope it for you, it names your three strongest mechanisms, and it pre-empts the "are you bluffing" question by inviting the audit.

2.4 The deployment claim, handled honestly

The README links a live deployment — `https://algoviz-2-0.vercel.app/` appears in eight places, and `vercel.json` supplies the SPA rewrite that keeps client routes working on refresh. That is the whole of what the repository establishes about deployment:

Claim	Verdict
A Vercel deployment is intended and configured	[VERIFIED] — `vercel.json` exists and the README links the URL
The site is reachable today	[NOT VERIFIED] — I did not fetch it; this handbook is grounded in the repository
CI deploys it	[VERIFIED as a negative] — `.github/workflows/ci.yml` has no deploy step; the deployment is external to the repository
There is a custom domain, uptime monitoring or analytics	[NOT VERIFIED] — nothing in the repository suggests any of them

How to answer. *"There's a Vercel deployment configured through `vercel.json` and linked from the README. Deployment happens outside the repository — CI only verifies, it doesn't publish, and that's deliberate: nothing merges and ships on its own. I'd be careful about claiming uptime or usage, because nothing in the repo measures either."*

If the interview is remote, have the live URL open in a tab and the repository open in another. **Showing the deployed app is stronger than describing it — but check the link the morning of the interview**, because a claim you cannot demonstrate is worse than a claim you scoped.

PART I

Project identity

What AlgoViz is, what problem it solves, who it is for, what it deliberately is not — and four pitches at four lengths.

CHAPTER 03

03 · What AlgoViz is and why it exists

The concept, the problem, the audience, and the boundary that keeps the project honest.

3.1 The problem the project actually solves

[VERIFIED] The repository states its own thesis in two places. The README's opening line is "Don't just watch. Understand the why." The Overview page renders: "An algorithm is a sequence of decisions. Slow it down, rewind it, and challenge every assumption."

> **KEY TAKEAWAY**
> **THE ENGINEERING PROBLEM IN ONE SENTENCE**
> A normal algorithm implementation returns a **result**. A learning tool needs a **sequence of explainable intermediate states** — and those two requirements pull in opposite directions, because everything that makes an algorithm fast (temporary variables, in-place writes, recursion, buffers) destroys the information a learner needs to see.

Think about what actually happens when you study sorting with a typical visualiser. The bars move, the array ends up sorted, and you have learned almost nothing, because the decisions flashed past at animation speed. A learner's real question is never "what does a sorted array look like?" — it is:

- Which two elements were compared, and why those two?
- Was a swap performed, or was the comparison false?
- What does the algorithm believe is true right now (its invariant)?
- What did it already know at frame 12 that it didn't know at frame 4?
- If two algorithms get the same input, do they make the same number of decisions?

[VERIFIED] AlgoViz answers these by making execution **inspectable** rather than merely animated. Each generator produces frames carrying: the full array state, which indices are currently active, which indices are already finalised, the pivot index where relevant, the recursion range for divide-and-conquer algorithms, a machine-readable `codeLine` tag, a human sentence in `msg`, and a snapshot of running counters in `currentStats`.

3.2 The five capabilities that define the product

[VERIFIED] All five exist in `src/lab/SortingLab.jsx` and `src/lab/engine.js`.

Capability	Mechanism in the source	What the learner gets
Recorded-frame playback	Each `run()` returns a complete `frames[]` array before playback begins (`engine.js` 8–62)	Any state in the execution can be revisited byte-for-byte
Transport control	`playbackReducer` with `load`/`toggle`/`tick`/`seek` (`engine.js` 80–105)	Play, pause, replay, step forward, step back, restart, seek
Explained steps	`msg` + `codeLine` + `activeIndices` on every frame	The reason for each state, not just the state
Same-input comparison	`otherFrames` runs a second algorithm on the identical input array (`SortingLab.jsx` 94–97)	A like-for-like comparison of recorded comparison counts
Counterexample lab	`findCounterexample()` bounded exhaustive search (`engine.js` 152–169)	A reproducible input that proves a plausible implementation wrong
Portable experiments	Save to `localStorage`, export/import JSON (`SortingLab.jsx` 156–197)	An interesting input and frame position can be revisited or shared as a file

3.3 Who it is for

[VERIFIED] The product copy, the README, and the twelve module routes make the audience explicit.

- **Primary audience:** a learner who can read the code of an algorithm but cannot yet predict its behaviour on a new input. The Overview calls the goal "your next aha moment".
- **Secondary audience:** someone preparing for interviews who needs to explain why two $O(n^2)$ sorts behave differently on the same array.
- **Tertiary audience (the one that makes the project impressive):** a sceptical reader of their own code. The Counterexample Lab exists precisely because "a working example is not proof".

> **NOTE**
> **WHO IT IS NOT FOR**
> [VERIFIED] It is not a teaching tool for absolute beginners who have never seen a loop — there is no guided lesson sequencing or progress tracking. It is also not a tool for practising writing algorithms: the algorithm set is fixed and closed. The README says so directly: "Not a general code debugger: execution is limited to built-in implementations."

3.4 What AlgoViz is NOT — the boundary list

[VERIFIED] This is the most valuable list in the whole handbook, because almost every aggressive interview question is an attempt to get you to claim one of these. Say them proactively.

AlgoViz is NOT	Evidence
Not full-stack / not MERN	`package.json` has zero backend dependencies; the only `fetch(` in the repository sits in a **dead, unimported** component (`src/components/CardsGrid.jsx` line 21)
Not a database application	Persistence is `localStorage` only, scoped to one browser and one origin
Not a benchmark platform	The comparison panel explicitly avoids wall-clock claims: "No wall-clock claims: animation speed is not algorithm performance."
Not an AI tutor	Explanations come from static `msg` strings and static tutorial data files
Not a general code debugger / not a sandbox for user code	Users supply numbers, never code; no evaluator, no worker, no `eval`
Not a formally verified system	Tests are examples and bounded search, not proofs
Not an unbounded engine	Input is capped at 24 integers by `parseInput`
Not multi-user	No accounts, no sync, no sharing server; export/import is a file, not a service
Not mobile-app	It is a responsive web app; the README documents a 390 px smoke check, nothing more
Not a security-audited system	No audit exists; the README states the dependency audit is "not a security certification"

> **TRAP**
> **THE TRAP DISGUISED AS A COMPLIMENT**
> **"This would be great with a backend so users could share experiments."** The trap is agreeing too enthusiastically and implying you had designed for it. The defensible answer: "A server would change the trust model — right now the app has no upload path and no accounts, so there is nothing to secure server-side. If sharing became a requirement, I'd add a small API for experiment documents with server-side schema validation and rate limiting, and I'd keep the client-side validation as a UX layer, not as the security boundary." That answer shows you understand that client-side validation is a UX mechanism, not a security control.

3.5 Feature inventory, verified against the source

Feature	Status	Source
Six sorting algorithms	[VERIFIED]	`engine.js` 8–62
Frame recording with metadata	[VERIFIED]	six generators under `src/modules/core/algorithm/sorting/algorithms/`
Play / pause / replay	[VERIFIED]	`toggle` action; button label switches to "Replay" at the last frame
Step forward / step back / restart	[VERIFIED]	`seek(cursor ± 1)`, `seek(0)`
Timeline scrubber	[VERIFIED]	`<input type="range" min=0 max={frames.length - 1}>`
Speed control (4 presets)	[VERIFIED]	600 / 300 / 160 / 50 ms; labels 0.5× / 1× / 2× / 6×
Keyboard shortcuts	[VERIFIED]	Space, ArrowLeft, ArrowRight; ignored inside form controls
Live state inspector	[VERIFIED]	comparisons, passes, exchanges + `activeIndices`, `pivotIndex`, `sortedIndices`
Per-algorithm invariant, idea, complexity	[VERIFIED]	`engine.js` metadata objects, rendered in the "What stays true?" panel
Same-input comparison with percentage alignment	[VERIFIED]	`compareCursor` computation
Comparison throughput meters	[VERIFIED]	`av-meter` width proportional to `comps`
Save / list / reload experiments (max 6)	[VERIFIED]	`localStorage` + filtering on read
JSON export / import with validation	[VERIFIED]	`download()` + `importExperiment()`
Seeded dataset generator + 4 presets	[VERIFIED]	`makeDataset(kind, seed)`
Counterexample lab with bounded search	[VERIFIED]	`findCounterexample()`
Twelve non-sorting visualisation modules	[VERIFIED]	`Home.jsx` routes 45–79, `constants.js`
Learning notes / pseudocode per algorithm	[VERIFIED]	`TutorialTab` rendered from `simplesortData` / `sortingTutorialsData`
Dark UI with responsive breakpoints	[VERIFIED]	`lab.css` media queries at 1600/1200/950/680 px; `prefers-reduced-motion` block at line 1570

NOT VERIFIED BY SOURCE

CLAIMS THE REPOSITORY CANNOT ESTABLISH

"v1" features, development timeline, how long the project took, who wrote which line, whether the algorithms were written from scratch or adapted, GPU/CPU performance figures, real user counts, deployment history beyond the `vercel.json` and README instructions, and whether any employer used it. None of this is in the repository. If asked about authorship history, answer factually about the work you did and do not invent a timeline.

CHAPTER 04

04 · The stack, and the four pitches

What is actually installed, why each choice is defensible, and four scripts you can deliver at four lengths.

4.1 The stack, verified from `package.json` and the lockfile

Layer	Declared	Installed	Notes that matter
UI library	`react` ^19.2.0	19.2.1	Uses `createRoot`, `StrictMode`, hooks only; no class components
DOM renderer	`react-dom` ^19.2.0	19.2.1	`react-dom/client` entry
Routing	`react-router-dom` ^7.11.0	7.18.3	`BrowserRouter` + `Routes`/`Route`; `useSearchParams`, `useLocation`, `useNavigate`, `NavLink`, `Link`
Styling	`tailwindcss` ^3.4.17 (dev)	3.4.17	Compiles `@tailwind` directives from `src/index.css`; used by legacy modules
Styling (lab)	—	hand-written	`src/lab/lab.css` 1,756 lines; `modules.css` 505; `lesson.css` 419; `styles/global.css` 258
Icons	`lucide-react` ^0.556.0	0.556.0	Tree-shaken icon components (Play, Pause, ChevronLeft…)
Build	`vite` aliased to `npm:rolldown-vite@7.2.5`	7.2.5	An npm alias, not a forked config. `overrides` pins the same version
Lint	`eslint` ^9.39.1 + `@eslint/js`, `eslint-plugin-react-hooks`, `eslint-plugin-react-refresh`, `globals`	9.39.1	Flat config in `eslint.config.js`
Tests	none in dependencies	—	`node --test tests/*.test.js`, built-in `node:test` + `node:assert/strict`
Runtime	`engines.node >= 22.12.0`, `.nvmrc` = 22	22.22.3 local	CI pins Node 22
Deploy	`vercel.json` SPA rewrite	—	README documents Vercel preset/commands

MUST KNOW

THE STACK SENTENCE TO SAY IN AN INTERVIEW

"React 19 with hooks and no state library, React Router 7 with lazy-loaded routes, Tailwind plus scoped CSS, Vite for build and dev server, ESLint for static checks, Node's built-in test runner for correctness checks, GitHub Actions for CI, Vercel for static hosting. There is no backend and no runtime dependency beyond React, React Router and an icon library."

4.2 Why each choice is defensible (and what the alternative would cost)

Choice	Why it fits AlgoViz	What you gave up
React	The UI is a function of one cursor value; re-rendering on state change is exactly the required behaviour	Bundle size, a build step, and framework churn
`useReducer` for playback	Four named transitions, all pure, all unit-testable without React	Slightly more ceremony than two `useState` calls — and that ceremony is the point

Choice	Why it fits AlgoViz	What you gave up
Vite	Instant dev server, native ESM, one config file, proper production bundling with code splitting	You inherit Rollup/Rolldown semantics (e.g. `rolldown-vite` alias must be understood)
React Router	Twelve deep-linkable module URLs plus query params (`?algorithm=quick`, `?compare=1`)	A routing dependency and the SPA-fallback requirement on the host
Tailwind + scoped CSS	Fast iteration for 12 legacy modules; precise control for the lab's bar/timeline visuals	Two styling systems in one codebase — an honest trade-off to acknowledge
`localStorage`	Zero infrastructure for a client-only tool; instant, offline, private	~5 MB per origin, synchronous API, no durability guarantee, no cross-device sync
Node test runner	Zero extra dependency; runs the actual generator modules in Node with no DOM	No DOM, no component tests, no coverage report, no watch UI
GitHub Actions	The checks already exist as npm scripts; CI is four lines	Nothing, for a public repo

INFERENCE

WHY THE SORT OF "WHY NOT REDUX / ZUSTAND / MOBX?" QUESTION YOU WILL GET

[INFERENCE] The code already keeps its state deliberately small: 12 `useState` calls and one `useReducer`, all local to a single component. There is no cross-component state except the URL, which is React Router's own. Adding a global store would introduce a second source of truth for data that only one screen uses. The honest answer: "State lives where it is used. The playback state is the only genuinely stateful thing, and a reducer already gives me the transition discipline a store would provide, without the dependency."

4.3 The four pitches

Deliver these from memory in your own words, not by reciting. The bracketed notes are the intent of each line — say the line in plain language, not the bracket.

SAY THIS OUT LOUD

30-SECOND PITCH (ELEVATOR / RECRUITER SCREEN)

"AlgoViz is a browser-based algorithm laboratory in React. The problem it solves is that most algorithm visualisers animate a result — you watch it finish, but you never see why the algorithm made each decision. So instead of animating execution, AlgoViz **records** it. Each algorithm runs to completion and produces a list of frames; every frame has the array, the active indices, the sorted region, a message and running counters. The UI plays those frames back through a small reducer, so play, pause, step, seek and rewind are all just moving an index into a recording. It has six sorting algorithms, same-input comparison using comparison counts rather than animation timing, local experiment saves with validated JSON import and export, and a test suite that checks correctness and trace reproducibility."

SAY THIS OUT LOUD

60-SECOND PITCH (HIRING MANAGER)

"The interesting part of AlgoViz isn't the visualiser UI, it's the execution model. A normal sorting function throws away everything interesting: comparisons, decisions, the moment the algorithm learns something. So I split the project into two halves. The engine — the six generators — runs the algorithm synchronously and returns a complete array of frames. A frame is a full snapshot plus metadata: the array at that step, which indices are being compared, which are already final, the pivot, the code line that produced it, a sentence explaining it, and the counters as they stood at that moment. The second half is playback: a `useReducer` with four actions — `load`, `toggle`, `tick`, `seek` — that owns only the cursor, the trace length and whether we're playing. A single effect schedules one timer and cleans it up. That split is what makes rewind possible: seeking backwards is selecting an earlier recorded state, not executing JavaScript backwards. On top of that, the same input can be run through two algorithms and compared by recorded comparison counts — never by animation speed, and the UI says so. Then there's the verification story: 150 sorting correctness and reproducibility cases, 768 sliding-window cases checked against an independent brute-force oracle in the test file, and a counterexample lab that searches a bounded domain for an input that proves an implementation wrong. CI runs tests, lint and build on every push."

SAY THIS OUT LOUD

2-MINUTE TECHNICAL PITCH (ENGINEER INTERVIEWER)

"AlgoViz is a client-side React 19 app, Vite-built, React Router 7 for twelve module routes. No backend at all.

The core abstraction is a recorded frame. Each of the six sorting algorithms is a generator function that takes a raw array of numbers and returns an array of frames. A frame carries the full array as objects with **stable IDs**, so equal values stay distinguishable; `activeIndices` for the elements under consideration; `sortedIndices` for the region already finalised; `pivotIndex` and `subArrayRange` where the algorithm has those concepts; a `codeLine` tag like `'compare'` or `'swap'`; a human `msg`; and `currentStats` — a snapshot of comparisons, passes and exchanges at that instant. The generators clone the array on every frame push, so frames are independent and immutable.

Playback is a separate concern. A single `useReducer` holds `{cursor, length, playing}` and accepts four actions. `load` resets for a new trace, `toggle` flips playing and rewinds if you're at the end, `tick` advances one frame and clears `playing` when it reaches the end, and `seek` clamps into `[0, length-1]` and pauses. One `useEffect` is the only timer owner: while playing, it schedules a `setTimeout` for the current speed and clears it on cleanup, so there is exactly one pending timer and it is cancelled when playback stops, speed changes or the component unmounts. Clamping happens twice — once in the reducer, once in the component when it derives `cursor` — so a stale cursor after an input change can't index out of bounds.

Comparison mode runs a second algorithm over the identical `input` array via a `useMemo`, so both traces come from the same data. Alignment is proportional: the preview frame is `round(cursor / (frames.length - 1) * (otherFrames.length - 1))`. That's a viewing convenience — I say explicitly in the UI that percentage alignment is not equivalent operations, and I compare recorded comparison counts, not wall-clock time, because animation speed is not algorithm performance.

Persistence is `localStorage` under `algoviz.experiments.v1`, capped at the latest six, filtered on read, and export/import is a versioned JSON document that is re-validated through the same `parseInput` the UI uses, with a 16 KB size cap.

Verification is `node:test` — six algorithm groups covering five edge inputs plus twenty seeded datasets each, which is the 150; assertions are sorted output, input preservation, deep-equal trace reproducibility and final ID uniqueness. Separately, the sliding-window implementation is checked against a brute-force oracle over 64 arrays × 12 targets — the 768 — and there's a bounded differential search that finds the first input where a deliberately buggy variant diverges. CI is four steps: clean install, tests, repository-wide lint, production build."

SAY THIS OUT LOUD

5-MINUTE DEEP TECHNICAL EXPLANATION (THE "TELL ME EVERYTHING" ROUND)

Use the 2-minute pitch as the spine, then extend it in this order — each extension has a matching chapter in this book:

1. **Why frames instead of live animation** (Ch. 08) — because you cannot rewind an animation you didn't record, and because computing the trace first makes the UI a pure function of `cursor`.
2. **What a frame contains, exactly** (Ch. 09) — the field table, which fields are optional per algorithm, and why the IDs are string keys `block-<original index>`.
3. **The reducer as a state machine** (Ch. 11) — the four actions, the boundary rules, and the three invariants (`0 <= cursor <= length-1`, `playing` only true below the last frame, `length` only changes on `load`).
4. **The timer contract** (Ch. 12) — one effect, one `setTimeout`, cleanup on every dependency change, why `setInterval` would need history to catch up and would still be wrong.
5. **The memory trade-off** (Ch. 05) — full snapshots are O(frames × n) element copies, which for a quadratic algorithm that copies per step is conceptually $O(n^3)$ storage; the project's answer is a hard 24-element input cap rather than a scalability claim. Measured: the reverse-sorted 24-element bubble trace is 1,154 frames and ~980 KB of JSON.
6. **Comparison without lying** (Ch. 12–14) — same input, separate traces, comparison counts only, percentage alignment for the preview, explicit on-screen disclaimers about what the numbers are not.
7. **The verification story** (Ch. 17–20) — what is tested, what an oracle is, why reproducibility is the property that matters for a frame-based tool, and precisely what finite testing does not prove.
8. **The honest weakness list** (Ch. 44) — the reducer only owns sorting playback; ten legacy modules still run their own `setTimeout`/`await` loops; there is dead code; there is no component or browser test in the repo.

DANGER ZONE — WHERE CANDIDATES DIE

IF YOU ONLY REMEMBER ONE THING FROM PART I

The sentence that makes this project sound senior rather than student: **"The algorithms run to completion and record their decisions; the UI replays the recording. Playback is indexing, not execution — which is why rewinding is free and why the reducer never needs to know anything about sorting."**

PART II

Complete project architecture

The real file map, the real data flow, and a colour code that tells you what to study first.

CHAPTER 05

05 · The repository, mapped

Every source file, what it does, whether it is live, and how much you need to know it.

5.1 The honest high-level map

The repository contains **two generations of code living side by side**. Understanding this is the difference between sounding like you know your own project and sounding like you inherited someone else's.

MUST KNOW
TWO GENERATIONS, ONE REPOSITORY

Generation 2 (the lab) — `src/lab/*`, `src/App.jsx`, `src/main.jsx`, `src/constants.js`, `src/pages/Home.jsx`, `src/components/*`, `src/styles/global.css`, `tests/*`. This is the React 19 + Vite + React Router application whose engine is `src/lab/engine.js`. Its centrepiece is `src/lab/SortingLab.jsx` (596 lines).

Generation 1 (the legacy modules) — `src/modules/**`. Twelve algorithm/pattern visualisations that each manage their own animation with `useState` + `async` loops and `setTimeout`-based waiting. They were kept working, share the new page chrome through `ModuleFrame.jsx`, and are **not** driven by the playback reducer. Their algorithm logic lives under a folder named `algorithm/sorting/` even when the algorithm is nothing to do with sorting.

This is a real, defensible narrative: "I rebuilt the sorting lab on a recorded-frame engine and gave the older modules the shared shell, but I did not migrate non-sorting playback to the reducer — the README lists that as the next migration. I'd rather say that than pretend the whole app uses one execution model."

5.2 The RED / YELLOW / GREEN file classification

Use this to allocate study time. **RED** = you must be able to open, explain line by line, and modify. **YELLOW** = you must know its role, inputs, outputs and one trade-off. **GREEN** = recognition only; know it exists and why.

File	Lines	Colour	Role
`src/lab/engine.js`	169	**RED**	The heart: algorithm registry, `parseInput`, `playbackReducer`, `makeDataset`, `windowTrace`, `findCounterexample`
`src/lab/SortingLab.jsx`	596	**RED**	The lab UI: state register, timers, keyboard, save/import/export, comparison
`src/modules/core/algorithm/sorting/algorithms/bubbleSort.js`	92	**RED**	Canonical frame generator; the one to walk through
`src/modules/core/algorithm/sorting/algorithms/mergeSort.js`	172	**RED**	Recursion + buffer + write-counting quirks
`src/modules/core/algorithm/sorting/algorithms/quickSort.js`	160	**RED**	Partition, pivot locking, sorted-index bookkeeping
`src/modules/core/algorithm/sorting/algorithms/insertionSort.js`	86	**RED**	`isLifting` flag, early-break frame
`src/modules/core/algorithm/sorting/algorithms/selectionSort.js`	73	**RED**	`minimumIndex` tracking

File	Lines	Colour	Role
src/modules/core/algorithm/sorting/algorithms/heapSort.js	120	**RED**	Two phases; recursive heapify; the "O(log n)*" footnote
tests/engine.test.js	140	**RED**	The 150, the 768, the oracle, the counterexample, playback tests
src/lab/Shell.jsx	116	**YELLOW**	Shared chrome: sidebar, nav, topbar, footer, scroll-to-top
src/lab/Overview.jsx	178	**YELLOW**	Landing page and library grid (also used as /library)
src/lab/Counterexample.jsx	203	**YELLOW**	Differential-execution UI with first-divergence display
src/lab/ModuleFrame.jsx	156	**YELLOW**	Breadcrumb, module switcher, Visualize/Learn segmented control
src/pages/Home.jsx	104	**YELLOW**	BrowserRouter + all routes + Suspense fallback + 404
src/constants.js	102	**YELLOW**	CORE_MODULES (6) + PATTERN_MODULES (6) metadata
package.json	36	**YELLOW**	Scripts, deps, engines, the Vite alias override
.github/workflows/ci.yml	15	**YELLOW**	The four CI steps
vite.config.js	9	**YELLOW**	React plugin, dev/preview host + allowedHosts
src/main.jsx	9	**YELLOW**	createRoot + StrictMode entry
src/App.jsx	12	**GREEN**	Thin wrapper that renders <Home />
src/index.css	12	**YELLOW**	The three @tailwind directives + body defaults — the only Tailwind entry
tailwind.config.js / postcss.config.js	9 / 6	**YELLOW**	Content globs and PostCSS pipeline
eslint.config.js	45	**YELLOW**	Flat config; the legacy set-state-in-effect exception is a great talking point
src/lab/lab.css	1756	**GREEN**	Lab visual system. Know: bar colours, timeline, breakpoints, reduced-motion
src/lab/modules.css	505	**GREEN**	Legacy module chrome
src/lab/lesson.css	419	**GREEN**	Tutorial/lesson layout
src/styles/global.css	258	**GREEN**	Root variables, dark theme, legacy page styles
src/modules/core/tutorials/*Data.js (10 files)	~700	**GREEN**	Static learning content per algorithm
src/modules/core/tutorials/TutorialTab.jsx	202	**GREEN**	Renders lesson sections from that data
src/modules/core/*.jsx (6) + src/modules/patterns/*.jsx (6)	~3,500	**GREEN**	Legacy visualisations; each self-manages playback
src/modules/core/algorithm/sorting/algorithms/*Logic.js (9)	~640	**GREEN**	Legacy frame producers for the non-sorting modules
vercel.json, .nvmrc, README.md, docs/*	—	**GREEN**	Deployment fallback, Node version, documentation
src/components/*.jsx (9 files)	~370	**GREEN — dead code**	Not imported anywhere; see 5.4
src/components/Layout.jsx	0	**GREEN — empty**	Zero bytes. It exists, it is empty, nothing imports it
src/App.css	43	**GREEN — orphan**	Not imported by any module

DANGER ZONE — WHERE CANDIDATES DIE
THE QUESTION YOU MUST BE READY FOR THE MOMENT YOU OPEN THE REPO

"What is `src/components/Layout.jsx`? It's empty." **And: "Who imports `CardsGrid.jsx`?"** Nobody. Nine components plus one empty file plus one orphan stylesheet are leftovers from an earlier structure. This is not a scandal — every evolving codebase has them — but you must know it and answer instantly: "Those are remnants from the earlier version's structure. They are not imported anywhere; Vite tree-shakes them out of the build entirely. They're on my cleanup list, and I keep them visible rather than deleting them blind so I don't lose the visit-counter logic that only exists there."

5.3 What happens to dead code in the build

[VERIFIED] I ran `npm run build`: 1,736 modules transformed, 19 emitted asset files (16 JavaScript chunks and 3 stylesheets). No chunk contains `CardsGrid`, `SimpleSorts` or `EfficientSorts` — unreferenced modules are eliminated. **But** the CSS is different: Tailwind scans `./src/**/*.{js,ts,jsx,tsx}` and emits any utility class it finds in source, including source that is never bundled. That is why utilities like `mt-12` from a dead component can still appear in the shipped CSS.

INFERENCE
WHY THIS MATTERS FOR AN INTERVIEW ANSWER

[INFERENCE] It demonstrates you understand the difference between **module elimination** (bundler-level, driven by the import graph) and **class extraction** (Tailwind-level, driven by a content glob over source text). Most candidates cannot articulate that distinction. Say it once, confidently, and move on.

5.4 The dead-code inventory, verified

Path	Evidence it is dead
`src/components/Algocards.jsx`, `CardsGrid.jsx`, `Header.jsx`, `Layout.jsx`, `ModeToggle.jsx`, `NQueensControls.jsx`, `NQueensplay.jsx`, `SearchControls.jsx`, `SortingControls.jsx`	No source file imports any of them
`src/context/ModeContext.jsx`	Only imported by the dead `CardsGrid.jsx` and `ModeToggle.jsx` — therefore effectively dead too
`src/App.css`, `src/assets/react.svg`, `public/vite.svg`	No imports / no references (only `favicon.svg` is referenced by `index.html`)
`src/modules/core/SimpleSort.jsx`, `EfficientSort.jsx`	Not routed; sorting routes render `SortingLab` instead. They are the v1 sorting UIs, superseded by the lab

WEAKNESS / LIMITATION
A SUBTLETY WORTH KNOWING BEFORE YOU GET ASKED

`CardsGrid.jsx` contains the only network call in the entire repository — an `await fetch('https://countapi.mileshilliard.com/api/v1/hit/algoviz-siva-2026')` for a visit counter. Because the component is not imported, **no request is ever made**. If you are asked "does your app call any external service?", the correct answer is: *"No. There is one `fetch` left in a component that isn't imported anywhere, so it never executes. It's dead code from the earlier version — and honestly, if it were live I'd remove it, because a third-party counter API in a client-side app is both a privacy leak and a supply-chain dependency I don't control."* That answer turns a landmine into evidence of judgement.

CHAPTER 06

06 · Architecture and data flow

The pipeline, redrawn to match the actual source rather than the idealised version.

6.1 The pipeline as implemented

The block diagram below is the real data flow. Every arrow is a function call or a state write that exists in the code.



… continued (2/2)

```
            |           |       +-----------------+
            |           v
            |      +-------------------------------+
            |      |  useEffect: if playing ->      |
            |      |  setTimeout(dispatch tick, speed)|
            |      |  cleanup clears the timer      |
            |      +-------------------------------+
            v
       +-----------------------------------------------------+
       |  DERIVATION (render time, no state)                  |
       |     cursor = min(playback.cursor, frames.length - 1)  |
       |     frame  = frames[cursor]        <- the only read   |
       |     stats  = frame.currentStats                       |
       |     last   = frames.at(-1).currentStats               |
       |     compareCursor = round(cursor / (frames.length-1))  |
       |                       * (otherFrames.length-1))       |
       +-----------------------------------------------------+
            |
            +-------------------+--------------------+
            v                   v                    v
+-----------------+  +-----------------+  +----------------------+
| ArrayView(frame) |  | State inspector |  | Explanation panel      |
| bars, active,    |  | comps/passes/   |  | codeLine + msg + step  |
| sorted, pivot    |  | swaps, indices  |  | badge "005"            |
+-----------------+  +-----------------+  +----------------------+

SIDE CHANNELS (all outside the reducer, all synchronous)
  Save   -> localStorage["algoviz.experiments.v1"] = JSON.stringify(latest 6)
  Export -> Blob + object URL + synthetic <a download>.click()
  Import -> FileReader via file.text() -> size check -> JSON.parse ->
            schema check -> parseInput -> load() -> seek()
  Dataset-> makeDataset(kind, seed)  (LCG, 10 values) -> load()
```

6.2 The nine stages, each with its guarantee

Stage	Input	Output	Guarantee it provides
1. Draft entry	keystrokes	`text` string	Nothing. It is deliberately unvalidated so the user can type freely
2. Validation	`text`	`number[]` or `Error`	Every value is a safe integer, `|n| <= 999`, and there are 2–24 of them
3. Commit	`number[]`	`input` state + reducer `load`	The analysed dataset is frozen until the next successful commit
4. Frame generation	`rawArray`	`frames[]`	Frame 0 is the initial state; the final frame is fully sorted with all indices marked
5. Memoisation	`(algo, input)`	cached `frames[]`	The generator runs once per distinct `(algorithm, input)` pair per render pass
6. Playback state	actions	`{cursor, length, playing}`	`cursor` is always within `[0, length-1]` when `length >= 1`
7. Scheduling	`playing`, `speed`, `cursor`	one pending timeout	At most one timer exists; it is cancelled on any dependency change or unmount
8. Derivation	`playback`, `frames`	`frame`, `stats`, `compareCursor`	Render happens for a valid frame index even if state is momentarily stale
9. Presentation	`frame`	bars, metrics, text	What is shown is exactly what was recorded — nothing is recomputed for display

6.3 Why this architecture, and what the alternative would have been

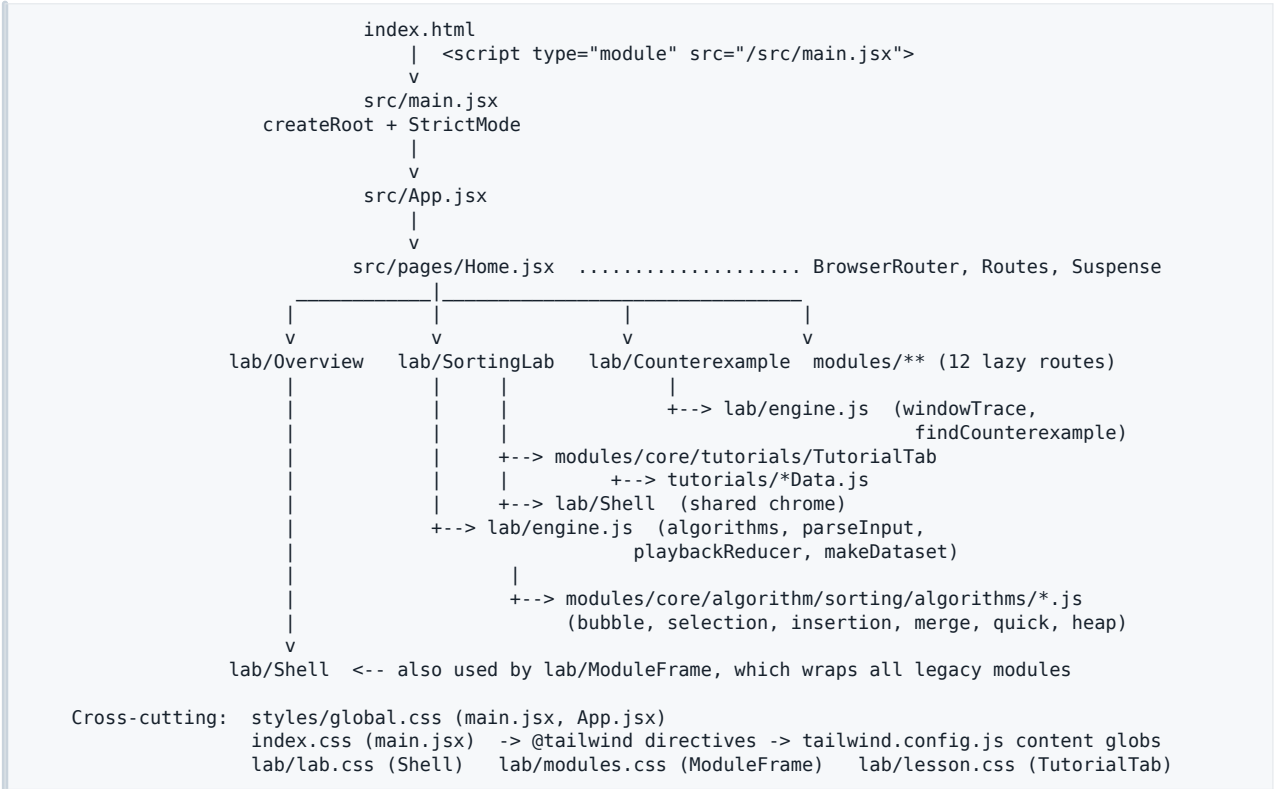
MUST KNOW

THE ARCHITECTURAL ARGUMENT IN THREE SENTENCES

If you animated while the algorithm ran — the classic approach — you could only ever move **forward**, because the algorithm's own variables are the animation state. To go backwards you would have to re-run the algorithm from the start and stop early, which is wasteful, and impossible to do at a precise earlier decision point for algorithms whose behaviour depends on comparisons performed in sequence. Recording frames inverts the problem: execution becomes a pure, testable, DOM-free function returning data, and navigation becomes an index into that data.

Aspect	Animate live (legacy modules)	Record frames (the lab)
Rewind	Impossible without re-running or storing history	Free — just set `cursor`
Seek to arbitrary frame	Impossible	`frames[i]`
Testable without a browser	No — needs a running animation	Yes — `run()` is a pure function returning data
Trace reproducibility	Not checkable	Directly checkable with `deepEqual`
Memory	O(1) extra	O(frames × n) snapshots
Speed control	Changes the wall-clock delay	Changes only how fast the cursor moves
Debuggability	Hard — state is spread across timers	Easy — the whole trace is inspectable data
Cancellation	Manual flags and refs	Clear the timeout

6.4 The dependency graph (who imports whom)



NOTE
THE STRUCTURAL FACT WORTH NOTING ALOUD

`engine.js` is the only file that imports all six sorting generators. It is a deliberate boundary: everything above it (React components) knows nothing about how frames are made, and everything below it (generators) knows nothing about React. That is why the algorithms are testable in plain Node with no DOM shim. [VERIFIED]

6.5 The two execution models side by side

This table is the single most useful thing to have memorised, because "why is your app inconsistent?" is a real question you will be asked.

Property	Generation 2: the lab	Generation 1: the twelve modules
Where frames are created	In `engine.js` via generator functions that return the whole array	Inside the component, via `generateXAnimations()`
When the UI updates	On cursor change	Inside an `async` loop after `await smartWait(...)`
Pause mechanism	`playing: false` in a reducer	`isPausedRef` checked in a 20 ms polling loop
Rewind / seek	Supported	Not supported
Speed control	`setTimeout` delay	`speedRef.current = 2100 - curspeed`
Testable in Node	Yes	Partly — the logic module is, the component is not
Duplicate-run protection	N/A — the effect owns one timer	`if (isVisualizing) return;` guard
State location	One reducer + `useState`	15–20 `useState` hooks per component

INFERENCE
WHY THE LEGACY CODE LOOKS LIKE THAT

[INFERENCE] The stored animation array plus `await`-in-a-loop is the natural first way most developers write a visualiser, and it is exactly the design the lab was built to replace. Keeping it is a deliberate engineering decision — the non-sorting visualisations work, and rewriting twelve of them carries regression risk with no test coverage to catch it. The README says as much: "The shared UI covers all modules; the shared sorting playback reducer does not yet cover every algorithm family."

CHAPTER 07

07 · Important files you must know

The table to print out. For each file: purpose, inputs, outputs, state, dependencies, the functions to know, and the question you will be asked.

7.1 `src/lab/engine.js` — 169 lines — RED

Aspect	Detail
Purpose	The non-React core: algorithm registry, input validation, playback state machine, seeded datasets, the sliding-window reference implementation, and the bounded counterexample search
Imports	The six sorting generators only
Exports	`algorithms` (object of 6), `parseInput`, `playbackReducer`, `makeDataset`, `windowTrace`, `findCounterexample`
State	None. Every export is pure — no module-level mutable state, no I/O
Read by	`SortingLab.jsx` (registry, parse, reducer, datasets), `Counterexample.jsx` (`windowTrace`, `findCounterexample`), `tests/engine.test.js` (everything)
Functions to know	`parseInput` (validation contract), `playbackReducer` (4 cases), `makeDataset` (LCG + 4 kinds), `windowTrace(values, target, buggy)` (two behaviours in one function), `findCounterexample` (bounded $4^3 \times 8$ search)
Likely question	"Walk me through `parseInput`. Why does it split on commas before validating?"
Likely follow-up	"Why is `windowTrace` allowed to be buggy?" — because differential testing needs a deliberately wrong implementation to prove the test can detect divergence

7.2 `src/lab/SortingLab.jsx` — 596 lines — RED

Aspect	Detail
Purpose	The entire Sorting Lab screen: input panel, transport, stage, inspector, comparison, notes, saved experiments
Local components	`ArrayView({frame, compact})` and a module-level `download(data)` helper
State	12 `useState` + 1 `useReducer` + 2 `useRef` (see the state register in Ch. 8)
Effects	(1) playback timer, (2) keyboard listener — both with cleanup
Derived values	`frames`, `otherFrames`, `cursor`, `frame`, `meta`, `stats`, `last`, `compareCursor`, `initial`
Handlers	`load`, `apply`, `importExperiment`, `save`, `seek`
Reads from URL	`?algorithm=` (validated against the registry), `?compare=`
Likely question	"Why is there both `text` and `input` state?"
Likely follow-up	"Why is the timer dependency array `[playing, cursor, speed]` and not just `[playing]`?"

7.3 The six generators — RED

All six share the same shape, which is itself a talking point: `rawArray -> [{id:'block-i', val}] -> frames[]`, with a local `cloneArray()` used on every push.

Generator	Lines	Frames pushed at	Distinctive machinery
`generateBubbleSort`	92	ready, each outer pass, each `inner` scan, each `compare`, each `swap` before + after, each pass-complete, final	`sortedIndices.push(n - i - 1)` after each pass; `isSwapping` toggles around the swap
`generateSelectionSort`	73	ready, each pass start, each `inner` scan, each `new_min`, each `swap` before + after, each `lock`, final	`minimumIndex` carried in every frame; loop runs to `n-1`, then `sortedIndices.push(n-1)`
`generateInsertionSort`	86	ready, each lift, each `compare`, each shift before + after, `break`, `drop`, final	`isLifting` flag on the lifted element; the only generator with an explicit early-`break` frame
`generateMergeSort`	172	ready, each `divide`, `base_case`, `mid`, `recurse_left`, `recurse_right`, `merge_call`, `buffer`, each `compare`, each write, `write_rem`, remaining-element pushes, final	Recursion; a temporary buffer; `yOffset` set to 80 while elements are "in the buffer"; a single final loop marking every index sorted
`generateQuickSort`	160	ready, each call, `base_case` (marks the index sorted when `low === high`), `partition`, each `compare`, each conditional swap, pivot placement, `recurse_left`, `recurse_right`, final	`pivotIndex` in most frames; the pivot index is pushed to `sortedIndices` immediately after partition
`generateHeapSort`	120	ready, `build_heap`, per-heapify `pointers` + `compare_children` + optional swap + recurse, `extract`, per-extraction `swap_root` + `heapify_root`, final	Recursive `heapify`; two comparison increments per call; `phaseMsg` distinguishes "Building Heap" from "Extracting"

7.4 The support files — YELLOW

File	Purpose	Inputs → Outputs	Likely question
`src/main.jsx`	Entry point	— → mounted `<App/>` inside `StrictMode`	"What does `StrictMode` do to your effects in development?"
`src/pages/Home.jsx`	Router, routes, lazy imports, Suspense fallback, 404	URL → matched element	"What happens on a direct visit to `/lab`?"
`src/lab/Shell.jsx`	Sidebar, top bar, footer, `window.scrollTo(0,0)` on path change	`children` → page chrome	"Why does the sidebar compute `pageName` from `pathname` instead of props?"
`src/lab/Overview.jsx`	Landing + library grid	`library` prop → different heading and hero	"How does one component serve two routes?"
`src/lab/Counterexample.jsx`	Prediction → reveal → divergence display → bounded search	`findCounterexample()` → example, `revealed`, `searched`	"How do you know the first divergence is meaningful?"
`src/lab/ModuleFrame.jsx`	Breadcrumb, module switcher (`navigate`), Visualize/Learn toggle	tab props → framed module content	"Why does the module switcher use `navigate` instead of `<Link>`?"
`src/constants.js`	Module metadata for the library and navigation	— → 12 module descriptors	"Where does `/modules/patterns/bs-variants` come from?"
`tests/engine.test.js`	All automated verification	generator modules → pass/fail	"Point at the 150. Point at the 768."
`.github/workflows/ci.yml`	CI pipeline	push/PR → 4 steps	"What happens if `npm test` fails?"

7.5 The files you must be able to name but not explain — GREEN

`src/modules/**` (12 visualisations + 13 logic modules + 10 tutorial data files + `TutorialTab.jsx`), `src/lab/lab.css` (1,756 lines), `modules.css`, `lesson.css`, `global.css`, `src/components/*` (dead), `docs/*`, `vercel.json`, `.nvmrc`, `postcss.config.js`, `public/favicon.svg`.

CHECKLIST

THE 20-FILE DRILL

Open a terminal and run this. If you can say one accurate sentence about every line it prints, you have passed the file-level part of the interview.

```
find src tests .github -type f \( -name '*.js' -o -name '*.jsx' -o -name '*.yml' \) | sort
```

PART III

Application startup and routing

From index.html to the first rendered frame of the Sorting Lab — the exact chain of files.

CHAPTER 08

08 · Startup: index.html → main.jsx → App → Home → routes

Six files, in order, with the reason each one exists.

8.1 Stage 1 — `index.html` (the only HTML file)

```
<link rel="icon" type="image/svg+xml" href="/favicon.svg" />
<meta name="viewport" content="width=device-width, initial-scale=1.0" />
<title>AlgoViz — The Algorithm Laboratory</title>
<meta name="description" content="Inspect, rewind and compare algorithms. A local-first algorithm laboratory with reproducible experiments and counterexample challenges." />
...
<div id="root"></div>
<script type="module" src="/src/main.jsx"></script>
```

[VERIFIED] What matters here:

- **One mount point:** `<div id="root">`. Nothing else in the document is React-managed.
- **No bundler script tag in the source:** the `<script>` points at `/src/main.jsx`, which browsers cannot execute directly. In development Vite serves and transforms it on the fly; in production the build rewrites the tag to a hashed, minified chunk. This is why the built `dist/index.html` looks different from the source file — the same reason "you can't just open `index.html`" (which your README warns about).
- **`<meta name="description">` is real product copy**, not filler. It contains the project's three honest self-descriptions: local-first, reproducible, counterexample.
- **No CSS `<link>`:** all styles arrive through JavaScript imports, which Vite extracts into CSS chunks at build time (one 72.37 kB stylesheet plus two smaller ones, per my build run).

> **INTERVIEW QUESTION**
> **"WHY IS THERE NO `<LINK REL="STYLESHEET">` IN YOUR HTML?"**
> [VERIFIED] Because the stylesheets are imported from JavaScript (`main.jsx` imports `global.css` and `index.css`; `Shell.jsx` imports `lab.css`; and so on). This gives per-route CSS: Vite emits `ModuleFrame-*.css` and `TutorialTab-*.css` only alongside the chunks that need them. The trade-off is that a `<link rel="stylesheet">` would let the browser start CSS download before the JS bundle is parsed — a real, if small, difference in first paint.

8.2 Stage 2 — `src/main.jsx` (9 lines)

`src/main.jsx (verbatim, 10 lines)`

```
import React from 'react';
import ReactDOM from 'react-dom/client';
import App from './App.jsx';
import './styles/global.css';
import './index.css';
ReactDOM.createRoot(document.getElementById('root')).render(
  <React.StrictMode>
    <App />
  </React.StrictMode>,
);
```

[VERIFIED] Four decisions are visible in nine lines:

1. **`createRoot`** — the React 18+ client API, replacing `ReactDOM.render`. It enables concurrent features and is the only supported entry for React 19.
2. **`document.getElementById('root')` is not null-checked** — a deliberate simplification that is safe only because `index.html` is in the same repository and does declare that element. It is a coupling between two files, and it is the classic "what if you misspell the id?" question.

3. **<React.StrictMode>** wraps the application. In development only, React intentionally double-invokes render functions, effects, and (in React 19) ref callbacks to surface non-idempotent code. This has a direct consequence for AlgoViz's timer effect, which you should be able to explain — see 8.6.
4. **Stylesheet import order:** `global.css` then `index.css`, and `index.css` is the Tailwind entry containing `@tailwind base`, which includes Tailwind's preflight reset. Import order therefore matters more than it looks: preflight loading after `global.css` means Tailwind's reset wins on conflicting base rules. [INFERENCE] That ordering is intentional enough to be worth mentioning, and it is a good answer to "does CSS order matter in a JS-bundled app?"

> **INTERVIEW QUESTION**
> **"WHERE DOES REACT START?"**
> [VERIFIED] *"`ReactDOM.createRoot(document.getElementById('root')).render(<StrictMode><App/></StrictMode>)`. That's it — the root and the render call are in `src/main.jsx`. Everything after that is ordinary component composition and routing."*

8.3 Stage 3 — `src/App.jsx` (12 lines)

`src/App.jsx (verbatim, 13 lines)`

```
import React from 'react';
import Home from './pages/Home';
import './styles/global.css';

function App() {
  return (
    <div>
      <Home />
    </div>
  );
}

export default App;
```

[VERIFIED] `App` is intentionally almost empty: it imports `global.css` (a second time — `main.jsx` already did) and renders `<Home />` inside a `<div>`. There is no provider, no theme context, no error boundary.

> **INFERENCE**
> **WHY THIS FILE EXISTS AT ALL IF IT ONLY RENDERS HOME**
> [INFERENCE] It is a conventional seam: the natural place to add providers, error boundaries, or global layout later, and the conventional filename that tooling and readers expect. It is also the only place a global error boundary could go today, which makes it a good answer to "how would you handle a render crash in one module?" — [PROPOSED]: add an `ErrorBoundary` component in `App.jsx` so a crash in one lazily loaded module does not blank the whole workspace.

Note the duplicate stylesheet import: `main.jsx` imports `./styles/global.css` and `App.jsx` imports it again. Vite deduplicates the module in the bundle, so there is no double download, but the redundancy is real. [VERIFIED]

8.4 Stage 4 — `src/pages/Home.jsx` (104 lines): routing

`src/pages/Home.jsx · lines 1–20 (verbatim)`

```
import { lazy, Suspense } from "react";
import { BrowserRouter, Routes, Route, Link } from "react-router-dom";
import Overview from "../lab/Overview";
import Shell from "../lab/Shell";
const SortingLab = lazy(() => import("../lab/SortingLab"));
const Counterexample = lazy(() => import("../lab/Counterexample"));
const BinarySearch = lazy(() => import("../modules/core/BinarySearch"));
const NQueens = lazy(() => import("../modules/core/Nqueens"));
const TwoPointers = lazy(() => import("../modules/patterns/TwoPointers"));
const FastSlowPointers = lazy(
  () => import("../modules/patterns/FastSlowPointers"),
);
const BinaryVariants = lazy(() => import("../modules/patterns/BinaryVariants"));
const PrefixSum = lazy(() => import("../modules/patterns/PrefixSum"));
const SlidingWindow = lazy(() => import("../modules/patterns/SlidingWindow"));
const MergeIntervals = lazy(() => import("../modules/patterns/MergeIntervals"));
const TreeTraversal = lazy(() => import("../modules/core/TreeTraversal"));
const Pathfinder = lazy(() => import("../modules/core/PathFinder"));
export default function Home() {
  return (
```

[VERIFIED] All twelve heavy module components are imported with `React.lazy(() => import(...))`. `Overview` and `Shell` are imported eagerly because they are needed on the very first paint — the shell would otherwise flash empty.

`src/pages/Home.jsx · lines 21–44 (verbatim)`

```
    <BrowserRouter>
      <Suspense
        fallback={
          <Shell>
            <main
              className="av-page av-loading"
              role="status"
              aria-label="Loading workspace"
            >
              <div className="av-eyebrow">PREPARING YOUR EXPERIMENT</div>
              <h1>One moment of curiosity…</h1>
              <div className="av-loading-block" />
              <div className="av-loading-block tall" />
              <p>Loading workspace</p>
            </main>
          </Shell>
        }
      >
        <Routes>
          <Route path="/" element={<Overview />} />
          <Route path="/library" element={<Overview library />} />
          <Route path="/lab" element={<SortingLab />} />
          <Route path="/counterexample" element={<Counterexample />} />
          <Route path="/modules/core/simple-sort" element={<SortingLab />} />
```

The complete route table, verified from lines 39–100:

Path	Element	Lazy?	Notes
`/`	`<Overview />`	no	Landing page
`/library`	`<Overview library />`	no	Same component, different presentation
`/lab`	`<SortingLab />`	yes	The Sorting Lab
`/counterexample`	`<Counterexample />`	yes	Counterexample Lab
`/modules/core/simple-sort`	`<SortingLab />`	yes	Routes to the lab, not to `SimpleSort.jsx`
`/modules/core/advanced-sort`	`<SortingLab key="advanced" />`	yes	The `key` forces a fresh mount so the initial algorithm differs
`/modules/core/binary-search`	`<BinarySearch />`	yes	Legacy module
`/modules/core/n-queens`	`<NQueens />`	yes	Legacy module
`/modules/patterns/two-pointers`	`<TwoPointers />`	yes	Legacy module
`/modules/patterns/fast-slow`	`<FastSlowPointers />`	yes	Legacy module
`/modules/patterns/bs-variants`	`<BinaryVariants />`	yes	Legacy module
`/modules/patterns/prefix-sum`	`<PrefixSum />`	yes	Legacy module

Path	Element	Lazy?	Notes
`/modules/patterns/sliding-window`	`<SlidingWindow />`	yes	Legacy module
`/modules/patterns/merge-intervals`	`<MergeIntervals />`	yes	Legacy module
`/modules/core/tree-traversal`	`<TreeTraversal />`	yes	Legacy module
`/modules/core/pathfinding`	`<Pathfinder />`	yes	Legacy module
`*`	404 panel inside `<Shell>`	no	"404 / OUTSIDE THE SEARCH SPACE"

MUST KNOW
THREE DETAILS IN THAT TABLE THAT EARN YOU CREDIBILITY

1. **`<SortingLab key="advanced" />`** — the `key` prop makes React treat the advanced-sort route as a different component instance, so state resets and the initial algorithm becomes `quick` instead of `bubble`. Without the key, navigating between `/modules/core/simple-sort` and `/modules/core/advanced-sort` would reuse the same mounted component and keep the previous algorithm. [VERIFIED: the default algorithm is derived from `location.pathname.includes("advanced-sort")` at line 65, which is only re-evaluated on mount.]
2. **The 404 route renders inside `<Shell>`** with nav and footer intact, rather than dumping a bare error page. That is a UX choice with a cost: the 404 page imports `Shell`, so it is not lazy.
3. **`/library` reuses `Overview` with a boolean prop** instead of duplicating a component. Together with the `*` 404 panel, those are the only three routes that never suspend: `Overview` and `Shell` are imported eagerly, so first paint never waits on a chunk for them. The other fourteen routes render a lazily-imported component.

8.5 Stage 5 — the Suspense boundary

`src/pages/Home.jsx · lines 22–38 (verbatim)`

```
      <Suspense
        fallback={
          <Shell>
            <main
              className="av-page av-loading"
              role="status"
              aria-label="Loading workspace"
            >
              <div className="av-eyebrow">PREPARING YOUR EXPERIMENT</div>
              <h1>One moment of curiosity…</h1>
              <div className="av-loading-block" />
              <div className="av-loading-block tall" />
              <p>Loading workspace</p>
            </main>
          </Shell>
        }
      >
```

[VERIFIED] One `<Suspense>` wraps **all** routes, with a fallback that is itself wrapped in `<Shell>`. Consequences worth stating:

- The navigation chrome stays visible while a lazy chunk downloads — the user is never looking at a blank page.
- The fallback is not per-route, so navigating between two lazy routes shows the same "PREPARING YOUR EXPERIMENT / One moment of curiosity…" screen.
- `<main role="status" aria-label="Loading workspace">` gives the loading state a live-region role and a label, which is the accessibility-minded way to announce loading.

INTERVIEW QUESTION
"WHAT HAPPENS DURING A ROUTE CHANGE?"

Trace it in this order: **(1)** `BrowserRouter` reads the new `location` and re-renders its children; **(2)** `<Routes>` matches the new pathname against the route table; **(3)** if the matched element is a lazy component whose chunk has not been fetched, React suspends and renders the `<Suspense>` fallback instead; **(4)** when the chunk resolves, the module component mounts; **(5)** inside it, `Shell`'s `useEffect` on `pathname` fires and calls `window.scrollTo(0, 0)`; **(6)** any component that reads `useLocation` or `useSearchParams` (like `SortingLab`) re-computes its derived values on the next render. [VERIFIED]

8.6 StrictMode and the timer: the question that catches candidates

INTERVIEW QUESTION
"YOU USE STRICTMODE. WHAT DOES THAT DO TO YOUR PLAYBACK EFFECT?"

[VERIFIED] In development, StrictMode mounts, unmounts and remounts each component to expose effects that are not safe to run twice. The playback effect is exactly the kind of code this is designed to test:

```
useEffect(() => {
  if (!playback.playing) return;
  const id = setTimeout(() => dispatch({ type: "tick" }), speed);
  return () => clearTimeout(id);
}, [playback.playing, playback.cursor, speed]);
```

The first mount schedules a timeout; the simulated unmount runs the cleanup, which clears it; the remount schedules a new one. **Net effect: one timer, because the cleanup always cancels before the replacement schedules.** If the effect had no cleanup — the classic bug — StrictMode would leave two live timers in development and the animation would visibly double-step. So the honest, impressive answer is: "StrictMode is why I have a cleanup that clears the timeout. Without it, double-mounting in development would leave an orphaned timer... and the same bug would appear in production whenever the effect re-ran, for example when the user changed speed mid-playback."

DANGER ZONE — WHERE CANDIDATES DIE
THE THREE TIMER BUGS STRICTMODE WILL EXPOSE IN YOUR CODE, IF YOU EVER REMOVE THE CLEANUP

1. Two pending timers after any dependency change → the cursor advances twice per interval.
2. A tick dispatched after unmount → a React warning, and in older React versions a state update on an unmounted component.
3. Speed changes that appear not to take effect, because the old timer is still the one firing.

PART IV

The Sorting Lab, end to end

Every state variable, the draft-versus-committed distinction, and the full path from a keystroke to a rendered bar.

CHAPTER 09

09 · The Sorting Lab state register

This table is the single most examinable artefact in the project. Learn it as a table, not as prose.

9.1 Why a state register matters

An interviewer who says "show me your state" is testing one thing: do you know which of your variables are inputs, which are derived, and which are copies that can drift out of sync. AlgoViz has ten `useState` calls, one `useReducer`, one `useRef`, two `useMemo` values and six derived render-time constants (plus two router hooks that supply read-only values). Knowing the category of each one is the difference between "I have some state" and "I can tell you why each piece exists".

9.2 The register (all [VERIFIED] from `src/lab/SortingLab.jsx`)

State	Type	Initial value	Who changes it	Who reads it	Why it exists
`fileInput`	`useRef` (`HTMLInputElement`)	`null`	React on mount	the hidden `<input type="file">`, and the Import button's `onClick`	The file input is visually hidden, so a button must programmatically trigger it. A ref is the DOM handle; a state variable would cause pointless re-renders
`params`	`URLSearchParams`	from the URL	navigation	`initial` algorithm, `compare` initial value	Deep links: `?algorithm=quick`, `?compare=1`
`location`	location object	from the router	navigation	`initial` (advanced-sort detection)	The pathname decides the default algorithm for the two sorting routes
`algo`	`string` (key of `algorithms`)	`"bubble"`, `"quick"` for advanced-sort, or a valid `?algorithm=`	`load()`; the algorithm `<select>`	`frames` memo, `meta`, `save()`, `download()`	Which generator produces the primary trace
`text`	`string`	`"38, 66, 12, 85, 24, 57, 43, 19"`	the text input's `onChange` **on every keystroke**; `load()` normalises it	the input element; `apply()`; `parseInput`	**The draft.** It must accept intermediate, invalid text like `"38, 66,"` — otherwise the user could not type a comma at the end
`input`	`number[]`	`[38, 66, 12, 85, 24, 57, 43, 19]`	`load()` only	`frames` memo, `otherFrames` memo, `save()`, `download()`, the "ARRAY STATE [n]" caption	**The committed dataset.** It is the only array the engine ever sees, and it is guaranteed valid by construction
`error`	`string`	`""`	`apply()` and `importExperiment()` (set), `load()` (cleared)	the red `role="alert"` banner	Validation feedback must survive until the next successful action
`notice`	`string`	`""`	`save()` (both branches), `importExperiment()` (success), `load()` (cleared)	the `role="status"` banner	Positive confirmation ("Saved on this device.") is separate from validation failure

State	Type	Initial value	Who changes it	Who reads it	Why it exists
`speed`	`number` (ms)	`160`	the speed `<select>`	the timer effect's delay, and its dependency array	Delay per frame. 160 ms ≈ the 2× label
`compare`	`boolean`	`params.has("compare")`	the Compare button (`setCompare(!compare)`)	whether `otherFrames` are computed, and whether the comparison panel renders	Comparison must be opt-in so the default view stays simple
`other`	`string`	`"merge"`	the comparison `<select>`	`otherFrames` memo	The second algorithm's key
`seed`	`number`	`42`	the shuffle button (`setSeed(seed + 1)`)	the next shuffle click	Makes "random" datasets reproducible: seed 42 always produces the same ten numbers (LCG: 27, 12, 56, 25, 38, 7, 45, 15, 83, 94)
`saved`	array of experiment objects	**lazily** read from `localStorage`, filtered, sliced to 6, or `[]` on throw	`save()`	the saved-experiments list	A cache of what is in `localStorage`, so reading and parsing happen once per mount, not once per render
`frames`	`object[]` (**derived**)	computed	`useMemo` on `[algo, input]`	almost everything: `ArrayView`, timeline max, tag, counters, comparison	The primary recorded trace
`otherFrames`	`object[]` (**derived**)	`[]` or computed	`useMemo` on `[compare, other, input]`	comparison panel	The second recorded trace, from the **same** `input`
`playback`	`{cursor, length, playing}`	`{cursor: 0, length: frames.length, playing: false}`	the reducer via `dispatch`	cursor, tag, play button label, timer effect	Transport state — the only genuinely stateful machine in the component
`cursor`	`number` (**derived**)	—	recomputed every render	`frame`, timeline value, step buttons, tag, `save()`	`Math.min(playback.cursor, frames.length - 1)` — a defensive clamp against a stale reducer cursor after an input change
`frame`	`object` (**derived**)	—	recomputed	`ArrayView`, inspector, explanation	`frames[cursor]` — the one and only read of the trace in the render path
`meta`	`object` (**derived**)	—	recomputed	headings, invariant panel, comparison labels	`algorithms[algo]`
`stats`	`object` (**derived**)	`{}`	recomputed	the three metric tiles	`frame.currentStats || {}`
`last`	`object` (**derived**)	—	recomputed	comparison panel	`frames.at(-1).currentStats` — the **final** counters for the complete run
`compareCursor`	`number` (**derived**)	`0`	recomputed	the compact `ArrayView` in comparison mode	`Math.round((cursor / (frames.length - 1)) * (otherFrames.length - 1))`

9.3 The two properties that make this register safe

MUST KNOW
PROPERTY 1 — `INPUT` CANNOT BE INVALID

`input` is written in exactly one place, inside `load()`, and every caller of `load()` either passes an already-validated `number[]` from `parseInput` or a `makeDataset()` result. There is no code path that assigns `text` (the unvalidated string) to `input`. [VERIFIED] This is why the engine never needs to defend against bad data: validation happens at the boundary and the interior is total.

MUST KNOW
PROPERTY 2 — `CURSOR` CANNOT BE OUT OF RANGE

Two independent clamps exist. The reducer clamps on `seek` (`Math.max(0, Math.min(action.cursor, state.length - 1))`) and on `tick` (`Math.min(state.cursor + 1, state.length - 1)`). The component additionally derives `cursor = Math.min(playback.cursor, frames.length - 1)`. [VERIFIED] The second clamp matters in a specific race: if `input` changes, `frames` shrinks in the same render that `playback` still holds the old cursor — because the reducer's `length` only updates when the `load` action is processed. Without the derived clamp, `frames[cursor]` could momentarily be `undefined` and `ArrayView` would throw on `frame.array.map`.

INTERVIEW QUESTION
"WHY IS THERE BOTH `TEXT` AND `INPUT` — ISN'T THAT DUPLICATED STATE?"

This is the most important state question in the project. Answer in three parts:

1. They have different types and different meanings. `text` is a string that is allowed to be invalid; `input` is a number array that is guaranteed valid. They are not two copies of one value — they are a draft and a commitment.
2. If they were one piece of state, the input field could not hold `"38, 66,"` while the user is typing, because the moment a comma is dangling the value is invalid. You would have to either reject the keystroke (hostile) or store the parsed array and lose the user's literal text.
3. The sync rule is one-directional: `text → input` happens only on submit, `input → text` happens on `load()` as normalisation. A user who types junk and presses Apply sees an error and **the previous committed dataset keeps being visualised** — the lab does not blank out. [VERIFIED]

9.4 The one place the register costs something

WEAKNESS / LIMITATION

THE DUPLICATED GENERATOR RUN ON `LOAD`

`load()` does this:

```
dispatch({ type: "load", length: algorithms[algorithm].run(values).length });
```

That call executes the whole generator — building every frame — purely to read `.length`. Then the `useMemo` for `frames` runs the same generator a second time to obtain the frames themselves. So every input change runs the selected generator **twice**. [VERIFIED]

Impact: for the capped input sizes (≤ 24 elements, ≤ 1,154 frames for the worst measured trace) this is sub-millisecond work and invisible to the user. It is still genuinely wasteful, and the fix is trivial — derive the length from the memo instead of computing it:

```
// [PROPOSED] not implemented
const frames = useMemo(() => algorithms[algo].run(input), [algo, input]);
useEffect(() => { dispatch({ type: "load", length: frames.length }); }, [frames]);
```

Or better, keep the reducer as the single owner of `length` by having it derive the length from the frame array it is handed. Either way, the honest answer is: "Yes, I run it twice. It's deliberate only in the sense that I wanted the reducer's length to be set at commit time so the first render after a load is already consistent. At this input cap it's free; at a 10,000-element cap I would restructure it." This is exactly the kind of answer that reads as senior: acknowledge, quantify, and give the fix.

CHAPTER 10

10 · The input pipeline: draft → validation → committed dataset

Six rules, five error messages, and the exact order in which they fire.

10.1 The five validation rules in `parseInput`

src/lab/engine.js · lines 63–79 (verbatim)

```
export function parseInput(text) {
  if (typeof text !== "string" || !text.trim())
    throw new Error("Enter 2–24 integers, separated by commas.");
  const parts = text.split(",");
  if (parts.length < 2 || parts.length > 24)
    throw new Error(
      "Use 2–24 values. Small inputs keep every operation inspectable.",
    );
  if (parts.some((p) => !/^-?\d+$/.test(p.trim())))
    throw new Error(
      "Use whole numbers separated by commas. Empty values are not allowed.",
    );
  const values = parts.map(Number);
  if (values.some((n) => !Number.isSafeInteger(n) || Math.abs(n) > 999))
    throw new Error("Each number must be between −999 and 999.");
  return values;
}
```

#	Rule	Code	Failure message	Why this rule exists
1	Non-empty string (and a string at all)	`typeof text !== "string" || !text.trim()`	"Enter 2–24 integers, separated by commas."	Guards against `null` from a missing query param, and against whitespace-only input
2	Between 2 and 24 values	`parts.length < 2 || parts.length > 24`	"Use 2–24 values. Small inputs keep every operation inspectable."	**The performance guard.** Bubble sort on 24 elements already produces 1,154 frames and ~980 KB of JSON; the cap is what keeps the trace usable and the memory bounded
3	Every part is an optionally-signed integer	`!/^-?\d+$/.test(p.trim())`	"Use whole numbers separated by commas. Empty values are not allowed."	Rejects `2.5`, `` ` ``, abc, 1e3, Infinity, 0x10`, and trailing commas — the regex is anchored, so partial matches fail
4	Safe integer and `|n| <= 999`	`!Number.isSafeInteger(n) || Math.abs(n) > 999`	"Each number must be between −999 and 999."	`Number.isSafeInteger` rejects values beyond 2^{53} even if they parse; the range cap keeps bar heights meaningful and prevents absurd labels
5	Returns `number[]`	`parts.map(Number)`	—	The single conversion point from text to numbers

MUST KNOW
WHY RULE 3 USES A REGEX INSTEAD OF `NUMBER()`/`ISNAN()`

`Number("")` is `0`, `Number("  ")` is `0`, `Number("1e3")` is `1000`, `Number("0x10")` is `16`, and `Number("Infinity")` is `Infinity`. Numeric coercion silently accepts all of these. An anchored regex `^-?\d+$` states the accepted grammar explicitly, and only then is the value converted. This is the standard "validate the shape, then parse" discipline, and it is worth saying out loud — it is the kind of detail that separates people who have been burned by implicit coercion from people who have not.

10.2 What actually happens on submit — the full trace

Trace `apply()` when the user types `"38, 66, 12"` and presses Enter:

1. The `<form onSubmit={apply}>` fires; `apply(e)` calls `e.preventDefault()` so the browser does not perform a native form submission and reload the page. [VERIFIED line 149]
2. `parseInput("38, 66, 12")` runs — note the spaces are preserved in the draft but each part is `.trim()`-ed during the regex test, and `Number(" 66 ")` is `66`, so spaces are harmless.
3. Validation passes → `[38, 66, 12]`.
4. `load([38, 66, 12])` is called with `algorithm = algo` as the default parameter.
5. `setInput([38, 66, 12])` — the committed dataset changes.
6. `setText("38, 66, 12")` — the draft is normalised (so `"38,66 , 12"` becomes `"38, 66, 12"`).
7. `setAlgo(algo)` — a no-op state set to the same value; React bails out of re-rendering for that call.
8. `dispatch({type: "load", length: 16})` — the generator ran once here to count frames for this input.
9. `setError("")`, `setNotice("")` — banners are cleared.
10. React re-renders: `frames` memo recomputes (because `input` is a **new array reference**, even though its contents differ from the old one anyway), `cursor` clamps into the new range, `frame = frames[0]`, and `ArrayView` renders the initial state.

Now trace the **failure** path. The user types `"38, 66, 2.5"`:

1. `parseInput` throws `new Error("Use whole numbers separated by commas. Empty values are not allowed.")`.
2. `catch (err) { setError(err.message); }`.
3. **Nothing else changes.** `input` still holds the previous dataset, `frames` is unchanged, the animation keeps its cursor, and the visualisation keeps rendering the last valid dataset while a red banner appears above it. [VERIFIED]

KEY TAKEAWAY
THE DESIGN PRINCIPLE TO NAME IN THE INTERVIEW

"A failed commit must not destroy the last good state." That is why the error path touches exactly one piece of state. It is the same principle that makes the input panel safe to experiment in: you can type nonsense into AlgoViz indefinitely without losing the trace you were studying.

10.3 The other three ways a dataset enters the lab

Entry point	Code path	Notes
Preset buttons (`random`, `sorted`, `reverse`, `duplicates`)	`load(makeDataset(kind, 42))`	Always seed **42** — the presets are fixed shapes, deliberately reproducible. [VERIFIED line 283]

Entry point	Code path	Notes
Shuffle button	`setSeed(seed + 1);` `load(makeDataset("random", seed))`	The seed increment is a state update, so the current render's `seed` value is what gets used. Consequence: the sequence of datasets is still 42, 43, 44…, but the visible `seed` state is always one ahead of the dataset actually loaded. Harmless, but if an interviewer asks "is your shuffle reproducible?", the precise answer is "The dataset for a given click is determined by the seed value at that click, and the seeds advance by one each click — so a session's shuffle sequence is reproducible even though I don't display the seed." [VERIFIED]
Saved experiment / imported JSON	`parseInput(item.input.join(","))` → `load(values, item.algorithm)`	Revalidated through the **same** `parseInput` — untrusted data gets no shortcuts. An invalid saved entry is caught and reported as "This saved experiment is invalid." [VERIFIED]

10.4 `makeDataset`: a hand-rolled deterministic generator

`src/lab/engine.js · lines 106–120 (verbatim)`

```
export function makeDataset(kind, seed = 42) {
  let state = seed >>> 0;
  const random = () => {
    state = (Math.imul(1664525, state) + 1013904223) >>> 0;
    return state / 4294967296;
  };
  const values = Array.from(
    { length: 10 },
    () => 5 + Math.floor(random() * 90),
  );
  if (kind === "reverse") return values.sort((a, b) => b - a);
  if (kind === "sorted") return values.sort((a, b) => a - b);
  if (kind === "duplicates") return values.map((n) => (n % 4) * 15 + 10);
  return values;
}
```

Three things worth knowing precisely:

1. **It is a linear congruential generator**, `state = (1664525 * state + 1013904223) mod 2³²`, using `Math.imul` for 32-bit-correct multiplication and `>>> 0` to keep the state unsigned. `Math.imul` matters: plain `*` on numbers above 2^{53} loses precision and the sequence would not be reproducible.
2. **It always produces exactly ten values**, in the range 5–94 (`5 + floor(random() * 90)`). Not two, not twenty-four — ten. [VERIFIED]
3. **Same seed → identical array, every time, in every browser**, because the arithmetic is integer-only and does not touch `Math.random()`. The test asserts exactly this: `assert.deepEqual(makeDataset("random", 12), makeDataset("random", 12))` and `assert.notDeepEqual(makeDataset("random", 12), makeDataset("random", 13))`. [VERIFIED]

Kind	Transformation	Purpose
`random`	as generated	Baseline unsorted input
`sorted`	`values.sort((a, b) => a - b)`	**Best case** for insertion sort and bubble sort's early-exit variant — the trace is dramatically shorter
`reverse`	`values.sort((a, b) => b - a)`	**Worst case** for bubble sort and quick sort with a last-element pivot

Kind	Transformation	Purpose
`duplicates`	`values.map(n => (n % 4) * 15 + 10)` → values from the set {10, 25, 40, 55}	Forces equal keys, which is the only way to observe whether equal values keep their relative order — and, in this codebase, the only way to see the stable-ID design matter

INTERVIEW QUESTION
"WHY IS `DUPLICATES` THE MOST INTERESTING PRESET?"

[VERIFIED] Because equal values are the case where an index-based visualisation breaks down and an ID-based one does not. Each element carries `id: "block-<original index>"`, so when two elements both hold the value 25, React's `key={item.id}` keeps them as distinct DOM nodes and the "active" highlight follows the element, not the position. The element-object clone on every frame preserves that id, so the identity of an element survives every swap it participates in. With `(n % 4) * 15 + 10` the array is guaranteed to contain repeats, which is exactly the input that would expose an implementation that identified elements by their value.

10.5 The input panel's defensive details

Detail	Source	Why it is there
`maxLength={150}` on the text input	line 260	A cheap client-side ceiling so a user cannot paste a 10 MB string and force a 24-slot parse attempt over millions of characters
`aria-describedby="input-help"`	line 259	Connects the input to the hint text "2–24 integers · −999 to 999"
`role="alert"` on the error banner	line 290	Screen readers announce validation failure immediately
`role="status"` on the notice banner	line 295	Polite announcement for save/import confirmations
The Apply button is `type="submit"` inside the form	line 263	Pressing Enter applies the input — the form provides the keyboard affordance for free
The shuffle button is `type="button"`	line 267	Without it, it would submit the form and apply the (unchanged) text instead of shuffling
Preset buttons are **outside** the form	lines 278–288	They are plain buttons in a `div`; being outside the form means they cannot accidentally submit it

TRAP
"WHY IS THERE A `MAXLENGTH` ON AN INPUT YOU VALIDATE ANYWAY?"

The trap is answering "because of validation". They are different controls at different layers: `maxLength` is a UI affordance that prevents the bad state from being entered; `parseInput` is a correctness guarantee that runs regardless of how the value got there (paste, autofill, saved experiment, imported JSON, or a programmatic call in a test). Remove `maxLength` and the app is still correct, just less pleasant. Remove `parseInput` and nothing is guaranteed. Never confuse a UI constraint with an invariant.

CHECKLIST
RED-LINE DRILLS FOR PART IV

- ■ Can you name all 12 `useState` variables and the reducer without looking?
- ■ Can you explain `text` vs `input` in under 30 seconds?
- ■ Can you state all five validation rules and the exact error message for each?
- ■ Can you trace a failed submit and explain why the visualisation keeps working?
- ■ Can you name the four dataset kinds and say what each one is good for?
- ■ Can you say the LCG formula and why `Math.imul` is necessary?
- ■ Can you point at the duplicated generator run and give the fix?

PART V

Recorded-frame architecture

What a frame is, how generators build them, why cloning matters, and why execution is not playback.

CHAPTER 11

11 · Frame anatomy

Ten possible fields, six required ones, and the exact meaning of each.

11.1 The definition

MUST KNOW
WHAT IS A FRAME?

[VERIFIED] A frame is a **plain JavaScript object** describing one meaningful moment in an algorithm's execution. It is created by the generator, never mutated afterwards, and rendered as-is. A trace is an array of frames: `frames[0]` is always the initial state before any work, and `frames.at(-1)` is always the completed, fully sorted state with every index marked sorted.

`src/lab/engine.js · lines 8–17 (verbatim)`

```
export const algorithms = {
  bubble: {
    name: "Bubble sort",
    run: generateBubbleSort,
    time: "O(n²)",
    space: "O(1)",
    idea: "Compare neighbors. Move the larger value right. Each pass locks one more element.",
    invariant:
      "After each complete pass, the rightmost processed elements are in their final positions.",
  },
```

11.2 The complete field table

The right-hand columns show which generators emit each field. A dot means "always present", a dash means "not present".

Field	Type	Meaning	bub	sel	ins	mer	qck	hea
`array`	`{id, val, ...}[]`	The full array state at this moment — a **fresh clone**, never a shared reference	●	●	●	●	●	●
`activeIndices`	`number[]`	The indices under consideration right now; drives the green highlight and the ● marker	●	●	●	●	●	●
`sortedIndices`	`number[]`	Indices considered finalised; drives the teal "done" styling and the ✓ marker	●	●	●	●	●	●
`isSwapping`	`boolean`	True only on the frame pushed immediately before a real swap is applied	●	●	●	●	●	●
`minimumIndex`	`number | null`	Selection sort's tracked minimum	●	●	●	–	–	–
`codeLine`	`string`	Machine-readable tag naming the operation (`'compare'`, `'swap'`, `'base_case'`…)	●	●	●	●	●	●
`currentStats`	`{passes, comps, swaps}`	A **clone** of the running counters at this moment	●	●	●	●	●	●
`msg`	`string`	The human sentence shown in the explanation panel	●	●	●	●	●	●
`pivotIndex`	`number | null`	Quick sort's pivot / heap sort's staging field	–	–	–	●	●	●

Field	Type	Meaning	bub	sel	ins	mer	qck	hea
`subArrayRange`	`[low, high]`	The recursion window currently being processed	–	–	–	●	●	●
`isLifting`	`boolean`	Insertion sort's "this element is being carried" flag	–	–	●	–	–	–

> **NOTE**
> **FOUR FIELDS THE LIVE UI DOES NOT CONSUME — AND THE SUBTLE REASON WHY**
>
> [VERIFIED] In the lab, `ArrayView` and the inspector read `array`, `activeIndices`, `sortedIndices`, `pivotIndex`, `codeLine`, `msg` and `currentStats`. They do not read `isSwapping`, `subArrayRange`, `minimumIndex` or merge sort's `yOffset`.
>
> **Be precise about this, because the careless version of the claim is false.** I grepped it: those four fields are read — by `src/modules/core/SimpleSort.jsx` (`isSwapping`, `minimumIndex`, `isLifting`) and `src/modules/core/EfficientSort.jsx` (`isSwapping`, `pivotIndex`, `subArrayRange`, `yOffset`, for a `translateY` on buffered elements). Both files have **zero importers** anywhere in the repository, so neither is in the bundle. The accurate statement is therefore: no component in the shipped application reads these fields; the only readers are two unimported files from an earlier generation of the UI.
>
> This is a real, honest finding and a superb thing to volunteer, because it shows you read your own data model critically: *"The frame schema carries four fields the app's UI doesn't consume — `isSwapping`, `subArrayRange`, `minimumIndex` and merge sort's `yOffset`. They were written for a richer renderer: the swap flag for a flash animation, `subArrayRange` for a nested-invocation bracket, `minimumIndex` for a corner marker. Two files from the earlier UI generation do read them, but both have zero importers and aren't in the bundle — so in the shipped app the data is recorded, cloned, compared for determinism, and not drawn."* [PROPOSED] Adding a sub-array bracket for merge/quick/heap and a swap pulse would consume the existing data with no engine changes at all.

11.3 The `codeLine` vocabulary — the complete list

[VERIFIED] Extracted from the six generators. This is the closest thing the project has to "which line of pseudocode are we on", and it is what the UI displays as "CURRENT OPERATION".

Generator	Tags
Bubble	`start`, `outer`, `inner`, `compare`, `swap`
Selection	`start`, `outer`, `inner`, `new_min`, `swap`, `lock`
Insertion	`start`, `outer`, `compare`, `swap`, `break`, `drop`
Merge	`start`, `base_case`, `mid`, `recurse_left`, `recurse_right`, `merge_call`, `buffer`, `compare`, `write_left`, `write_right`, `write_rem`
Quick	`start`, `base_case`, `partition`, `compare`, `swap`, `recurse_left`, `recurse_right`
Heap	`start`, `build_heap`, `extract`, `pointers`, `compare_children`, `swap_heapify`, `swap_root`, `heapify_root`

[VERIFIED] The renderer prints it with underscores replaced by spaces: `frame.codeLine?.replaceAll("_", " ")` — so `base_case` renders as "BASE CASE" in the operation line. The optional chaining (`?.`) is defensive: if a frame somehow lacked `codeLine`, the panel would render "CURRENT OPERATION · " rather than crashing.

11.4 The ID scheme, and why it is not an index

MUST KNOW

STABLE IDENTITY IS THE QUIET HERO OF THE WHOLE DESIGN

[VERIFIED] Every element is wrapped once, at the top of the generator:

```
let arr = rawArray.map((val, index) => ({ id: `block-${index}`, val }));
```

Then `ArrayView` renders `key={item.id}`. The ID is assigned **before** any sorting happens and it travels with the element object through every swap for the rest of the trace. `id` therefore encodes which original input position this value came from, and it is stable across the whole recording.

Three consequences, in increasing order of interview value:

1. **React reconciliation is correct.** If you used `key={i}` (the render index), React would reuse DOM nodes positionally and a swap would look like a value change rather than a movement — and any in-progress CSS transition would attach to the wrong element.
2. **Equal values stay distinguishable.** With `duplicates` input the array holds repeated values; only the ID can tell which of the two 25s is under comparison.
3. **It makes traces comparable as data.** `assert.deepEqual(frames, algorithm.run(input))` passes only if IDs are deterministic. Because they are derived from the original index, they are — which is the property that makes the reproducibility test possible at all.

TRAP

"SO IDS ARE JUST ARRAY INDICES WITH EXTRA STEPS?"

No, and the distinction is precise: an **index** is a position in the current array; an **ID** is a property of the element. After one swap, index 3 holds a different element while `block-3` sits somewhere else. The test that proves the difference is in the test file: `new Set(frames.at(-1).array.map(x => x.id)).size === input.length` — after sorting, every original element is still present exactly once. If IDs were reused or duplicated, that assertion fails.

DANGER ZONE — WHERE CANDIDATES DIE

THE ONE PLACE WHERE IDS GENUINELY BREAK — MERGE SORT, AND IT IS A REAL BUG

[VERIFIED — I reproduced this by running the generator] In `generateMergeSort`, the merge step copies the window into a temporary buffer with `tempArr.push({...arr[i]})` — a **shallow copy of the element object, including its `id`** — then writes each buffered object back into `arr[k]`, starting at `k = low`. Because the write-back starts at the left edge of the window while the original elements are still sitting at their old positions, the **same `id` exists twice inside `arr` at the same time** — and the frame pushed by `write_left` / `write_right` / `write_rem` clones `arr` in exactly that state.

Measured on the real code:

Input	Trace length	Frames containing a duplicate `id`
default 8 values	107	**23** (e.g. frame 39, `write_right`, duplicate `block-2`)
`[3,1,2]`	30	4 (e.g. frame 15, duplicate `block-1`, array renders as `[1,1,2]`)
seeded 10 values	142	24
`duplicates` preset	142	22
reverse 24 values	375	**141** (frame 24: duplicate `block-1`)

`ArrayView` renders each column with `key={item.id}`, so those frames are React **duplicate-key** frames. In development React logs "Encountered two children with the same key..."; in production reconciliation with duplicate keys is unspecified, so a column can momentarily fail to update. The **final** frame is always clean — every trace's last frame has unique ids, which is exactly what the test file asserts — so nothing corrupts permanently; the bug is confined to the transient mid-merge frames.

There is a second, visual half to the same root cause: the value that was moved into the buffer disappears from the rendered array until it is written back. Frame 15 of `[3,1,2]` renders `[1,1,2]` — the `3` is momentarily gone because it is sitting in `tempArr`, which no frame exposes. The generator already carries `yOffset = 80` and a `buffer` frame for exactly this visual, but `ArrayView` does not read `yOffset`, so the buffer state is invisible.

SAY THIS OUT LOUD

HOW TO ANSWER IF THE INTERVIEWER SPOTS IT

"That's a real bug and I'm glad you found it — it's in merge sort's write-back. The generator copies elements into a temporary buffer as shallow clones, so the clone keeps the same `id`, and then writes them back into the array from the left while the originals are still at their old positions. For the frames pushed during the write-back, the same id appears twice in the array, and my column key is `item.id`, so React sees a duplicate key. It never affects the final frame — the last frame always has unique ids and that's asserted in the test suite — so the damage is a React warning plus the theoretical risk of a mis-reconciled column for one frame. There are two candidate fixes and they trade off against each other. The minimal one is to make the render key unique per frame — `key={item.id + ':' + i}` — which removes the warning immediately, because every frame is a full snapshot so positional keys cannot corrupt anything across frames; the cost is that React then treats bars as position-identified rather than element-identified, which would matter if I ever added per-element movement animations. The other option is to keep element identity and give the buffered clone a fresh id at write time, e.g. `{...tempArr[i], id: tempArr[i].id + '-m' + low + '-' + high}`; that preserves identity semantics but changes what an id means for merge sort, and the final-frame uniqueness assertion would still pass either way. I would take the key fix first because it is one line and cannot affect the algorithm, and the deeper fix is that the renderer should show the buffer explicitly — that's also why the value appears to vanish mid-merge; the generator already emits `yOffset` and a `buffer` frame for it, but the renderer ignores them."

11.5 The `currentStats` clone

[VERIFIED] The generators keep one mutable `stats` object and clone it into every frame: `currentStats: { ...stats }`. This is the second-most important clone in the project.

Why it matters: if a frame stored a reference to `stats`, then every frame would show the **final** counters, because the object is mutated in place throughout the run. Frame 0 would display the total comparisons of the entire sort. The spread makes each frame's counters a true snapshot of "how many comparisons had happened at this moment" — which is what the "Comparisons 12" tile claims to mean.

> **MUST KNOW**
> **THE INVARIANT THAT THE CLONE PROTECTS**
>
> **Any value inside a frame must be immutable from the moment the frame is created.** There are exactly two cloning operations that enforce it, and you should name both: `cloneArray()` for the array, `{...stats}` for the counters. Everything else in a frame is either a primitive or a freshly allocated literal.

CHAPTER 12

12 · How a generator builds a trace

The canonical walkthrough: `generateBubbleSort` line by line, then the general pattern that all six follow.

12.1 The general shape (all six generators)

```
generate<Algo>(rawArray)
  |
  +-- 1. WRAP      rawArray.map((val, i) => ({ id: `block-${i}`, val }))
  |                (insertion adds isLifting:false; merge adds yOffset:0)
  |
  +-- 2. INIT      frames = []; n = arr.length; sortedIndices = [];
  |                stats = { passes: 0, comps: 0, swaps: 0 }
  |
  +-- 3. CLONE     const cloneArray = () => arr.map(item => ({ ...item }))
  |
  +-- 4. SEED      push the "System Ready" frame (empty active, empty sorted, zero stats)
  |
  +-- 5. LOOP      for each structural step:
  |                   push a frame  BEFORE the state changes
  |                   mutate arr and/or stats
  |                   push a frame  AFTER the state changed   (only for swaps)
  |                   update the bookkeeping lists (sortedIndices, minIndex, ...)
  |
  +-- 6. FINISH    push a terminal frame with sortedIndices = all indices and the final stats
  |
  +-- 7. RETURN    frames
```

12.2 Annotated source: the first twelve frames of bubble sort

`src/modules/core/algorithm/sorting/algorithms/bubbleSort.js · lines 1–29 (verbatim)`

```
export const generateBubbleSort = (rawArray) => {
    let arr = rawArray.map((val, index) => ({
        id: `block-${index}`,
        val: val
    }));

    const frames = [];
    const n = arr.length;
    const sortedIndices = [];
    let stats = { passes: 0, comps: 0, swaps: 0 };

    const cloneArray = () => arr.map(item => ({ ...item }));

    frames.push({
        array: cloneArray(), activeIndices: [], sortedIndices: [...sortedIndices],
        isSwapping: false, minimumIndex: null, codeLine: 'start',
        currentStats: { ...stats },
        msg: "System Ready. Initializing Bubble Sort."
    });

    for (let i = 0; i < n; i++) {
        stats.passes++;

        frames.push({
            array: cloneArray(), activeIndices: [], sortedIndices: [...sortedIndices],
            isSwapping: false, minimumIndex: null, codeLine: 'outer',
            currentStats: { ...stats },
            msg: `Pass ${stats.passes} of ${n}.`
        });
```

Reading it precisely, line by line:

Lines	What happens	Why it is written that way
2–5	Wrap each value in an object with a stable id	The id survives sorting; the value is what gets compared

Lines	What happens	Why it is written that way
7–10	`frames`, `n`, `sortedIndices`, `stats`	`sortedIndices` is a plain array that is copied into each frame with `[...sortedIndices]` — never shared
12	`cloneArray`	A shallow-per-element clone: a new array whose element objects are new objects. Any in-place write to `arr[j]` after this point cannot retroactively change an earlier frame
14–19	The seed frame	Guarantees `frames[0]` is a valid, renderable state — this is what makes "restart" trivially safe
21–23	Outer loop; `stats.passes++` **before** the frame is pushed	So the pass frame reports the pass it is starting, not the one it finished
24–29	Pass-start frame with `activeIndices: []`	An "empty-handed" frame: the pass is announced before any comparison happens. This is what gives the play-through a readable rhythm

`src/modules/core/algorithm/sorting/algorithms/bubbleSort.js · lines 31–45 (verbatim)`

```
        for (let j = 0; j < n - i - 1; j++) {
            frames.push({
                array: cloneArray(), activeIndices: [j, j + 1], sortedIndices: [...sortedIndices],
                isSwapping: false, minimumIndex: null, codeLine: 'inner',
                currentStats: { ...stats },
                msg: `Scanning window at index ${j} and ${j + 1}.`
            });

            stats.comps++;
            frames.push({
                array: cloneArray(), activeIndices: [j, j + 1], sortedIndices: [...sortedIndices],
                isSwapping: false, minimumIndex: null, codeLine: 'compare',
                currentStats: { ...stats },
                msg: `Comparing ${arr[j].val} and ${arr[j + 1].val}. Is ${arr[j].val} > ${arr[j + 1].val}?`
            });
```

Lines	What happens	Why
31	`for (let j = 0; j < n - i - 1; j++)`	The inner bound shrinks because the last `i` elements are already locked
32–37	A frame **before** the comparison, `codeLine: 'inner'`	Separates "moving the window" from "comparing the window"
39	`stats.comps++`	Incremented once per comparison pair — the counter is incremented before the compare frame so the frame reports this comparison as already counted
40–45	The `compare` frame, with `msg` interpolating the actual values	This is the sentence a learner reads to understand the decision

`src/modules/core/algorithm/sorting/algorithms/bubbleSort.js · lines 47–73 (verbatim)`

```
            if (arr[j].val > arr[j + 1].val) {
                stats.swaps++;
                frames.push({
                    array: cloneArray(), activeIndices: [j, j + 1], sortedIndices: [...sortedIndices],
                    isSwapping: true, minimumIndex: null, codeLine: 'swap',
                    currentStats: { ...stats },
                    msg: `Yes! ${arr[j].val} > ${arr[j + 1].val}. Swapping blocks...`
                });

                let temp = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = temp;

                frames.push({
                    array: cloneArray(), activeIndices: [j, j + 1], sortedIndices: [...sortedIndices],
                    isSwapping: false, minimumIndex: null, codeLine: 'swap',
                    currentStats: { ...stats },
                    msg: `Swap complete.`
                });
            } else {
                frames.push({
                    array: cloneArray(), activeIndices: [j, j + 1], sortedIndices: [...sortedIndices],
                    isSwapping: false, minimumIndex: null, codeLine: 'compare',
                    currentStats: { ...stats },
                    msg: `No swap needed. Moving window forward.`
                });
            }
```

Lines	What happens	Why
47	`if (arr[j].val > arr[j + 1].val)`	The decision. It is a strict `>` — equal values are **not** swapped
48	`stats.swaps++`	Counted once per physical swap
49–54	Pre-swap frame with `isSwapping: true`	The frame exists so the UI could render a "mid-swap" moment. It carries the pre-swap array, so it also proves what the array looked like before
56–58	The actual three-line swap	Classic temp-variable swap — no destructuring, which is a readability choice
60–65	Post-swap frame, `isSwapping: false`	The other half of the pair. Together, these two frames make the swap visible as a two-step event
66–73	The else branch pushes a `"No swap needed"` frame	**This is the frame most visualisers omit.** It records a negative decision — proof that the comparison happened and the answer was "no". Without it, a viewer cannot tell a skipped comparison from a comparison that returned false

Line 76 is the subtlest line in the file:

```
sortedIndices.push(n - i - 1);
```

After each outer pass, exactly one more index is finalised — the last unsorted position. The array `sortedIndices` is **mutated**, and the next frame that reads it copies it with `[...sortedIndices]`. Note that the number of elements marked sorted grows by one per pass, which is why the "sorted" region in the UI grows from the right. [VERIFIED]

12.3 The frame-count arithmetic (measured, not guessed)

I ran every generator against several inputs. These are real numbers from this repository:

Input	bubble	selection	insertion	merge	quick	heap
`[5, 1]` (n = 2)	10	8	7	16	16	10
default 8 values	118	65	76	107	75	84
seeded 10 values	172	76	79	142	136	122
reverse-sorted 24 values	**1,154**	492	876	375	632	348
`[]` (empty)	2	2	2	4	4	4

MUST KNOW

THE FORMULA FOR BUBBLE SORT'S FRAME COUNT

For an input where every adjacent pair swaps (`m` swaps total), each inner iteration produces: **1** `inner` frame + **1** `compare` frame + **2** swap frames (`swap` + `swap complete`) = 4 frames. A non-swapping iteration produces 1 + 1 + 1 = 3 frames. Plus **1** seed frame + **1** outer frame per pass + **1** "pass complete" frame per pass + **1** terminal frame.

For reverse-sorted `n = 24`: comparisons = n(n−1)/2 = 276, all of which swap. Frames = 1 (seed) + 24 (outer) + 276×4 (inner+compare+swap+swap-complete) + 24 (pass complete) + 1 (final) = **1,154**. [VERIFIED — this matches the measured value exactly, which is a beautiful thing to derive on a whiteboard if you are asked.]

INTERVIEW QUESTION

"HOW MANY FRAMES DOES BUBBLE SORT PRODUCE FOR 24 ELEMENTS, AND WHY THAT NUMBER?"

Answer with the derivation, not the number: "There's a seed frame, one frame per pass start, one per inner iteration for the window, one for the comparison, two for each swap — the pre-swap and the post-swap — one per pass completion, and one final frame. For a reverse-sorted 24-element array every comparison swaps, so 276 comparisons × 4 + 1 + 24 + 24 + 1 = 1,154. I've measured it; it matches."

12.4 The same pattern, seen in the other five

Algorithm	Structural frames that make it recognisable on screen
Selection	Every frame carries `minimumIndex`, so the "current best candidate" is visible while the scan continues. `new_min` frames mark the exact moment the candidate changes; `lock` frames mark the moment an index becomes final. The outer loop runs to `n-1` and the last index is force-marked sorted afterwards
Insertion	`isLifting` is set on the element being carried (line 23), and the `drop` frame is pushed after `isLifting = false` (line 70) — so the renderer could animate a lift-and-drop with data that already exists. The `break` frame is the only explicit "we are stopping early" record in the project
Merge	The recursion produces `recurse_left` / `recurse_right` frames before the recursive call, so the trace shows the call tree in order. `buffer` frames mark the moment elements are conceptually moved into temporary memory, and `yOffset = 80` is set on exactly the elements in the current merge window (lines 70–72) — a visual channel the lab's renderer does not use
Quick	`partition` frames bracket the partition work; the pivot index is carried on nearly every frame; after `partition()` returns, the pivot index is pushed onto `sortedIndices` immediately (line 54) because a pivot is final the moment its partition ends
Heap	Split into two phases with explicit `build_heap` and `extract` frames. `heapify` pushes a `pointers` frame showing parent and children, then a `compare_children` frame — two comparisons are counted per call (lines 35 and 40) even if the first condition short-circuits

MUST KNOW

THE ONE STATEMENT TO MAKE ABOUT ALL SIX GENERATORS

"They all share one contract: take a raw array of numbers, never mutate it, and return a complete immutable trace whose first frame is the initial state and whose last frame is the sorted state with all indices marked. That uniformity is what lets `engine.js` treat them interchangeably and what lets the tests loop over all six with a single assertion body." [VERIFIED]

CHAPTER 13

13 · Execution vs playback — the separation that defines the project

Two phases, two very different properties, one interface between them.

13.1 The two phases

```
PHASE 1 · EXECUTION                          PHASE 2 · PLAYBACK
-------------------------------------        -------------------------------------
when:  inside algorithms[algo].run(input)    when:  every render after that
cost:  one synchronous burst, O(n^2)         cost:  O(1) — read frames[cursor]
       comparisons + O(frames x n) copies
output: frames[]                              input:  frames[] + playback state
knows about: sorting                          knows about: nothing but an index
testable: yes, in plain Node                  testable: the reducer is; the UI is not
happens: once per (algo, input) pair          happens: on every tick, seek, keypress
```

MUST KNOW

WHY THIS SEPARATION IS THE WHOLE DESIGN

Playback does not know what a comparison is. It knows there are `length` things and a `cursor` pointing at one of them. That is why the same reducer could drive N-Queens, pathfinding, or a tree traversal with zero modification — it has no algorithm-specific knowledge to unlearn.

13.2 "Why does the algorithm not need to run backwards when you seek backwards?"

Because **nothing is being computed during playback at all.** The algorithm has already finished. Its entire history is sitting in `frames` as data. To show an earlier moment, React renders a different array element.

SAY THIS OUT LOUD

THE ANSWER, IN THE STRONGEST FORM

"The algorithm ran once, to completion, before you ever pressed Play. Every state it passed through was recorded as an object with the array cloned and the counters cloned. Playback is `frames[cursor]`. When you drag the timeline left, the cursor decreases and React renders a different element of an array that is already in memory. There is no reverse execution, no inverse operation, no re-run — and that's precisely why arbitrary seeking works: jumping from frame 900 to frame 40 costs exactly the same as jumping to frame 899."

That last clause is the part that demonstrates understanding. A naive implementation that re-ran the algorithm to reach a target frame would make seek cost O(target) — which is exactly the design's alternative, and it is worth naming as such.

13.3 What you give up — the honest trade-off

Consequence	Detail
Memory becomes a real cost	Full snapshots mean memory is proportional to frames × elements, not to the algorithm's own auxiliary space. The engine's space complexity and the application's space complexity are now different numbers.
Execution must complete before playback starts	There is no streaming, no "watch it think". For 24 elements that is imperceptible; for a large input it would be a visible freeze on the main thread.

Consequence	Detail
Traces need a bound	Hence the 24-element cap: it is the mechanism that keeps the memory consequence acceptable. [VERIFIED: `parseInput` enforces it, and the UI footnote says "Trace storage is bounded by the 24-element input limit."]
You cannot inspect a state you did not record	Frames only exist where the generator pushed one. If a step is missing from the generator, it is invisible to the user — the recording defines the observable history.

13.4 The alternative designs, and why they lose (for this use case)

Alternative	How seek would work	Verdict
Live animation, no history	Impossible — you cannot go back	Original v1 approach; cannot support the headline feature
Re-run to a target step	Re-execute from frame 0 and stop at step `k` → O(k) per seek, O(n) per scrub position	Simple to implement, but scrubbing becomes quadratic and non-deterministic under a mutable seed
Record operations, replay forward from a checkpoint	Store deltas + periodic snapshots; reconstruct by replaying from the nearest checkpoint	[PROPOSED] The right answer at scale — far less memory, still O(1)-ish seeks with periodic reconstruction
Inverse operations	Derive the reverse of each mutation	Fragile: `merge` and `heapify` have no clean inverse, and every algorithm needs bespoke undo code
Web Worker + streaming frames	Worker computes and posts frames; UI stays responsive	[PROPOSED] Solves the freeze, not the memory; the README lists workers as a future step before raising the input cap

DANGER ZONE — WHERE CANDIDATES DIE

DO NOT SAY "I CHOSE FULL SNAPSHOTS BECAUSE MEMORY IS CHEAP"

That is a non-answer. The defensible version: "I chose full snapshots because they make every frame independently renderable and independently testable, which is what I needed for reproducibility and for the test suite. That was a deliberate first implementation with a known cost, and I bounded the input size to 24 to keep the cost contained instead of claiming scalability I haven't tested. The next architecture step is compact events with periodic checkpoints, which the README lists as planned work." — [VERIFIED] that plan is literally in the README's "What's next" list.

CHAPTER 14

14 · The memory trade-off, quantified

Real measurements from this repository, and the asymptotic story behind them.

14.1 Measured trace sizes

I serialised every trace with `JSON.stringify` to get a concrete byte count (a fair proxy for retained memory, since these are plain data objects):

Input	Algorithm	Frames	JSON size
2 elements	quick	16	4.4 KB
default 8 elements	bubble	118	48.6 KB
seeded 10 elements	bubble	172	80.1 KB
reverse 24 elements	heap	348	311 KB
reverse 24 elements	merge	375	430 KB
reverse 24 elements	selection	492	420 KB
reverse 24 elements	quick	632	554 KB
reverse 24 elements	insertion	876	1.13 MB
reverse 24 elements	**bubble**	**1,154**	**981 KB**

MUST KNOW

THE NUMBER TO REMEMBER

The worst trace this application can produce is **~1 MB of frames** — bubble sort on a reverse-sorted 24-element array, 1,154 frames. That is the consequence of the input cap, and it is why the cap exists. Quote this number: it makes the trade-off concrete instead of hand-wavy.

14.2 The asymptotics, stated exactly

Let n be the input length and F the number of frames.

Quantity	Complexity	Reason
Frame count for bubble/insertion/selection	**$O(n^2)$**	The inner loop is quadratic and each iteration pushes a constant number of frames
Frame count for merge/quick/heap	$O(n \log n)$ typical, $O(n^2)$ worst for quick	Driven by the number of comparisons and recursive calls
Elements per frame	$O(n)$	`cloneArray()` copies every element object
Total retained data	**$O(F \times n)$**	Both factors multiply
For a quadratic sort with per-step cloning	**conceptually $O(n^3)$**	Because F itself is $O(n^2)$ and each frame is $O(n) \rightarrow O(n^2 \cdot n)$
Playback per render	**$O(1)$** to select the frame, $O(n)$ to render its bars	`frames[cursor]` is an array index; the DOM work is proportional to visible elements

Check the $O(n^3)$ claim against the measurements: n goes from 8 to 24 (3×). n^3 grows by 27×. Bubble's JSON goes from 48.6 KB to 981 KB — 20×, in the same ballpark once you account for fixed per-frame overhead. [INFERENCE — but the measured growth is consistent with it, and the README itself states the cubic-storage consequence.]

INTERVIEW QUESTION
"WHAT IS THE SPACE COMPLEXITY OF ALGOVIZ?"

This question has two correct answers and you must give both, in this order:

1. **The algorithms:** all six are in-place or near-in-place. Bubble/selection/insertion are O(1) auxiliary; merge sort's implementation allocates a temporary buffer per merge, up to O(n); heap sort's implementation recurses, so its auxiliary stack is O(log n) — which is exactly the footnote the UI displays as "* This implementation uses recursive heapify; its call stack uses O(log n) space." [VERIFIED: `engine.js` declares heap's space as `"O(log n)*"` and the UI renders that asterisk explanation.]
2. **The visualisation layer:** O(F × n), which for the quadratic sorts is conceptually $O(n^3)$. This is the application's cost, not the algorithm's, and it is the honest price of record-and-replay.

If you only give answer 1, you have missed the interesting part of your own project. If you only give answer 2, you have missed that the generators themselves are the standard textbook complexities.

14.3 "What happens with a huge trace?" — the failure sequence

If the 24-element cap were removed (say via a code change — there is no UI path around `parseInput`):

1. **First pain: synchronous main-thread freeze.** The generator is a plain synchronous loop. For n = 10,000, bubble sort performs ~50 million comparisons and clones a 10,000-element array on every push — hundreds of millions of object allocations, all before the first paint. The tab locks up.
2. **Second pain: memory.** Naive frame counts explode. At 10,000 elements a quadratic trace would need on the order of 10^{11} element copies. The tab crashes with an out-of-memory error long before it finishes.
3. **Third pain: render cost per frame.** Even if a frame existed, rendering 10,000 DOM nodes per frame — each with nested `div`/`strong`/`span` — would blow past any reasonable frame budget.
4. **Fourth pain: the timeline becomes useless.** A range input with 10^8 steps cannot be scrubbed meaningfully.
5. **Fifth pain: `localStorage`** would throw `QuotaExceededError` on the first save (typically ~5 MB per origin), which the `try/catch` in `save()` would catch and report as "Browser storage is unavailable. Use Export instead." [VERIFIED that this catch exists]

SAY THIS OUT LOUD
WHAT WOULD BREAK FIRST, AND HOW YOU WOULD FIX IT

"The main thread would lock up first, before memory even becomes the issue, because generation is synchronous. So the fixes come in this order: (1) put generation in a Web Worker so the UI never blocks; (2) change the frame representation from full snapshots to compact events with periodic checkpoints, so memory drops from O(frames × n) to something closer to O(operations + checkpoints × n); (3) keep an input cap, just a much larger one, because there is always a size where a browser tab is the wrong tool. The README lists exactly those first two as planned work — they are the designed next step, not something I'd invent on the spot." [VERIFIED for the README plan; the ordering is INFERENCE]

14.4 The delta-frame design, described properly [PROPOSED]

This is the answer to "how would you make this scale?", and it is worth being able to sketch precisely.

```
CURRENT (full snapshots)                PROPOSED (events + checkpoints)

frame 0: {array: [A B C D], ...}        frame 0:      SNAPSHOT {array: [A B C D]}
frame 1: {array: [A C B D], ...}        frame 1:      EVENT {swap: [1, 2], stats}
frame 2: {array: [A C B D], ...}        frame 2:      EVENT {compare: [1, 2], stats}
frame 3: {array: [A C D B], ...}        frame 3:      EVENT {swap: [2, 3], stats}
...                                     ...
frame k: full array again (every time)  frame k%50:   SNAPSHOT (re-anchor)

memory: O(frames x n)                   memory: O(events x constant) + O(n x n/50)
seek to frame j: O(1)                   seek to frame j: O(1) to the last checkpoint,
                                                         then replay <= 50 events
```

The trade-off, stated honestly: deltas make forward playback cheaper but make random seek cost a bounded amount of work (bounded by the checkpoint interval), and they make every frame's array derived rather than stored — which means the "render any frame directly" simplicity is lost, and the test suite would need a reconstruction function to assert against. The simplicity of full snapshots is the thing being traded away, and at 24 elements that trade was not worth making. [PROPOSED]

CHECKLIST

RED-LINE DRILLS FOR PART V

- ■ Recite the frame field table from memory (6 required + 4 optional fields).
- ■ Explain `cloneArray()` and `{...stats}` and what breaks without each.
- ■ Derive bubble sort's frame count for n = 5 if every pair swaps.
- ■ Quote the worst-case trace size: 1,154 frames / ~1 MB at n = 24.
- ■ State the memory complexity for the algorithms AND for the application, separately.
- ■ Explain what breaks first at 1 million frames, and give the three-step fix.

PART VI

React state management in AlgoViz

Only the React concepts this project actually uses — with the exact line they appear on and the alternative you rejected.

CHAPTER 15

15 · useState — ten independent pieces of state

What it does, where it is used, why, what the alternative was, and what it costs.

MUST KNOW
THE CONCEPT

`useState(initial)` returns a pair: the current value for this render, and a setter that schedules a re-render with a new value. State is **per component instance** and **persists across renders** but is **reset when the component unmounts**. Setters do not change the variable you already captured in this render — they queue an update for the next render. That single sentence explains 80% of "why is my state stale?" bugs.

15.1 Where AlgoViz uses it

[VERIFIED] Ten calls in `SortingLab.jsx` (`algo`, `text`, `input`, `error`, `notice`, `speed`, `compare`, `other`, `seed`, `saved`); the state register in Chapter 9 lists all of them with types and readers. Two other hook calls look like state but are not: `useSearchParams()` and `useLocation()`. The grouping that matters:

Group	Variables	Why they are separate state
Input	`text`, `input`	Draft (invalid-tolerant string) vs committed (guaranteed-valid array). Different types, different lifetimes
Feedback	`error`, `notice`	Failure and success need different roles (`alert` vs `status`) and different styling, and they must not overwrite each other's semantics
Configuration	`algo`, `other`, `speed`, `compare`, `seed`	Each is an independent user choice; none is derivable from the others
Persistence cache	`saved`	Read from `localStorage` **lazily** so the parse happens once per mount
Playback	none	Deliberately moved into `useReducer` (see Chapter 16)

15.2 The deep dive: initialising `saved` lazily

`src/lab/SortingLab.jsx · lines 77–92 (verbatim)`

```
  const [saved, setSaved] = useState(() => {
    try {
      const data = JSON.parse(
        localStorage.getItem("algoviz.experiments.v1") || "[]",
      );
      return Array.isArray(data)
        ? data
            .filter(
              (x) => x && Object.hasOwn(algorithms, x.algorithm) && Array.isArray(x.input),
            )
            .slice(0, 6)
        : [];
    } catch {
      return [];
    }
  });
```

MUST KNOW
THE LAZY-INITIALISER PATTERN, EXPLAINED

`useState(fn)` is not the same as `useState(fn())`. If you pass a function, React calls it **once**, during the first render, to compute the initial value. If you pass the result of a call, the expression executes on **every render** and the result is thrown away on all but the first.

So `useState(() => JSON.parse(localStorage.getItem(...)))` is a genuine optimisation, not a style preference. It matters here because the initialiser touches `localStorage` (synchronous, can throw, and is not free), then parses JSON, then filters and slices. Doing that on every keystroke-driven re-render would be pure waste.

`useState(42)` is fine with a primitive because there is nothing to compute.

Also inside that initialiser, three defensive layers, all of which you should be able to name:

1. `localStorage.getItem(...) || "[]"` — a `null` (missing key) becomes a valid JSON literal rather than a parse error.
2. `try { } catch { return []; }` — **a bare `catch` with no binding.** This is required because `JSON.parse` throws on malformed data, and `localStorage` itself can throw in restricted contexts (Safari private mode historically, or a `SecurityError` when storage is disabled).
3. `Array.isArray(data) ? data.filter(...).slice(0, 6) : []` — the stored value might be valid JSON but the wrong shape (an object, a string, a number). `Array.isArray` catches that, and `.filter()` re-checks every entry against the live `algorithms` registry so that a stored entry for a removed algorithm cannot crash the render.

INTERVIEW QUESTION
"WHY DOES THE FILTER CHECK `OBJECT.HASOWN(ALGORITHMS, X.ALGORITHM)` AGAIN, IF YOU ONLY EVER SAVE VALID ALGORITHMS?"

Because the stored data is **not trustworthy input**. It may be from an older version of the app with a different algorithm set, edited by the user in DevTools, or written by another page on the same origin. Validation happens at the boundary between storage and application state — that boundary is the point of trust, not the moment of writing. [INFERENCE on intent; the code is VERIFIED]

15.3 Alternative · trade-off · question

Aspect	Detail
Alternative	`useReducer` for a single `{text, input, error, notice}` object
Trade-off	A reducer would make the draft→commit transition explicit in one place, and make "clear both banners on load" a single action. It would also force every keystroke through a dispatch, adding indirection for a value that is genuinely just "the string in the box"
What AlgoViz chose	Independent `useState` for independent concerns; a reducer only where transitions are related (playback)
Interview question	*"Your component has ten `useState` calls. Isn't that a smell?"* See the model answer below

MODEL ANSWER
"TEN USESTATE CALLS — ISN'T THAT A SMELL?"

"Ten pieces of state, but only about eight conceptual concerns: the draft, the committed dataset, two kinds of feedback, the algorithm pair, speed, seed, and the storage cache. None of them is derived from another, and none of them is shared with a component that would need it lifted. The one place where the values are genuinely coupled — cursor, length and playing, which must change together under strict rules — is exactly where I did use a reducer. So I'd argue the smell test isn't the count, it's whether the state has any cross-field invariants. The playback state has three; the rest have none."

TRAP

"WHY NOT `useState` FOR PLAYBACK INSTEAD OF `useReducer`?"

Because the three playback values have **cross-field invariants**: `cursor` must stay within `[0, length-1]`, `playing` must be false on a seek, and `playing` must become false automatically at the last frame. With three `useState` calls, those rules would be re-implemented at every call site (the play button, the timeline, the arrows, the keyboard handler) and could drift apart. With a reducer, the rules exist once, are pure functions of `(state, action)`, and can be tested without rendering React — which is exactly what the test file does.

CHAPTER 16

16 · useReducer — the playback state machine

The single most important React decision in the project.

MUST KNOW
THE CONCEPT

`useReducer(reducer, initialArg)` returns `[state, dispatch]`. `dispatch(action)` queues an action; React calls `reducer(state, action)` during the next render to compute the new state and re-renders if it differs by `Object.is`. The reducer must be **pure** and must **not mutate** its arguments. Multiple dispatches in one event are batched. Because `dispatch` is stable across renders and the reducer is a standalone function, both are easy to test.

16.1 The exact call site

`src/lab/SortingLab.jsx · lines 98–105 (verbatim)`

```
  const [playback, dispatch] = useReducer(playbackReducer, {
    cursor: 0,
    length: frames.length,
    playing: false,
  });
  const cursor = Math.min(playback.cursor, frames.length - 1),
    frame = frames[cursor];
  const seek = (index) => dispatch({ type: "seek", cursor: index });
```

Three things to notice:

1. **The initial state is a value, not a function** — so `frames.length` is evaluated on every render even though it is only used on the first. Harmless, but a purist would wrap it: `useReducer(playbackReducer, {…}, init)`.
2. **`length` is initialised from `frames.length`**, which at the very first render is the trace for the default 8-element input. It is kept in sync afterwards by the `load` action.
3. **`cursor` is immediately re-derived** on the next line with `Math.min(playback.cursor, frames.length - 1)` — the defensive clamp discussed in Chapter 9.

16.2 Alternative · trade-off · question

Aspect	Detail
Alternative	Three `useState` calls, or a custom hook `usePlayback(length)` wrapping the reducer
Trade-off	A custom hook would be tidier and reusable — [PROPOSED] `usePlayback(frames.length)` would return `{cursor, playing, toggle, tick, seek, restart}` and would make the reducer reusable for the non-sorting modules (which the README lists as the next migration). It would also hide the reducer from a reader who wants to see the transition table
What AlgoViz chose	The raw `useReducer` in the component, so the state machine is visible where it is used
Interview question	"Where would you extract this?" → the hook. Say it as PROPOSED, not as existing work

DANGER ZONE — WHERE CANDIDATES DIE

NEVER CLAIM THE REDUCER IS "SHARED" OR "GENERIC"

[VERIFIED] The reducer is generic in design — it knows nothing about sorting — but it is currently used by exactly one component, `SortingLab`, and only one component imports it. The README says the migration to other algorithm families has not happened yet. Say: "The reducer is algorithm-agnostic and would drive any frame list, but today only the sorting lab consumes it — the migration of the other modules is the next step."

CHAPTER 17

17 · useEffect — two effects, both with cleanup

The most dangerous hook in the project, used carefully.

MUST KNOW
THE CONCEPT

`useEffect(fn, deps)` runs `fn` **after** the render commits. If `deps` is present, the effect re-runs only when a dependency changes by `Object.is`. Before each re-run — and on unmount — React calls the function returned by the previous run (the cleanup). `useEffect` has no dependency array = runs after every render; empty array `[]` = runs once on mount (twice in StrictMode development).

17.1 Effect #1 — the playback timer

`src/lab/SortingLab.jsx · lines 106–110 (verbatim)`

```
  useEffect(() => {
    if (!playback.playing) return;
    const id = setTimeout(() => dispatch({ type: "tick" }), speed);
    return () => clearTimeout(id);
  }, [playback.playing, playback.cursor, speed]);
```

Element	Explanation
`if (!playback.playing) return;`	The guard. When paused, the effect body does nothing — but note it **does not return a cleanup function**, so React has nothing to clean up. That is correct: no timer was scheduled
`setTimeout(() => dispatch({type: "tick"}), speed)`	One frame advance, scheduled once. Not an interval
`return () => clearTimeout(id)`	The cleanup. Fires before the next run and on unmount
`[playback.playing, playback.cursor, speed]`	`playing` to start/stop; `cursor` so each tick schedules the next one; `speed` so a mid-play speed change reschedules at the new delay

MUST KNOW
THE RE-SCHEDULING PATTERN, STATED PRECISELY

This effect is a **self-perpetuating chain, one link at a time**: playing → schedule a timeout → the timeout dispatches `tick` → `cursor` changes → the effect re-runs → the old timer is already fired, and the new one is scheduled → repeat. There is never more than one pending timer, because each run's cleanup clears the previous one. That is the whole "how do you prevent duplicate timers" answer: **the cleanup, plus a dependency array that includes everything the body reads.**

17.2 Effect #2 — the keyboard listener

`src/lab/SortingLab.jsx · lines 111–136 (verbatim)`

```
useEffect(() => {
  const key = (e) => {
    if (
      /INPUT|SELECT|TEXTAREA|BUTTON/.test(e.target.tagName) ||
      e.target.isContentEditable ||
      e.ctrlKey ||
      e.metaKey ||
      e.altKey
    )
      return;
    if (e.code === "Space") {
      e.preventDefault();
      dispatch({ type: "toggle" });
    }
    if (e.key === "ArrowRight") {
      e.preventDefault();
      dispatch({ type: "seek", cursor: cursor + 1 });
    }
    if (e.key === "ArrowLeft") {
      e.preventDefault();
      dispatch({ type: "seek", cursor: cursor - 1 });
    }
  };
  window.addEventListener("keydown", key);
  return () => window.removeEventListener("keydown", key);
}, [cursor]);
```

The interesting parts:

1. **The guard clause** (lines 113–120) ignores events when the target is an `INPUT`, `SELECT`, `TEXTAREA` or `BUTTON`, when the target is content-editable, or when any modifier key is held. [VERIFIED] Without it, typing a space in the dataset field would both insert a space and toggle playback — the classic global-shortcut bug.
2. **`e.code === "Space"`** vs `e.key === "ArrowRight"`. `code` is the physical key (so it works regardless of keyboard layout for the space bar), while `key` is the produced character. Mixing them here is deliberate but inconsistent-looking; a question worth anticipating.
3. **`e.preventDefault()`** on each handled key stops the page from scrolling when an arrow key is pressed.
4. **`[cursor]` as the dependency array.** The handler closes over `cursor`, so the listener must be re-attached whenever `cursor` changes, otherwise the arrows would read a stale cursor and appear to do nothing after the first press. This is the stale-closure trap in this codebase, and the correct fix is visible: the effect depends on the value the handler reads.
5. **The cleanup removes the listener** — without it, every cursor change would add another listener and the arrow keys would fire N times per press.

WEAKNESS / LIMITATION
THE INEFFICIENT-BUT-CORRECT DETAIL WORTH KNOWING

Because the dependency array contains `cursor`, this effect tears down and re-registers a window listener on **every frame advance during playback** — roughly 6 times per second at 6×. The correct and more efficient pattern is a functional update (`dispatch({type: "seek", cursor: cursorRef.current ± 1})` via a ref, or making the reducer accept a relative seek action like `{type: "step", delta: 1}`), which would let the dependency array be empty.

[VERIFIED] that this is the current behaviour; [INFERENCE] that the step action is the cleaner design. If asked "what would you change in this file?", this is one of the two best answers — the other being the duplicated generator run in `load()`.

17.3 What is not in an effect, and why that matters

MUST KNOW

THE EFFECTS THAT ARE DELIBERATELY ABSENT

There is **no effect that synchronises playback with the frame array.** When `input` or `algo` changes, the trace is recomputed by `useMemo` during render, and the cursor is clamped by a plain `Math.min(...)` during render. No `useEffect` participates. This is why the lab never shows a frame of a stale trace: the clamp is derived during render, not reconciled after the fact.

That is a genuine pattern worth naming: **"derive, don't synchronise."** If you had written `useEffect(() => setCursor(0), [frames])`, you would get one render with the old cursor against the new frames — the classic "flash of the wrong frame" and, in the worst case, an index out of bounds.

CHAPTER 18

18 · useMemo — two memoised traces

Caching that carries real semantics, not just performance.

18.1 Where it is used

src/lab/SortingLab.jsx · lines 93–97 (verbatim)

```
const frames = useMemo(() => algorithms[algo].run(input), [algo, input]);
const otherFrames = useMemo(
  () => (compare ? algorithms[other].run(input) : []),
  [compare, other, input],
);
```

Aspect	Detail
What it does	Returns a cached value, recomputed only when a dependency changes by `Object.is`
Why here	Each generator call allocates hundreds of objects. Without memoisation, `frames` would be rebuilt on every render: on every keystroke in the dataset field, every speed change, every hover, every error banner — for data that had not changed
Why `[algo, input]` and not `[text]`	`text` is the unvalidated draft. Deriving frames from it would mean running the generator on invalid input
Why `input` works as a dependency	Arrays compare by reference. `input` is replaced only by `load()`, which receives a freshly created array from `parseInput` or `makeDataset` — so the reference changes exactly when the dataset changes
The comparison memo	`[compare, other, input]`: `compare` gates the work (empty array when off), `other` selects the algorithm, `input` is the shared dataset — **the same reference for both traces**, which is precisely what makes the comparison honest

INTERVIEW QUESTION
"ISN'T `USEMEMO` JUST AN OPTIMISATION? WHY DOES IT MATTER ARCHITECTURALLY?"

Because here it also encodes a **semantic guarantee**: both memos depend on the same `input` reference, so the primary and comparison traces are provably derived from the same dataset. If the comparison ran on a copy of the input, the traces could still be correct but they would not be provably like-for-like. The single shared reference is the mechanism behind "same dataset" in the panel heading. [VERIFIED]

The honest caveat: **`useMemo` is a performance hint, not a semantic guarantee** — React is permitted to discard memoised values. It happens to be safe here because recomputing produces identical frames (the generators are pure), so a discarded cache costs time, not correctness. That is the distinction to make out loud.

18.2 Alternative · trade-off

Alternative	Result
No `useMemo` — build frames during render each time	Correct but wasteful; the worst trace is ~1 MB of objects per render
`useState` + an effect to build frames	Worse: an extra render pass with no frames, and the "flash of the wrong frame" problem
`useMemo` with `[algo, input, compare, other]` merged	Couples the two traces: toggling comparison off would needlessly rebuild the primary trace

Alternative	Result
Move generation into a Worker	[PROPOSED] the scale answer; adds async complexity that is unjustified at n ≤ 24

18.3 `useCallback` — used exactly once, and not here

[VERIFIED] `useCallback` appears once in the entire repository: `src/modules/core/Nqueens.jsx`, line 21, where `resetBoard` is memoised with the dependency `[gridSize]` and then invoked by an effect whose dependency is the memoised function itself.

```
const resetBoard = useCallback(() => { /* rebuild the board */ }, [gridSize]);
useEffect(() => { resetBoard(); }, [resetBoard]);
```

That pattern is worth understanding, because it is the reason `useCallback` exists in a codebase that mostly doesn't need it: **the effect depends on a function, and a function defined inline is a new object every render, so the effect would run on every render.** Memoising the function makes the effect's dependency stable, and the effect then runs only when `gridSize` genuinely changes — which is exactly when the board must be rebuilt.

The lab does not use `useCallback`, and the reason is structural rather than stylistic: its two effects depend on values (`playing`, `cursor`, `speed`), never on a function, so there is no unstable identity to stabilise. Adding `useCallback` where nothing consumes the function's identity is pure noise — React would allocate the wrapper and compare it against nothing.

INTERVIEW QUESTION
"SO WHY NOT MEMOISE `LOAD`, `APPLY` AND `SAVE` WITH `USECALLBACK`?"

Because nothing downstream depends on their identity. They are passed to `onClick` and `onSubmit` handlers, which are recreated every render anyway, and neither effect reads them. `useCallback` buys you a stable reference; if no dependency array and no memoised child consumes that reference, it buys you nothing. The correct order is to profile first and memoise only where an unstable identity is causing work. [GENERAL KNOWLEDGE, applied to this codebase]

Where it would start to matter: if `ArrayView` were memoised and given a callback prop, or if an effect took one of these functions as a dependency. Neither is true today.

CHAPTER 19

19 · useRef — two refs, two jobs

One for the DOM, one that is a trap.

[VERIFIED] There is exactly **one** `useRef` call in `SortingLab.jsx`:

1. **`fileInput`** (line 60) — holds the hidden `<input type="file">` element so the Import button can call `fileInput.current?.click()`. This is the canonical use: a mutable handle that must not trigger renders.

Two other values are easy to misremember as refs under pressure: **`params` **and** `location`** come from `useSearchParams()` and `useLocation()` — read-only router hooks, not refs, and not written to. Say the count precisely: one ref in the lab, two refs in the legacy modules (`isPausedRef` and `speedRef`).

Inside the legacy modules — the live example is `src/modules/core/Nqueens.jsx`, which is a lazy route — refs do a very different job: `isPausedRef` and `speedRef` mirror state so that `async` loops — which captured their own closure long ago — can read current values. That is the classic "state that the render cycle cannot reach" problem, and it is why the legacy code has synchronous pause behaviour without re-running effects.

MUST KNOW
THE `USEREF` FACTS AN INTERVIEWER IS CHECKING

- A ref's `.current` mutation **does not** re-render the component.
- The ref object is **stable** across renders (`useRef` returns the same object every time).
- Refs are the escape hatch for **values that must survive renders but must not drive them**.
- In React 19 + StrictMode, ref callbacks may be invoked twice in development; the ref object pattern used here is unaffected.

TRAP
"WHY IS `FILEINPUT` A REF AND NOT STATE?"

Because you never want to render based on the file input element. Putting it in state would cause a re-render the moment it attaches (and the element identity would change on every render, potentially causing an infinite loop if used as a dependency). Refs exist exactly for this: an imperative handle to a DOM node. [GENERAL KNOWLEDGE, applied to VERIFIED code]

CHAPTER 20

20 · Props, lifting, and derived state

How data moves between the components in this project.

20.1 Every significant prop in the project

Component	Props	Direction
`App`	none	—
`Home`	none	Reads URL via router hooks
`Overview`	`library` (boolean)	One prop switches hero vs library presentation
`Shell`	`children`	Layout slot
`ModuleFrame`	`children, activeTab, onTabChange, running, paused, onMouseLeave`	**This is the only component in the project with a callback prop** — the legacy modules own their own tab and running state and pass handlers down
`SortingLab`	none	Self-contained; gets everything from URL hooks
`ArrayView`	`frame`, `compact` (default `false`)	Pure presentation; no callbacks
`TutorialTab`	`data`	Renders whichever lesson object it is given

NOTE
WHAT IS NOTABLE BY ITS ABSENCE

There is **no prop drilling deeper than two levels**, no context provider in the live tree, and no state lifted above a page component. `ModeContext` exists but is only consumed by dead components. The consequence is a codebase with almost no indirection — which is a legitimate architectural answer to "how is your state organised?".

20.2 Derived state, done right

[VERIFIED] Seven values are computed during render and stored in no state at all: `initial`, `cursor`, `frame`, `meta`, `stats`, `last`, `compareCursor`. They are plain `const` declarations.

MUST KNOW
THE RULE ALGOVIZ FOLLOWS

If a value can be computed from existing state during render, it must not be its own state. Storing `frame` in state would create a second source of truth that could disagree with `frames` and `cursor` — for example after an input change. Deriving it guarantees consistency by construction: there is no moment when `frame` is not `frames[cursor]`.

INTERVIEW QUESTION
"GIVE ME AN EXAMPLE OF DERIVED STATE IN YOUR PROJECT AND WHAT WOULD GO WRONG IF YOU MADE IT REAL STATE."

"`frame` is derived — it's `frames[cursor]`. If I kept it in state and updated it in an effect on cursor change, then between the cursor change and the effect firing there would be a render where the cursor had moved but the rendered frame had not. The UI would show one frame while the timeline and step badge showed another. The dangerous half of that is when `frames` shrinks: for one render, `frame` would be an element of the previous trace — or `undefined` if the index no longer exists, which would crash `ArrayView` at `frame.array.map`. Deriving it makes that impossible." [VERIFIED reasoning from the code]

20.3 The cleanup checklist (everything that must be cleaned up)

Resource	Where it is acquired	How it is cleaned up	Verified
Playback timeout	the timer effect	`return () => clearTimeout(id)`	[VERIFIED] line 109
Keydown listener	the keyboard effect	`return () => window.removeEventListener("keydown", key)`	[VERIFIED] line 135
Generated object URL	`URL.createObjectURL(blob)` in `download()`	`setTimeout(() => URL.revokeObjectURL(url), 1000)`	[VERIFIED] line 57
Synthetic anchor	created in `download()`	never appended to the document, so nothing to remove	[VERIFIED] — it is clicked while detached; the browser still honours the download
Lazy chunks	React Router + `lazy()`	nothing to clean — module registry is process-wide	[GENERAL KNOWLEDGE]

MUST KNOW

THE REVOKEOBJECTURL DETAIL, AND WHY THE 1,000 MS DELAY IS DELIBERATE

`URL.createObjectURL` pins the blob in memory until it is revoked. Revoking **immediately** after `.click()` is a known failure mode in some browsers, because the download may not have started reading the blob yet. Delaying revocation by a second is the standard safe pattern. If asked "does your code leak memory?", this is the answer: "The only object URL I create is revoked on a timer, deliberately delayed so the download can start reading it first. A leak there would pin at most one small JSON blob per export." [VERIFIED code; the rationale is GENERAL KNOWLEDGE]

CHECKLIST

RED-LINE DRILLS FOR PART VI

- ■ Explain the lazy `useState` initialiser and why `useState(fn())` is wrong.
- ■ Explain the difference between the two effects' dependency arrays.
- ■ Explain why the keyboard effect must depend on `cursor`.
- ■ Explain "derive, don't synchronise" with the `frame` example.
- ■ Name two things that are cleaned up and one that is only cleaned up after a delay.
- ■ Say what the reducer is not: shared, generic-in-use, or used outside the lab.

PART VII

The playback reducer

The state machine, the transition tables, twenty worked simulations, and twenty-eight questions with answers.

CHAPTER 21

21 · The reducer, line by line

Twenty-five lines that carry the transport semantics of the entire application.

21.1 The complete source

`src/lab/engine.js · lines 80–105 (verbatim)`

```
export function playbackReducer(state, action) {
  switch (action.type) {
    case "load":
      return { cursor: 0, length: action.length, playing: false };
    case "toggle":
      return {
        ...state,
        cursor: state.cursor === state.length - 1 ? 0 : state.cursor,
        playing: !state.playing,
      };
    case "tick":
      return {
        ...state,
        cursor: Math.min(state.cursor + 1, state.length - 1),
        playing: state.cursor + 1 < state.length - 1,
      };
    case "seek":
      return {
        ...state,
        cursor: Math.max(0, Math.min(action.cursor, state.length - 1)),
        playing: false,
      };
    default:
      return state;
  }
}
```

21.2 Every line, annotated

Line	Code	What it does	Why it is written that way
80	`export function playbackReducer(state, action)`	A **pure standalone function**, exported separately from React	Exporting it means the test file can import and exercise it in plain Node with no renderer — this is why the playback rules are testable at all
81	`switch (action.type)`	Explicit action dispatch	A `switch` gives an exhaustive, greppable list of everything that can happen to transport state. Four cases, no more
82–83	`case "load": return { cursor: 0, length: action.length, playing: false };`	A **brand-new object**, not a spread of `state`	Loading a new trace discards everything. Spreading `...state` here would carry over a stale cursor, which could exceed the new `length`
84–89	`case "toggle"`	Spreads state, resets the cursor to 0 **if it is sitting on the last frame**, flips `playing`	This one line implements replay: pressing Play at the end starts again from the beginning. It also implements pause, because `!playing` is symmetric
90–95	`case "tick"`	Advances one frame, clamps at the end, and **derives** whether playback should continue	`playing: state.cursor + 1 < state.length - 1` uses the old cursor deliberately: it means "keep playing only if the frame we are moving to is not the last one"

Line	Code	What it does	Why it is written that way
96–101	`case "seek"`	Clamps into `[0, length-1]` and **always pauses**	Two separate decisions in one case: a clamp (safety) and a forced pause (semantics). See the danger box below for what is missing
102–103	`default: return state;`	Unknown actions return the identical object reference	React bails out of re-rendering when the reducer returns the same reference by `Object.is`. Returning `state` instead of `{...state}` is therefore a real optimisation, and it makes an accidental typo in an action type harmless rather than destructive

21.3 The three invariants the reducer maintains

MUST KNOW
THE TRANSPORT INVARIANTS

1. **Bound:** `0 <= cursor <= length - 1` — enforced on `seek` (clamp), `tick` (min), and `load` (reset to 0).
2. **Completion:** reaching the last frame always sets `playing: false`. There is no state in which playback is "running" at the end.
3. **Seek pauses:** `playing` is forced to `false` by every `seek`, so dragging the timeline never leaves the animation running underneath.

And the invariant it deliberately does **not** maintain: `length` is never computed from the frames — it is passed in by the caller. The reducer has no knowledge of the frame array at all. That is what makes it algorithm-agnostic, and it is also the reason `load()` must run the generator once just to count frames.

21.4 The state machine



21.5 The transition table — state × action

The table below is the complete truth table for the reducer. `F` = `length`, `L` = `length − 1` (the last index). Every entry was produced by actually running the reducer. [VERIFIED]

Current state	Action	New state	Notes
anything	`load(F')`	`{cursor: 0, length: F', playing: false}`	Discards previous state entirely; `length` is replaced, not merged
`{c, F, false}`, `c < L`	`toggle`	`{c, F, true}`	Plain play: resume from where you are
`{c, F, true}`	`toggle`	`{c, F, false}`	Plain pause
`{L, F, false}`	`toggle`	`{0, F, true}`	**Replay**: at the end, Play restarts
`{L, F, true}`	`toggle`	`{0, F, false}`	Unreachable in practice (completion always clears `playing`); if it occurred, the cursor rewinds but playback stays stopped
`{c, F, _}`	`tick`	`{min(c+1, L), F, (c+1 < L)}`	Advances; sets `playing` from the old cursor
`{L, F, _}`	`tick`	`{L, F, false}`	Already at the end: stays put, and stops
`{c, F, _}`	`seek(i)`, `0 ≤ i ≤ L`	`{i, F, false}`	Pauses unconditionally
`{c, F, _}`	`seek(i)`, `i < 0`	`{0, F, false}`	Clamped low
`{c, F, _}`	`seek(i)`, `i > L`	`{L, F, false}`	Clamped high
anything	unknown type	same object reference	`Object.is` bail-out: no re-render

21.6 What the reducer is not allowed to know

MUST KNOW

THE IGNORANCE IS THE DESIGN

The reducer never receives, reads, or references the frame array. It handles three numbers and a boolean. That is why:

- it can be tested with plain objects — `playbackReducer({}, {type: "load", length: 3})` — as the test file does;
- it would work unchanged for any other frame list (pathfinding visits, N-Queens placements, tree traversal orders) — which is the migration the README describes as future work;
- playback can never corrupt the trace: there is no code path from the reducer to the frames.

CHAPTER 22

22 · Twenty-two worked simulations

Every answer below was produced by executing the real reducer. Copy the table; rehearse the reasoning.

Common setup for 22.1–22.16: `length = 20`, so indices `0…19` and the last index `L = 19`.

#	Starting state	Action	Result	Why
1	(none)	`load(20)`	`{cursor:0, length:20, playing:false}`	A load always resets to the beginning and stops
2	`{5, 20, false}`	`seek(5)`	`{cursor:5, …, playing:false}`	Seek to where you already are is still a valid, simple state change (same values, new object)
3	`{5, 20, false}`	`tick`	`{cursor:6, playing:true}`	**Careful:** `tick` sets `playing` from the arithmetic, not from the previous value. It is not reachable from the UI while paused, because only the timer dispatches `tick`
4	`{5, 20, true}`	`tick`	`{cursor:6, playing:true}`	The normal forward step during playback
5	`{5, 20, false}`	`toggle`	`{cursor:5, playing:true}`	Play in place — it does **not** reset the cursor unless you are on the last frame
6	`{5, 20, true}`	`toggle`	`{cursor:5, playing:false}`	Pause in place
7	`{19, 20, false}`	`toggle`	`{cursor:0, playing:true}`	Replay from the top
8	`{19, 20, true}`	`toggle`	`{cursor:0, playing:false}`	The inconsistent state — under normal operation, reaching frame 19 has already set `playing:false`
9	`{5, 20, true}`	`seek(10)`	`{cursor:10, playing:false}`	Seeking pauses. This is the rule people forget in the interview
10	`{5, 20, true}`	`seek(-5)`	`{cursor:0, playing:false}`	Clamped low — a negative seek from a drag or a keyboard mashing session cannot go out of range
11	`{5, 20, false}`	`seek(99)`	`{cursor:19, playing:false}`	Clamped high
12	`{18, 20, true}`	`tick`	`{cursor:19, playing:false}`	The completion case: the cursor lands on the last frame and playback stops **in the same transition**
13	`{5, 20, false}`	`seek(2.5)`	`{cursor:2.5, …}`	**The gap.** The clamp does not floor or validate the type. `frames[2.5]` is `undefined`
14	`{5, 20, false}`	`seek(NaN)`	`{cursor:NaN, …}`	`Math.min(NaN, 19)` is `NaN`. Same class of problem
15	`{5, 20, false}`	`"nonsense"`	the same object (identity equal)	The `default` branch returns `state` itself, so React skips the re-render
16	`{19, 20, false}`	`tick`	`{cursor:19, playing:false}`	Ticking at the end is a no-op
17	`load(1)`	—	`{cursor:0, length:1, playing:false}`	A one-frame trace
18	`{0, 1, false}`	`toggle`	`{cursor:0, playing:true}`	Index 0 **is** the last index, so this is a replay request
19	`{0, 1, true}`	`tick`	`{cursor:0, playing:false}`	Advances to itself and immediately stops — a single-frame trace cannot animate. Not reachable: every generator emits ≥ 2 frames

#	Starting state	Action	Result	Why
20	`{5, 20, true}`	`load(7)`	`{cursor:0, length:7, playing:false}`	A new input **while playing** stops playback and resets. This is the answer to "what happens if the user changes the input mid-playback?"
21	`{19, 20, false}`	`seek(0)`	`{cursor:0, playing:false}`	Restart
22	`{5, 20, true}`	`seek(5)`	`{cursor:5, playing:false}`	Seeking to the current position still pauses — the most commonly missed consequence of an unconditional `playing: false`

MODEL ANSWER

"WALK ME THROUGH WHAT HAPPENS WHEN I PRESS PLAY ON FRAME 5 OF A 20-FRAME TRACE."

"Pressing Play dispatches `{type:'toggle'}`. The reducer spreads the state, checks whether the cursor is on the last frame — it is 5, not 19, so the cursor is left alone — and flips `playing` to true. That re-renders, the timer effect's dependency array sees `playing` change, so it schedules a `setTimeout` for the current speed. When it fires it dispatches `tick`; the reducer moves the cursor to 6 and sets `playing: state.cursor + 1 < state.length - 1`, which is `6 < 19`, so true. That re-renders, `cursor` changed, the effect re-runs, cleanup clears the already-fired timer, and a new one is scheduled. That chain continues until `cursor` is 18: the tick moves it to 19 and computes `19 < 19`, which is false — so `playing` becomes false in the same transition, the effect's next run returns early without scheduling, and playback stops with the final frame displayed."

DANGER ZONE — WHERE CANDIDATES DIE

THE TWO REAL WEAKNESSES IN THE REDUCER — VOLUNTEER THEM BEFORE YOU ARE ASKED

1. **`seek` does not validate or floor the index.** `seek(2.5)` yields `cursor: 2.5`, and `frames[2.5]` is `undefined`, which would throw at `frame.array.map`. Every current call site guards against this (the range input passes `Number(...)` of an integer step; saved and imported cursors are checked with `Number.isInteger`), so it is not reachable through the UI — but the reducer itself is not robust, and a reducer is exactly where such a guard belongs. The one-line fix: `cursor: Math.max(0, Math.min(Math.trunc(action.cursor) || 0, state.length - 1))`. [PROPOSED]
2. **`tick` sets `playing` rather than preserving it.** If a tick ever arrived while paused, playback would start. Today only the timer effect dispatches `tick`, and it only runs while playing, so it is unreachable — but it makes the reducer's contract harder to reason about. [PROPOSED] Either rename it (`advanceAndContinue`) or compute `playing: state.playing && (state.cursor + 1 < state.length - 1)`.

CHAPTER 23

23 · Twenty-eight reducer questions, with answers

Read the question, answer out loud, then check. Anything you can answer cold here, you can answer in the interview.

INTERVIEW QUESTION
1. WHY A REDUCER INSTEAD OF THREE `USESTATE` CALLS?
Because cursor, length and playing have cross-field rules — clamping, auto-stop at the end, pause-on-seek. A reducer states those rules once, in a pure function, and makes them testable without React. [Ch. 15]

INTERVIEW QUESTION
2. WHAT ARE THE FOUR ACTIONS?
`load`, `toggle`, `tick`, `seek`. Anything else hits `default` and returns the same state object.

INTERVIEW QUESTION
3. WHAT DOES `LOAD` CARRY?
Only `length`. The reducer never sees the frames themselves.

INTERVIEW QUESTION
4. WHY DOES `LOAD` RETURN A FRESH OBJECT INSTEAD OF SPREADING STATE?
To guarantee no stale cursor survives a trace change. Spreading could carry a cursor beyond the new length.

INTERVIEW QUESTION
5. WHAT DOES `TICK` DO AT THE LAST FRAME?
Clamps the cursor at `length - 1` and sets `playing: false`. It is idempotent at the boundary; the cursor never runs off the end.

INTERVIEW QUESTION
6. WHY DOES `TICK` COMPUTE `PLAYING` FROM THE OLD CURSOR?
Because the expression "the frame I am moving to is not the last one" is `cursor + 1 < length - 1` evaluated before the assignment. If you used the new cursor you would write `cursor < length - 1` and get the same result — but the comment-free version makes the intent clearer as written.

INTERVIEW QUESTION
7. WHY DOES `SEEK` SET `PLAYING: FALSE`?
Because dragging a timeline while the animation runs underneath is disorienting, and it would immediately move the cursor away from where the user just put it.

INTERVIEW QUESTION
8. WHAT HAPPENS IF YOU SEEK BEYOND THE END?
The cursor clamps to `length - 1` and playback stops. The test asserts this: `playbackReducer(s, {type:"seek", cursor:999}).cursor === 2` for a 3-frame trace.

INTERVIEW QUESTION
9. WHAT HAPPENS IF YOU SEEK BEFORE THE START?
Clamps to 0. Also asserted in the test file.

INTERVIEW QUESTION
10. WHERE IS THE REDUCER IMPORTED?

Exactly twice: `SortingLab.jsx` (line 20) and `tests/engine.test.js` (line 6).

INTERVIEW QUESTION
11. IS THE REDUCER PURE?

Yes — same inputs always produce the same output, and it never mutates `state` or `action`. Every branch returns a new object except `default`, which returns the `same` object (also pure, and better for React).

INTERVIEW QUESTION
12. WHAT DOES AN UNKNOWN ACTION DO?

Returns the identical state object. React compares with `Object.is` and skips re-rendering.

INTERVIEW QUESTION
13. WHAT IS THE INITIAL STATE?

`{cursor: 0, length: frames.length, playing: false}` — computed in the component from the first render's trace.

INTERVIEW QUESTION
14. WHAT HAPPENS WHEN THE USER CHANGES THE ALGORITHM WHILE PLAYING?

`load(values, algorithm)` dispatches `load`, which stops playback and resets the cursor to 0 for the new trace. [Simulation 20]

INTERVIEW QUESTION
15. WHAT HAPPENS WHEN THE USER EDITS THE TEXT INPUT WHILE PLAYING?

Nothing to playback. `text` is a draft; `input` only changes when Apply succeeds. The animation keeps running against the committed dataset. [Ch. 10]

INTERVIEW QUESTION
16. CAN THE CURSOR EVER EXCEED `LENGTH - 1`?

Through the reducer, no. Through the component, there is a brief window after an input change where the reducer's `length` is still the old value — which is why the component re-derives `cursor = Math.min(playback.cursor, frames.length - 1)`.

INTERVIEW QUESTION
17. WHY DOES THE REDUCER NOT NEED THE FRAME ARRAY?

Because every decision is about indices, not content. That ignorance is what makes it reusable.

INTERVIEW QUESTION
18. WHAT IS THE WEAKEST LINE IN THE REDUCER?

`cursor: Math.max(0, Math.min(action.cursor, state.length - 1))` — it clamps the range but not the domain. Non-integers and `NaN` pass through. Fix with `Math.trunc` and a numeric guard.

INTERVIEW QUESTION
19. HOW WOULD YOU ADD A `STEP` ACTION?

`case "step": return {...state, cursor: Math.max(0, Math.min(state.cursor + action.delta, state.length - 1)), playing: false};` This would let the keyboard effect use an empty dependency array. [PROPOSED]

INTERVIEW QUESTION
20. HOW WOULD YOU TEST THE REDUCER ON ITS OWN?

Import the pure function and drive it with a sequence of actions, asserting the resulting state — exactly what the test file does in the "playback keeps final state, pauses on seek and bounds its cursor" group.

INTERVIEW QUESTION
21. WHY IS `LENGTH` IN THE STATE AT ALL, RATHER THAN PASSED TO EACH ACTION?

Because `tick`, `toggle` and `seek` all need it, and the reducer cannot look it up. It is the minimal context needed for the rules.

INTERVIEW QUESTION
22. WHAT IF `LENGTH` IS STALE?

The derived `cursor` clamp in the component saves the render, and the next `load` repairs the state. It cannot corrupt the trace, because the trace is not in the reducer.

INTERVIEW QUESTION
23. COULD YOU REPLACE THE REDUCER WITH A `USEREF` AND IMPERATIVE CODE?

Technically yes — but then playback would not be a render input, the UI would not update, and the rules would live in event handlers instead of a tested function. That is a regression, not a simplification.

INTERVIEW QUESTION
24. DOES THE REDUCER HANDLE THE CASE WHERE FRAMES ARE REGENERATED IDENTICALLY (SAME INPUT, SAME ALGORITHM)?

Yes — but `load` is still dispatched because `load()` is called from `apply`/`load`, which is what resets the cursor. If the user re-applies the same value, the trace is identical and the cursor resets to 0. There is no memoisation of "same input → don't reset". [VERIFIED]

INTERVIEW QUESTION
25. WHAT IS THE REDUCER'S SPACE COMPLEXITY?

O(1) — a new three-field object per action. The trace lives elsewhere.

INTERVIEW QUESTION
26. IS THE REDUCER'S `LENGTH` THE NUMBER OF FRAMES OR THE NUMBER OF FRAMES MINUS ONE?

The number of frames. The last valid index is `length - 1`, which is why every clamp uses `state.length - 1`.

INTERVIEW QUESTION
27. WHY NOT STORE `CURSOR` AS A PERCENTAGE?

Because percentage is a presentation concern — it is computed on the fly for the comparison preview (`compareCursor`) and for the meters. Storing an index keeps the primary path exact and avoids rounding drift between cursor and frames.

INTERVIEW QUESTION
28. WHAT SINGLE PROPERTY MAKES THE WHOLE DESIGN WORK?

That the trace is immutable and complete before playback starts. Every reducer rule is then a pure index manipulation, and every render is a pure function of `cursor`.

CHECKLIST

RED-LINE DRILLS FOR PART VII

- ■ Write the reducer from memory, including the `default` branch.
- ■ State the three invariants.
- ■ Simulate: `length = 20, cursor = 5` → `toggle` → `tick` ×3 → `seek(0)` → `tick`.
- ■ Explain why `seek` pauses and why that is a deliberate choice.
- ■ Volunteer the two weaknesses and their fixes.
- ■ Explain what happens on `load` while playing.

PART VIII

Play, pause and the timer system

One timer, one owner, one cleanup — and the five bugs that pattern prevents.

CHAPTER 24

24 · The complete Play sequence, in nine steps

Anything that happens when you click Play, in the exact order React runs it.

24.1 The code involved

src/lab/SortingLab.jsx · lines 106–110 (verbatim)

```
  useEffect(() => {
    if (!playback.playing) return;
    const id = setTimeout(() => dispatch({ type: "tick" }), speed);
    return () => clearTimeout(id);
  }, [playback.playing, playback.cursor, speed]);
```

src/lab/SortingLab.jsx · lines 386–400 (verbatim)

```
                  <button
                    className="av-button primary"
                    onClick={() => dispatch({ type: "toggle" })}
                  >
                    {playback.playing ? (
                      <Pause size={17} />
                    ) : (
                      <Play size={17} />
                    )}{" "}
                    {playback.playing
                      ? "Pause"
                      : cursor === frames.length - 1
                        ? "Replay"
                        : "Play"}
                  </button>
```

24.2 The nine steps

Step	What happens	React/JS mechanism
1	The user clicks the Play button	A click handler runs inside a React event
2	`onClick={() => dispatch({type: "toggle"})}` is called	`dispatch` is passed the action object; React does not run the reducer yet
3	React schedules a re-render and calls `playbackReducer(prev, action)` during it	`toggle` returns a new state with `playing: true` (and the cursor unchanged, because the cursor is not on the last frame)
4	The component re-renders with `playback.playing === true`	The button's label becomes "Pause" and its icon switches to the Pause glyph; the status tag becomes "Running"
5	After the commit, React runs effects whose dependencies changed	The timer effect's dependency array `[playing, cursor, speed]` changed (`playing` went `false → true`)
6	The effect body runs: `if (!playback.playing) return;` does not fire	The guard exists for the pause case
7	`const id = setTimeout(() => dispatch({type: "tick"}), speed)` schedules exactly one callback	The browser holds the timer; `id` is captured in this effect run's closure
8	The effect returns `() => clearTimeout(id)`	React stores this cleanup against that effect run
9	After `speed` milliseconds, the callback fires and dispatches `tick`	The reducer advances the cursor, sets `playing` to `cursor + 1 < length - 1`, React re-renders, and the effect re-runs because `cursor` changed. Its cleanup clears the already-fired timer (a harmless no-op), and step 7 repeats.

MUST KNOW
THE ONE-SENTENCE SUMMARY

Play sets a boolean; a single effect owns the timer; each tick re-enters the effect; the cleanup guarantees at most one pending timer at any instant.

24.3 The pause sequence, and why it is clean

Step	What happens
1	The user clicks Pause (the same button)
2	`dispatch({type:'toggle'})` → the reducer flips `playing` to `false`, leaving the cursor untouched
3	React re-renders; the button label returns to "Play" (or "Replay" if the cursor is on the last frame)
4	The effect's dependency array changed, so React runs the previous run's cleanup **before** the new body
5	The cleanup calls `clearTimeout(id)`, cancelling the pending tick
6	The new body runs, hits `if (!playback.playing) return;`, and schedules nothing

Pause is therefore not "asking the timer to stop" — it is "removing the only thing that was going to schedule another one", plus cancelling the one already in flight. If the cleanup were missing, pause would stop scheduling new ticks but the pending one would still fire, producing one extra frame after the user pressed pause — the "pause stutters by one frame" bug.

TRAP
"DOES THE TIMER KEEP RUNNING WHILE PAUSED?"

No, and the correct nuance is: "There is no interval to keep running. A single timeout was pending, and the cleanup cancelled it. While paused, the effect body returns early and schedules nothing — so there are zero timers alive." The wrong answer is "the timer keeps running but does nothing".

CHAPTER 25

25 · The timer system: ownership, cleanup, and duplicates

Everything the interviewer wants to know about timers, asked and answered.

25.1 The ownership model

Question	Answer
Who owns the timer?	Exactly one `useEffect` in `SortingLab.jsx` (lines 106–110). No other code in the lab schedules playback
How many can exist at once?	At most one. Each effect run clears its predecessor's timer before scheduling its own
What re-schedules it?	A change to `playing`, `cursor`, or `speed`
What cancels it?	The cleanup function, which React calls before each re-run and on unmount
Who dispatches `tick`?	Only the timer callback
What if the component unmounts mid-playback?	React runs the cleanup, `clearTimeout` cancels the pending tick, and no state update occurs after unmount

25.2 The five timer bugs this pattern prevents

Bug	What it looks like	The mechanism that prevents it
Orphaned timer	The animation "runs" after you leave the page; React warns about a state update on an unmounted component	Cleanup on unmount
Duplicate timers	Each frame advances two or three steps at once — the classic "playback runs at double speed" bug	Cleanup runs before the next scheduling
Stale-delay timer	Changing speed has no effect until the next frame, or forever	`speed` is in the dependency array, so the effect re-runs and re-schedules with the new delay
Runaway chain	The cursor overshoots the last frame and `frames[cursor]` is `undefined`	`tick` clamps with `Math.min`, and completion sets `playing: false`
Ghost pause	One more frame advances after pressing pause	The sweep that clears the pending timeout

25.3 Why `setTimeout` and not `setInterval`

MODEL ANSWER
"WHY NOT SETINTERVAL?"

"Because the interval would be a second source of truth for 'where are we'. Four reasons in order of importance.

First, **state control**: with `setInterval` the timer keeps firing whether or not the cursor is at the end — I would still need to clear it from inside the reducer's effect, so I have not removed any complexity, I have added a second thing that must be kept in sync.

Second, **speed changes**: `setInterval` captures its delay at creation. Changing the speed would require clearing and recreating the interval anyway — which is exactly what the `setTimeout` chain does naturally on every re-render.

Third, **pause**: pausing would need an explicit `clearInterval`, and pausing mid-frame would be ambiguous. With the chain, pause simply stops the next link from being scheduled.

Fourth, and this is the one people miss: **an interval creates catch-up behaviour.** If the main thread is blocked for 500 ms, a 160 ms interval will queue several callbacks and then fire them back to back, and the animation jumps or stutters to catch up. A self-scheduling `setTimeout` chain always measures from when the previous frame actually rendered — so a slow frame simply delays the next one, which is what you want for a visualisation. There's also `requestAnimationFrame` as an alternative, but that is for smooth 60 fps motion, not for stepping through discrete recorded states half a second apart."

25.4 Stale closures — the concept, and where it would bite here

MUST KNOW
WHAT A STALE CLOSURE IS

A function remembers the variables from the scope in which it was **created**. If a React render creates a handler that reads `cursor`, and that handler is stored somewhere permanent (a window listener, a timer, a ref), it will keep reading the old `cursor` forever — because it closed over that render's value.

[VERIFIED] There are exactly two places in `SortingLab.jsx` where a closure captures state:

1. **The keyboard handler** reads `cursor` and calls `dispatch({type:"seek", cursor: cursor + 1})`. The effect's dependency array is `[cursor]`, so React removes the old listener and attaches a new one whenever the cursor changes — the handler is never stale. **The cost**: a remove/add pair on most frame advances.
2. **The timer callback** reads nothing — it dispatches a constant action. It is immune to staleness precisely because it captures no state. (This is a nice pattern to name: keep the callback dumb, let the reducer read the state.)

TRAP
"WHY DOES THE KEYBOARD EFFECT DEPEND ON `CURSOR`? WHY NOT JUST USE `[]`?"

Because the handler body reads `cursor`. With `[]`, the listener would be attached once with the cursor value from the first render, and after the first arrow-key press every subsequent press would dispatch a seek to the same index — the arrows would look broken. So the dependency is not decoration; it is what makes the closure current. The better design, which I would do next, is to add a `step` action with a `delta` so the handler never needs to read `cursor` at all, and then the array can be empty. [PROPOSED]

25.5 Race conditions and unmount

Scenario	What happens	Is it a problem?
User changes speed mid-playback	The effect re-runs with the new delay; the pending tick is cancelled and rescheduled	No — the next frame simply waits the new duration

Scenario	What happens	Is it a problem?
User drags the timeline mid-playback	`seek` pauses, so `playing` becomes false and the effect stops scheduling	No
User presses Apply with new input mid-playback	`load` resets `playing` to false; the trace changes; the derived cursor clamp keeps the render valid	No. **This is the important one** — the frame array and the cursor update in the same commit
Component unmounts mid-playback (route change)	Cleanup cancels the pending timeout	No
The timer fires in the same tick as a user's pause click	Either order is safe: if the tick lands first, the cursor advances and then pause stops it; if pause lands first, the cleanup cancels the timer and the tick never runs. There is no way to end up with `playing: true` and no timer	No
Two rapid Play clicks	`toggle` twice returns `playing` to `false`; the second render's effect cleanup cancels the timer	No — the button is idempotent-ish by symmetry

DANGER ZONE — WHERE CANDIDATES DIE

THE ONE GENUINE RISK, AND HOW TO TALK ABOUT IT

JavaScript's event loop is single-threaded, so there is no true parallelism here — but a `setTimeout` callback can become due at almost exactly the moment a click handler runs. The saving grace is that **both paths go through the reducer**, and the reducer is a pure function of the current state. Whichever order the two dispatches are processed in, each one sees the state left by the previous one. There is no shared mutable variable that both paths write to, which is precisely why reducers are used for this kind of coordination. [GENERAL KNOWLEDGE applied to VERIFIED code]

CHAPTER 26

26 · Speed control, and what it does not change

A one-selector feature with a disproportionately good interview answer.

26.1 The control

src/lab/SortingLab.jsx · lines 410–421 (verbatim)

```
                <label className="av-speed">
                  Speed
                  <select
                    value={speed}
                    onChange={(e) => setSpeed(Number(e.target.value))}
                  >
                    <option value={600}>0.5×</option>
                    <option value={300}>1×</option>
                    <option value={160}>2×</option>
                    <option value={50}>6×</option>
                  </select>
                </label>
```

Option	Value	Label
1	600 ms	0.5×
2	300 ms	1×
3	160 ms	2× (the default)
4	50 ms	6×

[VERIFIED] The `value` is a **delay in milliseconds**, and `setSpeed(Number(e.target.value))` converts the DOM string back to a number — the select's value is always a string, which is a real source of bugs in React apps and worth mentioning.

MUST KNOW

WHAT SPEED DOES AND DOES NOT CHANGE

It changes	It does not change
The delay between scheduled `tick` dispatches	The number of frames in the trace
How long the user waits between visible states	The number of comparisons, passes or exchanges
Perceived smoothness	The algorithm's time or space complexity
Nothing else	The trace itself — frames are computed once, before playback

INTERVIEW QUESTION

"DOES SPEED AFFECT ALGORITHM COMPLEXITY?"

This is a comprehension check disguised as a trick question. The answer: *"No — and the cleanest way to say why is that the algorithm has already finished by the time speed matters. Speed is a `setTimeout` delay in the playback effect. The trace was computed in one synchronous burst when the input was committed. Changing the delay changes how fast I walk through a recording, in the same way that rewinding a video faster doesn't make the actors perform better. The UI says exactly that: 'No wall-clock claims: animation speed is not algorithm performance.' Also — the complexity labels in the side panel are static metadata declared in `engine.js`, not measured from the run."*

The last sentence is the part that makes it airtight: complexity in AlgoViz is **declared, not measured**. [VERIFIED: `time: "O(n²)"`, `space: "O(1)"` etc. are literal strings in the algorithm registry.]

TRAP

"SO YOUR APP HAS NO WAY TO TELL ME WHICH ALGORITHM IS ACTUALLY FASTER?"

Correct, and you should say so without hedging: "It cannot, and it does not claim to. What it measures is recorded operation counts — comparisons, and separately the generator-specific pass and exchange counters. Those are a proxy for work done, not for wall-clock time, because real runtime depends on cache behaviour, constant factors, the JavaScript engine, and the data distribution. The project is explicit about that: it is listed as 'not a benchmark platform' in the README, and the panel text says it avoids wall-clock claims."

WEAKNESS / LIMITATION

WHAT "0.5×" AND "6×" ACTUALLY MEAN

The labels are relative to the 300 ms option (1×). 0.5× is 600 ms — twice as long per frame — and 6× is 50 ms. But note the ratios are not exact across the set: 600/300 = 2 (0.5× is right), 300/300 = 1 (1× is right), 300/160 = 1.875 (the "2×" label is approximate), and 300/50 = 6 (6× is right). So the labels are presentation, and only three of the four are exact. If an interviewer inspects the numbers, the honest answer is: "The labels are indicative — they're relative to the 300 ms option, and 160 ms is 1.875× rather than exactly 2×. Rounding that to a friendly label was a deliberate UI choice, but if I wanted exactness I'd either use values that divide evenly or display the millisecond delay directly." [VERIFIED arithmetic]

CHAPTER 27

27 · Keyboard control, second by second

A small feature with three real bugs to avoid, all of which are avoided here.

`src/lab/SortingLab.jsx · lines 111–136 (verbatim)`

```
  useEffect(() => {
    const key = (e) => {
      if (
        /INPUT|SELECT|TEXTAREA|BUTTON/.test(e.target.tagName) ||
        e.target.isContentEditable ||
        e.ctrlKey ||
        e.metaKey ||
        e.altKey
      )
        return;
      if (e.code === "Space") {
        e.preventDefault();
        dispatch({ type: "toggle" });
      }
      if (e.key === "ArrowRight") {
        e.preventDefault();
        dispatch({ type: "seek", cursor: cursor + 1 });
      }
      if (e.key === "ArrowLeft") {
        e.preventDefault();
        dispatch({ type: "seek", cursor: cursor - 1 });
      }
    };
    window.addEventListener("keydown", key);
    return () => window.removeEventListener("keydown", key);
  }, [cursor]);
```

Guard	Reason	What breaks without it
`INPUT|SELECT|TEXTAREA|BUTTON` test on `e.target.tagName`	Space must type a space when the user is in the dataset field, and Space must activate a focused button natively	Typing `38, 66, 12` in the input would toggle playback on the first space; pressing Space on a focused button would both click the button and toggle playback
`e.target.isContentEditable`	The same idea for rich-text-ish elements	Rare here, but it is the complete version of the check
`e.ctrlKey || e.metaKey || e.altKey`	Never hijack browser or OS shortcuts (Ctrl+R, Cmd+←, Alt+Tab)	`Ctrl+ArrowLeft` would step the animation and do whatever the browser does; `Cmd+←` would do both
`e.preventDefault()` inside each handled branch	Stop the page from scrolling on arrow keys, and stop Space from scrolling	The whole page jumps while stepping through frames
`window.addEventListener` in an effect with **return cleanup**	Register once per dependency change, remove exactly one	Every cursor change would add another listener; a single ArrowRight press would advance by an increasing number of frames

MUST KNOW

THE MEMORABLE NAME FOR THIS BUG

Listener accumulation. It is the same failure mode as the duplicate-timer bug, arriving through a different door, and it makes a perfect pairing in an answer: "Both of my effects return cleanup functions for the same reason — the timer effect clears its timeout, and the keyboard effect removes its listener. If either cleanup were missing, the bug would be the same shape: the old thing keeps running alongside the new one."

INTERVIEW QUESTION

"WHICH KEY IS CHECKED WITH `E.CODE` AND WHICH WITH `E.KEY` — AND IS THAT DELIBERATE?"

`Space` uses `e.code` (the physical key) while the arrows use `e.key` (the produced value). [VERIFIED] `code` is layout-independent, so it is the safer choice for a physical key like the space bar; `key` is more readable for named keys like `"ArrowRight"`. Mixing them is slightly inconsistent, and if I were reviewing my own code I would pick one convention — but both calls are correct, and on every standard layout `e.key === " "` matches the same physical key.

CHECKLIST

RED-LINE DRILLS FOR PART VIII

- ■ Recite the nine steps of Play without notes.
- ■ Explain exactly what pauses: the boolean, the cleanup, and the guard.
- ■ Give four reasons `setTimeout` beats `setInterval` here.
- ■ Name the five timer bugs the pattern prevents.
- ■ Explain the stale-closure risk in the keyboard effect and why `[cursor]` fixes it.
- ■ State precisely what speed does **not** change.

PART IX

Step, seek and the timeline

Why `frames[cursor]` is the most important expression in the project, and how to debug navigation.

CHAPTER 28

28 · Timeline navigation

Every way the user can move through a trace, and the exact code path each one takes.

28.1 The five navigation affordances

src/lab/SortingLab.jsx · lines 355–368 (verbatim)

```
            <div className="av-transport">
              <div className="av-timeline">
                <input
                  aria-label="Execution timeline"
                  type="range"
                  min="0"
                  max={frames.length - 1}
                  value={cursor}
                  onChange={(e) => seek(Number(e.target.value))}
                />
                <span>
                  {cursor} / {frames.length - 1}
                </span>
              </div>
```

src/lab/SortingLab.jsx · lines 369–409 (verbatim)

```
              <div className="av-transport-row">
                <div>
                  <button
                    className="av-icon-button"
                    onClick={() => seek(0)}
                    aria-label="Restart"
                  >
                    <SkipBack size={17} />
                  </button>
                  <button
                    className="av-icon-button"
                    disabled={!cursor}
                    onClick={() => seek(cursor - 1)}
                    aria-label="Step backward"
                  >
                    <ChevronLeft size={19} />
                  </button>
                  <button
                    className="av-button primary"
                    onClick={() => dispatch({ type: "toggle" })}
                  >
                    {playback.playing ? (
                      <Pause size={17} />
                    ) : (
                      <Play size={17} />
                    )}{" "}
                    {playback.playing
                      ? "Pause"
                      : cursor === frames.length - 1
                        ? "Replay"
                        : "Play"}
                  </button>
                  <button
                    className="av-icon-button"
                    disabled={cursor === frames.length - 1}
                    onClick={() => seek(cursor + 1)}
                    aria-label="Step forward"
                  >
                    <ChevronRight size={19} />
                  </button>
                </div>
```

Control	Code	Dispatches	Effect on state
Restart (`SkipBack`)	`onClick={() => seek(0)}`	`seek 0`	Cursor to the first frame, playback stops

Control	Code	Dispatches	Effect on state
Step backward (`ChevronLeft`)	`onClick={() => seek(cursor - 1)}, disabled={!cursor}`	`seek cursor-1`	One frame back, playback stops
Play / Pause / Replay	`dispatch({type:"toggle"})`	`toggle`	Flips playing, resets to 0 if on the last frame
Step forward (`ChevronRight`)	`onClick={() => seek(cursor + 1)}, disabled={cursor === frames.length - 1}`	`seek cursor+1`	One frame forward, playback stops
Scrubber	`onChange={e => seek(Number(e.target.value))}`	`seek value`	Jumps to the dragged frame, playback stops

Note that **four of the five controls are just different arguments to the same action.** There is no `next`, no `previous`, no `restart` action — the reducer has four actions in total and navigation is entirely expressed through `seek`.

28.2 The two disabled states, and why they are exactly right

```
disabled={!cursor}                      ->  disabled only at cursor === 0
disabled={cursor === frames.length - 1}  ->  disabled only at the last frame
```

MUST KNOW
THE `!CURSOR` IDIOM

[VERIFIED] `!cursor` is true when `cursor` is `0` — and also when it is `NaN`, `null` or `undefined`. For this variable, after the derived `Math.min` clamp, the reachable values are non-negative integers, so `!cursor` means exactly "we are at the first frame". It is a compact idiom that works only because of that invariant. The forward button uses the explicit comparison because there is no falsy value that means "at the end" — 19 is truthy.

That asymmetry is worth pointing out before an interviewer does: *"The back button uses `!cursor` and the forward button uses `cursor === frames.length - 1`. That's inconsistent-looking but correct — `!cursor` catches 0, and there's no falsy representation of 'last index'. If I refactored, I'd make both explicit comparisons for symmetry."*

28.3 `frames[cursor]` — the single most important expression

```
const cursor = Math.min(playback.cursor, frames.length - 1),
      frame  = frames[cursor];
```

KEY TAKEAWAY
WHY THIS IS THE PROJECT IN ONE LINE

Everything the user sees on the stage — the bars, the heights, the labels, the active ring, the sorted tint, the pivot marker, the explanation sentence, the operation name, the three counters, the index list — comes from the single object `frames[cursor]`. Nothing is recomputed, nothing is animated, nothing is stored in parallel state. Rendering the whole screen is a **pure function of one integer**.

If you can say that sentence and then prove it by pointing at `ArrayView` and the inspector, you have demonstrated that you understand your own architecture. [VERIFIED: every one of those values reads from `frame` or `stats = frame.currentStats || {}`.]

Read in the UI	Source	Line
Bar heights	`Math.abs(item.val) / max` where `max` is the largest absolute value **in this frame**	ArrayView
Active ring	`frame.activeIndices?.includes(i)`	ArrayView
Sorted tint	`frame.sortedIndices?.includes(i)`	ArrayView
Pivot marker	`frame.pivotIndex === i`	ArrayView

Read in the UI	Source	Line
Step badge	`String(cursor).padStart(3, "0")`	explanation panel
Operation label	`frame.codeLine?.replaceAll("_", " ")`	explanation panel
Explanation text	`frame.msg`	explanation panel
Counters	`stats.comps`, `stats.passes`, `stats.swaps`	state inspector
Index lists	`frame.activeIndices`, `frame.pivotIndex`, `frame.sortedIndices`	state inspector
Timeline position	`value={cursor}`, label `{cursor} / {frames.length - 1}`	transport
Status tag	`cursor === frames.length - 1` ? "Completed" : playing ? "Running" : `cursor` ? "Paused" : "Ready"	panel heading

> **NOTE**
> **THE BAR-HEIGHT DETAIL THAT SURPRISES PEOPLE**
> The height of every bar is computed **relative to the largest absolute value in the current frame** — `Math.max(1, ...frame.array.map(x => Math.abs(x.val)))`. Since the array contents are the same multiset in every frame (sorting only reorders), this maximum is constant across the whole trace for a given input. That is why the bars do not rescale as the animation plays: the visual scale is stable, and only the order changes. The `Math.max(1, …)` guard prevents a division by zero when every value is 0. [VERIFIED]

28.4 Completion, and why the final state stays visible

Question	Answer
What happens at the last frame?	`tick` clamps the cursor and sets `playing: false`; the status tag reads "Completed"; the Play button's label becomes "Replay"
Does the view reset to frame 0?	**No.** The cursor stays at the final frame and the sorted array remains on screen. The README calls this "final-state retention — study the completed result without it disappearing"
How do you watch it again?	Press the same button, now labelled "Replay": `toggle` sees `cursor === length - 1` and resets to 0 while starting playback
Why is that design better?	Most visualisers snap back to the initial state, so the one moment the learner wants to study — the finished result — is the one moment it is not on screen

> **INTERVIEW QUESTION**
> **"WHAT HAPPENS IF YOU PRESS PLAY WHILE THE TRACE IS COMPLETE?"**
> "It's the same `toggle` action. The reducer checks `state.cursor === state.length - 1`; that's true, so the cursor resets to 0 and `playing` flips to true. The render then shows frame 0 with the button labelled Pause. That one line in the reducer is what turns the Play button into a Replay button without a second action type or any conditional logic in the component."

CHAPTER 29

29 · Debugging navigation

Eight hand-built scenarios with symptom, cause, fix and regression test.

29.1 The debugging method for anything playback-related

```
1. Is the reducer wrong, or is the component wrong?
   -> import playbackReducer in a scratch Node file and replay the action sequence.
      If the state sequence is correct, the bug is in the React layer.

2. Is the cursor out of range, or is the frame wrong?
   -> log cursor, frames.length and frame.codeLine. Any cursor >= frames.length
      means a stale length; a correct cursor with a wrong frame means a stale trace.

3. Is the trace itself wrong?
   -> assert frames.at(-1).array.map(x => x.val) equals the sorted input.
      If that fails, the bug is in the generator, not in playback.

4. Is it a timing bug or a state bug?
   -> pause, step manually. If stepping is correct and only playing is wrong,
      the bug is in the timer effect, not the reducer.
```

MUST KNOW

THE 4-STEP DECISION TREE ABOVE IS THE ANSWER TO "HOW WOULD YOU DEBUG PLAYBACK?"

Say the tree, not a story. It is short, ordered from the pure layer to the impure one, and it tells the interviewer you know where the boundaries are. Every scenario below is an instance of it.

29.2 Scenario A — the cursor goes past the end of the trace

Symptom	After changing the input, the stage goes blank or the console shows `Cannot read properties of undefined (reading 'array')` at `ArrayView`
Likely causes	(1) The component is not clamping the cursor; (2) `load` was never dispatched, so the reducer still holds the old `length`; (3) a new input produced a shorter trace while the cursor was near the end
Debugging steps	Log `playback.cursor`, `playback.length`, `frames.length`, and the derived `cursor`. If `frames.length < playback.length`, the reducer's length is stale, which means a code path changed `input` without dispatching `load`
Fix	Dispatch `load` in every path that changes `input` — or better, make the trace the source of truth and derive `length` in one effect: `useEffect(() => dispatch({type:"load", length: frames.length}), [frames])`
Regression test	Drive the reducer directly: `load(20)` → `seek(19)` → `load(3)` → assert `cursor === 0`. Then a component-level check: change the input to a much shorter one while the cursor is at the end, and assert the render still shows a valid frame

29.3 Scenario B — playback advances two frames per step

Symptom	The animation runs at double speed; the frame counter jumps 0, 2, 4, 6
Likely causes	(1) The timer effect lost its cleanup, so two timers are alive; (2) an extra `tick` is dispatched somewhere; (3) StrictMode's double-invocation is exposing a missing cleanup
Debugging steps	Add a counter/log inside the effect body and its cleanup: a healthy run logs one cleanup per body. Also check for a second `useEffect` dispatching `tick`, and check the dependency array for a value that changes twice per frame

Fix	Restore `return () => clearTimeout(id)`; ensure exactly one effect dispatches `tick`
Regression test	Not directly testable in Node — but you can assert the invariant that makes it impossible: the effect's dependency array must contain everything the body reads, and the body must return a cleanup whenever it schedules

29.4 Scenario C — pause appears to stutter (one extra frame)

Symptom	Pressing Pause at frame 40 shows frame 41
Likely causes	The pending timeout was not cleared, so a tick that was already due still landed. This is the symptom of a missing cleanup, not of a wrong reducer
Debugging steps	Log inside the cleanup; compare the timestamp of the pause click with the tick's arrival
Fix	Ensure the cleanup runs on **every** dependency change, not only on unmount. A common mistake is `useEffect(() => { ... }, [])` with a manual clear inside an event handler
Regression test	Node-level: the reducer itself cannot produce this (pause is a pure state change). Component-level: step manual + pause sequencing, asserting the cursor does not move after the pause click

29.5 Scenario D — arrow keys stop working after the first press

Symptom	The first ArrowRight steps forward; every later press appears to do nothing (or seeks to the same index)
Likely cause	The keyboard effect's dependency array was changed to `[]`, so the handler closed over `cursor = 0` forever and kept dispatching `seek(1)`
Debugging steps	Log the action objects: you will see the same `cursor` value dispatched repeatedly. That confirms a stale closure rather than a broken reducer
Fix	Either restore `[cursor]` in the dependency array, or (cleaner, [PROPOSED]) add a relative `step` action so the handler no longer reads `cursor` at all
Regression test	Assert that one listener is attached: wrap `window.addEventListener`/`removeEventListener` in a spy and check the balance after ten cursor changes

29.6 Scenario E — the timeline slider jumps back while dragging

Symptom	Dragging to frame 80 then continuing to drag causes the slider to snap backwards
Likely causes	(1) `seek` clamps to a stale `length`; (2) the input is uncontrolled and React is resetting it; (3) frames are being regenerated mid-drag
Debugging steps	Log `e.target.value`, the dispatched cursor, and the resulting `playback.cursor`. If the dispatched value is right but the state clamps, the reducer's `length` is stale
Fix	This is the "`load` not dispatched" family again; the fix is to keep `length` synchronised with the trace. Note the range input also reflects `value={cursor}` — the derived cursor — so if the derived clamp fires, the thumb will visibly fight the drag
Regression test	`seek(999)` on a 3-frame trace must yield `cursor === 2` — which is exactly what the existing test asserts

29.7 Scenario F — the last frame never renders

Symptom	The animation stops at frame `n-2`; the last frame is only visible after pressing step-forward once
Likely causes	(1) A `tick` that stops one frame early (`cursor + 1 < length` instead of `< length - 1`); (2) a `<` that should be `<=`; (3) the timeline's `max` set to `frames.length - 2`
Debugging steps	Drive the reducer to completion in Node and assert the final cursor equals `length - 1`. If that passes, inspect the render path for an off-by-one in the timeline `max` or the disabled condition
Fix	The correct completion expression is `playing: state.cursor + 1 < state.length - 1`, which reaches the last frame and stops there. The existing test asserts exactly this: after two ticks on a 3-frame trace, `cursor === 2` and `playing === false`
Regression test	The existing playback test group. Add a case for `length = 2`

29.8 Scenario G — the frame shown does not match the step badge

Symptom	The badge says 012 but the explanation text is from the previous operation
Likely causes	The badge uses the derived `cursor` and the text uses `frame` — so this symptom means two different traces are in play, i.e. `frames` was recomputed while `cursor` was stale
Debugging steps	Log `frames.length`, `playback.cursor` and `cursor` in the same render. If `playback.cursor > frames.length - 1`, the clamp is doing its job and the mismatch is cosmetic; a content mismatch (right index, wrong message) means the memo returned a new array built from a different input
Fix	Keep `(algo, input)` as the memo dependencies and never derive frames from `text`. If a future feature needs a live preview while typing, that preview must be a separate value, not the same `frames`
Regression test	Assert `frames[cursor].codeLine` is defined for every cursor in `[0, frames.length-1]` for a set of inputs

29.9 Scenario H — the timeline shows `0 / 0`

Symptom	The transport reads `0 / 0` and the stage is empty
Likely cause	`frames.length === 1` (or 0), so `frames.length - 1` is 0. This means a generator produced a single frame
Debugging steps	Call the generator directly in Node for that input and print `frames.length` and `frames.map(f => f.codeLine)`. All six generators always push a seed frame and a terminal frame, so a length below 2 indicates a throwing or short-circuited generator
Fix	Depends on which generator misbehaved; the general guard is that `parseInput` guarantees at least two values, and every generator handles `n = 0` and `n = 1` (measured: bubble/selection/insertion return 2 frames for `[]`, merge/quick/heap return 4)
Regression test	The existing "correctness, input preservation and reproducible trace" test group already includes `[]` and `[1]` as inputs for all six algorithms

DANGER ZONE — WHERE CANDIDATES DIE

THE META-LESSON, AND THE SENTENCE TO SAY

Four of the eight scenarios above reduce to the same root cause: **the reducer's `length` and the memoised `frames.length` are two separate variables that must be kept in step.** That is a genuine design smell — not a bug, but a coupling that has to be maintained by hand. The refactor that removes it entirely is to stop storing `length` in the reducer and pass it in on each action, or to derive it. If an interviewer asks "what's the weakest part of your playback design?", this is the best answer in the project, because it is real, it is subtle, and you have a concrete fix. [PROPOSED]

CHECKLIST

RED-LINE DRILLS FOR PART IX

- ■ Name all five navigation affordances and the single action they share.
- ■ Explain the `!cursor` idiom and why the forward button uses a different test.
- ■ List everything on screen that comes from `frames[cursor]`.
- ■ Explain why bar heights do not rescale between frames.
- ■ Recite the four-step debugging decision tree.
- ■ Explain why "length is stored twice" is the weakest part of the playback design.

PART X

The six sorting algorithms

Code, traces, complexity, stability, edge cases and whiteboard implementations for every algorithm in AlgoViz — plus the ones you should not claim.

CHAPTER 30

30 · The algorithm set, and what is not in it

Establish the boundary before the interviewer does.

30.1 What "six algorithms" means, precisely

[VERIFIED] `src/lab/engine.js` registers exactly six algorithms in `algorithms`, and they are the six whose generators live in `src/modules/core/algorithm/sorting/algorithms/`:

Key	Display name	Generator file	Declared time	Declared space
`bubble`	Bubble sort	`bubbleSort.js`	O(n²)	O(1)
`selection`	Selection sort	`selectionSort.js`	O(n²)	O(1)
`insertion`	Insertion sort	`insertionSort.js`	O(n²)	O(1)
`merge`	Merge sort	`mergeSort.js`	O(n log n)	O(n)
`quick`	Quick sort	`quickSort.js`	O(n²) worst	O(n) worst
`heap`	Heap sort	`heapSort.js`	O(n log n)	O(log n)*

The `time` and `space` strings are **declared metadata rendered in the UI** — not measured. [VERIFIED]

DANGER ZONE — WHERE CANDIDATES DIE

FOUR ALGORITHMS THAT EXIST IN THE REPOSITORY BUT ARE NOT IN THE "SIX SORTING ALGORITHMS"

1. **Binary search, N-Queens, pathfinding (BFS/DFS), tree traversals, two pointers, fast/slow pointers, prefix sum, merge intervals** — these are the twelve module routes, not the sorting set. Each has its own logic module (some confusingly located under `algorithm/sorting/`).
2. **The sliding-window implementation** is in the sorting algorithms folder but is not registered in `algorithms` — it is used by the Sliding Window module and by the counterexample lab.
3. There is **no** counting sort, radix sort, bucket sort, shell sort, timsort, or introspective sort. [VERIFIED by exhaustive file listing]
4. There is **no** Dijkstra (the README names "weighted Dijkstra" explicitly as not a current feature) and **no** Floyd cycle detection — the Fast & Slow Pointers module demonstrates Move Zeroes.

If an interviewer asks "which sorts did you implement?", answer with the six and add the boundary in the same breath: "Six sorts in the lab. The other twelve modules are different algorithm families — those exist, but they don't use the frame recorder, so I'd keep the two stories separate."

30.2 The colour code for study priority

Colour	Algorithms	What you must be able to do
RED — code from scratch under pressure	Bubble, Insertion, Selection, Merge, Quick	Write the algorithm, the generator version, and state complexity — without notes
ORANGE — code with minor help	Heap	Understand sift-down/heapify and the two-phase structure; code it if given the array-index formulas

Colour	Algorithms	What you must be able to do
GREEN — recognition	The twelve non-sorting modules	Describe the idea of each and point to its file

MUST KNOW

THE HONEST ANSWER TO "WHICH ONE SHOULD I BE ABLE TO CODE FROM SCRATCH?"

All five RED ones, because they are the standard interview set and AlgoViz gives you a story for each. Heap sort is worth being able to code too — it is the one that separates candidates — and the only reason it is ORANGE here is that its generator subtleties (two comparisons per heapify call, extraction bookkeeping) are harder to reproduce perfectly under pressure than the algorithm itself.

CHAPTER 31

31 · Bubble sort

The algorithm you must be able to explain not as "the slow one" but as a specific sequence of compare-and-swap decisions.

31.1 Problem and intuition

Problem. Given an array of numbers, produce a permutation in non-decreasing order.

Intuition. Walk left to right comparing neighbours. If a pair is out of order, swap them. After one full walk, the largest value is guaranteed to be at the end — because it will have won every comparison it entered and been carried rightward, one swap at a time. That single fact gives the algorithm its name and its invariant.

31.2 The invariant

[VERIFIED from `engine.js` metadata] **"After each complete pass, the rightmost processed elements are in their final positions."**

This is the sentence displayed in the lab's "What stays true?" panel, and it is the correct invariant: after pass `i`, the last `i+1` positions hold the `i+1` largest values in sorted order, and they will never be touched again. The inner loop bound `j < n - i - 1` is the direct expression of the invariant. [VERIFIED in the code.]

31.3 The actual implementation

`src/modules/core/algorithm/sorting/algorithms/bubbleSort.js · lines 1–45 (verbatim)`

```
export const generateBubbleSort = (rawArray) => {
    let arr = rawArray.map((val, index) => ({
        id: `block-${index}`,
        val: val
    }));

    const frames = [];
    const n = arr.length;
    const sortedIndices = [];
    let stats = { passes: 0, comps: 0, swaps: 0 };

    const cloneArray = () => arr.map(item => ({ ...item }));

    frames.push({
        array: cloneArray(), activeIndices: [], sortedIndices: [...sortedIndices],
        isSwapping: false, minimumIndex: null, codeLine: 'start',
        currentStats: { ...stats },
        msg: "System Ready. Initializing Bubble Sort."
    });

    for (let i = 0; i < n; i++) {
        stats.passes++;

        frames.push({
            array: cloneArray(), activeIndices: [], sortedIndices: [...sortedIndices],
            isSwapping: false, minimumIndex: null, codeLine: 'outer',
            currentStats: { ...stats },
            msg: `Pass ${stats.passes} of ${n}.`
        });

        for (let j = 0; j < n - i - 1; j++) {
            frames.push({
                array: cloneArray(), activeIndices: [j, j + 1], sortedIndices: [...sortedIndices],
                isSwapping: false, minimumIndex: null, codeLine: 'inner',
                currentStats: { ...stats },
                msg: `Scanning window at index ${j} and ${j + 1}.`
            });

            stats.comps++;
            frames.push({
                array: cloneArray(), activeIndices: [j, j + 1], sortedIndices: [...sortedIndices],
                isSwapping: false, minimumIndex: null, codeLine: 'compare',
                currentStats: { ...stats },
                msg: `Comparing ${arr[j].val} and ${arr[j + 1].val}. Is ${arr[j].val} > ${arr[j + 1].val}?`
            });
```

31.4 The real trace: `[3, 1, 2]` → 19 frames

This is the actual output of the generator, not a reconstruction:

#	codeLine	array	active	sorted	comps	passes	swaps	msg
0	start	[3,1,2]	[]	[]	0	0	0	System Ready. Initializing Bubble Sort.
1	outer	[3,1,2]	[]	[]	0	1	0	Pass 1 of 3.
2	inner	[3,1,2]	[0,1]	[]	0	1	0	Scanning window at index 0 and 1.
3	compare	[3,1,2]	[0,1]	[]	1	1	0	Comparing 3 and 1. Is 3 > 1?
4	swap	[3,1,2]	[0,1]	[]	1	1	1	Yes! 3 > 1. Swapping blocks...
5	swap	[1,3,2]	[0,1]	[]	1	1	1	Swap complete.
6	inner	[1,3,2]	[1,2]	[]	1	1	1	Scanning window at index 1 and 2.
7	compare	[1,3,2]	[1,2]	[]	2	1	1	Comparing 3 and 2. Is 3 > 2?
8	swap	[1,3,2]	[1,2]	[]	2	1	2	Yes! 3 > 2. Swapping blocks...
9	swap	[1,2,3]	[1,2]	[]	2	1	2	Swap complete.
10	outer	[1,2,3]	[]	[2]	2	1	2	Pass complete. Block locked into its final sorted position.
11	outer	[1,2,3]	[]	[2]	2	2	2	Pass 2 of 3.

#	codeLine	array	active	sorted	comps	passes	swaps	msg
12	inner	[1,2,3]	[0,1]	[2]	2	2	2	Scanning window at index 0 and 1.
13	compare	[1,2,3]	[0,1]	[2]	3	2	2	Comparing 1 and 2. Is 1 > 2?
14	compare	[1,2,3]	[0,1]	[2]	3	2	2	No swap needed. Moving window forward.
15	outer	[1,2,3]	[]	[2,1]	3	2	2	Pass complete. Block locked into its final sorted position.
16	outer	[1,2,3]	[]	[2,1]	3	3	2	Pass 3 of 3.
17	outer	[1,2,3]	[]	[2,1,0]	3	3	2	Pass complete. Block locked into its final sorted position.
18	start	[1,2,3]	[]	[0,1,2]	3	3	2	Bubble Sort Complete! Array is fully sorted.

MUST KNOW

THREE THINGS THIS TRACE PROVES, THAT YOU SHOULD POINT OUT

1. **Frame 18 shows the entire stats as `c3 p3 s2`** — 3 comparisons, 3 passes, 2 swaps for a 3-element array. The theoretical minimum comparisons for the optimal bubble variant is n−1 = 2; this implementation performs 3 because it does not early-exit.
2. **`sortedIndices` at frame 17 is `[2, 1, 0]` — descending. The sorted set is filled from the right and is** not ordered**, which matters because the UI only ever asks `.includes(i)`, never iterates it for display order. [VERIFIED]
3. **Frame 16 and 17 are the "wasted" third pass.** With early-exit, the pass at frame 16 would be skipped entirely. Its presence is visible evidence that this implementation runs exactly n passes.

31.5 Complexity

Case	Comparisons	Swaps	Time	Why
Best	n(n−1)/2 = 276 for n = 24	0	**Θ(n²)** in this implementation	No early-exit flag, so a sorted array still performs every pass and every comparison
Average	≈ n²/2	≈ n²/4	Θ(n²)	Each pair is out of order with probability ½
Worst (reverse)	n(n−1)/2	n(n−1)/2	Θ(n²)	Every comparison swaps — measured: 276 comparisons, 276 swaps for n = 24

TRAP

"ISN'T BUBBLE SORT'S BEST CASE O(N)?"

Not in this implementation. The textbook O(n) best case requires an early-exit flag:

```
// [PROPOSED] the optimisation this project does not implement
let swapped = false;
for (let j = 0; j < n - i - 1; j++) {
  if (arr[j].val > arr[j+1].val) { swap; swapped = true; }
}
if (!swapped) break;   // <-- absent from bubbleSort.js
```

[VERIFIED] `bubbleSort.js` has no `swapped` flag and no `break`, so it always performs all n passes. Measured proof: a fully sorted 24-element array produces **878 frames and 276 comparisons** — the same 276 comparisons as the reverse-sorted case. If you claim "bubble sort is O(n) on sorted input", an interviewer who has read the file can falsify it by running the app. The correct phrasing: "Bubble sort can be Θ(n) best-case with an early-exit flag; this implementation deliberately omits it, so it is Θ(n²) on every input. That's visible in the comparison counter — a sorted input still shows 276 comparisons, and the trace still shows every pass."

31.6 Stability, in-place, edge cases

Property	Answer
Stable?	**Yes.** The comparison is `arr[j].val > arr[j+1].val` — strict. Equal elements never swap, so their relative order is preserved
In-place?	Yes for the array itself. The visualisation is not in-place, which is a different question
Adaptive?	Only in the sense that it performs fewer swaps on a sorted input; the comparisons are fixed
n = 0	2 frames (`start` + `Complete!`), loop bodies never execute. [VERIFIED]
n = 1	2 frames. [VERIFIED]
All equal	No swaps at all; frames are still produced for every comparison. This is the case where the `duplicates` preset is useful
Negative values	Fine — comparisons use `val`, and the UI uses `Math.abs(val)` only for bar height
Large values	Bounded by `parseInput` to ±999

31.7 Bugs to watch for

Bug	How it would appear
Off-by-one in the inner bound (`j < n - i` instead of `j < n - i - 1`)	Compares past the sorted region, causing a frame where an active index sits inside `sortedIndices` — visible as a bar that is both green and teal
Using `>=` instead of `>`	Destroys stability: equal elements swap, and the ID order in the final frame changes. The reproducibility test would still pass (it only compares two runs of the same implementation), which is why stability needs a separate assertion
Forgetting `sortedIndices.push(n - i - 1)`	The sorted region never grows; the "sorted" tint would never appear, and the final frame would still be correct — a silent visual bug
Swapping but not pushing the post-swap frame	The trace would show the array before each swap and jump at the next frame; the learner would never see the result
Mutating `rawArray` instead of cloning	The caller's array changes; the test `assert.deepEqual(input, before)` would fail

31.8 Interview questions

INTERVIEW QUESTION
"WHAT DOES BUBBLE SORT'S INNER LOOP BOUND `N - I - 1` MEAN?"

"After pass `i`, the last `i+1` elements are in their final positions, so the inner loop only needs to scan the unsorted prefix. The `- 1` is there because the loop body compares `arr[j]` with `arr[j+1]`; if `j` could reach `n - i - 1`, the last comparison would read past the unsorted region — and for the final pass with `i = n - 1` on a 2-element remainder, `j+1` would address an element that is already locked."

INTERVIEW QUESTION
"WHY DOES YOUR BUBBLE SORT PUSH A FRAME EVEN WHEN NO SWAP HAPPENS?"

"Because a negative decision is information. If I only recorded swaps, a viewer could not tell a comparison that returned false from a comparison that never happened. The `No swap needed. Moving window forward.` frame is what makes the trace a record of decisions rather than a record of mutations."

INTERVIEW QUESTION

"BUBBLE SORT IS O(N²). WHY INCLUDE THE WORST ALGORITHM IN A PRODUCT?"

"For exactly the reason it's the worst algorithm: it is the clearest one to read. It is the algorithm people learn first, and having it in the same lab as merge sort is what makes the comparison feature meaningful — comparing merge against quick tells you something subtle, but comparing bubble against merge against the same array tells you something you can feel: 276 comparisons versus 16 on the same eight numbers. The lab is not trying to be fast; it is trying to make the difference visible."

31.9 Whiteboard

MUST KNOW

BUBBLE SORT — THE VERSION TO WRITE, AND THE VERSION ALGOVIZ WOULD NEED

```
// Classic, with early exit — what an interviewer expects first
function bubbleSort(a) {
  const arr = [...a];                     // never mutate the caller's array
  const n = arr.length;
  for (let i = 0; i < n - 1; i++) {
    let swapped = false;
    for (let j = 0; j < n - i - 1; j++) {
      if (arr[j] > arr[j + 1]) {
        [arr[j], arr[j + 1]] = [arr[j + 1], arr[j]];
        swapped = true;
      }
    }
    if (!swapped) break;                  // best case θ(n)
  }
  return arr;
}
```

Then say the sentence that shows you understand both worlds: "For the visualiser I'd write it differently — the early exit makes the animation shorter and hides the fact that the algorithm would otherwise keep scanning, so I kept the n-pass version and I'd probably show the early-exit variant as a second option rather than replacing this one." [PROPOSED for the second option]

CHAPTER 32

32 · Selection sort

The algorithm that makes "find the minimum" visible — and whose trace has the cleanest bookkeeping of the six.

32.1 Problem and intuition

Problem. Same as before.

Intuition. Divide the array into a sorted prefix (empty at first) and an unsorted suffix. Repeatedly scan the entire unsorted suffix to find its smallest element, then swap it into the first unsorted position. The prefix grows by one; the suffix shrinks by one. Crucially, the scan does not move anything — it only remembers an index.

32.2 The invariant

[VERIFIED] **"The sorted prefix contains the smallest elements in increasing order."**

By construction, the prefix holds the k smallest values of the whole array, in order, so it can never be disturbed by a later step.

32.3 The actual implementation

src/modules/core/algorithm/sorting/algorithms/selectionSort.js · lines 16–57 (verbatim)

```
    for(let i=0;i<n-1;i++){
        let minIndex=i;
        stats.passes++;
        frames.push({
            array: cloneArray(), activeIndices: [i], sortedIndices: [...sortedIndices],
            isSwapping: false, minimumIndex: minIndex, codeLine: 'outer',
            currentStats: { ...stats }, msg: `Pass ${stats.passes}: Assuming index ${i} (value ${arr[i].val}) is the minimum.`
        });
        for(let j=i+1;j<n;j++){
            stats.comps++;
            frames.push({
                array: cloneArray(), activeIndices: [j], sortedIndices: [...sortedIndices],
                isSwapping: false, minimumIndex: minIndex, codeLine: 'inner',
                currentStats: { ...stats }, msg: `Scanning index ${j} (value ${arr[j].val}). Current min is ${arr[minIndex].val}.`
            });
            if(arr[minIndex].val>arr[j].val){
                minIndex = j;
                frames.push({
                    array: cloneArray(), activeIndices: [j], sortedIndices: [...sortedIndices],
                    isSwapping: false, minimumIndex: minIndex, codeLine: 'new_min',
                    currentStats: { ...stats }, msg: `Found new minimum: ${arr[minIndex].val} at index ${minIndex}.`
                });
            }
        }
        if (minIndex !== i) {
            stats.swaps++;
            frames.push({
                array: cloneArray(), activeIndices: [i, minIndex], sortedIndices: [...sortedIndices],
                isSwapping: true, minimumIndex: minIndex, codeLine: 'swap',
                currentStats: { ...stats }, msg: `Pass complete. Swapping current index ${i} with minimum element ${arr[minIndex].val}.`
            });

            let temp = arr[i];
            arr[i] = arr[minIndex];
            arr[minIndex] = temp;

            frames.push({
                array: cloneArray(), activeIndices: [i, minIndex], sortedIndices: [...sortedIndices],
                isSwapping: false, minimumIndex: i, codeLine: 'swap',
                currentStats: { ...stats }, msg: `Swap complete.`
            });
        }
```

32.4 The real trace: `[3, 1, 2]` → 15 frames

#	codeLine	array	active	sorted	minIndex	comps	passes	swaps
0	start	[3,1,2]	[]	[]	–	0	0	0
1	outer	[3,1,2]	[0]	[]	0	0	1	0
2	inner	[3,1,2]	[1]	[]	0	1	1	0
3	new_min	[3,1,2]	[1]	[]	**1**	1	1	0
4	inner	[3,1,2]	[2]	[]	1	2	1	0
5	swap	[3,1,2]	[0,1]	[]	1	2	1	1
6	swap	[1,3,2]	[0,1]	[]	0	2	1	1
7	lock	[1,3,2]	[]	[0]	–	2	1	1
8	outer	[1,3,2]	[1]	[0]	1	2	2	1
9	inner	[1,3,2]	[2]	[0]	1	3	2	1
10	new_min	[1,3,2]	[2]	[0]	**2**	3	2	1
11	swap	[1,3,2]	[1,2]	[0]	2	3	2	2
12	swap	[1,2,3]	[1,2]	[0]	1	3	2	2
13	lock	[1,2,3]	[]	[0,1]	–	3	2	2
14	start	[1,2,3]	[]	[0,1,2]	–	3	2	2

Comparisons: 3 for n = 3. That is exactly n(n−1)/2 = 3 — selection sort's comparison count is **input-independent**. Verify against the measured data: for the default 8-element input it reports 28 = 8×7/2, and for the seeded 10-element input it reports 45 = 10×9/2. [VERIFIED] That is a beautiful fact to state: "Selection sort's comparison count is a function of n alone — it is the same for a sorted array as for a reversed one. Only the swap count is data-dependent."

32.5 Complexity

Case	Comparisons	Swaps	Time
Best	n(n−1)/2	**0**	Θ(n²)
Average	n(n−1)/2	≤ n−1	Θ(n²)
Worst	n(n−1)/2	n−1	Θ(n²)

MUST KNOW

THE ONE GENUINELY INTERESTING PROPERTY OF SELECTION SORT

It performs at most n − 1 swaps, the minimum possible for a comparison sort that only swaps. That is why it was historically attractive when writes were expensive. Bubble sort on the same reverse-sorted input performs 276 swaps where selection performs 23 (for n = 24). Measured from the traces: selection on reverse-24 reports 492 frames with 23 swaps; bubble reports 1,154 frames with 276 swaps. [VERIFIED]

32.6 Stability, in-place, edge cases

Property	Answer
Stable?	**No.** The swap `arr[i] ↔ arr[minIndex]` can jump an element over an equal one. Classic example: `[3a, 3b, 1]` — the first pass swaps index 0 with index 2, putting `3a` after `3b`. In this codebase you can see it: sort the `duplicates` preset and watch the `block-*` ids in the final frame — they will not be in ascending order for equal values
In-place?	Yes
n = 0 / 1	2 frames each [VERIFIED]

Property	Answer
All equal	`minIndex` never changes, so `minIndex === i` always and **zero swaps** occur. The trace is a pure scan
The trailing `sortedIndices.push(n - 1)`	[VERIFIED line 66] After the loop ends at `i = n - 2`, the final index is never visited by the outer loop, so it is marked sorted explicitly. Without it the sorted region would be missing one index in every final frame

INTERVIEW QUESTION
"WHY DOES YOUR SELECTION SORT LOOP TO N−1 AND THEN MARK THE LAST INDEX SEPARATELY?"

"Because the last element is sorted by definition — after n−1 passes, exactly one element remains unsorted, and it is necessarily the largest of the remaining, so it is already in place. Marking it explicitly keeps the last frame's `sortedIndices` complete and ordered, which the final-frame assertions depend on."

32.7 Bugs to watch for, and interview questions

Bug	Symptom
Comparing against `i` instead of `minIndex` (`arr[i].val > arr[j].val`)	The scan would find the minimum of the pair rather than the running minimum — the array would be partially sorted and the `new_min` frames would be meaningless
Forgetting `stats.passes++`	The "Passes / calls" tile stays at 0 while the trace visibly has passes
Swapping unconditionally	Extra frames and an inflated swap count; also destroys the "0 swaps on equal input" property
Not marking the last index	The final frame shows `n-1` sorted indices instead of `n`

INTERVIEW QUESTION
"IS SELECTION SORT EVER THE RIGHT CHOICE IN PRODUCTION?"

Almost never for arrays. Its modern relevance is niche: it needs only O(1) extra space, it does the minimum number of writes, and it is trivially interruptible (the sorted prefix is valid at every point). Where writes are the dominant cost — for example writing to flash memory — the write-minimal property can matter. But it is not adaptive, so it cannot exploit partial sortedness.

32.8 Whiteboard

```
function selectionSort(a) {
  const arr = [...a], n = arr.length;
  for (let i = 0; i < n - 1; i++) {
    let minIdx = i;
    for (let j = i + 1; j < n; j++) {
      if (arr[j] < arr[minIdx]) minIdx = j;
    }
    if (minIdx !== i) [arr[i], arr[minIdx]] = [arr[minIdx], arr[i]];
  }
  return arr;
}
```

MUST KNOW
THE THREE DETAILS THAT MAKE THIS ANSWER STRONG

The `minIdx !== i` guard (avoids a self-swap, which is why equal inputs report zero swaps), the `<` comparison (which would preserve stability if the swap were not used — say this precisely and you look sharp), and the fact that the inner loop starts at `i + 1` rather than 0.

CHAPTER 33

33 · Insertion sort

The only generator with an early exit, an explicit `break` frame, and a "lifted" element flag.

33.1 Problem and intuition

Intuition. Keep a sorted prefix. Take the next element, remember its value, and walk backwards through the prefix shifting larger elements one slot to the right until you find the element's correct home; drop it there. It is exactly how most people sort a hand of playing cards.

33.2 The invariant

[VERIFIED] **"The prefix before the current insertion is sorted."**

33.3 The actual implementation

src/modules/core/algorithm/sorting/algorithms/insertionSort.js · lines 21–70 (verbatim)

```
    for (let i = 1; i < n; i++) {
        stats.passes++;
        arr[i].isLifting = true;

        frames.push({
            array: cloneArray(), activeIndices: [i], sortedIndices: [...sortedIndices],
            isSwapping: false, minimumIndex: null, codeLine: 'outer',
            currentStats: { ...stats }, msg: `Pass ${stats.passes}: Lifting index ${i} (value ${arr[i].val}).`
        });

        let j = i;

        while (j > 0) {
            stats.comps++;
            frames.push({
                array: cloneArray(), activeIndices: [j, j - 1], sortedIndices: [...sortedIndices],
                isSwapping: false, minimumIndex: null, codeLine: 'compare',
                currentStats: { ...stats }, msg: `Comparing lifted value ${arr[j].val} with ${arr[j - 1].val}.`
            });

            if (arr[j].val < arr[j - 1].val) {
                stats.swaps++;
                frames.push({
                    array: cloneArray(), activeIndices: [j, j - 1], sortedIndices: [...sortedIndices],
                    isSwapping: true, minimumIndex: null, codeLine: 'swap',
                    currentStats: { ...stats }, msg: `${arr[j].val} < ${arr[j - 1].val}. Shifting block right.`
                });

                let temp = arr[j];
                arr[j] = arr[j - 1];
                arr[j - 1] = temp;

                frames.push({
                    array: cloneArray(), activeIndices: [j, j - 1], sortedIndices: [...sortedIndices],
                    isSwapping: false, minimumIndex: null, codeLine: 'swap',
                    currentStats: { ...stats }, msg: `Shift complete.`
                });

                j--;
            } else {
                frames.push({
                    array: cloneArray(), activeIndices: [j, j - 1], sortedIndices: [...sortedIndices],
                    isSwapping: false, minimumIndex: null, codeLine: 'break',
                    currentStats: { ...stats }, msg: `${arr[j].val} >= ${arr[j - 1].val}. Correct position found.`
                });
                break;
            }
        }

        arr[j].isLifting = false;
```

33.4 The real trace: `[3, 1, 2]` → 14 frames, including the `break`

#	codeLine	array	active	sorted	comps	passes	swaps
0	start	[3,1,2]	[]	**[0]**	0	0	0
1	outer	[3,1,2]	[1]	[0]	0	1	0
2	compare	[3,1,2]	[1,0]	[0]	1	1	0
3	swap	[3,1,2]	[1,0]	[0]	1	1	1
4	swap	[1,3,2]	[1,0]	[0]	1	1	1
5	drop	[1,3,2]	[]	[0,1]	1	1	1
6	outer	[1,3,2]	[2]	[0,1]	1	2	1
7	compare	[1,3,2]	[2,1]	[0,1]	2	2	1
8	swap	[1,3,2]	[2,1]	[0,1]	2	2	2
9	swap	[1,2,3]	[2,1]	[0,1]	2	2	2
10	compare	[1,2,3]	[1,0]	[0,1]	**3**	2	2
11	**break**	[1,2,3]	[1,0]	[0,1]	3	2	2
12	drop	[1,2,3]	[]	[0,1,2]	3	2	2
13	start	[1,2,3]	[]	[0,1,2]	3	2	2

MUST KNOW

FRAME 0 IS DIFFERENT FROM EVERY OTHER GENERATOR'S FRAME 0

Look at the `sorted` column: **`[0]`**. Insertion sort's seed frame already declares index 0 as sorted, because a single element is trivially sorted. Compare the seed frames: bubble and selection and merge and quick and heap all start with `[]`; insertion starts with `[0]`. [VERIFIED — this is initialised as `const sortedIndices = [0]` at line 10.]

That is a superb detail to volunteer, because it shows the sorting regions are algorithm-specific, not a generic "sorted so far" counter. It also means an interviewer checking "does every algorithm's final frame show all n indices sorted?" gets the same answer for all six — but the path there differs.

TRAP

THE INSERTION SORT TRACE CONTAINS A FRAME THAT WILL CONFUSE YOU IF YOU DO NOT READ IT CAREFULLY

At frame 10 the array is already `[1,2,3]` — fully sorted — and the algorithm **still performs one more comparison** before breaking. That comparison (comps goes 2 → 3) is the check that stops the shift loop: `2 >= 1`. That single extra comparison is why this implementation reports 3 comparisons instead of 2 for `[3,1,2]`.

It is also why the algorithm's "best case is Θ(n)" claim is precise rather than free: on a sorted input, insertion sort performs exactly n − 1 comparisons — one per element to discover it is already in place — and then breaks. [VERIFIED: the `break` frame is pushed with `codeLine: 'break'` and the message `"2 >= 1. Correct position found."`]

And note the direction of the active indices on a shift: `[2,1]` then `[1,0]`. `activeIndices` is `[j, j-1]`, so as `j` decrements the highlight walks leftwards through the prefix. That is the visual signature of insertion sort.

33.5 Complexity

Case	Comparisons	Shifts (the generator's "swaps")	Time
Best (sorted)	n − 1	0	Θ(n)
Average	≈ n²/4	≈ n²/4	Θ(n²)
Worst (reverse)	n(n−1)/2 = 276 for n = 24	n(n−1)/2 = 276	Θ(n²)

[VERIFIED against the measurements: insertion on reverse-24 produces 876 frames with 276 comparisons and 276 shifts; on the sorted 24-element input it performs far fewer.]

33.6 Stability, in-place, edge cases

Property	Answer
Stable?	**Yes**, and this implementation's strict `<` is what makes it so: an element stops shifting the moment it meets an equal one (`arr[j].val < arr[j-1].val` is false), so it is inserted after its equal, preserving the original order
In-place?	Yes
Adaptive?	Yes — the only one of the three O(n²) sorts in this project that is genuinely adaptive
n = 0	2 frames; the loop never runs, so the seed frame's `sortedIndices = [0]` refers to a non-existent index. [VERIFIED — that is a harmless but real artefact: for `[]`, the `start` frame claims index 0 is sorted]
All equal	Zero shifts; one comparison per pass then break. Fastest possible path

DANGER ZONE — WHERE CANDIDATES DIE

THE EMPTY-INPUT ARTEFACT, AND HOW TO ANSWER IT

[VERIFIED] For an empty array, insertion sort's seed frame carries `sortedIndices: [0]` even though index 0 does not exist. The final frame overwrites it with `[...Array(0).keys()] = []`, so the terminal state is correct — but the initial frame is internally inconsistent.

In practice `parseInput` requires at least 2 values, so this is unreachable through the UI and only appears in the test suite (which does sort `[]` for all six algorithms). The honest answer: *"The tests sort an empty array, and insertion sort's seed frame marks index 0 as sorted. That is harmless because 'sorted' is only ever queried with `.includes(i)` for real indices, and the final frame is correct — but it's an inconsistency, and the cleanest fix is to initialise `sortedIndices` to `n > 0 ? [0] : []`."* [PROPOSED]

33.7 Interview questions

INTERVIEW QUESTION

"WHY IS INSERTION SORT THE ONE SORTING ALGORITHM YOU WOULD ACTUALLY USE ON SMALL INPUTS?"

"Because it is adaptive and has tiny constant factors. On a nearly-sorted array it does almost no work, and below roughly 16 elements it beats the O(n log n) sorts in practice because it has no recursion overhead and no allocation. That is exactly why real library sorts — Timsort in Java and Python, and introsort implementations — switch to insertion sort below a size threshold. It is also online: it can sort a stream as elements arrive, one at a time, which is a property none of the other five have."

INTERVIEW QUESTION

"YOUR 'SHIFT' IS IMPLEMENTED AS A SWAP. IS THAT THE SAME THING?"

"No — and it is a real inefficiency in the generator. A true insertion sort shifts each larger element one slot right and writes the held value once, which is one write per shift plus one final write. This implementation swaps, which is two writes per shift. The number of comparisons is identical, which is why the comparison counter is unaffected, but the exchange counter over-reports relative to a shift-based insertion sort. That is a good example of why I tell people not to compare the swap counter across algorithms — and why the comparison panel only uses `comps`."

33.8 Whiteboard

```
function insertionSort(a) {
  const arr = [...a];
  for (let i = 1; i < arr.length; i++) {
    const key = arr[i];
    let j = i - 1;
    while (j >= 0 && arr[j] > key) {   // strict > preserves stability
      arr[j + 1] = arr[j];             // shift, not swap
      j--;
    }
    arr[j + 1] = key;                  // drop into place
  }
  return arr;
}
```

MUST KNOW

THE TWO SENTENCES THAT MAKE THIS A SENIOR ANSWER

"Shifting rather than swapping halves the writes, and the strict `>` is what keeps it stable — with `>=` an equal element would be moved behind the insertion point." Then, unprompted: "In my generator I used swaps, so my exchange counter is not the same measure as a shift-based implementation's — which is why the comparison panel deliberately only uses comparisons." That is the difference between knowing the algorithm and knowing your own code.

CHAPTER 34

34 · Merge sort

The only recursive divide-and-conquer sort with an explicit buffer, and the source of the one genuine bug in the frame model.

34.1 Problem and intuition

Intuition. Sorting a whole array is hard; merging two already sorted arrays is easy — repeatedly take the smaller front element. So split the array in half, sort each half (recursively), then merge. The recursion bottoms out at single elements, which are sorted by definition.

34.2 The invariant

[VERIFIED] **"Each merge consumes two sorted ranges and produces a sorted range."**

34.3 The actual implementation

src/modules/core/algorithm/sorting/algorithms/mergeSort.js · lines 21–67 (verbatim)

```
    function mergeSortHelper(low, high) {
        frames.push({
            array: cloneArray(), activeIndices: [], sortedIndices: [...sortedIndices],
            isSwapping: false, pivotIndex: null, subArrayRange: [low, high], codeLine: 'start',
            currentStats: { ...stats }, msg: `Dividing sub-array [${low} to ${high}].`
        });

        if (low >= high) {
            frames.push({
                array: cloneArray(), activeIndices: [], sortedIndices: [...sortedIndices],
                isSwapping: false, pivotIndex: null, subArrayRange: [low, high], codeLine: 'base_case',
                currentStats: { ...stats }, msg: `Base case reached (Single element).`
            });
            return;
        }

        const mid = Math.floor((low + high) / 2);
        stats.passes++;

        frames.push({
            array: cloneArray(), activeIndices: [mid], sortedIndices: [...sortedIndices],
            isSwapping: false, pivotIndex: null, subArrayRange: [low, high], codeLine: 'mid',
            currentStats: { ...stats }, msg: `Calculated midpoint at index ${mid}.`
        });

        frames.push({
            array: cloneArray(), activeIndices: [], sortedIndices: [...sortedIndices],
            isSwapping: false, pivotIndex: null, subArrayRange: [low, mid], codeLine: 'recurse_left',
            currentStats: { ...stats }, msg: `Recursively calling left half [${low} to ${mid}].`
        });
        mergeSortHelper(low, mid);

        frames.push({
            array: cloneArray(), activeIndices: [], sortedIndices: [...sortedIndices],
            isSwapping: false, pivotIndex: null, subArrayRange: [mid + 1, high], codeLine: 'recurse_right',
            currentStats: { ...stats }, msg: `Recursively calling right half [${mid + 1} to ${high}].`
        });
        mergeSortHelper(mid + 1, high);

        frames.push({
            array: cloneArray(), activeIndices: [], sortedIndices: [...sortedIndices],
            isSwapping: false, pivotIndex: null, subArrayRange: [low, high], codeLine: 'merge_call',
            currentStats: { ...stats }, msg: `Merging halves [${low} to ${mid}] and [${mid + 1} to ${high}].`
        });

        merge(low, mid, high);
    }
```

src/modules/core/algorithm/sorting/algorithms/mergeSort.js · lines 69–122 (verbatim)

```
    function merge(low, mid, high) {
        for (let i = low; i <= high; i++) {
            arr[i].yOffset = 80;
        }

        frames.push({
            array: cloneArray(), activeIndices: [], sortedIndices: [...sortedIndices],
            isSwapping: false, pivotIndex: null, subArrayRange: [low, high], codeLine: 'buffer',
            currentStats: { ...stats }, msg: `Pulling elements into temporary O(N) memory buffer.`
        });

        let tempArr = [];
        for(let i = low; i <= high; i++) tempArr.push({...arr[i]});

        let leftIdx = 0;
        let rightIdx = mid - low + 1;
        let k = low;

        while (leftIdx <= mid - low && rightIdx <= high - low) {
            stats.comps++;
            const actualLeftIdx = low + leftIdx;
            const actualRightIdx = low + rightIdx;

            frames.push({
                array: cloneArray(), activeIndices: [actualLeftIdx, actualRightIdx], sortedIndices: [...sortedIndices],
                isSwapping: false, pivotIndex: null, subArrayRange: [low, high], codeLine: 'compare',
                currentStats: { ...stats }, msg: `Comparing ${tempArr[leftIdx].val} and ${tempArr[rightIdx].val}.`
            });

            if (tempArr[leftIdx].val <= tempArr[rightIdx].val) {
                stats.swaps++;
                arr[k] = tempArr[leftIdx];
                arr[k].yOffset = 0;

                frames.push({
                    array: cloneArray(), activeIndices: [k], sortedIndices: [...sortedIndices],
                    isSwapping: true, pivotIndex: null, subArrayRange: [low, high], codeLine: 'write_left',
                    currentStats: { ...stats }, msg: `${arr[k].val} is smaller. Writing to main array.`
                });
                leftIdx++;
            } else {
                stats.swaps++;
                arr[k] = tempArr[rightIdx];
                arr[k].yOffset = 0;

                frames.push({
                    array: cloneArray(), activeIndices: [k], sortedIndices: [...sortedIndices],
                    isSwapping: true, pivotIndex: null, subArrayRange: [low, high], codeLine: 'write_right',
                    currentStats: { ...stats }, msg: `${arr[k].val} is smaller. Writing to main array.`
                });
                rightIdx++;
            }
            k++;
        }
```

34.4 The real trace: `[3, 1, 2]` → 30 frames

#	codeLine	array	active	sorted	range	comps	swaps
0	start	[3,1,2]	[]	[]	[]	0	0
1	start	[3,1,2]	[]	[]	[0,2]	0	0
2	mid	[3,1,2]	[1]	[]	[0,2]	0	0
3	recurse_left	[3,1,2]	[]	[]	[0,1]	0	0
4	start	[3,1,2]	[]	[]	[0,1]	0	0
5	mid	[3,1,2]	[0]	[]	[0,1]	0	0
6	recurse_left	[3,1,2]	[]	[]	[0,0]	0	0
7	start	[3,1,2]	[]	[]	[0,0]	0	0
8	base_case	[3,1,2]	[]	[]	[0,0]	0	0
9	recurse_right	[3,1,2]	[]	[]	[1,1]	0	0
10	start	[3,1,2]	[]	[]	[1,1]	0	0
11	base_case	[3,1,2]	[]	[]	[1,1]	0	0
12	merge_call	[3,1,2]	[]	[]	[0,1]	0	0

#	codeLine	array	active	sorted	range	comps	swaps
13	buffer	[3,1,2]	[]	[]	[0,1]	0	0
14	compare	[3,1,2]	[0,1]	[]	[0,1]	1	0
15	write_right	**[1,1,2]**	[0]	[]	[0,1]	1	1
16	write_rem	[1,1,2]	[]	[]	[0,1]	1	1
17	write_rem	[1,3,2]	[1]	[]	[0,1]	1	2
18–20	recurse_right → base_case	[1,3,2]	[]	[]	[2,2]	1	2
21	merge_call	[1,3,2]	[]	[]	[0,2]	1	2
22	buffer	[1,3,2]	[]	[]	[0,2]	1	2
23	compare	[1,3,2]	[0,2]	[]	[0,2]	2	2
24	write_left	[1,3,2]	[0]	[]	[0,2]	2	3
25	compare	[1,3,2]	[1,2]	[]	[0,2]	3	3
26	write_right	**[1,2,2]**	[1]	[]	[0,2]	3	4
27	write_rem	[1,2,2]	[]	[]	[0,2]	3	4
28	write_rem	[1,2,3]	[2]	[]	[0,2]	3	5
29	start	[1,2,3]	[]	**[0,1,2]**	[]	3	5

34.5 Three things this trace exposes

DANGER ZONE — WHERE CANDIDATES DIE

1. FRAMES 15 AND 26 RENDER DUPLICATE ELEMENT IDS — A REAL REACT BUG

Look at frame 15: the array renders as `[1,1,2]`. The original `[3,1,2]` had a `3` at index 0; after the merge writes the buffered `1` into slot 0, the array genuinely holds two copies of the value 1 — and, more importantly, **two elements with the same `id`**, because `tempArr` holds shallow clones that preserve `id`.

I measured this across inputs: **23 of the 107 default-input frames** contain a duplicate `id`, and **141 of the 375 frames** for a reverse-sorted 24-element input. `ArrayView` keys its columns by `item.id`, so these are React duplicate-key frames — a warning in development and unspecified reconciliation in production, confined to the transient write-back frames. The final frame is always unique (that is asserted in the test suite).

The mechanism, stated precisely: the write-back starts at `k = low` and overwrites slots that still hold the original elements, while the source of the written value is a clone sitting in `tempArr`. Until the window is fully rewritten, the same identity exists twice inside `arr`.

The fixes (see the script box): make the render key unique per frame (`item.id + ':' + i`), which is safe because every frame is a full snapshot; or re-id the buffered clone at write time. [PROPOSED — not implemented]

WEAKNESS / LIMITATION

2. THE MERGE VALUE APPEARS TO VANISH, AND `YOFFSET` IS DRAWN BY NOBODY IN THE APP

At frame 15 the `3` is not on screen at all — it is sitting in `tempArr`. The generator knows this: it sets `yOffset = 80` on every element in the current merge window (lines 70–72) and pushes a `buffer` frame with the message "Pulling elements into temporary O(N) memory buffer." But `ArrayView` never reads `yOffset`, so the buffer is invisible and the animation looks like values are being duplicated and deleted rather than moved through a staging area. (The only file in the repository that does read it is `src/modules/core/EfficientSort.jsx`, which has zero importers and is not in the bundle — so the field was designed for a renderer, just not for this one.) Wiring `yOffset` into the bar's `transform: translateY(...)` would make merge sort's animation correct with zero engine changes. [PROPOSED]

MUST KNOW

3. `SORTEDINDICES` STAYS EMPTY FOR THE ENTIRE MERGE SORT TRACE

Every frame from 0 to 28 has `sorted: []`. The `sorted` region appears only in the **final** frame, because the only code that fills it is:

```
if (low === 0 && high === n - 1) {          // the outermost merge only
  for (let i = 0; i < n; i++) sortedIndices.push(i);
}
```

[VERIFIED at lines 159–161] Compare this with the other five: bubble grows its sorted region one index per pass, selection locks one index per pass, insertion adds one per pass, quick locks pivots as partitions complete, heap grows from the right on every extraction. **Merge sort is the only algorithm that shows no partial sortedness on screen.**

Is that a bug? It is defensible — after the first merge of two single elements, positions 0–1 are sorted but not final, because later merges can move things through them. So marking them would be a lie. The honest framing: *"Merge sort's sorted region is only globally final at the end, so the generator marks it all at once. It's technically correct but it means merge sort's animation carries less information than the other five — the sub-array range is the signal, and that's what `subArrayRange` is for."* [INFERENCE on intent; the code behaviour is VERIFIED]

34.6 Complexity

Case	Comparisons	Writes (the generator's "swaps")	Time	Space
Best	≈ (n log n)/2	n log n	Θ(n log n)	**O(n)** for the buffer
Average	Θ(n log n)	n log n	Θ(n log n)	O(n)
Worst	Θ(n log n)	n log n	Θ(n log n)	O(n)

[VERIFIED note on the counter] The generator increments `stats.swaps` on every element written back from the buffer — including the "remaining" elements that need no comparison. For the default 8-element input the trace ends with `swaps: 24` = 8 elements × $\log_2 8$ levels × 1 write each. So merge sort's "swaps" is really a **write count**, and comparing it against bubble sort's swap count is comparing two different units. This is exactly what the UI footnote says: "Legacy generators count exchanges differently; comparison mode deliberately uses comparisons only." Quote that line in the interview — it is your own documentation defending you.

INTERVIEW QUESTION

"MERGE SORT IS O(N LOG N) IN EVERY CASE. WHY DOES IT LOSE TO QUICK SORT IN PRACTICE?"

"Three reasons. Constant factors: merge does more data movement and allocates a buffer, while partition moves data in place. Cache behaviour: merge's access pattern during a full merge pass is less predictable than quicksort's partition walk over a window. And writes: every element gets written back at every level, whereas a partition only writes the elements that actually move. On the other hand, quicksort's average is only n log n — its worst case is $\Theta(n^2)$, which merge sort never has. That's why real implementations mix strategies, and why stability pushes you towards merge: it's the stable Θ(n log n) comparison sort."

34.7 Stability, edge cases, bugs

Property	Answer
Stable?	**Yes** — the comparison is `tempArr[leftIdx].val <= tempArr[rightIdx].val`, so on a tie the left element is taken first, preserving original order
In-place?	No. The generator allocates `tempArr` per merge: O(n) auxiliary. The lab's declared `space: "O(n)"` is accurate
n = 0	4 frames: seed, divide `[0,-1]`, base case, final. [VERIFIED] The `low >= high` guard handles the empty and single-element cases

Property	Answer
n = 1	4 frames, same shape
Recursion depth	$\lceil \log_2 n \rceil + 1$, so ~5 levels for 24 elements. No stack-risk at this input cap
The duplicate-id bug	See 34.5. Symptom: React key warning during merge playback, and transient duplicate bars

INTERVIEW QUESTION

"WALK ME THROUGH THE MERGE FUNCTION'S INDEX ARITHMETIC — WHY `RIGHTIDX = MID - LOW + 1`?"

"`tempArr` is a window-local copy: index 0 of `tempArr` is element `low` of the array. So the left run occupies `tempArr[0 .. mid-low]` and the right run starts immediately after it, at `tempArr[mid-low+1 .. high-low]`. The loop condition `leftIdx <= mid - low && rightIdx <= high - low` is the same two ranges expressed as inclusive bounds. I use the `actualLeftIdx = low + leftIdx` conversion only to build the `activeIndices` for the frame — the comparison itself works on window-local indices."

That answer demonstrates the one thing interviewers are actually testing with merge sort: do you know the difference between window-local and array-global indices.

34.8 Whiteboard

```
function mergeSort(a) {
  if (a.length <= 1) return [...a];
  const mid = a.length >> 1;
  const left = mergeSort(a.slice(0, mid));
  const right = mergeSort(a.slice(mid));
  return merge(left, right);
}

function merge(L, R) {
  const out = [];
  let i = 0, j = 0;
  while (i < L.length && j < R.length) {
    out.push(L[i] <= R[j] ? L[i++] : R[j++]);   // <= keeps it stable
  }
  while (i < L.length) out.push(L[i++]);
  while (j < R.length) out.push(R[j++]);
  return out;
}
```

MUST KNOW

WHY ALGOVIZ'S VERSION LOOKS DIFFERENT

The generator does **not** return new arrays — it sorts in place using a buffer, because a frame must show a single canonical array, not a tree of sub-arrays. If merge sort returned fresh arrays at each level, there would be no single "current array" to render, and the trace would be a tree rather than a sequence. That is a genuine, defensible engineering reason for deviating from the textbook version, and it is worth saying explicitly: "I used the in-place-with-buffer variant because the visualisation needs one array whose state can be shown at each moment. The recursive merge-returns-new-array version is easier to read but has no single current state to display."

CHAPTER 35

35 · Quick sort

Lomuto partition with a last-element pivot — the algorithm whose worst case the user can produce by pressing the "sorted" preset.

35.1 Problem and intuition

Intuition. Pick an element and call it the pivot. Rearrange the array so that everything ≤ the pivot is on its left and everything > is on its right. Once that is done, the pivot is in its **final** position — nothing in the rest of the algorithm can move it. Then recurse on the two sides.

The whole algorithm is: choose, partition, recurse. Everything hard about quick sort is in the partition, and the choice of pivot.

35.2 The invariant

[VERIFIED] **"After partition, the pivot is final; left values are ≤ it and right values are greater."**

35.3 The actual implementation

src/modules/core/algorithm/sorting/algorithms/quickSort.js · lines 27–60 (verbatim)

```
        if (low >= high) {
            frames.push({
                array: cloneArray(), activeIndices: [], sortedIndices: [...sortedIndices],
                isSwapping: false, pivotIndex: null, subArrayRange: [low, high], codeLine: 'base_case',
                currentStats: { ...stats }, msg: `Base case reached.`
            });

            if (low === high) {
                sortedIndices.push(low);
                frames.push({
                    array: cloneArray(), activeIndices: [], sortedIndices: [...sortedIndices],
                    isSwapping: false, pivotIndex: null, subArrayRange: [], codeLine: 'base_case',
                    currentStats: { ...stats }, msg: `Index ${low} is sorted.`
                });
            }
            return;
        }

        stats.passes++;
        frames.push({
            array: cloneArray(), activeIndices: [], sortedIndices: [...sortedIndices],
            isSwapping: false, pivotIndex: null, subArrayRange: [low, high], codeLine: 'partition',
            currentStats: { ...stats }, msg: `Partitioning sub-array [${low} to ${high}].`
        });

        const pi = partition(low, high);

        sortedIndices.push(pi);
        frames.push({
            array: cloneArray(), activeIndices: [], sortedIndices: [...sortedIndices],
            isSwapping: false, pivotIndex: pi, subArrayRange: [low, high], codeLine: 'partition',
            currentStats: { ...stats }, msg: `Partition complete. Pivot locked at index ${pi}.`
        });
```

src/modules/core/algorithm/sorting/algorithms/quickSort.js · lines 76–122 (verbatim)

```
    function partition(low, high) {
        const pivot = arr[high].val;
        let i = low - 1;

        frames.push({
            array: cloneArray(), activeIndices: [], sortedIndices: [...sortedIndices],
            isSwapping: false, pivotIndex: high, subArrayRange: [low, high], codeLine: 'partition',
            currentStats: { ...stats }, msg: `Selecting ${pivot} at index ${high} as the pivot.`
        });

        for (let j = low; j < high; j++) {
            stats.comps++;
            frames.push({
                array: cloneArray(), activeIndices: [j], sortedIndices: [...sortedIndices],
                isSwapping: false, pivotIndex: high, subArrayRange: [low, high], codeLine: 'compare',
                currentStats: { ...stats }, msg: `Comparing ${arr[j].val} with pivot ${pivot}.`
            });

            if (arr[j].val <= pivot) {
                i++;

                if (i !== j) {
                    stats.swaps++;
                    frames.push({
                        array: cloneArray(), activeIndices: [i, j], sortedIndices: [...sortedIndices],
                        isSwapping: true, pivotIndex: high, subArrayRange: [low, high], codeLine: 'swap',
                        currentStats: { ...stats }, msg: `${arr[j].val} <= ${pivot}. Swapping elements.`
                    });

                    let temp = arr[i];
                    arr[i] = arr[j];
                    arr[j] = temp;

                    frames.push({
                        array: cloneArray(), activeIndices: [i, j], sortedIndices: [...sortedIndices],
                        isSwapping: false, pivotIndex: high, subArrayRange: [low, high], codeLine: 'swap',
                        currentStats: { ...stats }, msg: `Swap complete.`
                    });
                } else {
                    frames.push({
                        array: cloneArray(), activeIndices: [i, j], sortedIndices: [...sortedIndices],
                        isSwapping: false, pivotIndex: high, subArrayRange: [low, high], codeLine: 'swap',
                        currentStats: { ...stats }, msg: `${arr[j].val} <= ${pivot}, but already in correct position.`
                    });
                }
            }
        }
```

35.4 The real trace: `[3, 1, 2]` → 20 frames

#	codeLine	array	active	sorted	pivot	range	comps	passes	swaps
0	start	[3,1,2]	[]	[]	–	[]	0	0	0
1	start	[3,1,2]	[]	[]	–	[0,2]	0	0	0
2	partition	[3,1,2]	[]	[]	–	[0,2]	0	1	0
3	partition	[3,1,2]	[]	[]	**2**	[0,2]	0	1	0
4	compare	[3,1,2]	[0]	[]	2	[0,2]	1	1	0
5	compare	[3,1,2]	[1]	[]	2	[0,2]	2	1	0
6	swap	[3,1,2]	[0,1]	[]	2	[0,2]	2	1	1
7	swap	[1,3,2]	[0,1]	[]	2	[0,2]	2	1	1
8	swap	[1,3,2]	[1,2]	[]	2	[0,2]	2	1	2
9	swap	[1,2,3]	[1,2]	[]	**1**	[0,2]	2	1	2
10	partition	[1,2,3]	[]	**[1]**	1	[0,2]	2	1	2
11	recurse_left	[1,2,3]	[]	[1]	–	[0,0]	2	1	2
12	start	[1,2,3]	[]	[1]	–	[0,0]	2	1	2
13	base_case	[1,2,3]	[]	[1]	–	[0,0]	2	1	2
14	base_case	[1,2,3]	[]	**[1,0]**	–	[]	2	1	2
15–18	recurse_right → base cases	[1,2,3]	[]	[1,0,2]	–	[2,2]	2	1	2

#	codeLine	array	active	sorted	pivot	range	comps	passes	swaps
19	start	[1,2,3]	[]	[0,1,2]	–	[]	2	1	2

MUST KNOW

FOUR DETAILS THAT ONLY A PERSON WHO READ THE CODE WOULD KNOW

1. **`sortedIndices` can be out of order.** At frame 14 it reads `[1, 0]` — the pivot index 1 was locked at frame 10, then the single-element sub-array `[0,0]` added index 0. Order does not matter for `.includes()`, but it is a visible asymmetry between algorithms.
2. **The base case has two kinds of frames.** `if (low >= high)` pushes a bare `base_case` frame; then a nested `if (low === high)` pushes a `second` frame ("Index N is sorted.") and pushes the index onto `sortedIndices`. So a single-element sub-array produces two frames; an empty one (`low > high`, which happens for `pi - 1` when the pivot is 0) produces one. [VERIFIED at lines 27–42]
3. **The pivot swap is conditional.** `if (i + 1 !== high)` avoids a self-swap, and the else branch pushes a distinct message: "Pivot is already in its correct position."
4. **`stats.passes` increments once per `quickSortHelper` call that reaches the recursion step** — for `[3,1,2]` it ends at exactly 1, because only the root call partitioned. That is why the "Passes / calls" tile reads 1 for this input and 4 for the default 8-element input.

35.5 The complexity, and the worst case you can demonstrate live

Case	Comparisons	Time	When it happens
Best	Θ(n log n)	Θ(n log n)	The pivot lands near the median every time
Average	Θ(n log n)	Θ(n log n)	Random input
Worst	**n(n−1)/2**	**Θ(n²)**	**Already-sorted input with a last-element pivot** — the classic Lomuto trap

MUST KNOW

THE DEMONSTRATION THAT MAKES THIS UNFORGETTABLE

With `[1,2,3,…,24]` already sorted and the pivot chosen as `arr[high]`, every partition places the pivot at `high` itself: the `while (arr[j].val <= pivot)` condition is true for every `j`, so `i` walks all the way to `high - 1` and the "swap pivot into position" step does nothing. The recursion is then `[low, high-1]` — a **single** subproblem, one element smaller. That is the worst case exactly: n levels of recursion, each doing an O(n) partition.

Measured in this codebase: quick sort on the default 8-element input produces 75 frames with 16 comparisons; on the reverse-sorted 24-element input it produces 632 frames with 276 comparisons. The sorted-24 case is the pathological one — and the user can trigger it by pressing the "sorted" preset, which is a legitimate interactive demonstration of an algorithmic worst case. [VERIFIED]

The fix, if you want to discuss it: median-of-three pivot selection (choose the median of `arr[low]`, `arr[mid]`, `arr[high]`), or randomise the pivot by swapping `arr[high]` with a random index before partitioning. Both reduce the probability of quadratic behaviour to negligible on real inputs. [PROPOSED — not implemented]

35.6 Recursion depth and space

[VERIFIED: declared as `space: "O(n) worst"`] Because the recursion is not tail-call optimised by the JavaScript engine and each frame of work is a real stack frame, the deepest path is n−1 levels in the worst case — for the sorted-24 input that is 23 nested `quickSortHelper` calls, each holding `low`, `high`, `pi`. Hence O(n) worst-case auxiliary space, honestly declared in the UI.

INTERVIEW QUESTION
"IS QUICK SORT IN-PLACE?"

"The partition is in-place — it rearranges within the array using a single scan and one extra index. But the recursion is not free: the call stack is O(log n) on average and O(n) in the worst case, which is why the panel says O(n) worst-case space rather than O(1). If you recurse on the smaller side first and loop on the larger, the depth is bounded by O(log n) — that's the standard 'tail-call elimination by hand' trick, and it is not implemented here."

35.7 Stability, edge cases, bugs

Property	Answer
Stable?	**No.** The long-distance swaps in the partition are what break it, exactly as with selection sort
In-place?	The partition is; the recursion costs O(log n) average / O(n) worst
`[3,1,2]` walked by hand	Frames 4–9: `3 <= 2`? No. `1 <= 2`? Yes → `i` becomes 0 ≠ `j`=1 → swap → `[1,3,2]`. Then pivot swap (`i+1=1` ≠ `high`=2) → `[1,2,3]`. Pivot index 1 locked. Recurse `[0,0]` → sorted; recurse `[2,2]` → sorted. Total 2 comparisons — beating bubble's 3 on the same input
Duplicates	Lomuto with `<=` puts equal elements on the left of the pivot, which is correct but degrades badly when the array is mostly equal: every element goes left, and the subproblem shrinks by one. This is the "all elements equal" quadratic case
Duplicate-id frames	[VERIFIED] quick sort does **not** reproduce the merge-sort duplicate-id problem, because it only ever swaps whole element objects between positions — no cloning into a buffer

INTERVIEW QUESTION
"WHAT'S THE DIFFERENCE BETWEEN LOMUTO AND HOARE PARTITIONING, AND WHICH DID YOU USE?"

"Lomuto, the simpler scheme: a single scan index `i` that marks the boundary of the ≤-region, and a final swap that places the pivot. Hoare's is faster in practice because it does fewer swaps — roughly a third fewer on average — but it's harder to get right and doesn't place the pivot in its final position at the end of the partition. Lomuto was the right choice here because 'the pivot is now final, lock it' is a beautifully clear teaching moment, and the code shows exactly that: `sortedIndices.push(pi)` immediately after the partition returns."

35.8 Whiteboard

```
function quickSort(a, low = 0, high = a.length - 1) {
  if (low >= high) return a;
  const p = partition(a, low, high);
  quickSort(a, low, p - 1);
  quickSort(a, p + 1, high);
  return a;
}

function partition(a, low, high) {
  const pivot = a[high];
  let i = low - 1;
  for (let j = low; j < high; j++) {
    if (a[j] <= pivot) { i++; [a[i], a[j]] = [a[j], a[i]]; }
  }
  [a[i + 1], a[high]] = [a[high], a[i + 1]];
  return i + 1;
}
```

DANGER ZONE — WHERE CANDIDATES DIE

THE TWO THINGS INTERVIEWERS WATCH FOR WHEN YOU WRITE THIS

1. **The pivot must not be included in either recursive call.** `p - 1` and `p + 1`, not `p` — otherwise infinite recursion on a sub-array of size 1.
2. **`j < high`, not `j <= high`.** The pivot itself is excluded from the scan and placed by the final swap; including it would compare the pivot with itself and place it incorrectly.

CHAPTER 36

36 · Heap sort

Two phases, an array-as-tree, and the only algorithm here whose auxiliary space comes from recursion rather than a buffer.

36.1 Problem and intuition

Intuition. An array can represent a complete binary tree for free: for index `i`, the children are `2i+1` and `2i+2`, and the parent is `⌊(i−1)/2⌋`. If you enforce "every parent is at least as large as its children" — the max-heap property — then the root is the maximum. Swap the root to the end, shrink the heap by one, restore the property, and repeat. The array fills with sorted values from the right.

36.2 The invariant

[VERIFIED] **"After heap restoration, the root is the largest value in the active heap."**

36.3 The actual implementation

`src/modules/core/algorithm/sorting/algorithms/heapSort.js · lines 20–71 (verbatim)`

```
    function heapify(size, i, phaseMsg) {
        let maxIdx = i;
        const left = 2 * i + 1;
        const right = 2 * i + 2;

        let nodesToHighlight = [i];
        if (left < size) nodesToHighlight.push(left);
        if (right < size) nodesToHighlight.push(right);

        frames.push({
            array: cloneArray(), activeIndices: nodesToHighlight, sortedIndices: [...sortedIndices],
            isSwapping: false, pivotIndex: null, subArrayRange: [], codeLine: 'pointers',
            currentStats: { ...stats }, msg: `${phaseMsg}: Evaluating Parent [${arr[i].val}] at index ${i}.`
        });

        stats.comps++;
        if (left < size && arr[left].val > arr[maxIdx].val) {
            maxIdx = left;
        }

        stats.comps++;
        if (right < size && arr[right].val > arr[maxIdx].val) {
            maxIdx = right;
        }

        frames.push({
            array: cloneArray(), activeIndices: nodesToHighlight, sortedIndices: [...sortedIndices],
            isSwapping: false, pivotIndex: null, subArrayRange: [], codeLine: 'compare_children',
            currentStats: { ...stats }, msg: `Comparing with children. Largest value is ${arr[maxIdx].val}.`
        });

        if (maxIdx !== i) {
            stats.swaps++;
            frames.push({
                array: cloneArray(), activeIndices: [i, maxIdx], sortedIndices: [...sortedIndices],
                isSwapping: true, pivotIndex: null, subArrayRange: [], codeLine: 'swap_heapify',
                currentStats: { ...stats }, msg: `Sinking down: Swapping ${arr[i].val} with ${arr[maxIdx].val}.`
            });

            let temp = arr[i];
            arr[i] = arr[maxIdx];
            arr[maxIdx] = temp;

            frames.push({
                array: cloneArray(), activeIndices: [i, maxIdx], sortedIndices: [...sortedIndices],
                isSwapping: false, pivotIndex: null, subArrayRange: [], codeLine: 'swap_heapify',
                currentStats: { ...stats }, msg: `Recursively heapifying the affected sub-tree.`
            });

            heapify(size, maxIdx, phaseMsg);
        }
    }
```

36.4 The real trace: `[3, 1, 2]` → 14 frames

#	codeLine	array	active	sorted	comps	passes	swaps
0	start	[3,1,2]	[]	[]	0	0	0
1	build_heap	[3,1,2]	[]	[]	0	0	0
2	pointers	[3,1,2]	[0,1,2]	[]	0	0	0
3	compare_children	[3,1,2]	[0,1,2]	[]	**2**	0	0
4	extract	[3,1,2]	[]	[]	2	0	0
5	swap_root	[3,1,2]	[0,2]	[]	2	1	1
6	heapify_root	[2,1,3]	[0]	[2]	2	1	1
7	pointers	[2,1,3]	[0,1]	[2]	2	1	1
8	compare_children	[2,1,3]	[0,1]	[2]	**4**	1	1
9	swap_root	[2,1,3]	[0,1]	[2]	4	2	2
10	heapify_root	[1,2,3]	[0]	[2,1]	4	2	2
11	pointers	[1,2,3]	[0]	[2,1]	4	2	2
12	compare_children	[1,2,3]	[0]	[2,1]	**6**	2	2

#	codeLine	array	active	sorted	comps	passes	swaps
13	start	[1,2,3]	[]	[0,1,2]	6	2	2

MUST KNOW

WHY THE COMPARISON COUNT JUMPS BY 2 EVERY HEAPIFY, AND WHY THAT IS WORTH DEFENDING

[VERIFIED lines 35 and 40] `heapify` performs **two unconditional comparison increments**:

```
stats.comps++;
if (left < size && arr[left].val > arr[maxIdx].val) maxIdx = left;

stats.comps++;
if (right < size && arr[right].val > arr[maxIdx].val) maxIdx = right;
```

Both increments happen even when the child does not exist (for a leaf, both `left < size` and `right < size` are false, so no real comparison occurs). The trace proves it: at frame 11 the parent at index 0 is a leaf in a heap of size 1 — there are no children at all — and the counter still rises from 4 to 6.

Why this matters, and how to defend it: the counter is a generator instrumentation choice — it counts "comparison slots evaluated", not "value comparisons executed". For most algorithms the difference is invisible, but for heap sort it inflates the number.

The honest answer if challenged: *"Yes — heap sort's comparison counter increments twice per heapify call unconditionally, so leaf calls inflate it. Bubble, selection and insertion count exactly one comparison per actual value comparison. That's a real inconsistency between generators, and it's precisely why I tell people the counters are per-implementation instruments and not comparable across algorithms — and why the comparison panel only uses `comps`, and even then with the disclaimer on screen. If I were to fix it, I'd increment inside the guards, and then update the expected values in the tests."* [PROPOSED fix]

36.5 The two phases, and the sorting-region behaviour

Phase	Code	What the frames show
1 — build heap	`for (let i = Math.floor(n/2) - 1; i >= 0; i--) heapify(n, i, 'Building Heap')`	The heap property is established bottom-up. For n = 3 only one heapify call is needed (i = 0), which is why the trace jumps straight to `extract` at frame 4
2 — extract	`for (let i = n - 1; i > 0; i--)` — swap root with `arr[i]`, mark `i` sorted, then `heapify(i, 0, 'Extracting')`	Each extraction pushes one more index into `sortedIndices`, so the sorted region **grows from the right**, one index per pass — the same visual signature as bubble sort but arriving at the ordering by a completely different route

[VERIFIED] Unlike bubble and selection, heap sort's `sortedIndices` is filled exactly once per extraction and in strictly decreasing index order (n−1, n−2, ..., 1), with `sortedIndices.push(0)` after the loop for the final element. The completed region is therefore `[n-1, n-2, ..., 1, 0]`.

36.6 Complexity

Case	Comparisons	Time	Space
Best / average / worst	Θ(n log n)	**Θ(n log n) always**	Build phase O(n); recursive heapify O(log n) stack
Two-phase cost	Build is O(n) (not O(n log n))	Each of the n−1 extractions costs O(log n)	Hence Θ(n log n) total

MUST KNOW

THE FACT THAT MAKES YOU LOOK LIKE YOU KNOW HEAP SORT

Building a heap bottom-up is O(n), not O(n log n). The intuition: most nodes are near the bottom, and the depth available to sift them down is small, so the sum Σ (number of nodes at depth d) × O(height) telescopes to O(n). That is why heap sort's build phase does not dominate the total.

And the "space O(log n)\" in the panel is honest about this implementation*: it uses a **recursive** `heapify`, so each level of sifting holds a stack frame. An iterative sift-down loop would make it O(1) auxiliary, which is the version usually described as "heap sort is in-place". The asterisk in the UI exists precisely to point this out — and if an interviewer asks "is heap sort in-place?", the answer that earns credit is: "The swaps are in-place, but this implementation recurses in heapify, so the call stack is O(log n). The panel says O(log n) with an asterisk for exactly that reason — the footnote is on screen. To make it truly O(1) I'd convert the recursion into a while loop." [PROPOSED]

36.7 Stability, edge cases, bugs

Property	Answer
Stable?	**No.** The root-to-end swap moves an element a long distance, and sift-down swaps across the tree — both reorder equal keys
In-place?	The data is; the heapify recursion is not
n = 0	4 frames. The build loop `i = Math.floor(0/2)-1 = -1` does not run, and the extract loop `i = -1` does not run, but the two phase-announcement frames and the terminal frame are still pushed, plus the seed. [VERIFIED]
n = 1	4 frames, same shape
`pivotIndex` is always null	[VERIFIED] Heap sort's frames carry `pivotIndex: null` — the field exists in the schema only because merge and quick use it. `activeIndices` is what shows the parent-and-children triple

INTERVIEW QUESTION

"WHY DOES YOUR HEAP SORT SHOW THREE ACTIVE INDICES AT ONCE?"

"Because a heapify call has to compare a parent with both children, so the frame's `activeIndices` is built as `[i, left, right]` filtered to those that exist — `let nodesToHighlight = [i]; if (left < size) nodesToHighlight.push(left); if (right < size) nodesToHighlight.push(right);`. That is the array-as-tree visual: the three nodes involved in one sift step, highlighted together. The parent/child index formulas are `2i+1` and `2i+2`, which is why the highlight can jump around the array rather than sliding next to it."

36.8 Whiteboard

```
function heapSort(a) {
  const arr = [...a], n = arr.length;

  for (let i = (n >> 1) - 1; i >= 0; i--) siftDown(arr, n, i);   // build: O(n)

  for (let end = n - 1; end > 0; end--) {
    [arr[0], arr[end]] = [arr[end], arr[0]];                     // max to its final slot
    siftDown(arr, end, 0);                                       // restore on the shrunken heap
  }
  return arr;
}

function siftDown(a, size, i) {
  while (true) {                        // iterative on the whiteboard: O(1) space
    let largest = i;
    const l = 2 * i + 1, r = 2 * i + 2;
    if (l < size && a[l] > a[largest]) largest = l;
    if (r < size && a[r] > a[largest]) largest = r;
    if (largest === i) return;
    [a[i], a[largest]] = [a[largest], a[i]];
    i = largest;
  }
}
```

MUST KNOW

THE TWO SENTENCES THAT CLOSE OUT HEAP SORT

"Build is O(n) because most nodes are shallow, and each of the n−1 extractions costs O(log n), so the total is Θ(n log n) in every case — no input distribution is bad for it, which is its one real advantage over quick sort. Here I've written the sift as a loop so the space is O(1); the version inside AlgoViz recurses instead, which is why my UI declares O(log n) with a footnote about the call stack." [VERIFIED: the app's own panel says exactly this.]

CHECKLIST

RED-LINE DRILLS FOR PART X

- ■ For each of the six: complexity table, stability, in-place-ness, and the invariant sentence.
- ■ Write all five RED algorithms on paper in under ten minutes total.
- ■ Explain why this bubble sort is $\Theta(n^2)$ even on sorted input (no early exit).
- ■ Explain why selection sort's comparison count is input-independent.
- ■ Explain the insertion-sort `break` frame and why it costs one extra comparison.
- ■ Explain merge sort's write-based "swap" counter and the duplicate-id bug.
- ■ Explain quick sort's worst case with a last-element pivot and how to trigger it in the app.
- ■ Explain heap sort's two-phase structure, the O(n) build, and the recursive-heapify space footnote.

PART XI

Algorithm visualization design

The engineering problem behind the product: turning an algorithm that returns a result into a sequence of explainable states.

CHAPTER 37

37 · From result to recorded explanation

The design problem, the instrumentation model, and why this is not CSS animation.

37.1 The mismatch that defines the project

```
WHAT AN ALGORITHM NATURALLY PRODUCES          WHAT A LEARNER NEEDS
-------------------------------------        -------------------------------------
sorted([3, 1, 2])  ->  [1, 2, 3]             "at step 4, elements at index 1 and 2
                                              were compared; 3 > 2 was true so they
                                              swapped; after that the region from
                                              index 2 rightward is final"

one value in, one value out                  a sequence of (state, reason) pairs
no memory of the path                        the path is the product
optimised to forget intermediates            the intermediates are the whole point
```

MUST KNOW

THE THESIS OF THE WHOLE APPLICATION

An algorithm is a sequence of decisions, and a normal implementation discards that sequence. Every optimisation an implementation makes — temporary variables, in-place writes, a merge buffer, recursion — is work done to avoid keeping the intermediate states. AlgoViz's contribution is not the sorting; it is the decision to run each algorithm in a mode that keeps them, and then to treat the resulting sequence as a first-class, inspectable artefact.

37.2 Instrumentation: how execution becomes data

The generators are not a separate implementation of the algorithms. They are the algorithms, with three instrumentation techniques applied at every step.

Technique	Mechanism in the code	What it produces
Snapshot before mutation	`cloneArray()` is called before the state changes, so the frame captures the pre-state	The frame that explains why the next action happens
Snapshot after mutation	A second frame is pushed with a fresh `cloneArray()` after a swap or a write	The frame that shows the result of the decision
Metadata annotation	`activeIndices`, `sortedIndices`, `pivotIndex`, `subArrayRange`, `codeLine`, `msg`, `currentStats`	The explanation, carried with the state rather than derived from it later
Counter snapshot	`{...stats}` on every frame	"How much work had happened by this moment"
Bookkeeping lists	`sortedIndices.push(...)` at the algorithmic moment when a position becomes final	The invariant, rendered

KEY TAKEAWAY

THE CRITICAL INSIGHT ABOUT ANNOTATION

`activeIndices` is **not derivable** from the array state. If you only had the array at each step, you could not tell which two elements were being compared — after the fact, all you know is the order. **The indices are knowledge about the algorithm's intent, and intent leaves no trace in the data.** That is why the instrumentation exists at all, and it is the sharpest possible answer to "why not just diff the arrays?"

37.3 Why "just animate the CSS" is a different (and weaker) design

Aspect	CSS/transition animation	Recorded frames
What is the source of truth?	The DOM's current visual state	The `frames` array
Can you pause mid-animation at an exact logical step?	Only if the animation duration happens to align with a step boundary — CSS interpolates, it does not know about steps	Yes, by definition: the frame is the step
Can you step backwards one logical operation?	No — you would have to reverse a transition	Yes
Can you ask "what were the counters at this moment?"	No — the counters are not in the DOM's animation state	Yes — `frame.currentStats`
Can you test the sequence without a browser?	No	Yes — it is a pure function returning an array
Can you render the same trace twice and compare?	No	Yes — `deepEqual`
What breaks first at scale?	The compositor	Memory

MODEL ANSWER

"WHY NOT JUST USE CSS TRANSITIONS BETWEEN ARRAY STATES?"

"Because a CSS transition interpolates between two visual states — it has no idea what a logical operation is. If I wanted the user to be able to stop at the exact moment a comparison was evaluated, the animation would have to be quantised to logical steps anyway, at which point I am building a step engine and animating it. And the deeper problem is that the interesting information isn't visual: which elements are being compared, whether the comparison returned true, what the counter values were, which sub-array we're inside. None of that is expressible as a transition between two bar charts.

There is also a testing argument. Because my trace is data produced by a pure function, I can assert properties on it in Node with no browser: sortedness, input preservation, determinism, and ID uniqueness. That is only possible because the visualisation is downstream of the data rather than being the data."

37.4 The visual encoding, mapped to the data

Every visual channel in the lab is driven by a specific field. This table is the bridge between the frame schema and what the user sees.

Visual channel	Frame field	CSS / logic	Meaning for the learner
Bar **height**	`array[i].val` → `25 + (|val| / max) * 75` %	inline `style`	Magnitude. `Math.abs` means negative values are shown by size, with the sign preserved in the label
Bar **label**	`array[i].val`	`<strong>{item.val}</strong>`	The actual value — including its sign
Green fill	`activeIndices.includes(i)` → `.av-bar.active` (`#a6df71`)	the elements under consideration right now	"Look here"
Teal fill	`sortedIndices.includes(i)` → `.av-bar.sorted` (`#204c41`)	elements the algorithm considers final	"This part is done"
Amber fill	`pivotIndex === i` → `.av-bar.pivot` (`#63532e`)	quick sort's pivot; also used for heap's staging	"This is the reference value"
Marker glyph	`pivot ? "P" : active ? "●" : sorted ? "✓" : ""`	inside the bar	A redundant channel, so colour is not the only signal
Index label	`i` (render index)	`<small>{i}</small>` under each bar	The position — **not** the element's identity

Visual channel	Frame field	CSS / logic	Meaning for the learner
Operation line	`frame.codeLine.replaceAll("_", " ")`	"CURRENT OPERATION · BASE CASE"	Which line of the algorithm produced this state
Explanation sentence	`frame.msg`	the paragraph under the stage	Why this state exists, in words
Step badge	`cursor` → `String(cursor).padStart(3, "0")`	`005`	Where in the recording we are
Counters	`frame.currentStats.{comps, passes, swaps}`	three tiles	Work done so far
Index lists	`activeIndices`, `pivotIndex`, `sortedIndices`	the inspector	The raw metadata, for people who want numbers
Sub-array range	`subArrayRange`	**not rendered by the lab**	[PROPOSED] the missing channel for merge/quick/heap (a dead legacy file reads it)

MUST KNOW

THE ONE VISUAL CHANNEL THAT IS MISSING, AND WHY IT MATTERS

`subArrayRange` is present on every merge, quick and heap frame, and `ArrayView` ignores it. The only reader anywhere in the repository is `src/modules/core/EfficientSort.jsx`, a 498-line component with zero importers — which is a useful reminder that "unused" needs a scope: the field is unused by the shipped app, not by the codebase. That is the reason recursion-heavy algorithms are harder to read in the lab than the O(n²) ones: the user can see what changed but not which sub-problem is being worked on. Rendering a bracket or a background tint over `[low, high]` would make divide-and-conquer legible with zero engine changes. It is the single highest-value visual improvement available, and it requires no new data. [PROPOSED]

37.5 What makes an explanation explainable

The `msg` strings are not decoration; they follow a discipline. Comparing them across the six generators reveals four distinct message types:

Type	Example	Why it is written that way
Decision	`"Comparing 3 and 1. Is 3 > 1?"`	States the predicate as a question, so the reader can answer it before the next frame reveals the outcome. This is a deliberate pedagogical device: the app asks, the user predicts
Outcome	`"Yes! 3 > 1. Swapping blocks..."` / `"No swap needed. Moving window forward."`	Both branches are voiced, so a false comparison is as informative as a true one
Structural	`"Pass 1 of 3."`, `"Dividing sub-array [0 to 2]."`, `"Base case reached."`	Orients the reader in the algorithm's control flow, not just its data
Completion	`"Bubble Sort Complete! Array is fully sorted."`	Marks the terminal state unambiguously
Self-critical	`"5 ≠ 4: incorrectly skip a qualifying window."` (buggy variant in `windowTrace`)	Names the mistake at the exact frame it is made — the counterexample lab's key trick

KEY TAKEAWAY

THE RULE THE MESSAGES FOLLOW

A message must tell you what the algorithm is doing and why, without requiring you to read the array. If the sentence only restated the data ("array is now [1,3,2]") it would add nothing. Every message in this codebase either states a predicate, reports its outcome, or names the control-flow position. [INFERENCE from reading all ~64 message strings; the strings themselves are VERIFIED]

37.6 Comparison with the alternative designs

Design	How it would represent an execution	Why AlgoViz did not choose it
Instrumented live run (legacy modules)	The algorithm runs and pushes state into React as it goes, with `await` for pacing	Cannot seek backwards; state is spread across timers; untestable without a browser
Video / screen recording	Record the animation and play it back	Not queryable, not inspectable, not testable, not searchable — you cannot ask a video for the comparison count at frame 40
Step-through debugger	Pause real execution with breakpoints	Requires either a real interpreter or a code transformation pipeline; and the user must understand the source. Also: the README explicitly rules out arbitrary user-code execution
Operation log	Record only the operations, replay from scratch to reach a frame	[PROPOSED at scale] Much smaller memory, but every seek costs a replay and frames stop being independently renderable. See Chapter 14
Frame recording (chosen)	Run to completion, record every state, index into it	The one design where seeking is free, frames are testable data, and the algorithm code stays ordinary

DANGER ZONE — WHERE CANDIDATES DIE

NEVER SAY "I MADE A VISUALISER"

It is the weakest possible description of this project, and it invites the question "so what did you build that isn't in every other visualiser?" Use this instead: **"I built an execution recorder with a playback layer on top, and the algorithms are instrumented to report their decisions as data rather than to animate themselves. The visualisation is the least interesting part — it's a pure function of one integer."**

CHECKLIST

RED-LINE DRILLS FOR PART XI

- ■ Explain why `activeIndices` cannot be derived from the array snapshots.
- ■ Give three reasons CSS transitions cannot support step-accurate backwards navigation.
- ■ Name the five message types and why both branches of a decision are voiced.
- ■ Name the visual channel that exists in the data and is not rendered.
- ■ Say the one-sentence description that beats "I made a visualiser".

PARTS XII–XIV

Comparison, alignment and operation counts

What the comparison feature really measures — and the three disclaimers already written into your own UI.

CHAPTER 38

38 · Same-input comparison

One input array, two traces, four pieces of on-screen disclosure.

38.1 The code

src/lab/SortingLab.jsx · lines 74–76 (verbatim)

```
  const [compare, setCompare] = useState(params.has("compare"));
  const [other, setOther] = useState("merge");
  const [seed, setSeed] = useState(42);
```

src/lab/SortingLab.jsx · lines 93–97 (verbatim)

```
  const frames = useMemo(() => algorithms[algo].run(input), [algo, input]);
  const otherFrames = useMemo(
    () => (compare ? algorithms[other].run(input) : []),
    [compare, other, input],
  );
```

src/lab/SortingLab.jsx · lines 198–203 (verbatim)

```
  const meta = algorithms[algo],
    stats = frame.currentStats || {},
    last = frames.at(-1).currentStats;
  const compareCursor = otherFrames.length
    ? Math.round((cursor / (frames.length - 1)) * (otherFrames.length - 1))
    : 0;
```

38.2 What "same input" is guaranteed to mean

Guarantee	Mechanism
Both algorithms receive **the identical array**	`algorithms[algo].run(input)` and `algorithms[other].run(input)` pass the same reference — not a copy — so there is no possibility of a divergence introduced by copying
Neither mutates it	The generators wrap values in new objects (`rawArray.map(...)`) and only ever sort their own `arr`. This is asserted for all six by the test suite: `assert.deepEqual(input, before)`
Both traces are recomputed together when the input changes	`otherFrames`'s dependency array includes `input`
Comparison is opt-in	`compare` defaults to `params.has("compare")`, and `otherFrames` is `[]` when comparison is off, so no work happens unless the user asked for it

KEY TAKEAWAY

WHY PASSING THE SAME REFERENCE MATTERS MORE THAN IT SOUNDS

If the second algorithm had been given a copy, the two runs would still be correct — but the claim "same dataset" would rest on the developer having remembered to copy faithfully. By passing the identical reference and relying on the generators' purity, the guarantee becomes structural. The test suite proves the purity property for all six generators, so the comparison's core assumption is itself tested. [VERIFIED]

38.3 The four pieces of disclosure in the UI

This is the most important part of the chapter, because it is where the project demonstrates intellectual honesty — and every one of these lines is already written in your own code, so you can

quote them.

#	What the UI says	Where	Why it matters
1	"Preview aligned by percentage of trace—not equivalent operations."	under the comparison array	Disclaims the alignment
2	"Actual comparison events from the same input. No wall-clock claims: animation speed is not algorithm performance."	in the comparison results panel	Disclaims the metric
3	"COMPLETE-RUN COMPARISONS" as the panel's eyebrow	above the meters	States that the numbers describe the whole run, not the current frame
4	"Passes/calls are implementation-specific and should not be compared across algorithms. Legacy generators count exchanges differently; comparison mode deliberately uses comparisons only. Trace storage is bounded by the 24-element input limit."	the page footnote	Disclaims the other two counters and documents the memory bound

MUST KNOW
THE SENTENCE THAT WINS THIS PART OF THE INTERVIEW

"If you look at the comparison panel in the app, the disclaimers are already there — I wrote them into the product, not into the README. The panel says percentage alignment is not equivalent operations, and it says there are no wall-clock claims. The reason is that a comparison feature is exactly where a project is most tempted to lie, and I'd rather the UI be honest than the demo be impressive."

That is a genuinely differentiating answer, because it demonstrates that you anticipated the criticism and encoded the response.

38.4 What the feature compares, and what it refuses to

The feature DOES show	The feature does NOT show
Recorded **comparison counts** for the complete run of each algorithm	Wall-clock time
The two numbers side by side, as labelled figures	Memory or allocation counts
Two proportional meter bars, scaled to the larger of the two	A "winner" or a ranking
A second array preview at the proportionally corresponding frame	A claim that equivalent frames mean equivalent work
The primary algorithm's live counter as you step	A per-step difference between the two algorithms

[VERIFIED] There is no sorting of the results, no "faster/slower" label, no verdict. The panel renders the two counts and two meters and stops.

38.5 The five comparison questions, answered

INTERVIEW QUESTION
"WHY THE SAME INPUT?"

"Because otherwise the numbers are not comparable at all. Comparison counts depend heavily on the input — insertion sort's count on a sorted array is n−1, while on a reversed array it's n(n−1)/2. If the two algorithms were given different arrays, any difference in the counters could be explained by the data rather than the algorithm. Using the identical array removes the input as a variable. The fact that the generators don't mutate their input is what makes passing the same reference safe, and that's asserted in the test suite."

INTERVIEW QUESTION
"WHAT EXACTLY ARE YOU COMPARING?"

"One recorded quantity: the number of comparison operations each algorithm performed on the complete run. Not time, not swaps, not memory. I chose comparisons because it's the one counter that is semantically the same across all six implementations — every generator increments `stats.comps` exactly at a value comparison. The other counters are not: the exchange counter means a swap in bubble sort, a shift in insertion sort, and a buffer write in merge sort. That's why the footnote says the pass and exchange counters are implementation-specific and why the panel uses comparisons only."

INTERVIEW QUESTION
"DOES COMPARISON PROVE RUNTIME SUPERIORITY?"

"No, and the app says so explicitly. Comparison count is a proxy for the number of decisions, not the cost of them. Runtime depends on constant factors, memory access patterns, branch prediction, how the engine optimises each function, and the actual data distribution — none of which the counter captures. A concrete example from my own traces: merge sort does 16 comparisons and 24 writes on the default eight-element input, while selection sort does 28 comparisons and 6 swaps. Merge has fewer comparisons and more data movement — which one is faster in a browser depends on whether a comparison or a write costs more on that machine, and I don't measure either."

INTERVIEW QUESTION
"CAN TWO ALGORITHMS HAVE DIFFERENT TRACE LENGTHS?"

"Always, and that's exactly why the alignment question exists. On the default eight-element input: selection 65 frames, quick 75, insertion 76, heap 84, merge 107, bubble 118. Measured, not estimated. The reason is that frame count is a function of how many states the generator records, and different algorithms produce different numbers of intermediate states per unit of work — merge sort records recursion-entry, buffer, comparison and write frames, while bubble records a frame per comparison plus two per swap. So frame count encodes structure, not speed."

INTERVIEW QUESTION
"ARE OPERATION COUNTS NORMALIZED?"

"No. They are raw counts for the complete run. There is no per-element normalisation and no attempt to weight operations by cost — the meters are scaled against the larger of the two counts purely for visual proportion, which is a presentation decision. Part of the reason I don't normalise is that I'd have to answer 'normalised by what?' For different n? Both algorithms already saw the same n. By cost? I'd have to decide the relative cost of a comparison and a write, which is machine-dependent. Raw counts with an explicit disclaimer are more honest than a normalised number with a hidden assumption inside it."

CHAPTER 39

39 · Percentage-aligned replay, explained precisely

Ten numerical examples, one formula, and the single most important "does not equal" in the project.

39.1 The formula

src/lab/SortingLab.jsx · lines 201–203 (verbatim)

```
  const compareCursor = otherFrames.length
    ? Math.round((cursor / (frames.length - 1)) * (otherFrames.length - 1))
    : 0;
```

Symbol	Meaning
`cursor`	The primary algorithm's current frame index, `0 … frames.length - 1`
`frames.length - 1`	The primary trace's last valid index — the denominator, i.e. "how far is the primary playback from the end"
`otherFrames.length - 1`	The same for the secondary trace
`compareCursor`	The secondary frame index to render

The expression converts the primary position into a **fraction of progress**, then converts that fraction into an index in the other trace:

```
progress        = cursor / (frames.length - 1)
compareCursor   = round( progress * (otherFrames.length - 1) )
```

39.2 Worked examples

Example A — Algorithm A has 100 frames, Algorithm B has 40 frames.

Primary cursor	Progress	`compareCursor`	What you see of B
0	0 / 99 = 0.000	`round(0 × 39) = 0`	B's first frame
10	0.101	4	B's 5th frame
25	0.253	10	B's 11th frame
50	0.505	20	B's 21st frame
75	0.758	30	B's 31st frame
90	0.909	35	B's 36th frame
99	1.000	39	B's last frame

Example B — A has 40 frames, B has 100 (the reverse).

Primary cursor	Progress	`compareCursor`
0	0.000	0
10	0.256	26
20	0.513	51
30	0.769	77
39	1.000	99

Example C — the doubling case. If B's trace is exactly twice A's and A is at 50%, then `compareCursor = round(0.5 × (2×A - 1))`. With A = 100, B = 199: A at cursor 50 → progress

0.505 → `round(0.505 × 198) = 100`. So at the halfway point of a 100-frame trace you are shown frame 100 of a 199-frame trace — B's halfway point. The alignment is proportional and symmetric.

39.3 What percentage alignment is actually for

Purpose	Explanation
Visual sanity	Two arrays of wildly different lengths cannot be index-aligned: frame 50 of a 118-frame bubble trace and frame 50 of a 375-frame merge trace are at completely different points in their execution. Index alignment would show, at the start, a nearly-initial bubble trace next to a nearly-finished merge trace
A single slider for two traces	The user drives one control; the second view follows. Proportional mapping is the simplest rule that keeps both views near their ends at the same time
Honest by construction	The transform is obviously a proportion, so nobody can mistake it for a claim about equivalent operations — especially with the on-screen caption

39.4 Why direct frame-index alignment is worse

Problem with `otherFrames[cursor]`	Consequence
The secondary array is `undefined` when `cursor >= otherFrames.length`	Merge sort has 375 frames and selection 492 on the same 24-element input; when the shorter trace is primary you would read past the end of the longer one — actually the safe direction — but in the reverse case, driving bubble (1,154 frames) against merge (375), 779 of the primary cursors would have no secondary frame at all. The array preview would be blank for two-thirds of the playback
Different work per frame	Step 10 of bubble sort is roughly "one comparison"; step 10 of merge sort is "enter recursion, compute a midpoint, recurse left" — index equality implies nothing
The end states drift apart	Index-aligned playback would finish the shorter trace long before the longer one, so the user would see a completed array beside an unfinished one for most of the run

39.5 The limitations, stated without hedging

WEAKNESS / LIMITATION

FIVE LIMITATIONS OF PERCENTAGE ALIGNMENT

1. **The mapping is not injective.** With A = 1,154 and B = 375, several primary cursors map to the same secondary frame (the ratio is ~3:1). Scrolling through the primary trace, the secondary preview will visibly stall on some frames.
2. **`round` introduces quantisation.** Small traces lose resolution: with B = 7 frames, only 7 distinct secondary positions exist, so most primary steps show an unchanged preview.
3. **Equal percentages are not equal work.** At 50% of a bubble sort you are halfway through ~276 comparisons; at 50% of a merge sort you are halfway through ~16 comparisons. The two views are at comparable completion of their own trace, nothing more.
4. **Frame counts are generator artefacts.** A generator that pushed fewer frames would appear "faster" if anyone mistakenly read playback length as speed. Merge sort's 375 frames vs bubble's 1,154 on the same input does **not** mean merge is 3× faster — it means the generators emit different numbers of states per unit of work.
5. **The alignment is not reversible.** The secondary view cannot drive the primary; there is no reverse mapping in the code.

DANGER ZONE — WHERE CANDIDATES DIE
THE SENTENCE YOU MUST NOT GET WRONG

Percentage-aligned replay ≠ same execution time.

Aligned percentage means: "both views are at the same fraction of their own recorded trace."

Aligned percentage does **not** mean: "the same amount of work has been done", "the same number of operations has elapsed", "the algorithms are at the same point in the array", or "one is ahead of the other."

And separately: **frame count ≠ CPU runtime.** A frame is a recorded state, and the number of states a generator emits is a property of how that generator was instrumented — how many frames it pushes per comparison, whether it records both halves of a swap, whether it emits recursion-entry frames. CPU runtime is a property of executed machine instructions, cache behaviour and the engine's optimisations. There is no monotone relationship between them. **Concretely: merge sort has 375 frames to bubble sort's 1,154 on the same reversed 24-element input — and on any real machine merge sort is thousands of times faster.** If frame count predicted runtime, that measurement would be backwards.

39.6 Ten interviewer attacks on alignment, with answers

INTERVIEW QUESTION
ATTACK 1. "SO AT 50% BOTH ALGORITHMS HAVE DONE HALF THE WORK?"

No — at 50% both recordings are half-played. Half of bubble sort's 276 comparisons is 138 comparisons; half of merge sort's 16 comparisons on that input is 8. The percentage is a position in a recording, not a fraction of work.

INTERVIEW QUESTION
ATTACK 2. "THEN WHAT IS THE ALIGNMENT FOR?"

For the human watching. One slider has to drive two views; proportional mapping is the only simple rule that keeps both views near their beginnings and near their ends at the same time. With index alignment, the shorter trace would sit finished on screen for most of the longer one's playback.

INTERVIEW QUESTION
ATTACK 3. "WHY NOT ALIGN BY OPERATION COUNT INSTEAD? THAT WOULD BE MEANINGFUL."

That would be more meaningful, and it is a real design option: you would record the cumulative comparison count on each frame (which the data already has — `currentStats.comps`) and map the primary cursor to the secondary frame whose counter is closest to the same value. It is harder because comparison counts are not monotone per frame index in a predictable way, and you would need a lookup. It is a legitimate improvement, and I'd describe it as a change I considered rather than something I implemented. [PROPOSED]

INTERVIEW QUESTION
ATTACK 4. "DOES THE ALIGNMENT EVER SHOW ONE ALGORITHM AHEAD?"

No, and it cannot: proportional alignment guarantees both views reach 100% together. Any impression of "ahead" comes from reading the arrays, not the alignment.

INTERVIEW QUESTION
ATTACK 5. "WHAT HAPPENS WHEN THE SECONDARY TRACE HAS ONLY ONE FRAME?"

`otherFrames.length - 1 = 0`, so `Math.round(progress × 0) = 0` for every cursor — the preview is pinned to frame 0. The code guards the empty case with `otherFrames.length ? ... : 0`. Since every generator emits at least two frames for any valid input, the single-frame case is not reachable from the UI. [VERIFIED]

INTERVIEW QUESTION
ATTACK 6. "WHAT IF `FRAMES.LENGTH === 1`?"

Then `cursor / (frames.length - 1)` divides by zero → `Infinity` or `NaN` when the cursor is 0. `Math.round(NaN)` is `NaN`, and `otherFrames[NaN]` is `undefined`, which would crash `ArrayView`. This is unreachable because `parseInput` requires at least two values and every generator produces ≥ 2 frames even for empty input (measured: bubble/selection/insertion produce 2, merge/quick/heap produce 4). But it is an unguarded division, and the defensive fix is one line — clamp the denominator. [VERIFIED as a latent risk]

INTERVIEW QUESTION
ATTACK 7. "IS THE COMPARISON PANEL SHOWING ME THE CURRENT FRAME'S COUNTERS OR THE WHOLE RUN'S?"

The whole run's. `last = frames.at(-1).currentStats` and the panel renders `last.comps` against `otherFrames.at(-1).currentStats.comps`. The live counters in the State Inspector are the current frame's, from `frame.currentStats`. Two different things on the same screen, deliberately labelled: "COMPLETE-RUN COMPARISONS" versus "State inspector / LIVE".

INTERVIEW QUESTION
ATTACK 8. "SO SWITCHING WHICH ALGORITHM IS PRIMARY CHANGES THE NUMBERS?"

No — the counts are properties of complete runs. Swapping primary and secondary swaps the labels and the meters, not the values. `frames.at(-1).currentStats.comps` is the same number regardless of which trace is currently being scrubbed.

INTERVIEW QUESTION
ATTACK 9. "CAN I COMPARE AN ALGORITHM WITH ITSELF?"

Yes, and that is a gap: the comparison `<select>` lists all six algorithms and is not filtered against the primary selection. Selecting the same algorithm twice produces two identical traces and two identical counts. It is harmless — nothing breaks, and the result is trivially consistent — but it is not guarded, and a self-comparison is not a meaningful experiment. [VERIFIED]

INTERVIEW QUESTION
ATTACK 10. "DOES THE METER WIDTH MEAN ANYTHING?"

Only proportion: `Math.max(2, (count / max(1, last.comps, otherFrames.at(-1).currentStats.comps)) * 100)%`. The `Math.max(2, …)` floor ensures a non-zero count always shows a visible sliver instead of vanishing. It is a bar chart of two numbers, not a measurement. [VERIFIED]

CHAPTER 40

40 · Operation counts: what is recorded, and what it is not

Three counters, three different meanings, one of which is comparable across algorithms.

40.1 The counter contract

[VERIFIED] Every generator maintains `let stats = { passes: 0, comps: 0, swaps: 0 }` and clones it into each frame as `currentStats`. The three fields are rendered in the State Inspector as "Comparisons", "Passes / calls[1]" and "Recorded exchanges[2]" — the superscripts are real, and the page footnote defines them.

Counter	Incremented where	Semantic meaning	Comparable across algorithms?
`comps`	Immediately before each value comparison	"A comparison of two values was evaluated"	**Yes** — this is the counter comparison mode uses
`passes`	Once per outer-loop iteration (bubble, selection, insertion), per recursion node (merge, quick), per extraction (heap)	"A structural unit of the algorithm occurred"	**No** — the unit differs per algorithm; the UI labels it "Passes / calls[1]"
`swaps`	On each named exchange event	Bubble/selection/insertion: a real swap. Merge: a write from the buffer. Quick: a swap or pivot placement. Heap: a root exchange or a sift swap	**No** — the UI labels it "Recorded exchanges[2]" and the footnote says legacy generators count exchanges differently

40.2 The exact increment sites, verified across all six generators

Algorithm	`comps` incremented	`swaps` incremented	`passes` incremented
Bubble	once per inner iteration, before the `compare` frame	once per real swap	once per outer pass
Selection	once per `j` in the inner scan (i.e. once per element scanned)	once per actual exchange (guarded by `minIndex !== i`)	once per pass
Insertion	once per `while` iteration, before the comparison frame	once per shift	once per lift
Merge	once per element **taken from the buffer**	once per element **written back**, including the remaining element	once per non-base recursion call
Quick	once per `j` in the partition scan	once per swap inside the partition, plus once for the pivot placement	once per `quickSortHelper` call that partitions
Heap	**twice per heapify call, unconditionally**	once per sift swap, plus once per extraction	once per extraction

DANGER ZONE — WHERE CANDIDATES DIE

THE TWO COUNTER BEHAVIOURS THAT WILL BE PROBED

1. **Heap sort double-counts.** Both increments happen even when the guard `left < size` / `right < size` is false, so a leaf's heapify still adds 2. Verified in the trace: on `[1,2,3]` the counter rises from 4 to 6 at a frame where the parent has no children at all.
2. **Merge sort counts writes, not swaps.** On the default 8-element input merge reports `swaps: 24` — that is 8 elements written back per level across roughly 3 levels of merging. Nothing was "swapped" in the bubble-sort sense; elements were copied out of a buffer into position.

Both are correct as instrumentation, and both would be wrong as cross-algorithm measurements. That is why the comparison panel uses only `comps` — and even then with an on-screen disclaimer.

40.3 What the numbers do not mean

MUST KNOW

FOUR THINGS THE COUNTERS ARE NOT

1. **Not runtime.** No timing code exists anywhere in the project. The counters are increments inside a synchronous function.
2. **Not memory.** Nothing measures allocation, frame size, or heap usage.
3. **Not portable across implementations.** A different bubble sort that counts a swap's two writes separately, or an insertion sort that shifts instead of swaps, would produce different numbers for the same algorithm.
4. **Not normalised.** The panel displays raw complete-run counts; the meter width is a proportion of the larger count, not a normalised score.

40.4 Worked example — why "merge has fewer comparisons" is a statement about this input

Using the **measured** default input `[38, 66, 12, 85, 24, 57, 43, 19]`:

Algorithm	Comparisons	Passes/calls	Exchanges	Frames
Selection	28	7	6	65
Merge	**16**	7	24	107
Quick	16	4	8	75
Insertion	22	7	16	76
Bubble	28	8	16	118
Heap	44	7	18	84

KEY TAKEAWAY

HOW TO READ THAT TABLE IN AN INTERVIEW

"Merge sort does the fewest comparisons here — 16 against bubble's 28 — and the most writes, 24. So if you asked me which one is faster on this input, the counter data doesn't answer it: it tells me merge made fewer decisions and moved more data. In practice merge is much faster, because n log n versus n^2 dominates these constant-factor differences at any realistic n — but the counters don't show that, they show what was counted. This is precisely why I refuse to put a 'winner' in the comparison panel."

Notice also that **selection sort reports the same 28 comparisons as bubble** — and selection's count is provably input-independent ($n(n-1)/2$), while bubble's varies with the data. Two algorithms with equal counters that behave completely differently is the cleanest possible demonstration that a single counter is not a performance measurement.

40.5 The counter questions

INTERVIEW QUESTION
"WHICH COUNTER WOULD YOU USE TO COMPARE TWO ALGORITHMS, AND WHY?"

Comparisons, because it is the only one with the same definition in all six generators: one evaluation of `a.val ? b.val`. Passes and exchanges are structurally different per algorithm — a "pass" is an outer-loop iteration in bubble sort, a recursion node in merge sort, and an extraction in heap sort. Comparing those is comparing units.

INTERVIEW QUESTION
"SO YOUR COMPARISON FEATURE IS FAIR?"

It is fair in the sense that both algorithms did the same kind of thing the same number of times on the same input, and the count reaches the end of the complete run. It is not a performance verdict, and the UI says so in two places. I'd say it is honest rather than fair — and the difference is that I've disclosed what it doesn't mean instead of letting the meter bars imply a ranking.

INTERVIEW QUESTION
"HOW WOULD YOU MAKE THE COUNTERS MORE RIGOROUS?"

Three steps, in order of value: first, move every increment inside the guard that performs the actual comparison, so heap sort stops double-counting leaves; second, rename `swaps` per generator (or add a separate `writes` field for merge sort) so the field's meaning is explicit rather than "whatever this generator counts"; third, add per-algorithm metadata declaring what each counter means, and render that declaration next to the number. All three are data changes, not architecture changes. [PROPOSED]

INTERVIEW QUESTION
"IS `CURRENTSTATS` A COPY OR A REFERENCE?"

A copy — `{...stats}` on every frame push. If it were a reference, every frame would display the final counters, because the object is mutated in place throughout the run. This is the second of the two clones that make the frame model correct. [VERIFIED]

CHECKLIST
RED-LINE DRILLS FOR PARTS XII–XIV

- ■ State the four disclosures the UI already makes, from memory.
- ■ Compute `compareCursor` for A = 100, B = 40 at primary cursor 25 and 75.
- ■ Say the "percentage ≠ time" sentence and the "frame count ≠ runtime" sentence.
- ■ Quote the merge-vs-bubble frame counts (375 vs 1,154) as proof that frame count is not speed.
- ■ Name the two counter quirks (heap double-count, merge write-counting).
- ■ Explain why `comps` is the only comparable counter.

PARTS XV–XVI

Persistence and portability

localStorage, the six-experiment cap, and a JSON format with a version field and four validation gates.

CHAPTER 41

41 · Browser-local experiment saves

What is persisted, where, how it is read back, and every way it can fail.

41.1 The storage contract

src/lab/SortingLab.jsx · lines 181–197 (verbatim)

```
  function save() {
    const experiment = {
      version: 1,
      algorithm: algo,
      input: [...input],
      cursor,
      createdAt: new Date().toISOString(),
    };
    const next = [experiment, ...saved].slice(0, 6);
    try {
      localStorage.setItem("algoviz.experiments.v1", JSON.stringify(next));
      setSaved(next);
      setNotice("Saved on this device.");
    } catch {
      setNotice("Browser storage is unavailable. Use Export instead.");
    }
  }
```

src/lab/SortingLab.jsx · lines 77–92 (verbatim)

```
  const [saved, setSaved] = useState(() => {
    try {
      const data = JSON.parse(
        localStorage.getItem("algoviz.experiments.v1") || "[]",
      );
      return Array.isArray(data)
        ? data
            .filter(
              (x) => x && Object.hasOwn(algorithms, x.algorithm) && Array.isArray(x.input),
            )
            .slice(0, 6)
        : [];
    } catch {
      return [];
    }
  });
```

Aspect	Value
Storage mechanism	`localStorage` — synchronous, string-only, per-origin
Key	`algoviz.experiments.v1` — the `.v1` suffix matches the `version: 1` field in the records
Shape	A JSON **array** of experiment objects
Record fields	`version`, `algorithm`, `input`, `cursor`, `createdAt`
Cap	`slice(0, 6)` — the newest six, prepended
Read strategy	Lazy `useState` initialiser, parsed once per mount
Failure handling	`try/catch` around both read and write

MUST KNOW
THE COMPLETE EXPERIMENT RECORD, FIELD BY FIELD

```
{
  version: 1,                                  // schema version, checked on import
  algorithm: "bubble",                         // a key of the `algorithms` registry
  input: [38, 66, 12, 85, 24, 57, 43, 19],     // the committed dataset — a copy via [...input]
  cursor: 42,                                  // the frame the user was looking at
  createdAt: "2026-09-29T14:12:31.004Z"        // ISO-8601 timestamp
}
```

What is deliberately absent: the frames, the algorithm implementation, any engine version identifier, the counters, the comparison algorithm. The record stores what to re-run, not the result. That is the right call — it keeps records tiny (a few hundred bytes instead of ~1 MB), and it means a saved experiment re-runs the current algorithm code. The cost is the one the README admits: a future change to frame ordering would change the meaning of a stored cursor.

41.2 The read path, step by step

```
localStorage.getItem("algoviz.experiments.v1")
     |
     |  null?  ->  "[]"      (so JSON.parse never sees null)
     v
JSON.parse(...)              -- can throw -> catch -> return []
     |
     v
Array.isArray(data) ?
     |
     |-- no  -> []           (valid JSON, wrong shape: an object, a number, a string)
     |
     |-- yes -> data
     |            .filter(x => x && Object.hasOwn(algorithms, x.algorithm) && Array.isArray(x.input))
     |            .slice(0, 6)
     v
saved  (component state)
     |
     v
rendered as a list of buttons; clicking one re-validates through parseInput
```

MUST KNOW
FOUR INDEPENDENT DEFENCE LAYERS ON THE READ PATH — AND WHY EACH IS NECESSARY

1. **`|| "[]"`** — protects against the missing key, the single most common case (first visit).
2. **`try/catch`** — protects against malformed JSON. `localStorage` is user-writable text; `{"a":` is a legal string to store and an illegal one to parse.
3. **`Array.isArray`** — protects against well-formed JSON with the wrong structure. `"null"`, `"42"`, `"{}"` all parse successfully and would each break the subsequent `.filter`.
4. **`.filter(...)`** — protects against entries whose `algorithm` no longer exists in the registry. This matters because saved data outlives code: if an algorithm were renamed or removed in a future version, a stored record would reach `algorithms[item.algorithm].name` on the render path and throw. The filter turns "the app crashes on load for one user forever" into "the stale entry silently disappears".

There is a fifth layer at the click site: `parseInput(item.input.join(","))` inside a `try/catch` that sets "This saved experiment is invalid." So even an entry that survives the filter is re-validated before it can reach state.

41.3 The write path, and the two failure modes it handles

`src/lab/SortingLab.jsx · lines 189–196 (verbatim)`

```
    const next = [experiment, ...saved].slice(0, 6);
    try {
      localStorage.setItem("algoviz.experiments.v1", JSON.stringify(next));
      setSaved(next);
      setNotice("Saved on this device.");
    } catch {
      setNotice("Browser storage is unavailable. Use Export instead.");
    }
```

Concern	How it is handled
`input` mutation	`[...input]` — the record gets a copy, not the live array
Ordering	Newest first, via `[experiment, ...saved].slice(0, 6)`
The in-memory list and storage getting out of sync	`setSaved(next)` is called inside the `try`, after the successful `setItem`, so state only advances if the write actually succeeded
Quota exceeded / storage disabled	Caught; the user is told "Browser storage is unavailable. Use Export instead." — which is the correct remediation, since export does not depend on storage
Silent data loss on read	A malformed store returns `[]` and the UI shows the empty state; the user is not told their data was discarded. [WEAK — see below]

WEAKNESS / LIMITATION

THREE HONEST LIMITATIONS OF THE SAVE FEATURE

1. **A corrupt store is silently discarded.** If `JSON.parse` throws, the user sees an empty list with no explanation, and the next save overwrites the (possibly recoverable) data. A message like "Saved experiments on this device could not be read" would be more honest. [PROPOSED]
2. **No migration path.** The `version` field exists and is checked on import, but the read path does not consult it — it only checks that `algorithm` is known and `input` is an array. A future v2 record read by v1 code would be filtered by the algorithm check rather than migrated. [VERIFIED]
3. **Six is a hard cap.** The seventh save silently drops the oldest. That is a deliberate space bound, but there is no warning and no "archive" action — the only escape is Export. [VERIFIED]

41.4 Reload behaviour, precisely

Action	Result
Save, then reload	The list is restored from `localStorage`; the visualisation resets to the default input and algorithm, because those are `useState` initial values, not persisted
Click a restored entry	`parseInput` re-validates, `load()` sets `input`/`text`/`algo` and dispatches `load` for the new trace, then `seek(item.cursor)` jumps to the saved frame
The saved cursor's meaning	The frame index, not the frame content. For the same input and an unchanged generator, index 42 is the same frame. If the generator changed in a later release, index 42 could be a different moment — this is exactly the caveat the README documents
Two tabs open	Each has its own `saved` state. Saving in tab A writes to the shared store, but tab B does not re-read it. On tab B's next successful save, it writes its own list — and if it had loaded before tab A saved, it will overwrite tab A's entry. **There is no `storage` event listener and no conflict resolution.** [VERIFIED as an absence; it is a genuine multi-tab limitation]

41.5 The persistence questions

INTERVIEW QUESTION
"WHY NOT A BACKEND?"

"Because the feature doesn't need one, and a backend would add trust problems that don't currently exist. Nothing here needs to be shared, synchronised, or durable beyond one browser — a user saves an experiment to come back to it. A server would mean accounts or anonymous identifiers, a database schema and its migrations, an upload path that has to validate untrusted JSON server-side, rate limiting, and a privacy story. And it would make the app unusable offline, which it currently isn't. The README states the scope explicitly: 'There is no application backend, authentication, cloud database, AI service, or arbitrary user-code execution.'"

INTERVIEW QUESTION
"WHY NOT POSTGRESQL?"

"PostgreSQL is the right answer to 'I have relational data that must be durable, queryable and shared' — none of which is true here. My data is a handful of small documents, owned by one browser, never queried by anything but its own owner, and never read concurrently. There's no relationship to model: an experiment belongs to the device, not to a user, because there are no users. Using a database would mean adding a server, a connection, a schema, a migration story and an operations burden to store six JSON records — and the record is already a portable JSON document, which is why export/import works with no server at all."

INTERVIEW QUESTION
"CAN ANOTHER DEVICE ACCESS IT?"

"No. `localStorage` is scoped to an origin on a device — it isn't even shared across browsers on the same machine. That's why export exists: the JSON file is the only cross-device path, and it's a manual one. If synchronisation were a requirement, I'd have to add a server and an identity, and then I'd also have to answer what happens when two devices edit the same experiment."

That last clause is the senior answer: it names the problem that arrives with the feature, not just the missing feature.

INTERVIEW QUESTION
"IS LOCALSTORAGE SECURE?"

"No, and it's important to be precise about what that means. It provides confidentiality from nothing — any JavaScript running on the origin can read it, its contents are visible in DevTools, and it isn't encrypted at rest by the browser in any meaningful sense. It provides a small amount of integrity protection by accident: you cannot read it from another origin, and `localStorage` is not sent with requests, so it isn't exposed to CSRF or to network interception.

For this project the data is non-sensitive — an array of integers and a frame index that the user typed. The realistic threats are (a) the user tampering with their own saved data, which is handled by re-validating on read and again on click, and (b) any XSS on the origin being able to read or rewrite experiments — and for an app with no accounts and no secrets, an attacker reading your sorting history gains nothing. The one thing I'd never do is store a token or a secret there, and nothing in this codebase does."

INTERVIEW QUESTION
"WHAT IF THE USER CLEARS STORAGE?"

"Their saved experiments are gone, silently — there's no warning and no recovery. `localStorage` is explicitly not durable: browsers clear it on 'clear browsing data', in some private-browsing modes it may not persist at all, and under storage pressure the browser may evict it. The mitigation is already in the product: the Export button produces a portable JSON file, and the README says so — 'Export provides the portable backup.' The honest framing is that Export is the primary storage mechanism and localStorage is a convenience cache."

INTERVIEW QUESTION
"WHAT IF THE STORED JSON IS MALFORMED?"

"Four things could happen and all four are handled. If `getItem` returns `null`, I substitute `'[]'`. If the string doesn't parse, the `try/catch` returns an empty list. If it parses to a non-array, `Array.isArray` rejects it. If it's an array of objects that don't match my shape, `.filter` drops them — checking the record exists, that its `algorithm` is a currently known key, and that `input` is an array. And if a record survives all of that but has a bad value, the click handler re-validates through `parseInput` and reports 'This saved experiment is invalid.' So the boundary is: anything from storage is untrusted input, and it is validated twice before it can reach application state."

INTERVIEW QUESTION
"COULD A CRAFTED LOCALSTORAGE VALUE EXECUTE CODE IN YOUR APP?"

"No — there's no code path from stored data to evaluation. The stored fields are used as: a key looked up in a frozen set (`Object.hasOwn`), an array passed through `parseInput` (parsed with a regex and `Number`), a number passed to `Number.isInteger`, and a string rendered as a timestamp. I never `eval`, never construct a function, never build a URL from it, and React escapes the rendered string. The worst a crafted record can do is fail validation. That said, one thing I'd change: `createdAt` is never rendered anywhere, so the field is currently dead weight — and storing anything I don't use is a bad habit, because it's data I'd have to defend without having a purpose for it." [VERIFIED — `createdAt` is written and never read]

CHAPTER 42

42 · JSON import and export

A versioned, size-bounded, four-gate import path — and fifteen questions about it.

42.1 The export path

src/lab/SortingLab.jsx · lines 49–58 (verbatim)

```
function download(data) {
  const url = URL.createObjectURL(
    new Blob([JSON.stringify(data, null, 2)], { type: "application/json" }),
  );
  const a = document.createElement("a");
  a.href = url;
  a.download = "algoviz-experiment.json";
  a.click();
  setTimeout(() => URL.revokeObjectURL(url), 1000);
}
```

Step	Detail
Payload	`{version: 1, algorithm: algo, input, cursor}` — note the export payload is smaller than the saved record (no `createdAt`)
Serialisation	`JSON.stringify(data, null, 2)` — pretty-printed for human readability
Blob	`new Blob([json], {type: "application/json"})`
URL	`URL.createObjectURL(blob)` — a `blob:` URL valid only in this document
Download	A detached `<a>` with `download="algoviz-experiment.json"`, clicked programmatically
Cleanup	`URL.revokeObjectURL(url)` after 1,000 ms

MUST KNOW

WHY THE EXPORT PAYLOAD DIFFERS FROM THE SAVED RECORD — AND THE TRAP IN IT

The saved record has `createdAt`; the export does not. So a saved experiment and an exported experiment carry different field sets under the same `version: 1`. That is defensible — `createdAt` is device-local metadata with no use in a portable file — but it means the version number does not describe one canonical shape. If someone later added a required field, `version` would be the place to signal it. Worth knowing before you are asked "is your schema consistent?"

Also note: **`export` does not include the frames, the counters, or any engine identifier.** So an exported file describes how to reproduce a run, not the run itself. If the algorithm code changes, the same file produces a different trace — which is precisely the caveat the README documents about cursor meaning.

42.2 The import path, gate by gate

src/lab/SortingLab.jsx · lines 156–180 (verbatim)

```
async function importExperiment(event) {
  const file = event.target.files?.[0];
  event.target.value = "";
  if (!file) return;
  try {
    if (file.size > 16384)
      throw new Error("Experiment files must be smaller than 16 KB.");
    const data = JSON.parse(await file.text());
    if (
      data?.version !== 1 ||
      !Object.hasOwn(algorithms, data.algorithm) ||
      !Array.isArray(data.input)
    )
      throw new Error("Not a supported AlgoViz experiment.");
    const values = parseInput(data.input.join(","));
    load(values, data.algorithm);
    dispatch({
      type: "seek",
      cursor: Number.isInteger(data.cursor) ? data.cursor : 0,
    });
    setNotice("Experiment imported. Ready to inspect.");
  } catch (err) {
    setError(err.message);
  }
}
```

Gate	Check	Failure message	What it defends against
0	A file was actually selected, and the input is reset (`event.target.value = ""`)	silent return	An empty selection, and the "select the same file twice does nothing" bug — clearing the value makes the change event fire again for a repeated selection
1	`file.size > 16384` → reject	"Experiment files must be smaller than 16 KB."	Oversized files: a denial-of-service vector against the main thread (`file.text()` and `JSON.parse` both run synchronously) and against the tab's memory
2	`data?.version !== 1`	"Not a supported AlgoViz experiment."	Wrong-format files, other JSON entirely, and future schemas
3	`!Object.hasOwn(algorithms, data.algorithm)`	same message	An algorithm key that doesn't exist — including prototype keys, because `Object.hasOwn` does not walk the prototype chain
4	`!Array.isArray(data.input)`	same message	A non-array input
5	`parseInput(data.input.join(","))`	the message from `parseInput` itself	**Every value-level rule, re-used.** The imported array is flattened to a comma string and run through the exact same validator the UI uses — so the ±999 bound, the 2–24 count rule, the integer regex and the safe-integer check all apply
6	`Number.isInteger(data.cursor) ? data.cursor : 0`	falls back to 0	A missing, non-numeric, or fractional cursor. A fraction would produce `frames[2.5] === undefined` and crash the render

KEY TAKEAWAY

THE DESIGN PRINCIPLE BEHIND THIS PATH

An imported file is untrusted input and gets no privileges. It is not "our own data" even though we produced it, because it may have been edited, hand-written, truncated, corrupted in transit, or produced by a different version. So it is validated through exactly the same function the UI uses (`parseInput`), plus a schema gate, plus a size gate. Nothing about the import path assumes good faith.

MUST KNOW

GATE 5 IS THE STRONGEST THING IN THIS CHAPTER — SAY IT EXPLICITLY

"I don't have a second validator for imports. The imported input is joined into a comma-separated string and passed to `parseInput` — the same function the form calls. That means there is exactly one definition of a valid dataset in the codebase, and the file format cannot drift away from it. The cost is a tiny inefficiency (I serialise then re-parse), and the benefit is that I can't have a validator that's accidentally more permissive than the UI."

42.3 What is NOT validated — the honest gap list

Gap	Consequence
`data` being an array, a string, or `null`	Handled implicitly: `data?.version` is `undefined` for a non-object, and `undefined !== 1`, so gate 2 rejects it. This is correct but relies on the `?.` operator rather than an explicit `typeof` check
Extra unexpected properties	Ignored silently. `{version:1, algorithm:"bubble", input:[1,2], cursor:0, evil:"payload"}` import is accepted and `evil` is simply never used. Not a vulnerability here, but an explicit allowlist of fields would be more rigorous
`cursor` beyond the trace length	Not checked at import time. `dispatch({type:"seek", cursor})` catches it: the reducer clamps to `length - 1`. So the outcome is safe, but the clamping happens in the reducer rather than the validator
JSON with duplicate keys, depth bombs, or huge nesting	Not specifically defended. `JSON.parse` handles depth, and the 16 KB cap bounds the damage. A nested-array `input` would fail `Array.isArray` only if it weren't an array — `[[1,2],[3,4]]` passes gate 4, then `.join(",")` produces `"1,2,3,4"` and imports as `[1,2,3,4]`. **That is a real quirk**: a 2-D array imports as its flattened values. It fails safely (the result is a valid dataset the user can see), but it is not what the schema says. [VERIFIED]
Schema version downgrade	A `version: 2` file is rejected outright rather than migrated, which is the safe default

42.4 Fifteen import/export questions

INTERVIEW QUESTION

1. WHAT DOES THE EXPORTED FILE CONTAIN?

`{version: 1, algorithm, input, cursor}` — pretty-printed with a 2-space indent. No frames, no counters, no engine version. [VERIFIED]

INTERVIEW QUESTION

2. WHY `JSON.STRINGIFY(DATA, NULL, 2)` AND NOT PLAIN `STRINGIFY`?

Readability. The file is meant to be inspectable and hand-editable — you can open it in any editor and see the array. The size cost is irrelevant for a sub-kilobyte document.

INTERVIEW QUESTION

3. WHY IS THE FILE CAPPED AT 16 KB?

To bound a synchronous parse on the main thread, and to keep the import path from being a simple way to hang the tab. A legitimate file is a few hundred bytes, so 16 KB is roughly 50× headroom.

INTERVIEW QUESTION
4. WHERE DOES THE SIZE CHECK HAPPEN?

`file.size > 16384` — before `file.text()` is even awaited, so an oversized file is never read into memory.

INTERVIEW QUESTION
5. WHAT IF THE FILE ISN'T JSON AT ALL?

`JSON.parse` throws, the `catch` sets the error banner to the parser's message. The message is the engine's, not mine — so a malformed JSON file shows something like "Unexpected token < in JSON at position 0". That is honest but not friendly; a nicer version would catch the parse separately and say "This file isn't valid JSON." [PROPOSED]

INTERVIEW QUESTION
6. WHAT IF THE JSON IS VALID BUT IS AN ARRAY?

`data?.version` is `undefined` → `undefined !== 1` → rejected with "Not a supported AlgoViz experiment."

INTERVIEW QUESTION
7. WHAT IF `ALGORITHM` IS `"CONSTRUCTOR"` OR `"__PROTO__"`?

Rejected. `Object.hasOwn(algorithms, "constructor")` is `false`, because `Object.hasOwn` checks own properties only and does not consult the prototype chain. Using `algorithms[key]` alone — or `key in algorithms` — would have been the vulnerable version. This is a good, concrete example of a subtle JavaScript security detail handled correctly. [VERIFIED]

INTERVIEW QUESTION
8. WHAT IF `INPUT` CONTAINS A NESTED ARRAY?

It passes the `Array.isArray` gate, then `.join(",")` flattens it, and the flattened values are validated by `parseInput`. So `[[1,2],[3,4]]` imports as `[1,2,3,4]`. It fails safe — the user sees a valid, correctly validated dataset — but it is more permissive than the schema implies. [VERIFIED]

INTERVIEW QUESTION
9. WHAT IF `CURSOR` IS 5000 AND THE TRACE HAS 100 FRAMES?

It is dispatched as a seek, and the reducer clamps it to 99. Safe, but the clamp lives in the reducer rather than in the validator.

INTERVIEW QUESTION
10. WHAT IF `CURSOR` IS 2.5?

`Number.isInteger(2.5)` is false, so it falls back to 0. [VERIFIED]

INTERVIEW QUESTION
11. WHAT IF `INPUT` HAS 30 VALUES?

`parseInput` throws "Use 2–24 values...", and the error is shown. The file's own values never reach state.

INTERVIEW QUESTION
12. DOES IMPORT OVERWRITE EXISTING DATA?

No. Import only sets `input`, `text`, `algo` and the cursor. It does not touch `saved` — importing a file does not add it to the saved list. To keep it, the user must press Save.

INTERVIEW QUESTION

13. DOES EXPORT INCLUDE THE COMPARISON ALGORITHM?

No. `compare` and `other` are not in the payload, so an exported comparison experiment restores only the primary algorithm and the cursor. The comparison panel would be off unless the URL has `?compare=1`.

INTERVIEW QUESTION

14. COULD THE IMPORT PATH BE USED FOR CODE EXECUTION?

No. The only consumers of the parsed data are `Object.hasOwn` (a property lookup), `Array.isArray`, `Array.prototype.join`, `parseInput` (regex + `Number`), `Number.isInteger`, and `dispatch`. Nothing evaluates a string; nothing builds a URL; React escapes the rendered values. The strongest thing an attacker can do with 16 KB of crafted JSON is make the app display a validation error.

INTERVIEW QUESTION

15. HOW WOULD YOU TEST THE IMPORT PATH?

These are the cases I'd assert, in order of value: (1) valid file → input/algo/cursor restored; (2) `version: 2` → rejected; (3) unknown algorithm → rejected; (4) `input: "abc"` → rejected; (5) `input: [1,1000]` → rejected by the ±999 bound; (6) `input: [1]` → rejected by the 2–24 rule; (7) 20 KB file → rejected by size; (8) `cursor: 9999` → clamped by the reducer; (9) `algorithm: "__proto__"` → rejected. **Of these nine, none is currently automated** — the import path has no test coverage in the repository. That is the single biggest testing gap in the project, and the honest answer is to say so and describe the first three you would write. [VERIFIED absence]

WEAKNESS / LIMITATION

THE ONE-LINE SUMMARY OF THE IMPORT STORY'S WEAKNESS

Everything about the import path is robust but nothing about it is tested. It has six gates and zero assertions. If an interviewer asks "what's the biggest gap in your tests?", this is a better answer than "coverage is low", because it is specific, actionable, and shows you distinguished between validation that exists and validation that is verified.

PARTS XVII–XX

Testing and verification

The real test runner, the 150 cases, trace reproducibility, and the 768 brute-force comparisons.

CHAPTER 43

43 · The testing architecture, as it actually is

No Jest. No Vitest. No Playwright. Node's built-in test runner — and that is a deliberate, defensible choice.

43.1 The mechanism, verified from `package.json`

```
"scripts": {
  "test": "node --test tests/*.test.js"
}
```

Aspect	Reality
Runner	**Node.js built-in test runner** (`node:test`), available since Node 18 and stable in Node 20+
Assertion library	**`node:assert/strict`** — Node's built-in strict assertion module
Test file	One: `tests/engine.test.js`, 140 lines
Test groups	**14** (verified by running: `# tests 14`, `# pass 14`, `# fail 0`)
Dependencies added	**Zero** — there is no test dependency in `package.json`
Watch mode	Not used (`node --test --watch` exists, but no script uses it)
Coverage tooling	None
DOM environment	None. Tests run in plain Node, which is why they can import the generators directly
Snapshot testing	None
Mocking library	None

MUST KNOW

THE SENTENCE THAT MAKES THIS A STRENGTH, NOT A SHORTFALL

"AlgoViz tests with Node's built-in runner and `node:assert/strict` — there is no test dependency in `package.json` at all. That was possible because I deliberately kept the engine framework-free: `engine.js` and the six generators import nothing, not even React. A pure module that takes an array and returns an array is testable by importing it in Node. If I had put the frame generation inside a component, I would have needed a DOM environment, a JSX transformer and a test framework — and I'd have been testing React instead of testing the algorithms."

43.2 What the test file covers, group by group

#	Test group	Assertions	Category
1–6	`<id>: correctness, input preservation and reproducible trace` — one group per algorithm	25 inputs each: sorted output, input unchanged, `deepEqual` against a second run, final-frame ID uniqueness	Unit + property
7	`input validation rejects ambiguous, unbounded and malformed values`	7 `assert.throws` cases + 1 positive case	Unit (boundary)
8	`playback keeps final state, pauses on seek and bounds its cursor`	7 state assertions on the reducer	Unit (state machine)
9	`datasets reproduce from a seed`	Same seed equal; different seeds unequal	Unit (determinism)

#	Test group	Assertions	Category
10	`sliding window regression: at least is not exactly`	3 assertions pinning the corrected behaviour	Regression
11	`bounded differential search finds a reproducible failure`	The search finds a mismatch, is reproducible, and returns exactly `[2]` with target `1`	Differential search
12	`window reference matches brute force within bounded domain`	768 assertions against an in-test oracle	Differential (oracle)
13	`pathfinding is repeatable, preserves input and reports unreachable goals`	BFS and DFS paths, input preservation, a known 5-node path, and an unreachable case	Integration
14	`prefix sum validates ranges and handles full-range query`	A correct full-range sum and an invalid range	Unit (validation)

43.3 Unit vs integration vs end-to-end — what exists here

Layer	Exists?	Evidence
Unit tests	**Yes**	`parseInput`, `playbackReducer`, `makeDataset`, `windowTrace`, `generatePrefix` are all called directly
Property-style tests	**Yes, in a limited form**	20 generated inputs per algorithm with assertions that must hold for all of them. This is a hand-rolled property test without a framework (no shrinking, no random generation — the inputs come from a fixed seed list)
Differential / oracle tests	**Yes**	The sliding-window implementation checked against an independent brute-force implementation over 768 cases
Regression tests	**Yes**	The `at least is not exactly` group pins a previously-fixed bug
Integration tests	**Partially**	The pathfinding test assembles a grid and runs the full entry point; the sorting tests run the complete generator
Component tests	**No**	No test renders a React component; no `react-dom`, no `@testing-library`, no DOM environment
End-to-end / browser tests	**No, not in the repository**	No Playwright, Cypress, Puppeteer or Selenium dependency exists. The browser verification described in `docs/VERIFICATION.md` was performed manually with external tooling, and the README is explicit that it is "not a repository-contained automated browser suite"
Accessibility tests	**No**	`docs/VERIFICATION.md` states it is not a WCAG certification
Visual regression tests	**No**	Documented as not performed

DANGER ZONE — WHERE CANDIDATES DIE

THE THREE SENTENCES THAT MAKE THIS ANSWER SAFE

1. "There is no browser automation in the repository, and I don't claim there is." [VERIFIED]
2. "The browser checks in `docs/VERIFICATION.md` were run manually with headless Chromium at 1440 px and 390 px, outside the app's stack — they're smoke checks, not a suite." [VERIFIED — that document says exactly this]
3. "What the repository automates is the engine: 14 groups covering sorting correctness, reproducibility, playback transitions, validation, seeded datasets, the window oracle, the bounded search, pathfinding and prefix sums."

43.4 What the tests deliberately do not cover, and why

Not covered	Reason	Risk it leaves
`SortingLab.jsx` (596 lines)	Would need a DOM environment and a component testing library	The timer effect, the keyboard effect, save/import/export and the derived cursor clamp are all untested
The import/export path	Same	Six validation gates with zero assertions — the project's biggest test gap
`Counterexample.jsx`, `Overview.jsx`, `Shell.jsx`, `ModuleFrame.jsx`	Presentation	Low algorithmic risk
The twelve legacy module components	Would require simulating `async` timing	Their logic modules are partly covered (pathfinding, prefix sum)
Cross-module interactions	Not in scope	The route/URL behaviours (`?algorithm=`, `?compare=1`)
Performance	No benchmark harness	The 24-element cap makes it moot, and the README says it is not a benchmark platform

INTERVIEW QUESTION
"YOUR TEST SUITE HAS NO COMPONENT TESTS. WHY?"

"Because I'd have needed to add a DOM environment and a component testing library, and I made a judgement that the value was highest at the engine layer. The engine is where correctness lives — if a trace is wrong, the app is wrong regardless of how it renders, and a wrong trace is invisible: the UI would faithfully show you the wrong thing. The component layer is mostly presentation plus two effects, and I covered the effects by reasoning about them plus manual browser checks. I'm not claiming that's sufficient — save/import has six validation gates and no test, and that's the first thing I'd add, along with a test for the playback effect using fake timers."

INTERVIEW QUESTION
"WOULD YOU ADD A TEST FRAMEWORK NOW?"

"Only if I was adding component tests, and I'd look at Vitest first because the project is already on Vite — it reuses the same transform pipeline and config, so the marginal setup is small. For the engine tests I wouldn't change anything: `node --test` costs zero dependencies, runs in under a second, and reports TAP output that CI reads without configuration. Adding Jest to test pure functions would be adding 300 packages to prove that `[3,1,2].sort()` works." [PROPOSED]

CHAPTER 44

44 · The 150 sorting cases

Where the number comes from, what is asserted, and exactly how much it proves.

44.1 The generator of the 150

`tests/engine.test.js · lines 12–36 (verbatim)`

```
for (const [id, algorithm] of Object.entries(algorithms)) {
  test(`${id}: correctness, input preservation and reproducible trace`, () => {
    for (const input of [
      [],
      [1],
      [2, 2, 1],
      [-4, 0, 3, -1],
      [5, 4, 3, 2, 1],
      ...Array.from({ length: 20 }, (_, seed) => makeDataset("random", seed)),
    ]) {
      const before = [...input],
        frames = algorithm.run(input);
      assert.deepEqual(
        frames.at(-1).array.map((x) => x.val),
        [...input].sort((a, b) => a - b),
      );
      assert.deepEqual(input, before);
      assert.deepEqual(frames, algorithm.run(input));
      assert.equal(
        new Set(frames.at(-1).array.map((x) => x.id)).size,
        input.length,
      );
    }
  });
}
```

Component	Count	Detail
Algorithms	**6**	`bubble, selection, insertion, merge, quick, heap` — iterated with `Object.entries(algorithms)`
Inputs per algorithm	**25**	5 hand-written + 20 generated
Total algorithm-input cases	**150**	6 × 25
Assertions per case	**4**	Output correctness, input preservation, trace determinism, final ID uniqueness
Total assertions in this group	**~600**	150 × 4

44.2 The 25 inputs, broken down by category

#	Input	Category	What it exercises
1	`[]`	Empty	Every loop guard; the `n = 0` path in all six generators
2	`[1]`	Single element	Base cases: quick sort's `low >= high`, merge sort's `low >= high`
3	`[2, 2, 1]`	Duplicates	Equal keys, stability-relevant behaviour, `minIndex` not changing in selection sort
4	`[-4, 0, 3, -1]`	Negatives and zero	Comparisons on negative values; the UI's `Math.abs` for height; the `-?` branch of the validation regex
5	`[5, 4, 3, 2, 1]`	Reverse-sorted	The worst case for bubble and quick (last-element pivot)
6–25	`makeDataset("random", seed)` for `seed = 0…19`	20 seeded random datasets of exactly 10 values each	Bulk coverage of ordinary inputs, all reproducible

MUST KNOW

THE PRECISE CLAIM, IN ONE SENTENCE

"150 is 6 algorithms × 25 inputs. Five inputs are hand-picked edge cases — empty, single, duplicates, negatives and zero, and reverse-sorted — and the other twenty are seeded random datasets of ten values each, generated deterministically from seeds 0 through 19. Because the same seed produces the same array, all six algorithms are tested on the same twenty-five arrays, which means the six groups are comparable to each other."

44.3 The four assertions, precisely

```
FOR each algorithm in {bubble, selection, insertion, merge, quick, heap}:
  FOR each input in the 25:
     before := copy of input
     frames := algorithm.run(input)

     (1) SORTED OUTPUT
         assert.deepEqual(
            frames.at(-1).array.map(x => x.val),
            [...input].sort((a,b) => a - b)
         )
         -> the final frame holds the correctly sorted values
         -> note: compares values only, using the same comparator as the oracle

     (2) INPUT PRESERVATION
         assert.deepEqual(input, before)
         -> the generator did not mutate the caller's array
         -> this is what makes "same input" in comparison mode structurally true

     (3) TRACE REPRODUCIBILITY
         assert.deepEqual(frames, algorithm.run(input))
         -> a second run produces a deep-equal trace, frames and metadata included
         -> this is the strongest assertion in the suite: it pins the whole recording

     (4) FINAL ID UNIQUENESS
         assert.equal(new Set(frames.at(-1).array.map(x => x.id)).size, input.length)
         -> every original element appears exactly once in the final frame
         -> catches dropped, duplicated or merged elements
```

KEY TAKEAWAY

WHY ASSERTION (3) IS THE MOST INTERESTING

`deepEqual` on `frames` compares the entire recording: every frame, every field — the array contents and order, the ids, `activeIndices`, `sortedIndices`, `isSwapping`, `minimumIndex`, `codeLine`, `currentStats` (all three counters at that moment) and `msg` (the exact string, with its interpolated values). If a generator used `Math.random()`, a `Date`, a `Set` iteration order that varied, or an object key order that depended on insertion path, this assertion would fail.

So the assertion is really a **test of determinism as a property**, not just "the output matches". That is the difference between a sorting test and a trace test — and it is the part that is specific to this project. [VERIFIED]

44.4 Attack: "Does 150 prove correctness?"

MODEL ANSWER

THE NUANCED ANSWER — MEMORISE THIS SHAPE

"No. And I'd be suspicious of anyone who said it did. What 150 cases prove is that the final frame of every trace is correctly sorted, that the generators don't mutate their input, that the traces are reproducible, and that no element is lost or duplicated — for those 150 algorithm-input pairs. What they cannot prove is anything about the infinitely many other inputs.

The honest framing has three parts. **First, what it does prove:** correctness on a stated set, plus four properties — sortedness, purity, determinism, and element conservation — and the property assertions are the more valuable half, because they generalise beyond the specific values even though they don't generalise beyond the tested sizes.

Second, what it's weak at. Coverage is thin exactly where bugs live in sorts: the twenty random datasets are all ten values drawn from 5–94, so the suite tests almost nothing at n = 2 or 3 (where off-by-one errors actually occur), nothing with mass duplicates beyond `[2,2,1]`, and nothing at 24 elements — the app's own maximum. The `duplicates` preset that the UI offers is not in the test inputs at all, except as `[2,2,1]` and seed 7's dataset, which happens to contain a repeated value. Sizes above 10 are exercised only by the hand-written 5-element and 4-element cases.

Third, what would strengthen it: add inputs of every length from 2 to 24 rather than 4 distinct lengths; add an all-equal array like `[5,5,5,5]` and a mostly-equal array; add the `duplicates` preset; and add a property test that generates random arrays of random length 2–24 from a seeded generator, so the sizes are not fixed. The cheapest single improvement is one line — include `makeDataset("duplicates", seed)` in the input list — because the duplicates path is where stability and `minIndex` bugs appear, and it is one of the four presets the product advertises."

Then close with the boundary sentence: **"A finite test suite demonstrates the presence of correctness, never its absence. Testing shows that bugs are present, not that they are absent — and with 150 cases in a space that has more than 10^{50} arrays of length 24, the coverage rate is a rounding error, which is exactly why the assertions are properties rather than a golden-output file."**

TRAP

THE FOLLOW-UP TRAP: "THEN WHY 150 AND NOT MORE?"

The bad answer is "more would be better, I just didn't have time." The good answer: "Because at $n \leq 24$ a generator can run the whole space for small sizes if I want — 4^3 arrays of length 3 with values 1–4 is 64 exhaustive cases, which is what I did for the sliding window. The reason the sorting group is 150 rather than exhaustive is that the exhaustive space explodes: all arrays of length 24 with distinct values is $24! \approx 6\times10^{23}$. So for sorting I chose breadth through seeded sampling and depth through properties. The place where exhaustive testing was actually feasible — the window problem with values 1–4 and length 3 — I did exhaustively. That's the same reasoning a test designer should apply, and it's the reason I can point at two different coverage strategies in one suite and explain why each fits its problem."

CHAPTER 45

45 · Trace reproducibility

The property that makes a frame-based tool trustworthy — and that an ordinary sorting function never needs.

45.1 The concept, precisely

```
run(input) -> trace A
run(input) -> trace B
A must be deep-equal to B

deep-equal means: every frame, every field, every nested value —
array order, element ids, values, activeIndices, sortedIndices,
isSwapping, minimumIndex, pivotIndex, subArrayRange, codeLine,
currentStats.{passes,comps,swaps}, and every character of msg.
```

45.2 Why this matters more here than in a sorting utility

In a sorting utility	In AlgoViz
You observe one thing: the output array	You observe every intermediate state
The output is a permutation — easy to check independently by comparison	Intermediate states have no external oracle; the recording is the definition of correct behaviour
Non-determinism that still produces the correct output is usually harmless	Non-determinism in intermediate states means the user sees different decisions for the same input
Determinism is a nice property	**Determinism is the product promise**: "reproducible experiments" is in the page metadata, the README, and the save/export feature

MUST KNOW

THE SHARPEST WAY TO PUT IT

"If the final array is right but the intermediate frames differ between runs, then the visualisation is lying about the algorithm's behaviour — the learner would see one thing and someone else would see another for the same input and the same code. And my saved experiments store a frame index: if frame 42 means a different moment on the next run, my saved and exported files become meaningless. So reproducibility isn't a nice-to-have in this project, it's the reason the save format can store just a cursor instead of a whole trace."

45.3 What could break reproducibility — the checklist

Source of non-determinism	Present in AlgoViz?	Evidence
`Math.random()` in a generator	**No**	The generators contain no `Math.random()`; `makeDataset` uses a seeded LCG instead
`Date.now()` / timestamps in a frame	**No** in frames. `createdAt` exists only in the saved record, which is not part of the trace	`new Date().toISOString()` appears once, in `save()`
Object key iteration order	**No** — frames are object literals with a fixed key order, and `deepEqual` on objects is order-insensitive in Node's strict module for `deepStrictEqual`... but the string comparison of `msg` is order-sensitive and the messages are pure templates	—
`Set` / `Map` iteration	**No** — neither appears in the generators	—

Source of non-determinism	Present in AlgoViz?	Evidence
Floating point	**No** — inputs are integers and the only arithmetic is comparison; `makeDataset` uses integer LCG arithmetic with `Math.imul` and `>>> 0`	—
Mutating shared state between runs	**No** — each `run()` allocates its own `arr`, `frames`, `stats` and `sortedIndices`. There is no module-level mutable state in any generator, so two calls cannot interfere even if interleaved	—
Reading the clock or environment	**No**	—
Iteration order of an object being sorted	**No**	—

KEY TAKEAWAY

WHY THE TEST IS ABLE TO BE THIS STRICT

Because every input to a frame is either a number, a string from a fixed template, a boolean, or a freshly allocated literal. There is no ambient state, no I/O, no clock. So `deepEqual` is not testing "the same thing happened twice" — it is testing "the function is a pure function of its input", which is the property a test can establish. If a generator read the clock, the test would fail — and that is exactly the value of writing it.

45.4 The question in this area you should expect

INTERVIEW QUESTION

"WHAT IF THE FINAL OUTPUT MATCHES BUT THE INTERMEDIATE FRAMES DIFFER?"

"That would be a genuine bug that only the trace assertion catches — and it is a bug this project would actually care about, unlike a sorting library. Two cases: if the difference is in the data (a different sequence of swaps that happens to sort correctly), then the visualisation teaches a different algorithm than the code under test. If the difference is in the metadata — say `activeIndices` or the counters at some frame — then the explanations attached to the frames are wrong. Both are user-visible, and neither is caught by asserting only on the final array.

That's the reason the test asserts `deepEqual(frames, algorithm.run(input))` rather than `deepEqual(finalValue, sorted)`. The final-value assertion is one line; the trace assertion is what makes the recording a testable artefact. And it's the assertion that would catch the two things most likely to sneak into this codebase: a `Math.random()` used for a nice-looking jitter, or a frame that captured a mutable reference to something still being modified."

Then add the boundary, because it is the part that shows depth: "What the assertion does not establish is that the trace is the same trace for a different engine version, or across Node versions, or after I refactor a generator. It establishes determinism within one implementation, not stability of the recording across implementations — that's the caveat that makes my saved cursor potentially mean something different after a code change."

CHAPTER 46

46 · The 768 sliding-window cases and the brute-force oracle

The differential test, explained end to end — including why an independent implementation is the point.

46.1 The problem under test

Find the length of the **shortest contiguous subarray whose sum is at least a target**, or 0 if none exists.

Both implementations in AlgoViz expand and shrink a window over positive numbers:

- **Reference (correct)** — records every window whose sum satisfies `sum >= target`.
- **Buggy variant (deliberate)** — records only windows whose sum is exactly `target`, silently ignoring larger sums.

The buggy variant lives inside `windowTrace` behind a `buggy` flag, so both behaviours are produced by the same function and can be executed side by side. [VERIFIED in `engine.js` lines 121–151]

46.2 The 768 cases

`tests/engine.test.js · lines 82–101 (verbatim)`

```
test("window reference matches brute force within bounded domain", () => {
  for (let code = 0; code < 64; code++) {
    const a = [
      (code % 4) + 1,
      (Math.floor(code / 4) % 4) + 1,
      (Math.floor(code / 16) % 4) + 1,
    ];
    for (let target = 1; target <= 12; target++) {
      let best = Infinity;
      for (let l = 0; l < a.length; l++) {
        let sum = 0;
        for (let r = l; r < a.length; r++) {
          sum += a[r];
          if (sum >= target) best = Math.min(best, r - l + 1);
        }
      }
      assert.equal(windowTrace(a, target).result, best === Infinity ? 0 : best);
    }
  }
});
```

Component	Count	Detail
Arrays	**64**	All combinations of three values, each in `{1,2,3,4}` — that is $4^3 = 64$, **exhaustive** over that domain
Targets	**12**	`1` through `12`
Total cases	**768**	64 × 12

How the array is built — the digit-extraction trick:

Iteration	Digit produced	Expression
code % 4	1st value (least significant)	`(code % 4) + 1`
⌊code / 4⌋ % 4	2nd value	`(Math.floor(code / 4) % 4) + 1`
⌊code / 16⌋ % 4	3rd value	`(Math.floor(code / 16) % 4) + 1`

So `code = 0` → `[1,1,1]`, `code = 1` → `[2,1,1]`, `code = 63` → `[4,4,4]`. Every array of length 3 with values from 1–4 appears exactly once. [VERIFIED]

46.3 What an oracle is

MUST KNOW
DEFINITION

An **oracle** is an independent mechanism that can decide whether a given answer is correct — without using the implementation being tested. The word means "something that gives the right answer", not "a magic solution". In testing, the oracle is usually the slow, obviously-correct implementation: brute force, a reference model, or a specification.

The rule that makes an oracle useful: **it must not share code with the implementation under test.** If the oracle uses the same helper functions, a bug in a helper infects both sides and the test passes while both are wrong. This is called correlated failure, and it is the standard way differential tests give false confidence.

46.4 The oracle, line by line

```
let best = Infinity;
for (let l = 0; l < a.length; l++) {        // every start position
  let sum = 0;
  for (let r = l; r < a.length; r++) {      // every end position >= l
    sum += a[r];                            // running sum of a[l..r]
    if (sum >= target) best = Math.min(best, r - l + 1);
  }
}
// expected = best === Infinity ? 0 : best
```

Property	Why it matters
Time complexity **O(n³)** as written (O(n²) subarrays, and the running-sum accumulation makes counting the inner work O(n) per start — overall O(n³) worst case; here n = 3, so at most ~27 additions per case)	Deliberately naive; clarity over speed
It enumerates **every** contiguous subarray	Nothing can be missed by a windowing error, because there is no window
It is written **in the test file**, not imported	Zero shared code with `windowTrace` — which is what makes this a real oracle
It uses the same condition `sum >= target`	The specification is "at least", so the oracle must express "at least". This is the one thing the oracle and the implementation must agree on, because it is the requirement
It returns 0 for "no qualifying window"	Matches the implementation's `mini === Number.MAX_VALUE ? 0 : mini` convention

DANGER ZONE — WHERE CANDIDATES DIE
THE CRITICAL DISTINCTION THAT EARNS YOU THE POINT

The oracle must share the **specification** with the implementation but not the **mechanism**.

It is correct — indeed essential — that both use `sum >= target`, because that is the definition of the problem. If the oracle used `sum > target`, the test would fail on the boundary case where the sum equals the target, and it would be the oracle that was wrong.

What must not be shared is the how: the reference walks every window with two nested loops and a running sum; the implementation slides a single window with a shrinking left pointer. Those are genuinely different algorithms. So a bug in the sliding logic — an off-by-one in the shrink condition, forgetting to subtract the departing element, comparing with `===` instead of `>=` — cannot appear in the oracle, because the oracle has no sliding logic at all.

46.5 Why brute force, specifically

Reason	Explanation
Obviousness	Nested loops enumerating every subarray are almost impossible to get subtly wrong. The value of an oracle is inversely proportional to its cleverness
Feasibility	At n = 3 and targets 1–12 the brute force is trivially cheap: 768 cases complete in ~2 ms (measured: the whole test file runs in 226 ms)
It defines the answer independently	The expected value is computed, not hard-coded. So the test does not need a maintainer to know the right answer for each of the 768 cases
It generalises the assertion	If someone changes the target range from 1–12 to 1–50, the test still works — the oracle recomputes the truth rather than consulting a stored list

46.6 The assumptions the sliding window requires

MUST KNOW

THE CORRECTNESS PRECONDITION OF THE WINDOW TECHNIQUE

All values must be non-negative (here: strictly positive). The reason is the monotonicity that makes the technique valid:

When you move `right` forward, the sum **increases** — so once the sum reaches or exceeds the target, adding more elements can only keep it ≥ target. That means the current window is a candidate, and the only way to find something shorter is to shrink from the left. That logic is the entire algorithm.

The shrink step relies on the mirror property: when you remove `arr[left]`, the sum strictly **decreases**, so eventually it falls below the target and you must expand again. The two-pointer walk is therefore monotone — `left` and `right` each advance at most n times, giving O(n).

With a negative value, the increase is not monotone. Adding an element can reduce the sum, so "shrink while sum ≥ target" can terminate while a shorter qualifying window still exists to the right — and the walk will miss it, because the left pointer only ever moves forward and can never come back to sit on a later element alone.

I ran this against the real implementation to be sure rather than asserting it:

Input	Target	`windowTrace` result	Brute-force result	Agreement
`[2, 3]`	4	2	2	✔ (positive input)
`[-1, 3, 2]`	3	**2**	**1**	**✘ disagrees**
`[1, -1, 1, 1]`	2	**4**	**2**	**✘ disagrees**
`[3, -1, 2]`	3	1	1	✔ (by luck)

Walking `[-1, 3, 2]` with target 3: `right` reaches index 1 with a running sum of 2 — below target, so no shrink. `right` reaches index 2, the sum becomes 4 ≥ 3, and the window `[0..2]` is recorded as length 3; then the left pointer advances and the window `[1..2]` sums to 5, recorded as length 2. The correct answer is length **1** — the single element `3` at index 1 — but the window never sits on index 1 alone, because by the time the left pointer reaches 1 the right pointer is already at 2. The algorithm therefore over-reports: it says 2 where the truth is 1.

This is a real, measured demonstration that the precondition is not a formality, and it is the strongest possible support for the next paragraph. [VERIFIED by execution]

One subtlety worth knowing before you are asked: `windowTrace` itself does **not** guard against this — it will happily return a wrong answer for non-positive input. It is the library entry point, `generateWindow`, that rejects non-positive input with an explanatory frame. So the guard exists at the product boundary, not in the reference function the tests import. [VERIFIED]

[VERIFIED] The project enforces the precondition explicitly. `generateWindow` returns a single explanatory frame when the input is not all-positive:

```
'Use positive numbers and a positive target.'
```

And the counterexample lab's page states the precondition on screen: "Preconditions: positive integers and a positive target." The README also describes the problem as "shortest qualifying positive-number window with sum ≥ target". Three separate places, so the assumption is documented, enforced, and disclosed — which is the correct handling of a precondition.

46.7 The 768 cases: what they prove and what they do not

They prove	They do not prove
For every length-3 array with values in {1,2,3,4}, and every target 1–12, the sliding implementation returns the same answer as an independent enumeration	Anything about arrays of length 4, 5, 10, or 1,000
The domain is **exhaustive** within those bounds — this is not sampling	Anything about values outside 1–4 (no zero, no negatives, no large values)
The specific regression (`>=` vs `===`) is caught	That any other bug is absent — the assertion only compares final results, not the traces
The implementation and the oracle agree on the boundary case `sum === target`	Anything about arrays where the answer is 0 and the target is unreachable — though many cases in the range do produce 0, so that path is exercised

MODEL ANSWER

"COULD BOTH IMPLEMENTATIONS BE WRONG?"

"Yes — that is the fundamental limit of differential testing, and I can be precise about where it applies here. A differential test can only detect a disagreement. If both implementations shared a misconception, they would agree and the test would pass.

Concretely, in this case the shared belief is the specification: both say the answer is the shortest window whose sum is at least the target, and both return 0 when none exists. If that specification were wrong — say the intended problem was 'exactly equal to the target', or 'at least the target but the window must include the last element' — then both would be consistently wrong and all 768 cases would pass. The tests cannot validate the requirement; they validate agreement with the requirement as I encoded it. That is why the oracle being independent in mechanism matters so much: it eliminates the class of bugs where the sliding logic is wrong, but it cannot eliminate a wrong reading of the problem."

Then add the second half, because the interviewer is waiting for it: *"There is a second correlated-failure risk that I did close off: the assertion only compares final results, so if `windowTrace` were also returning a subtly wrong trace — right answer, wrong intermediate frames — these tests would not notice. The trace is checked separately, by the counterexample lab's first-divergence display and by the fact that the buggy variant's trace is asserted to differ at the sentence level in the regression group."*

MODEL ANSWER

"DOES 768 PROVE UNIVERSAL CORRECTNESS?"

"No, and the enumeration is honestly bounded: 64 arrays of length exactly 3, values 1–4, and targets 1–12. It is exhaustive within that box, which is stronger than sampling, but the box itself is small. The domain it leaves untested includes all lengths except 3, all values outside 1–4, zero and negative values (which the technique cannot handle at all, and which the code explicitly refuses), and the full input range the product allows.

So the correct claim is: **the implementation agrees with an independent oracle on all 768 cases in a bounded domain that was chosen to be small enough to enumerate completely.** That is a meaningful statement, and it is a very different statement from 'the sliding window is correct'. And I'd add one thing that matters more than the count: the bug that motivated these tests was found by differential search, not by these tests — the search returned [2] with target 1, which is the smallest possible case, and the 768 cases would also have caught it. The search is the mechanism that finds bugs; the 768 is the mechanism that pins correctness in a box. Both are in the suite for that reason."

46.8 The bounded differential search — the most interesting test in the project

`src/lab/engine.js · lines 152–169 (verbatim)`

```
export function findCounterexample() {
  // Bounded exhaustive search in increasing length and lexicographic order.
  for (let length = 1; length <= 3; length++) {
    for (let code = 0; code < 4 ** length; code++) {
      const values = Array.from(
        { length },
        (_, i) => 1 + (Math.floor(code / 4 ** (length - i - 1)) % 4),
      );
      for (let target = 1; target <= 8; target++) {
        const correct = windowTrace(values, target),
          buggy = windowTrace(values, target, true);
        if (correct.result !== buggy.result)
          return { values, target, correct, buggy };
      }
    }
  }
  return null;
}
```

Aspect	Detail
Domain	Arrays of length 1–3, values 1–4 (so 4 + 16 + 64 = 84 arrays), targets 1–8
Order	Increasing length, then increasing `code` (lexicographic by the digit-extraction scheme), then increasing target
Predicate	Return the **first** case where `correct.result !== buggy.result`
Result	`{values: [2], target: 1, correct, buggy}` — asserted in the tests
Claim	"The first failure in that bounded enumeration order" — **not** "the globally minimal counterexample"
Cost	84 arrays × 8 targets × 2 runs = 1,344 pairs of executions; the test completes in under 1 ms

MUST KNOW

WHY A DELIBERATELY BUGGY IMPLEMENTATION IS IN THE PRODUCTION MODULE

This is the part an interviewer will find genuinely unusual, and the answer is good: **you cannot test that a test can detect a bug unless you have a bug to detect.** Including a second, wrong-but-plausible implementation behind a `buggy` flag makes the detector testable. If `findCounterexample()` ever stopped finding failures — because a refactor made it compare the same function twice, or the predicate got inverted — the test would fail immediately, because it asserts that a mismatch exists and that it is reproducible.

That is a "test the detector" pattern, and together with the first-divergence display it is the most original engineering idea in the repository.

DANGER ZONE — WHERE CANDIDATES DIE

THE PRECISION REQUIRED WHEN DESCRIBING THIS

Say **"the first failure in the defined enumeration order within a bounded domain"**. Do NOT say "the smallest counterexample", "the minimal counterexample", or "the bug finder". The code's own comment says "Bounded exhaustive search in increasing length and lexicographic order", and the Counterexample page says "This is the first failure in that bounded domain—not a universal minimality proof." Your own UI already has the correct wording; use it.

CHECKLIST

RED-LINE DRILLS FOR PARTS XVII–XX

- ■ Name the runner, the assertion library, and the fact that there is no test dependency.
- ■ Recite the 14 test groups at a high level.
- ■ Explain 150 = 6 × 25 and name the five hand-written edge inputs.
- ■ List the four assertions in the sorting group.
- ■ Say the "testing shows the presence of bugs, not their absence" sentence, then the specific coverage weaknesses.
- ■ Explain determinism as a product requirement via the saved cursor.
- ■ Explain why the oracle must not share code with the implementation.
- ■ State the sliding window's positivity precondition and give a negative-value counterexample.
- ■ Say the exact correct phrasing for `findCounterexample`.

PARTS XXI–XXVII

The toolchain

CI, Vite, React Router, Tailwind, the JavaScript that actually matters, performance, and the real trust boundaries.

CHAPTER 47

47 · GitHub Actions, line by line

Fifteen lines of YAML that enforce the same three commands you run locally.

47.1 The workflow

`.github/workflows/ci.yml (verbatim, 15 lines)`

```
name: Verify AlgoViz
on: [push, pull_request]
jobs:
  verify:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with:
          node-version: 22
          cache: npm
      - run: npm ci
      - run: npm test
      - run: npm run lint
      - run: npm run build
```

Line	Meaning	Why it is there
`name: Verify AlgoViz`	The workflow's display name	Shows in the Actions tab and on PR checks
`on: [push, pull_request]`	Triggers on every push to any branch, and every PR	Deliberately broad: a push to a feature branch gets verified before it becomes a PR. The cost is duplicate runs when a push is part of an open PR
`runs-on: ubuntu-latest`	A fresh GitHub-hosted Linux VM per run	No state carries between runs — a clean-room guarantee
`actions/checkout@v4`	Clones the repository into the runner	Pinned to a major version tag — the conventional practice; the rigorous version is a full commit SHA pin, which defends against a compromised tag
`actions/setup-node@v4` with `node-version: 22`	Installs Node 22	Matches `.nvmrc` (`22`) and `engines.node: ">=22.12.0"` — three places agree on the runtime version
`cache: npm`	Caches the npm download cache keyed on the lockfile	Makes `npm ci` fast without skipping installation
`npm ci`	Clean, reproducible install straight from `package-lock.json`	The critical choice: `npm ci` **fails** if the lockfile and `package.json` disagree, and it deletes `node_modules` first. `npm install` would silently update the lockfile and hide drift
`npm test`	`node --test tests/*.test.js`	Fails the job on any failing assertion
`npm run lint`	`eslint .`	Fails on any lint error
`npm run build`	`vite build`	Fails if the app cannot be bundled

MUST KNOW

WHY `NPM CI` MATTERS, AND HOW TO SAY IT

"`npm ci` is the CI-correct install: it requires the lockfile to be in sync with `package.json`, it removes `node_modules` first so nothing can leak in from a previous run, and it installs the exact resolved versions rather than the newest matching a semver range. `npm install` in CI would happily update the lockfile and mask a version drift that would break a teammate's machine. The one-line version: **`npm ci` verifies the lockfile; `npm install` edits it.**"

47.2 The pipeline as a gate

```
push / pull_request
       |
       v
ubuntu-latest  ->  checkout  ->  setup-node 22 (with npm cache)
       |
       v
npm ci  ............ fails if lockfile != package.json
       |
       v
npm test .......... 14 groups; the 150 cases and the 768 comparisons
       |            (exits non-zero on the first failure)
       v
npm run lint ..... eslint flat config, whole repository
       |
       v
npm run build .... vite build -> dist/
       |
       v
PASS  ->  green check on the commit / PR
FAIL  ->  red check, and the job stops at the first failing step
```

Step order matters. Tests run before lint and build: if a correctness test fails, you learn that first, and you do not wait for a build to discover it. Build runs last because it is the slowest and the least specific.

47.3 What CI catches, and what it cannot

CI catches	CI cannot catch
A generator change that breaks sorted output, purity, determinism or ID uniqueness	A rendering bug — nothing in CI renders a component
A reducer change that breaks cursor bounds or the pause-on-seek rule	A CSS regression, a broken layout, a contrast problem
A validation change that lets through `"1,"`, `"2.5"`, `"1,1000"` or 25 values	A stale closure, a missing cleanup, or a timer bug — none of these are reachable without a DOM and a clock
A dependency or lockfile drift	A runtime error in a route that is never executed by tests
Lint violations (unused variables, hook-dependency mistakes, react-refresh export rules)	Anything about the twelve legacy modules' behaviour — only pathfinding and prefix sum are exercised
A build break: a bad import path, invalid JSX, a missing module	A broken user flow that type-checks and builds fine
A failing snapshot of the sliding-window regression	Whether the UI actually shows what the code intends

MODEL ANSWER

"SO A GREEN CI BUILD COULD STILL CONTAIN BUGS?"

"Absolutely, and I can name the bug classes it would sail past. CI runs tests, lint and build — none of which touch a browser. So a green run tells me that the engine behaves, that the code satisfies the lint rules, and that it compiles. It says nothing about whether the playback effect schedules one timer or three; whether the keyboard listener accumulates; whether `ArrayView` renders the merge sort buffer correctly; whether a saved experiment reloads; whether the layout holds at 390 px.

There's a concrete example in this project. The merge-sort duplicate-key frames — 23 of 107 frames on the default input — pass CI completely, because the test asserts ID uniqueness on the final frame and the duplicate ids only occur in the intermediate `write_left`/`write_right` frames. And no test renders a component, so React would warn about duplicate keys only in a browser. If I wanted CI to catch that class of bug, I'd need to add a component test with a DOM environment, or a browser test driver, or at minimum a property test that asserts ID uniqueness across every frame rather than just the last — and that third option is the cheapest, so it's the first thing I'd add."

47.4 The CI questions

INTERVIEW QUESTION
"WHY GITHUB ACTIONS?"

"Because the checks were already npm scripts and the repository was already on GitHub. The workflow is fifteen lines: checkout, set up Node, install, then run the three commands I run locally. There's no service to host, no runner to maintain, no configuration beyond a YAML file — and for a public repository the minutes are free. A more heavyweight choice like Jenkins would have meant running and securing a server to execute three shell commands."

INTERVIEW QUESTION
"WHAT HAPPENS WHEN A TEST FAILS?"

"The `npm test` step exits with a non-zero status, GitHub marks that step failed and stops the job — so lint and build do not run. The check appears against the commit and any PR, red, with the failing assertion printed in the log. If branch protection were enabled, a red check would block the merge."

INTERVIEW QUESTION
"CAN THE BUILD PASS WHILE THE UI IS BROKEN?"

"Yes, in at least four ways. A component can throw at runtime on a route the tests never visit. A CSS change can make the layout unusable without failing anything. An effect can have a stale-closure bug that only manifests when you interact. Or a feature can be entirely absent — nothing asserts that a button exists or does what its label says. Build success means 'the bundler resolved every import and transformed every module'; it says nothing about behaviour."

INTERVIEW QUESTION
"WHY DOES THE WORKFLOW RUN ON `PUSH` AND NOT JUST `PULL_REQUEST`?"

"A push trigger verifies every commit, including ones pushed directly to a branch before a PR exists, so a broken commit is caught within a minute rather than at review time. The cost is that a branch with an open PR gets two runs — one for the push and one for the PR event. If I wanted to trim the cost, I'd restrict the push trigger to the default branch."

INTERVIEW QUESTION
"IS ANYTHING IN THE WORKFLOW PINNED FOR SUPPLY-CHAIN SAFETY?"

"Partially. The actions are pinned to major-version tags (`@v4`), which is the community norm but not the hardened choice — a tag can be retargeted by a compromised maintainer, and the rigorous practice is to pin a full commit SHA. Node is pinned precisely, to 22, matching `.nvmrc` and `engines`. Dependencies are pinned through `package-lock.json` with `npm ci`. So the dependency graph is reproducible; the workflows are not yet hardened. That's an honest gap rather than a design decision." [PROPOSED: SHA-pin the actions]

CHAPTER 48

48 · Vite, in the amount this project needs

Dev server, build pipeline, and the one detail that makes this repository unusual: the Rolldown alias.

48.1 The scripts

Script	Command	What it does
`npm run dev`	`vite`	Starts the dev server with native ESM and on-demand transform; no bundling until a request arrives
`npm run build`	`vite build`	Produces the production bundle in `dist/`
`npm run preview`	`vite preview`	Serves the built output locally, to check the production artefact
`npm test`	`node --test tests/*.test.js`	**Not Vite.** Node runs the tests directly

48.2 The measured production build

I ran `npm run build` in this repository. The output is the most concrete Vite artefact you can quote:

```
rolldown-vite v7.2.5 building client environment for production...
transforming... ✓ 1736 modules transformed.
dist/index.html                          0.66 kB │ gzip:  0.40 kB
dist/assets/TutorialTab-*.css            6.19 kB │ gzip:  1.79 kB
dist/assets/ModuleFrame-*.css            8.55 kB │ gzip:  2.14 kB
dist/assets/index-*.css                 72.37 kB │ gzip: 14.03 kB
dist/assets/ModuleFrame-*.js             3.87 kB │ gzip:  1.72 kB
dist/assets/TutorialTab-*.js             5.58 kB │ gzip:  1.92 kB
dist/assets/Counterexample-*.js          5.84 kB │ gzip:  2.24 kB
dist/assets/TwoPointers-*.js            12.44 kB │ gzip:  4.42 kB
dist/assets/FastSlowPointers-*.js       13.74 kB │ gzip:  4.39 kB
dist/assets/BinarySearch-*.js           16.40 kB │ gzip:  5.16 kB
dist/assets/SlidingWindow-*.js          16.76 kB │ gzip:  5.31 kB
dist/assets/MergeIntervals-*.js         17.01 kB │ gzip:  5.34 kB
dist/assets/PrefixSum-*.js              17.28 kB │ gzip:  5.68 kB
dist/assets/PathFinder-*.js             18.41 kB │ gzip:  6.29 kB
dist/assets/engine-*.js                 18.65 kB │ gzip:  3.92 kB
dist/assets/Nqueens-*.js                18.69 kB │ gzip:  6.13 kB
dist/assets/TreeTraversal-*.js          20.16 kB │ gzip:  6.15 kB
dist/assets/BinaryVariants-*.js         20.43 kB │ gzip:  5.83 kB
dist/assets/SortingLab-*.js             29.85 kB │ gzip: 10.64 kB
dist/assets/index-*.js                 251.78 kB │ gzip: 80.90 kB
✓ built in 1.18s
```

(The chunk sizes are stable across runs; the total build time is not — it moves by a few tenths of a second depending on the machine and what is already warm. Quote it as "about a second", not as a decimal.)

MUST KNOW
HOW TO READ THAT OUTPUT IN AN INTERVIEW

"1,736 modules transformed, 19 emitted asset files — sixteen JavaScript chunks and three stylesheets — in about a second." Then three observations:

1. **The initial bundle is 251.78 kB / 80.90 kB gzipped** — that is React, React DOM, React Router and Lucide, plus the shell and the overview. Nothing else is in it.
2. **Every heavy module has its own chunk** — `SortingLab` is 29.85 kB, and the twelve legacy modules range from 12 to 20 kB. That is `React.lazy` doing real work: a visitor to the landing page never downloads the N-Queens solver or the pathfinder. This is the kind of claim you can only make because you measured it.
3. **`engine.js` gets its own 18.65 kB chunk**, shared between `SortingLab`, `Counterexample` and the test-adjacent code paths. That is a Rollup/Rolldown-level consequence of the module boundary: `engine.js` imports the six generators and nothing else imports them directly.

48.3 The Rolldown alias — the detail that shows you read your own config

```
"devDependencies": {
  "vite": "npm:rolldown-vite@7.2.5"
},
"overrides": {
  "vite": "npm:rolldown-vite@7.2.5"
}
```

[VERIFIED] This is an npm **alias**: the package name `vite` resolves to the package `rolldown-vite` at version 7.2.5. `vite.config.js` still says `import { defineConfig } from 'vite'` and `import react from '@vitejs/plugin-react'`, and nothing in the config mentions Rolldown — because the alias does the substitution at install time.

Rolldown is the Rust-based bundler intended as Vite's eventual Rollup replacement; the `rolldown-vite` package is the drop-in bundle that swaps the bundler while keeping the Vite API.

DANGER ZONE — WHERE CANDIDATES DIE
WHAT TO SAY, AND WHAT NOT TO SAY

Do not say "I built it with Rolldown" as if you had chosen a custom bundler, and do not claim any performance numbers for it.

Do say: *"Vite is aliased to `rolldown-vite` in `package.json` — the `vite` package name resolves to that instead. It's a drop-in: the config is still `defineConfig` with the React plugin, and the command is still `vite build`. The alias is duplicated in `overrides` to force every transitive dependency to the same version. It's an experiment with the new bundler rather than a requirement — nothing in the code depends on it, and I could revert by removing those two lines. What I can tell you is that the build transforms 1,736 modules in about 1.2 seconds, but I have not benchmarked it against the standard bundler, so I won't claim it's faster."*

That answer is unimpeachable: it states what the config does, why it is safe, and explicitly refuses to make a performance claim you have not measured.

48.4 Vite questions

INTERVIEW QUESTION
"IS VITE YOUR BACKEND?"

"No — Vite is a development server and a bundler, and in this project it's used only for local development and for producing static files. In production there is no Node process at all: Vercel serves `dist/` as static assets. The `vite preview` server is a local convenience for checking the built output, not a deployable server."

INTERVIEW QUESTION
"WHAT HAPPENS DURING `NPM RUN BUILD`?"

"Vite resolves the module graph from `index.html`, transforms every JSX and CSS file through the plugin pipeline, resolves the seventeen dynamic `import()` calls into separate chunks, minifies and hashes the assets, and writes them to `dist/` along with a rewritten `index.html` whose script tag points at the hashed entry chunk. It also runs PostCSS and therefore Tailwind's content scan, which is why the CSS bundle contains only the utility classes actually used in source. The output is pure static files: an `index.html` and `assets/` containing hashed JS and CSS."

INTERVIEW QUESTION
"WHAT IS THE `VERCEL.JSON` REWRITE FOR?"

`"{"rewrites":[{"source":"/((?!assets/|favicon.svg).*)","destination":"/index.html"}]}` — an SPA fallback. Because routing is client-side, a direct request to `/lab` would 404 on a static host, since no such file exists. The rewrite serves `index.html` for any path that is not a real asset, so the router can see the URL and render the right route. The negative lookahead keeps requests for hashed assets and the favicon out of the fallback."

INTERVIEW QUESTION
"WHAT ARE `SERVER.HOST` AND `ALLOWEDHOSTS` DOING IN YOUR CONFIG?"

[VERIFIED] `{ host: '0.0.0.0', allowedHosts: ['.e2b.app'] }` in both `server` and `preview`. Binding to `0.0.0.0` makes the dev server reachable from outside the container rather than only on loopback, and `allowedHosts` permits the hostname pattern used by the hosted preview environment. The README also warns about it: "The development server binds to all interfaces for hosted previews. Use it only in a trusted development environment; production hosting should serve the built application." That is a **development-environment convenience with a documented security caveat**, not a production setting.

INTERVIEW QUESTION
"DOES THE APP USE ENVIRONMENT VARIABLES?"

"No. There is no `.env` file, no `import.meta.env` reference in the source, and the README states that no environment variables or API keys are needed. That is a natural consequence of having no backend and no third-party services — there is nothing to configure, and nothing to leak."

CHAPTER 49

49 · React Router, as this project uses it

Sixteen routes, seventeen lazy imports, one Suspense boundary, two URL parameters.

49.1 The pattern

[VERIFIED] `BrowserRouter` (not `HashRouter`) wrapped around a single `Routes` block inside `Suspense`, in `src/pages/Home.jsx`. Router hooks used across the app: `useSearchParams` and `useLocation` in `SortingLab`, `useLocation` in `Shell` and `ModuleFrame`, `useNavigate` in `ModuleFrame`, and `Link`/`NavLink` for navigation.

Hook / component	Where	What it reads or does
`<BrowserRouter>`	`Home.jsx`	Provides history-based routing using the real URL path
`<Routes>`	`Home.jsx`	Matches the current location against the route list
`<Route path element>`	`Home.jsx`	Declarative route definition; `*` is the catch-all 404
`<Suspense fallback>`	`Home.jsx`	Renders the loading page while a lazy chunk downloads
`<Link>` / `<NavLink>`	`Shell`, `Overview`, `ModuleFrame`	Client-side navigation; `NavLink` adds `active` state for the sidebar
`useSearchParams`	`SortingLab`	Reads `?algorithm=` and `?compare=`
`useLocation`	`Shell`, `SortingLab`, `ModuleFrame`	Reads `pathname` for titles, page names and default algorithm selection
`useNavigate`	`ModuleFrame`	Programmatic navigation from the module switcher `<select>`

49.2 The URL parameters, precisely

Parameter	Where it is read	Effect
`?algorithm=<id>`	`SortingLab` line 63	Sets the initial algorithm **only if** `Object.hasOwn(algorithms, value)` — an unknown value silently falls back to `bubble` (or `quick` on the advanced-sort route). So `?algorithm=hack` is safe
`?compare`	`SortingLab` line 74	`params.has("compare")` — presence is enough, so `?compare=1`, `?compare=true` and `?compare=` all enable comparison mode. This is what makes the landing page's `/lab?compare=1` link work

MUST KNOW
THE DETAIL THAT WILL IMPRESS AN INTERVIEWER

[VERIFIED] `initial` is computed like this:

```
const initial = Object.hasOwn(algorithms, params.get("algorithm"))
  ? params.get("algorithm")
  : location.pathname.includes("advanced-sort")
    ? "quick"
    : "bubble";
```

Three things are happening. **First, the query parameter is validated against the registry** rather than trusted, so a hand-typed URL cannot put an unknown key into `algorithms[algo]` and crash the render with `Cannot read properties of undefined`. **Second, the precedence is deliberate:** an explicit `?algorithm=` beats the route's default, so `/modules/core/advanced-sort?algorithm=bubble` shows bubble sort. **Third, it is computed from `useSearchParams`, not `useState`** — but then stored in `useState`, which means changing the URL after mount does not change the algorithm. That is a real, subtle limitation: the parameter is an initial value, not a source of truth.

Note also that these are read **in the component body**, so the value is recomputed on every render — but since only the first render's value reaches `useState`, later recomputations are discarded. Harmless, and typical of this pattern.

49.3 Client-side routing questions

INTERVIEW QUESTION
"WHY CLIENT-SIDE ROUTING?"

"Because the application is already a client-side React app and every 'page' is a different view over the same data and the same engine. Client-side routing gives me three things a multi-page app would not: state survives navigation, so the shell and the shared CSS stay mounted; navigating between modules does not re-download the application; and the URL stays meaningful — `/modules/patterns/sliding-window` is a real, shareable, bookmarkable address, and `/lab?algorithm=quick&compare=1` restores a specific experiment configuration. The trade-off is that the server must serve `index.html` for unknown paths, which is what the `vercel.json` rewrite does."

INTERVIEW QUESTION
"WHAT HAPPENS WHEN SOMEONE OPENS `/MODULES/PATTERNS/PREFIX-SUM` DIRECTLY?"

"The static host has no such file, so the rewrite returns `index.html`. The browser parses that document and requests the entry JS chunk. React mounts, `BrowserRouter` reads `window.location.pathname` — which is still `/modules/patterns/prefix-sum` — and `Routes` matches it to the lazily imported `PrefixSum` component. React suspends while that chunk downloads and renders the `Suspense` fallback inside `Shell`; when the chunk resolves, the module mounts. Total extra round trips compared with an in-app navigation: the chunk fetch, unless it is already cached. No server-side rendering happens at any point."

INTERVIEW QUESTION
"WHY ARE SUB-ROUTE PATHS SO DEEP — `/MODULES/PATTERNS/TWO-POINTERS`?"

"Because they encode a hierarchy that the UI actually mirrors: `Shell` groups the sidebar into workspace sections, `ModuleFrame` shows a breadcrumb with the category, and `constants.js` holds the same category field used in the library grid. It costs nothing at runtime — the path is just a string that has to match — and it makes the URLs self-describing. If I were optimising for brevity I'd flatten them, but then the breadcrumb would have to invent a category that the URL doesn't contain."

INTERVIEW QUESTION

"WHAT DOES THE `KEY` ON ONE OF YOUR ROUTES DO?"

"`<SortingLab key="advanced" />` on `/modules/core/advanced-sort`. Adding a distinct `key` makes React treat it as a different element from the `<SortingLab />` on `/modules/core/simple-sort`, so navigating between the two routes unmounts and remounts the component instead of reusing it. That matters because the initial algorithm is computed from `location.pathname` at mount time — without the key, the component would keep the algorithm it started with, and the advanced-sort route would still show bubble sort."

CHAPTER 50

50 · Tailwind, in three pages instead of thirty

What it does in this project, where it is used, and the honest description of the split.

50.1 The configuration

`tailwind.config.js (verbatim, 12 lines)`

```
/** @type {import('tailwindcss').Config} */
export default {
  content: [
    "./index.html",
    "./src/**/*.{js,ts,jsx,tsx}", // <--- CRITICAL LINE
  ],
  theme: {
    extend: {},
  },
  plugins: [],
}
```

`src/index.css (verbatim, 13 lines)`

```
@tailwind base;
@tailwind components;
@tailwind utilities;

body {
  margin: 0;
  padding: 0;
  background-color: #000000;
  color: white;
  font-family: system-ui, -apple-system, sans-serif;
  width: 100%;
  min-height: 100vh;
}
```

Piece	Detail
Entry point	`src/index.css` — the only file containing `@tailwind` directives, imported by `main.jsx`
Content globs	`./index.html` and `./src/**/*.{js,ts,jsx,tsx}` — Tailwind scans source text for class-name candidates
PostCSS pipeline	`postcss.config.js` runs `tailwindcss` then `autoprefixer`
Theme customisation	None — `theme.extend` is empty, so only default Tailwind scales exist
Plugins	None
Output (measured)	`dist/assets/index-*.css` at 72.37 kB, 14.03 kB gzipped

50.2 Where Tailwind is actually used — the honest split

Area	Styling approach	Evidence
`src/modules/**` (the twelve legacy visualisations)	**Tailwind utility classes**, heavily	e.g. `SlidingWindow.jsx` uses `flex flex-col md:flex-row justify-between items-center bg-gray-800/60 border border-gray-700 p-4 rounded-xl mb-4 gap-4 shadow-lg` on one element
`src/lab/**` (the lab, shell, overview, counterexample)	**Hand-written CSS** with an `av-` prefix	`lab.css` 1,756 lines, `modules.css` 505, `lesson.css` 419, `styles/global.css` 258

Area	Styling approach	Evidence
Dead components	Tailwind classes	`CardsGrid.jsx` has a fully utility-styled footer — scanned by Tailwind and emitted into the CSS bundle, though the component is tree-shaken out of the JS

MUST KNOW

THE SENTENCE TO USE, AND THE SENTENCE TO AVOID

Avoid: "AlgoViz is built with Tailwind CSS." It is technically defensible — Tailwind is a dependency and it ships — but it misrepresents the lab, which is the part of the project you want to talk about.

Use: *"Tailwind is in the stack and it styles the twelve module pages — that's where most of the utility classes are. The lab itself, which is the rebuilt part, uses scoped CSS with an `av-` prefix: about 1,750 lines in `lab.css`. I kept Tailwind for the modules to avoid rewriting them, and used plain CSS for the lab because I needed precise control over the bar geometry, the timeline and the responsive breakpoints. So it's two systems, and I'd call that a deliberate trade-off rather than a design."*

If pushed on whether two systems are a problem: "It's a real cost — there are two places a style can live, and the bundle ships utilities for pages that are dead code. The benefit is that I didn't have to touch twelve working visualisations to rebuild the sixteenth screen. If I were starting over I'd pick one."

50.3 The three Tailwind facts that matter here

Fact	Why it is relevant
Tailwind is a **build-time** tool, not a runtime library	There is no Tailwind JavaScript in the browser bundle. It runs during the build, scans source files for class names, and emits matching CSS rules. That is why `dist/assets/index-*.css` contains only the utilities used in source
The content globs decide what is emitted	A class name that appears nowhere in `index.html` or `src/**` produces no CSS. That is also why classes inside `dead` components are still emitted — the globs scan source text, and the files exist
`@tailwind base` includes Preflight, a CSS reset	It is imported by `index.css`, which `main.jsx` imports after `global.css` — so Tailwind's reset wins on conflicting base rules

INTERVIEW QUESTION

"WHY DID THE PROJECT USE TAILWIND AT ALL?"

"Speed of iteration for the module pages. Twelve visualisations each need controls, grids, panels and responsive behaviour, and utility classes let those be built in place without inventing component CSS for each one. For the lab I wanted a small, explicit style vocabulary — the bars, timeline and panel chrome are all in one file with an `av-` namespace, which makes them easy to reason about and easy to change in one place."

CHAPTER 51

51 · JavaScript, only where it matters to AlgoViz

Ten language features, each tied to a specific line in this repository.

Concept	Where it appears in AlgoViz	Why it matters here
Arrays and objects	`frames` is an array of objects; each element is `{id, val}`	The whole engine is array-of-object manipulation. Understanding that `.map` returns a new array while `arr[i] = x` mutates is the difference between a correct and a corrupt trace
Spread for shallow copy	`cloneArray = () => arr.map(item => ({...item}))`, `{...stats}`, `[...sortedIndices]`, `[...input]`	Three different copies with three different purposes: element objects, the counters, the bookkeeping lists. All three are shallow and all three are sufficient because the values inside are primitives
Reference vs copy	`useMemo(() => algorithms[algo].run(input), [algo, input])`	`input` is compared by reference, so `load()` must create a new array for the memo to recompute — which `parseInput` does
Immutability	The generators never mutate `rawArray`; `input` is only replaced	This is what the `assert.deepEqual(input, before)` test asserts, and what makes same-input comparison structurally valid
**`Array.prototype.map`	`rawArray.map((val, i) => ({id: 'block-'+i, val}))`, every `cloneArray()`	The identity-assignment step and the per-frame clone
`Array.prototype.at(-1)	`frames.at(-1).currentStats`, `otherFrames.at(-1)`	Negative indexing, added in ES2022. Node 22 and all modern browsers support it
**`Array.prototype.includes`	`frame.activeIndices?.includes(i)`	The O(n) membership test that drives every highlight. At n ≤ 24 it is free; at n = 100,000 it would be the bottleneck, and a `Set` would be the fix
Closures	The keyboard handler closing over `cursor`; the timer callback dispatching a constant action	The stale-closure trap and the deliberate avoidance of it
Optional chaining / nullish	`frame.codeLine?.replaceAll("_", " ")`, `event.target.files?.[0]`, `frame.array.map(...)`	Defensive reads on fields that only some generators emit. `pivotIndex ?? "—"` distinguishes "no pivot" from "pivot at index 0" — a bug `||` would introduce
**`Object.hasOwn`	`Object.hasOwn(algorithms, params.get("algorithm"))`, and twice in the save/import paths	The correct prototype-safe existence check. `key in obj` walks the prototype chain and would accept `"constructor"`; `obj[key] !== undefined` would accept nothing useful and reject nothing dangerous
Template literals	Every `msg` string; `` `block-${index}` ``	The explanation text is built at generation time, which is why the messages can contain the actual values
`Number.isSafeInteger` / `Number.isInteger`	`parseInput`, and the cursor guard on save/import	Distinguishes "a number" from "a whole number in the representable integer range". `Number.isInteger(2.5)` is false, which is what stops `frames[2.5]`

Concept	Where it appears in AlgoViz	Why it matters here
JSON	`JSON.stringify(next, null, 2)`, `JSON.parse(await file.text())`, `JSON.stringify(frame.activeIndices)` in the inspector	The persistence format and the display format
ES modules	`import`/`export` throughout; no `require` in `src/`	Enables tree-shaking: the six generators are only pulled into the `engine` chunk because `engine.js` imports them
Timers	`setTimeout` in the playback effect; `clearTimeout` in the cleanup; a `setTimeout` for `revokeObjectURL`	Discussed at length in Chapter 25
Async/await	`await file.text()` in the import handler; no `async` anywhere in the engine	The engine is deliberately synchronous. Only file reading is async

51.1 Three language questions AlgoViz makes concrete

INTERVIEW QUESTION
"WHY IS `{...ITEM}` A SUFFICIENT CLONE FOR A FRAME ELEMENT?"

"Because an element is `{id: string, val: number}` — two primitives. A shallow copy of a two-primitive object is a complete copy; there is no nested object that could still be shared. Insertion sort adds a third primitive (`isLifting`) and merge sort a fourth (`yOffset`), both primitives, so the shallow copy remains sufficient in all six generators. If I ever attached a nested object to an element — say a history array — the shallow copy would suddenly be insufficient and frames would share that nested object. That is the kind of change that would silently break the trace, and it is exactly the kind of thing a test asserting deep equality across runs would not catch. It would need a test that mutates a frame and checks that an earlier frame is unaffected."

INTERVIEW QUESTION
"WHAT'S THE DIFFERENCE BETWEEN `PIVOTINDEX ?? '—'` AND `PIVOTINDEX || '—'`?"

"`??` only falls back on `null` or `undefined`; `||` falls back on any falsy value — including `0`. Since the pivot index is 0 whenever the pivot is the first element, `||` would display '—' for a real pivot at index 0 and the app would look broken on a specific class of inputs. That's a one-character bug that only appears on some inputs, which makes it the worst kind. The code uses `??`, and it is the right choice here."

INTERVIEW QUESTION
"WHY `OBJECT.HASOWN` RATHER THAN `IN` OR A TRUTHINESS CHECK?"

"Because the key comes from a user-controlled string — a URL query parameter, or a field in an imported JSON file. `'constructor' in algorithms` is `true` because `in` walks the prototype chain, so `algorithms['constructor']` would resolve to a function. `Object.hasOwn` only checks own properties, so `'constructor'` and `'__proto__'` are correctly rejected. And `.run` would then be called on a function that has no `.run`, which is a crash. So the choice is the difference between a validation error and an unhandled exception on a crafted URL or a crafted file."

CHAPTER 52

52 · Performance: the real limits and the real fixes

Measured numbers, the first thing that breaks, and the four-step plan to make it scale.

52.1 The measured baseline

Quantity	Measured value
Worst trace the app can produce	1,154 frames (bubble, reverse-sorted 24 elements)
Its serialised size	~981 KB
Typical trace (default 8 elements, bubble)	118 frames, ~49 KB
Generator runtime for the worst case	Milliseconds — well under one animation frame
Whole test suite	226 ms for 14 groups including 150 sorting runs and 768 window cases
Production build	1,736 modules, ~1.2 s, initial bundle 251.78 kB / 80.90 kB gzipped
Bundle for the lab route	29.85 kB / 10.64 kB gzipped, downloaded only when visited

52.2 Where the time actually goes (in order)

#	Cost	When it happens	Why it is bounded today
1	**Frame generation**	Once per `(algorithm, input)` pair, synchronously	n ≤ 24, so at most ~29,000 element comparisons for the worst quadratic case times the copying overhead — milliseconds
2	**`cloneArray` allocations**	Per frame push, inside generation	Same bound; the dominant allocation source but still trivial at n = 24
3	**Rendering a frame**	Every cursor change: ~6 times/second at 6× speed, or every frame while scrubbing	≤ 24 columns, each a few DOM nodes
4	**`useMemo` cache misses**	When `algo` or `input` changes, or when `compare`/`other` change	Guarded by the memo; the second generator runs only in comparison mode
5	**The duplicated generator run in `load()`**	Every input change	A real 2× waste on the generation path; invisible at this scale
6	**The keyboard effect's re-registration**	Every cursor change (~6/s during playback)	A `removeEventListener`/`addEventListener` pair against `window` — microseconds
7	**`localStorage` writes**	On save only	Six small records

MUST KNOW

THE PERFORMANCE SENTENCE TO LEAD WITH

"The input cap is a performance decision, not an arbitrary restriction. The worst trace the application can produce is 1,154 frames and about 1 MB, and generation is a single synchronous burst in the milliseconds — so the cap is what makes 'run to completion, then replay' safe on the main thread. I've measured the worst case rather than assumed it."

52.3 What would break first, in order

1. **Synchronous generation blocks the main thread.** The generators are plain synchronous loops with no yielding. At n = 10,000 a quadratic sort would perform tens of millions of comparisons and clone a 10,000-element array on every push, before the first paint. The tab freezes and the browser may offer to kill the page.
2. **Memory explodes.** Retained data is O(frames × n), and frames is $O(n^2)$ for the quadratic sorts — so conceptually $O(n^3)$ elements. At n = 1,000 a bubble trace could need on the order of 10^9 element copies.
3. **Rendering becomes the bottleneck.** 1,000 DOM columns per frame, re-rendered as the cursor moves, is far beyond a frame budget. `ArrayView` has no virtualization and no `memo`.
4. **The timeline stops being usable.** A range input with a million steps cannot be scrubbed to a meaningful position.
5. **`localStorage` throws.** The 5 MB per-origin quota would be exceeded by the first save; the existing `try/catch` would catch the `QuotaExceededError` and show "Browser storage is unavailable. Use Export instead."

MODEL ANSWER
"COULD ALGOVIZ HANDLE 1 MILLION FRAMES?"

"Not today, and I can tell you exactly what fails. At a million frames you're looking at roughly 800 MB to a gigabyte of retained objects, since each frame is about 800 bytes of array-of-objects, and the timeline alone would have a million steps. The main thread would freeze before any of that even mattered, because generation is synchronous — a million-frame trace can't be built one frame at a time without blocking the UI for seconds. So the answer is no, and the honest follow-up is how I'd get there, which is a four-step plan rather than one fix."

MUST KNOW
THE OPTIMIZATION PLAN, IN THE ORDER IT SHOULD BE DONE

1. **Move generation into a Web Worker.** The generators have no DOM dependency — they take an array and return an array, which is exactly what a worker does well. This removes the freeze. The cost is that traces become async, so `frames` needs a state machine (loading/ready/error) and the reducer's `load` has to wait.

Do not call this a security sandbox. A worker is a concurrency tool; it shares the origin and can still make requests.

2. **Replace full snapshots with compact events plus checkpoints.** Store operations (`{op:'swap', i, j}`, `{op:'compare', i, j}`) and a full snapshot every k frames. Memory drops from O(frames × n) to O(frames) events plus O((frames/k) × n) checkpoints — with k = 50 that is a ~50× reduction on the snapshot component.
3. **Virtualise the array view.** Render only the visible columns for large n, or switch to a canvas-based bar chart.
4. **Only then** raise the input cap, and keep one — there is always a size at which a browser tab is the wrong tool, and a visualisation of 100,000 bars is not readable by a human anyway.

Steps 1 and 2 are listed in the README's own "What's next" section as "Introduce compact trace events and checkpoints" and "Add worker execution before increasing input limits" — so the plan is the project's declared direction, not something improvised in an interview.

52.4 The optimizations already present

Optimization	Where	Effect
`useMemo` for both traces	`SortingLab` 93–97	The generator runs once per `(algo, input)` pair, not once per render
Lazy routes with `Suspense`	`Home.jsx`	Twelve module chunks are not downloaded until visited

Optimization	Where	Effect
`useState` lazy initialiser for `saved`	`SortingLab` 77	`localStorage` is read once per mount, not once per render
`Object.is` bail-out on unknown actions	reducer `default`	No re-render for an unrecognised dispatch
Tree-shaking	Vite/Rolldown, verified by inspecting `dist/`	Dead components are absent from the bundle
Tailwind's content scan	build time	Only used utility classes are emitted
Bounded input	`parseInput`	The single most effective performance control in the project

TRAP

"WHY DON'T YOU MEMOISE ARRAYVIEW WITH REACT.MEMO?"

The honest answer: *"Because at n ≤ 24 the render is not the bottleneck, and memoising prematurely would add a dependency comparison for every cursor change with no measurable benefit — and the props are never actually equal, since `frame` changes on every tick. It becomes worth doing at step 3 of the scale plan, together with virtualization, when n is large enough that the render dominates. Adding it now would be cargo-culting."* The key point: `React.memo` on `ArrayView` would be useless here because `frame` is a different object every tick, so the shallow comparison always fails. That is a strong, specific answer.

CHAPTER 53

53 · Security and trust boundaries

Only the boundaries that actually exist in this project — and an explicit list of what is not claimed.

53.1 The trust boundary diagram

```
+-------------------------------------------------------------+
|  TRUSTED (same origin, shipped by me)                       |
|                                                             |
|   frames[]        <- produced by my generators              |
|   algorithms{}    <- a frozen-in-practice registry          |
|   playback state  <- owned by my reducer                    |
|   route table     <- literal paths in Home.jsx              |
+-------------------------------------------------------------+
           ^                                  ^
           |  validated                       |  validated
           |                                  |
+--------------------------+    +-------------------------------+
|  SEMI-TRUSTED            |    |  UNTRUSTED                    |
|                          |    |                               |
|  localStorage contents   |    |  A file the user imports      |
|  (?algorithm= URL param) |    |  (arbitrary bytes, <= 16 KB)  |
|  saved experiment records |    |  Text typed into the input    |
+--------------------------+    +-------------------------------+
           |                                  |
           +---------------+------------------+
                           v
             BOUNDARY CONTROLS (the only validation points)
                Object.hasOwn(algorithms, ...)   <- prototype-safe key check
                Array.isArray(...)               <- shape check
                parseInput(...)                  <- 5 value rules, one definition
                Number.isInteger(...)            <- cursor type check
                file.size > 16384                <- size limit
                try/catch everywhere             <- parse + storage failures
                React's escaping                 <- all rendered values
```

53.2 What each control defends against

Control	Threat	Failure mode it prevents
`Object.hasOwn(algorithms, key)`	A crafted `algorithm` value from a URL or file resolving via the prototype chain	`algorithms["constructor"].run(...)` → `TypeError`, crashing the component and blanking the route
`parseInput`'s anchored regex	Type-confusion via implicit coercion (`""` → 0, `"1e3"` → 1000, `"0x10"` → 16, `"Infinity"` → `Infinity`)	Values that are not what they look like reaching the algorithm and the display
`Number.isSafeInteger` + `|n| ≤ 999`	Out-of-range values, and numbers beyond 2^{53} that lose precision	Silently wrong comparisons; absurd bar labels; division-by-zero risk in the height calculation
The 2–24 length rule	Unbounded work	A synchronous generator run that freezes the tab
The 16 KB file cap	Oversized parse on the main thread	A crafted or accidental large file hanging the tab before any validation runs
`Number.isInteger(cursor)` on save/import	Fractional or non-numeric cursors	`frames[2.5] === undefined` → `ArrayView` throws on `frame.array.map`
`try/catch` around `JSON.parse` and `localStorage`	Malformed or hostile stored bytes	An exception during the lazy `useState` initialiser, which is the worst place for one — it would make the route unrenderable

Control	Threat	Failure mode it prevents
React's default escaping	Injected markup from any string value	The imported `input` values, the `algorithm` name and the stored data are all rendered through JSX children, which React escapes. There is no `dangerouslySetInnerHTML` anywhere in the codebase
`.filter()` on the read path	Records referring to removed algorithms	A crash at `algorithms[item.algorithm].name` during render

DANGER ZONE — WHERE CANDIDATES DIE

FOUR THINGS TO NEVER CLAIM

1. **Never claim a security audit.** None was performed. The README is explicit: the dependency audit result is "not a security certification".
2. **Never claim localStorage is secure.** It is plaintext, readable by any script on the origin, visible in DevTools, and not a secret store.
3. **Never claim a Web Worker is a sandbox.** A worker shares the origin, can `fetch`, and can access `indexedDB`. It is a threading and responsiveness tool. The README's interview guide says this explicitly: "Do not call a Worker a security sandbox."
4. **Never claim the import validator is a security boundary for a server.** It is a **client-side** control. Its job is to protect the user's own session from bad data. If a server ever accepted uploads, that server would need its own validation, because a client-side check protects nothing against a client that simply doesn't run it.

53.3 What the project is safe from, by construction

Vector	Why it does not exist here
SQL injection	There is no SQL and no database
Command injection	There is no server and no shell
Server-side request forgery	No server
Stored XSS	No server-side storage; saved data is re-validated, and React escapes everything rendered
Reflected XSS	The only URL inputs are `algorithm` (validated against a registry) and the presence of `compare` (a boolean). Neither is rendered as markup
Arbitrary code execution via the import path	No `eval`, no `new Function`, no dynamic `import()` of user data, no URL built from user input, no `dangerouslySetInnerHTML`. The imported values pass through a regex and `Number()`
Credential leakage	No credentials exist. No `.env`, no API keys, no tokens, no cookies set by the app
CSRF	No authenticated state-changing endpoints, and `localStorage` is not sent with requests
Prototype pollution via the algorithm key	`Object.hasOwn` blocks prototype-chain access
Open redirect	The app never navigates to a URL derived from user input

53.4 What is genuinely exposed

Exposure	Reality	Mitigation available
Any script on the origin can read saved experiments	True. `localStorage` has no access control within an origin	Store nothing sensitive there — and nothing in this app is sensitive. If it ever became sensitive, the answer is not "encrypt in localStorage" (the key would have to live in the same place) but "don't store it client-side"

Exposure	Reality	Mitigation available
The user's own saved data is editable via DevTools	True, and treated as untrusted	The read path validates twice: filter on load, `parseInput` on click
A dependency could be compromised	True for every npm project. 300+ transitive packages are installed	Lockfile pinning via `npm ci`, the `overrides` pin for the Vite alias, and a periodic `npm audit`. Actions are pinned to major tags, not SHAs — an identified gap
Prompts from the browser's own warnings	`vite.config.js` binds the dev server to all interfaces and allows `.e2b.app` hosts	Documented in the README as a development-only convenience. Production serves static files with no dev server
No CSP / security headers	`vercel.json` contains only the SPA rewrite	[PROPOSED] Add a `Content-Security-Policy` with `default-src 'self'`, plus `X-Content-Type-Options: nosniff` and `Referrer-Policy`. For a static SPA with no inline scripts this is cheap and would meaningfully raise the cost of an XSS
No Subresource Integrity for the CDN	Nothing is loaded from a CDN — there are no external script tags at all	Not applicable; this is a strength worth stating

MODEL ANSWER

"WHAT IS YOUR THREAT MODEL?"

"A single user, on their own machine, running an offline-capable client-side app with no accounts and no server. So the assets worth protecting are: the user's browser session (no XSS), their typed data (not being corrupted by malformed input), and their own saved experiments (not becoming a crash on load). I've hardened exactly three boundaries — the imported file, the URL parameters, and the localStorage contents — because those are the only places where data crosses from untrusted into application state. Beyond that, the honest position is that there's nothing here to steal: no credentials exist, no personal data is collected, and no requests leave the origin. That's why I'd describe this project as having a small attack surface rather than a hardened one."

CHECKLIST

RED-LINE DRILLS FOR PARTS XXI–XXVII

- ■ Recite the four CI steps and why `npm ci` is used.
- ■ Name three bug classes a green build cannot catch, with the merge-sort duplicate-key example.
- ■ Quote the build figures: 1,736 modules, 19 asset files, ~1.2 s, 251.78 kB initial (80.90 kB gzipped).
- ■ Explain the `rolldown-vite` alias without claiming a performance win.
- ■ Explain the `vercel.json` rewrite and the `allowedHosts` setting.
- ■ Say the Tailwind split sentence without overclaiming.
- ■ Explain `Object.hasOwn` vs `in` with the `constructor` example.
- ■ List the five first-to-break performance limits in order.
- ■ Say the four things you must never claim about security.

PART XXVIII

Debugging master section

Forty realistic AlgoViz bug scenarios, each with symptom, causes, steps, fix and regression test.

CHAPTER 54

54 · Forty debugging scenarios

How to use this chapter: cover the answer, read the symptom, say your diagnosis out loud, then check. This is the single highest-value rehearsal in the book, because a debugging question is the one interview question that cannot be bluffed.

54.0 The universal method

Before the scenarios, the method — it answers every one of them, and interviewers notice when you apply a framework instead of guessing.

```
1. CLASSIFY   Is the defect in the ENGINE (frames), the REDUCER (transport state),
              or the RENDER (what is displayed)?
              -> Test: open the same input in a Node scratch file and inspect frames.

2. LOCATE     Bisect the pipeline: input -> parseInput -> generator -> frames
              -> memo -> cursor -> reducer -> frame -> ArrayView.
              Print the value at each boundary and find the first one that is wrong.

3. REPRODUCE  Reduce to the smallest input and the fewest actions.
              "Bubble sort on [3,1,2] with three ticks" beats "sometimes it breaks".

4. DISTINGUISH Is this a STATE bug or a TIMING bug?
              -> If manual stepping is correct and only playback misbehaves,
                 it is the timer effect, not the reducer.

5. FIX        Change the smallest thing. Prefer fixing the layer that owns the
              invariant (the reducer clamps; the component derives).

6. GUARD      Add the assertion that would have caught it. If it is assertable in
              Node, add it to tests/engine.test.js.
```

Playback and transport

1 — Playback advances two frames per step

Symptom	The animation advances 2 frames per interval; the step counter jumps 0, 2, 4, 6
Possible causes	(a) The timer effect lost its cleanup, so two timeouts are live; (b) a second effect or handler also dispatches `tick`; (c) `speed` or `cursor` was accidentally omitted from the dependency array so the effect never re-schedules and an old timer survives; (d) StrictMode double-mount with no cleanup
Debugging steps	Instrument the effect: log on schedule, log on cleanup. A healthy run logs one cleanup per body. Count the dispatches: wrap `dispatch` in a logger and check whether each `tick` is followed by exactly one `cursor` change
Fix	Restore `return () => clearTimeout(id)` and ensure the array is `[playback.playing, playback.cursor, speed]`
Regression test	Not directly testable in Node. The proxy assertion: the effect must always return a cleanup in the branch where it schedules

2 — Pause does not stop the animation

Symptom	Clicking Pause flips the button label but frames keep advancing
Possible causes	(a) A `setInterval` was introduced somewhere, which pause cannot cancel; (b) the effect body ignores `playing` (missing the guard); (c) `playing` is read from a stale closure in the timer callback; (d) the button dispatches something other than `toggle`

Debugging steps	Log `playback.playing` on each render and each effect run. If `playing` is `false` and the effect body still schedules a timer, the guard or the dependency array is wrong
Fix	The guard `if (!playback.playing) return;` must be the first line, and `playing` must be in the dependency array
Regression test	Reducer level: `toggle` twice must return `playing: false`. The timer behaviour needs a fake-timer component test, which does not exist yet

3 — The cursor exceeds the frame count

Symptom	Blank stage, or `Cannot read properties of undefined (reading 'array')` in `ArrayView`
Possible causes	(a) `input` changed without dispatching `load`, so the reducer's `length` is stale and larger than `frames.length`; (b) the derived clamp was removed; (c) a saved/imported cursor was dispatched unclamped after a shorter trace was loaded
Debugging steps	Log `playback.cursor`, `playback.length`, `frames.length`, `cursor`. If `playback.length > frames.length`, something changed the trace without a `load`. Also log the order of `setInput` and `dispatch` in `load()` — they are batched, so they land in the same commit
Fix	Dispatch `load` in every path that changes the trace, and keep the derived clamp as a second line of defence
Regression test	`load(20)` → `seek(19)` → `load(3)` must leave `cursor === 0`. Plus a property assertion: for every cursor in range, `frames[cursor]` is defined

4 — The last frame never renders

Symptom	Playback stops one frame short; the completed array only appears after a manual step
Possible causes	(a) Completion computed as `cursor + 1 < state.length` instead of `< state.length - 1`; (b) the timeline's `max` is `frames.length - 2`; (c) the step-forward button's disabled condition is off by one; (d) the generator's final frame is missing
Debugging steps	First check the engine: call the generator in Node and assert `frames.at(-1)` has all indices sorted. If that passes, the bug is in the transport
Fix	The correct completion expression is `playing: state.cursor + 1 < state.length - 1`
Regression test	The existing playback group asserts exactly this for a 3-frame trace

5 — Seek jumps to the wrong frame

Symptom	Dragging the timeline to a displayed position lands on a different frame
Possible causes	(a) The range input's `value` is the derived cursor while `onChange` dispatches the raw event value — a mismatch if the derived clamp is active; (b) the value is not converted with `Number()` so a string reaches the reducer and `Math.min("10", 19)` coerces; (c) `max` does not match `frames.length - 1`
Debugging steps	Log `e.target.value`, the dispatched cursor, and the resulting `cursor`. If the dispatched value is right and the state is wrong, the reducer clamped against a stale `length`
Fix	`seek(Number(e.target.value))`, `max={frames.length - 1}`, `value={cursor}`, with the reducer and the derived cursor agreeing on the same length
Regression test	`seek(-10) → 0` and `seek(999) → length-1` — both already asserted

6 — Step backward does nothing at frame 5

Symptom	The back arrow is unresponsive except at the start
Possible causes	(a) The button is `disabled={cursor === 0}` on the wrong side; (b) the handler dispatches `{type:'step'}` on an action the reducer does not implement, so `default` returns the same state; (c) the handler reads a stale `cursor`
Debugging steps	Log the action object. If the reducer logs an unknown action, the `default` branch is silently swallowing it — that is a reminder that the reducer always returns a state, which makes typos quiet
Fix	Use `seek(cursor - 1)`; make sure the disabled condition is `!cursor` or `cursor === 0`
Regression test	Drive a `seek` sequence in Node and assert each resulting cursor

7 — The keyboard arrows stop working after the first press

Symptom	First arrow press steps once; subsequent presses do nothing
Cause	Stale closure: the effect's dependency array is `[]`, so the handler permanently holds `cursor = 0` and keeps dispatching the same seek
Debugging steps	Log the action objects — you will see an identical `cursor` value every time
Fix	Add `cursor` to the dependency array, or add a relative `step` action so the handler never read `cursor` [PROPOSED]
Regression test	Spy on `addEventListener`/`removeEventListener` and assert the balance after ten cursor changes

8 — Multiple keyboard listeners accumulate

Symptom	One ArrowRight press advances by 3 or 4 frames, and the amount grows over time
Cause	The keyboard effect has no cleanup, so every re-registration adds another live listener
Debugging steps	In DevTools, use `getEventListeners(window)` (Chrome console) and count `keydown` handlers; or log inside the effect and count registrations vs removals
Fix	`return () => window.removeEventListener("keydown", key);`
Regression test	The listener-balance spy described above

9 — Space toggles playback while typing in the dataset field

Symptom	Typing a space in the input both inserts a space and starts playback
Cause	The tag-name guard was removed or is incomplete — the handler must ignore events whose target is `INPUT`, `SELECT`, `TEXTAREA` or `BUTTON`
Debugging steps	Log `e.target.tagName` at the top of the handler before the guard
Fix	Keep the guard, and keep `e.target.isContentEditable` and the modifier-key checks
Regression test	Dispatch a synthetic `keydown` with `target` an input and assert no dispatch occurs

10 — Speed changes have no effect until playback is toggled

Symptom	Changing the speed selector mid-playback does not change the cadence
Cause	`speed` is missing from the effect's dependency array, so the effect never re-runs to schedule with the new delay
Debugging steps	Log the delay inside the effect body; it will keep printing the old value
Fix	Add `speed` to the dependency array

Regression test	Assert the dependency list contains every value the body reads (a lint rule can enforce this)

11 — The animation continues after navigating away

Symptom	Console warning about a state update on an unmounted component, or the timer firing after a route change
Cause	The cleanup is not returned, so React has nothing to cancel on unmount
Debugging steps	Log inside the cleanup; it should print on every dependency change and on unmount
Fix	Always return the cleanup when the effect schedules anything
Regression test	Mount, play, unmount manually and assert no timer was pending (a fake-timer component test)

12 — Play does nothing at the very start (frame 0 of a 2-frame trace)

Symptom	With a minimal trace, Play appears to do nothing
Cause	Not a bug in most cases: `toggle` at `cursor = length - 1` is a replay request. With `length = 2` and `cursor = 0`, play starts, the first `tick` moves to frame 1 and immediately sets `playing: false` — a two-frame trace finishes in one interval
Debugging steps	Check `frames.length`. Every generator emits at least 2 frames even for an empty input, so a short trace is expected for tiny inputs
Fix	No fix required; the observable behaviour is correct. If it is confusing in the UI, that is a UX issue, not a defect
Regression test	Assert `run([]).length >= 2` for all six generators

Engine and frames

13 — The final frame is not sorted

Symptom	The trace ends with an unsorted array
Possible causes	(a) The generator's final `cloneArray()` happens before the last mutation; (b) a loop bound excludes the last element; (c) a swap writes to the wrong index
Debugging steps	Assert `frames.at(-1).array.map(x => x.val)` against `[...input].sort((a,b)=>a-b)` in a Node scratch file. Then bisect: which frame first diverges from the expected state?
Fix	Depends on the generator; the invariant to restore is that the final frame is the fully sorted state
Regression test	The first assertion of the existing sorting group — it covers all six algorithms and 25 inputs

14 — The algorithm returns correct results but the frames are wrong

Symptom	The final frame is sorted but intermediate frames show impossible states (e.g. a value that appears from nowhere, or a sorted region that shrinks)
Possible causes	(a) The frame captured a reference to the live array instead of a clone; (b) `cloneArray` was applied after the mutation instead of before; (c) `sortedIndices` was shared by reference and mutated retroactively
Debugging steps	This is the classic symptom of a missing clone. Check all three clone sites: `cloneArray()`, `{...stats}`, `[...sortedIndices]`. A quick test: mutate `frames[0].array[0].val` and re-read `frames[1].array[0].val` — if it changed, the frames share objects

Fix	Restore the clone at the frame-construction site
Regression test	A "frames are independent" test: mutate one frame and assert its neighbours are unchanged. **This test does not exist yet** — it is the natural companion to the deep-equality test

15 — Duplicate element IDs in a frame

Symptom	React logs "Encountered two children with the same key" during merge sort playback; a bar momentarily fails to update or the array appears to contain two identical values
Cause	Merge sort's buffer write-back: shallow clones preserve `id`, and the main array still holds the originals during the partial write. Measured: 23 of 107 frames on the default input, 141 of 375 on the reverse-24 input
Debugging steps	Log the frame's ids and find duplicates: `new Set(ids).size !== ids.length`. Then look at the frame's `codeLine` — it will be `write_left`, `write_right` or `write_rem`
Fix	Minimal: make the render key unique per frame, `key={item.id + ':' + i}`, which is safe because every frame is a full snapshot. Alternative: re-id the buffered clone at write time. [PROPOSED — not implemented]
Regression test	Extend the existing assertion from the final frame to **every** frame: for each frame, `new Set(frame.array.map(x => x.id)).size === input.length`. That single change would have caught this bug in CI

16 — A merge-sort value appears to vanish mid-animation

Symptom	During the merge phase the rendered array shows a duplicate value and the "missing" value is absent
Cause	The value is in `tempArr`, which no frame exposes. The generator sets `yOffset = 80` to flag it and pushes a `buffer` frame, but `ArrayView` ignores `yOffset`
Debugging steps	Compare the frame's rendered values to the pre-merge frame: the multiset is wrong by exactly the buffered element
Fix	Render the buffer: apply `translateY` when `yOffset` is non-zero, or render a "buffer lane" under the array. Engine change not required. [PROPOSED]
Regression test	Assert that the multiset of values in every frame equals the multiset of the input — a strong property that would also catch bugs 13 and 14

17 — Trace reproducibility fails intermittently

Symptom	`deepEqual(frames, run(input))` fails only occasionally, or between machines
Possible causes	(a) `Math.random()` or `Date.now()` inside a generator; (b) object key iteration order affecting a string built from an object; (c) a `Set`/`Map` with insertion-order dependence; (d) floating-point arithmetic on non-integers
Debugging steps	Diff the two traces frame by frame and print the first index and field that differs. A difference in `msg` points at interpolation; a difference in `array` order points at iteration; a difference only in `currentStats` points at shared-state mutation
Fix	Remove the non-determinism — generators must be pure functions of their input
Regression test	The existing determinism assertion, run repeatedly, plus a test that asserts no generator's source contains `Math.random` (a crude but effective guard)

18 — The generator mutates the caller's array

Symptom	After a run, the original array is reordered; the same-input comparison becomes invalid
Cause	The generator sorted `rawArray` directly, or an element object from `rawArray` was retained and mutated
Debugging steps	`const before = [...input]; algorithm.run(input); assert.deepEqual(input, before)`
Fix	Wrap the values into new objects first — `rawArray.map((val, i) => ({id, val}))` — and mutate only that new array
Regression test	The existing input-preservation assertion, for all six algorithms

19 — Empty input produces a frame claiming a non-existent index is sorted

Symptom	Sorting `[]` with insertion sort yields a seed frame with `sortedIndices: [0]`
Cause	`const sortedIndices = [0]` is unconditional
Debugging steps	Run the generator on `[]` and inspect frame 0
Fix	`const sortedIndices = n > 0 ? [0] : [];` [PROPOSED]
Regression test	For every generator and every input, assert every index in every frame's `sortedIndices` and `activeIndices` is `< input.length`

20 — Frame count is unexpectedly large

Symptom	A trace has 1,154 frames where you expected a few dozen
Cause	Not a bug — it is the instrumentation density. Bubble sort pushes 3–4 frames per inner iteration (window, compare, and two per swap) plus per-pass frames
Debugging steps	Group frames by `codeLine` and count each group: `frames.reduce((acc,f) => (acc[f.codeLine] = (acc[f.codeLine]||0)+1, acc), {})`. The counts will explain the total exactly
Fix	No fix; this is the design. If the count is a problem, the answer is coarser instrumentation or delta frames
Regression test	A frame-count budget assertion per algorithm per input size, so an accidental instrumentation change is caught

Validation and input

21 — Invalid input is accepted

Symptom	`"1, 2, abc"` or a 30-value input is visualized
Possible causes	(a) The generator is called directly instead of through `load()`; (b) a new entry point bypasses `parseInput`; (c) the regex was weakened to allow decimals or exponents
Debugging steps	Search for every call to `algorithms[...].run(` and confirm each is reached from a validated path
Fix	Route all entry points through `parseInput`; the import path already does this by joining to a string first
Regression test	The existing validation group covers `""`, `"1,"`, `"1,,2"`, `"2.5,3"`, `"1,Infinity"`, `"1,1000"` and 25 values

22 — A valid input is rejected

Symptom	A legitimate entry such as `" 4, 8, 15 "` is rejected

Possible causes	(a) The trimming happens after the regex test instead of before; (b) the count check runs on the untrimmed split; (c) an over-strict character class
Debugging steps	Call `parseInput` with the string in Node and read the thrown message — it identifies which of the five rules fired
Fix	The regex is applied to `p.trim()`; the count uses the raw split length. Both must agree on what a "value" is
Regression test	A positive case with surrounding whitespace, as the existing test does for `" -2, 0, 3 "`

23 — Applying the same input resets the animation unexpectedly

Symptom	Pressing Apply on an unchanged dataset restarts playback from frame 0
Cause	Not a bug: `load()` always dispatches `load`, which resets the cursor and stops playback. `parseInput` returns a **new array**, so `useMemo` recomputes even though the contents are identical — because the dependency is compared by reference
Debugging steps	Confirm the array reference changed: `frames` recomputes whenever `load()` runs
Fix	Expected behaviour. If a no-op re-apply were desired, compare array contents before committing [PROPOSED]
Regression test	Not needed; the behaviour is deterministic and correct

24 — The commit is invisible because the animation is still running

Symptom	A user changes the input while playing and does not notice the trace changed
Cause	`load` stops playback and resets the cursor, but there is no explicit visual cue
Debugging steps	Trace `playing` across the commit: it goes `true → false`
Fix	UX: a brief notice ("Trace reloaded") or a transition. The state machine is correct [PROPOSED]
Regression test	Assert `load` yields `playing: false` regardless of prior state — already implied by the reducer's `load` case

Comparison

25 — Comparison starts from different data

Symptom	The two previews look like they are sorting different arrays
Possible causes	(a) The second memo does not depend on `input`, so it is stale; (b) the generator mutates its input, corrupting the second run; (c) the comparison algorithm was changed without re-running the trace
Debugging steps	Log `frames.at(-1).array.map(x=>x.val)` and `otherFrames.at(-1).array.map(x=>x.val)` — both must be the sorted version of the same input
Fix	Ensure `otherFrames`'s dependencies include `input`, and that generators do not mutate. The input-preservation test is the guard
Regression test	For each pair of algorithms, assert both traces' final values equal the same sorted array

26 — Operation counts look inconsistent between panels

Symptom	The inspector shows 12 comparisons while the comparison panel shows 28

Cause	Not a bug: the inspector shows the **current frame's** `currentStats`; the comparison panel shows the **complete run's** `frames.at(-1).currentStats`. Two different quantities, deliberately labelled "LIVE" and "COMPLETE-RUN COMPARISONS"
Debugging steps	Step to the last frame and confirm the inspector matches the panel
Fix	Expected. If it confuses users, label them more emphatically
Regression test	Assert `frames.at(-1).currentStats` is monotone non-decreasing along the trace for all three counters

27 — The comparison preview goes blank

Symptom	The secondary array disappears partway through playback
Cause	Index out of range on `otherFrames[compareCursor]`. The formula can produce `NaN` if `frames.length === 1` (division by zero)
Debugging steps	Log `compareCursor` and `otherFrames.length`. If `compareCursor` is `NaN` or `>= length`, the denominator was zero or the lengths diverged
Fix	Guard the denominator: `frames.length > 1 ? ... : 0`. Practically unreachable because every generator emits ≥ 2 frames, but unguarded
Regression test	For every algorithm pair and a set of inputs, assert `compareCursor` is an integer in `[0, otherFrames.length - 1]` for every primary cursor

28 — Selecting the same algorithm twice

Symptom	Two identical traces and two equal counts
Cause	The comparison `<select>` is not filtered against the primary selection
Fix	Filter the options, or show a hint. Harmless but meaningless as an experiment [PROPOSED]
Regression test	If filtered, assert the option list excludes the current primary

29 — The meters look wrong for very small counts

Symptom	A count of 1 draws a bar as wide as a count of 2
Cause	The `Math.max(2, …)` floor exists so a non-zero count is visible at all
Fix	Expected; the floor is deliberate. A log scale would be misleading
Regression test	Assert the width expression never exceeds 100%

30 — Swapping which algorithm is primary changes the numbers

Symptom	The counts appear to change after swapping primary/secondary
Cause	Should be impossible — the counts come from each trace's final frame and do not depend on which is primary. If it happens, one of the memos is not recomputing, so a stale trace is being displayed against the new `algo`
Debugging steps	Log `algo`, `other` and both `frames.length` values across the swap
Fix	Ensure both memos depend on their own algorithm key and on `input`
Regression test	Assert counts are symmetric: the pair of counts is the same set regardless of order

Persistence and import

31 — A saved experiment disappears after reload

Symptom	The saved list is empty after a refresh
Possible causes	(a) The write threw and was swallowed — but the catch sets a notice, so check for it; (b) storage is disabled (private mode, third-party-cookie blocking) so `setItem` silently fails or throws; (c) the read path's filter rejected the record because the algorithm key changed; (d) the page is on a different origin than when it was saved — e.g. `localhost` versus `127.0.0.1`
Debugging steps	In DevTools → Application → Local Storage, read `algoviz.experiments.v1` directly and inspect its contents. Then evaluate the filter's three conditions against each record by hand
Fix	Depends on the cause. For (d), the honest answer is that localStorage is origin-scoped, which is why export exists
Regression test	Save → clear the in-memory list → re-run the read path → assert the record survives. Requires a DOM environment; not covered today

32 — localStorage contains corrupted data

Symptom	The saved list is empty, or the route crashes on mount
Possible causes	Hand-edited or partially-written JSON; a record whose `input` is not an array; a record whose `algorithm` no longer exists
Debugging steps	Read the key in DevTools and try `JSON.parse` on it in the console
Fix	Already handled — four defence layers on the read path. The gap is that a corrupt store is discarded silently, with no user-facing message [PROPOSED]
Regression test	Seed `localStorage` with `"{"`, `"null"`, `"{}"`, `'[{"algorithm":"nope"}]'` and `'[{"algorithm":"bubble","input":"x"}]'` and assert the component renders an empty list rather than throwing

33 — JSON import crashes the app

Symptom	A red error banner with a parser message, or an unhandled exception
Possible causes	(a) `file.text()` rejected and the `catch` set a message — fine; (b) `data.input` is valid but produces a cursor that is dispatched unclamped; (c) an exception thrown outside the `try`
Debugging steps	Log at each gate in `importExperiment` to find which one fired. If all gates pass but the render throws, the cursor path is the suspect
Fix	The cursor guard `Number.isInteger(data.cursor) ? data.cursor : 0` plus the reducer's clamp should make this unreachable. If a throw still escapes, the `try` block does not cover the render — which is correct, so the fix belongs at the data boundary
Regression test	The nine import cases listed in Chapter 42; none is automated today

34 — Exported file cannot be re-imported

Symptom	The app says "Not a supported AlgoViz experiment" for a file it exported
Possible causes	(a) The file was edited and lost a required field; (b) the export payload and the import schema have drifted — note the export omits `createdAt`, which import does not require, so this is currently consistent; (c) the file is larger than 16 KB
Debugging steps	Open the file and check `version === 1`, that `algorithm` is a current key, and that `input` is an array
Fix	Keep the schema in one place — a shared `EXPERIMENT_VERSION` constant would prevent drift [PROPOSED]

Regression test	A round-trip test: export an in-memory experiment object, re-validate it through the import gates, and assert the values survive

35 — Save reports failure even though storage works

Symptom	"Browser storage is unavailable" when storage is fine
Possible causes	(a) Quota exceeded by another key on the same origin; (b) the payload is not serialisable — it always is here, but a future non-JSON field would throw in `stringify`, which is inside the `try`; (c) storage is disabled for third-party frames
Debugging steps	Log the actual exception — the `catch` currently discards it with a bare `catch {}`, so you have to add a log to see it. That is a real debuggability weakness [PROPOSED: `catch (e) { console.warn(e); ... }`]
Fix	Surface the reason, and distinguish quota from access errors
Regression test	Mock `setItem` to throw `QuotaExceededError` and assert the notice text

Rendering and React

36 — Excessive re-rendering

Symptom	React DevTools Profiler shows the whole component re-rendering on every keystroke
Possible causes	(a) `text` changes on every keystroke by design, so the component re-renders — that is expected; (b) the memo is missing so `frames` rebuilds; (c) an effect dispatches on every render causing a loop
Debugging steps	Profile with "highlight updates", then check whether the expensive work (the `useMemo` bodies) re-executes — instrument the memo body with a counter
Fix	If the memo is working, nothing is wrong: re-rendering a small component tree is cheap. The one genuine inefficiency is the keyboard effect re-registering on every cursor change
Regression test	Assert the generator is called once per input change (spy on the generator)

37 — The wrong frame renders for one paint

Symptom	A flash of the previous trace's frame immediately after changing the input
Cause	Storing `frame` in state instead of deriving it, or synchronising with an effect
Debugging steps	Log `frames.length`, `playback.cursor` and `cursor` in the same render — a render where `cursor >= frames.length` proves the derivation is not in place
Fix	Derive during render: `const cursor = Math.min(playback.cursor, frames.length - 1); const frame = frames[cursor];`
Regression test	A component test asserting no render ever receives `frames[undefined]`; today the invariant is protected only by the derivation itself

38 — An error in one module blanks the whole app

Symptom	One route's exception unmounts everything
Cause	There is no error boundary anywhere in the tree
Fix	Mount a boundary per route, inside `Suspense`, so a crashing module shows a fallback while the shell survives [PROPOSED]
Regression test	A test that throws inside a child and asserts the boundary renders its fallback

39 — A lazy route shows the fallback forever

Symptom	"One moment of curiosity..." never resolves
Possible causes	(a) The chunk 404s after a deploy because the hashed filename changed and the browser holds the old HTML; (b) a dynamic import path typo; (c) the network request failed
Debugging steps	Check the Network tab for a failed `.js` request and the console for a chunk-load error
Fix	For (a), a deploy-version check that reloads on a chunk error; for (b), correct the path — note the build would normally fail on a bad static import, but a dynamic one is only resolved at build time for path existence. Error boundaries plus a reload prompt is the standard remedy [PROPOSED]
Regression test	A build-level assertion that every lazy import produces a chunk (visible in the build output)

40 — CI passes but the feature is broken

Symptom	Green checks, broken behaviour in the browser
Possible causes	(a) The feature lives in a component, and no test renders a component; (b) the assertion is too weak — e.g. only the final frame's IDs are checked, so the merge duplicate-key bug passes; (c) the feature is only reachable through a route nothing tests; (d) a CSS-only break
Debugging steps	Ask "which assertion would have failed if this were broken?" If the answer is none, that is the finding — not a mystery, a coverage gap
Fix	Add the missing assertion at the lowest layer that can express it. The merge bug is the perfect example: the fix is to extend an existing assertion from `frames.at(-1)` to every frame, which is a two-line change that turns a browser-only warning into a CI failure. **That single change is the highest-value test improvement available in this repository**
Regression test	The new assertion itself

MUST KNOW

THE THREE DEBUGGING ANSWERS THAT IMPRESS MOST

1. **Scenario 15/40** — "the assertion is too weak; I check IDs on the last frame but the bug lives in the middle frames." This is the rare answer that identifies a testing defect rather than a code defect, and it comes with a two-line fix that you can describe precisely.
2. **Scenario 14** — "the frames share objects with the live array", because it demonstrates you understand that `cloneArray` per element is what makes a recorded frame immutable.
3. **Scenario 3** — "`length` lives in two places", because it identifies an architectural smell and proposes the structural fix rather than a patch.

CHECKLIST

RED-LINE DRILLS FOR PART XXVIII

- ■ Recite the six-step universal method.
- ■ Explain the engine/reducer/render classification and how to test for it in Node.
- ■ Give the two-line test fix for the merge duplicate-key bug.
- ■ Name three scenarios that are not bugs (12, 23, 26) and why.
- ■ Explain why the `default` branch of a reducer can swallow typos.

PART XXIX

Code walkthrough master section

Thirty code blocks, explained line by line, with the question an interviewer will ask about each.

CHAPTER 55

55 · Thirty walkthroughs

Format: the code, then **What it does** · **Line by line** · **Why it matters** · **The question** · **The trap**.

Block 01 · The algorithm registry

`src/lab/engine.js · lines 1–17 (verbatim)`

```
import { generateBubbleSort } from "../modules/core/algorithm/sorting/algorithms/bubbleSort.js";
import { generateSelectionSort } from "../modules/core/algorithm/sorting/algorithms/selectionSort.js";
import { generateInsertionSort } from "../modules/core/algorithm/sorting/algorithms/insertionSort.js";
import { generateMergeSort } from "../modules/core/algorithm/sorting/algorithms/mergeSort.js";
import { generateQuickSort } from "../modules/core/algorithm/sorting/algorithms/quickSort.js";
import { generateHeapSort } from "../modules/core/algorithm/sorting/algorithms/heapSort.js";

export const algorithms = {
  bubble: {
    name: "Bubble sort",
    run: generateBubbleSort,
    time: "O(n²)",
    space: "O(1)",
    idea: "Compare neighbors. Move the larger value right. Each pass locks one more element.",
    invariant:
      "After each complete pass, the rightmost processed elements are in their final positions.",
  },
```

- **What it does** — declares the six algorithms as data: a generator function plus display metadata.
- **Why it matters** — this object is the only place algorithms are registered, so everything else iterates it: the algorithm `<select>`, the comparison `<select>`, the save/import validators (`Object.hasOwn(algorithms, …)`), the test loop (`Object.entries(algorithms)`), and the label lookup `algoviz.experiments.v1` → display name on the saved list.
- **The question** — "What happens if you add a seventh algorithm?" Add a generator, add an entry here, and everything follows: the dropdowns, the comparison panel, the tests (the loop is data-driven, so it grows automatically), the default-algorithm fallback, and the import validator. The only thing you would have to add by hand is tutorial content, because `simpleSortData[algo] || sortingData[algo]` needs an entry.
- **The trap** — someone will ask whether `time` and `space` are measured. They are **declared strings**, rendered in the panel as static text. Say so before you are asked: "Those are hand-written metadata, not measurements — the app does not benchmark anything."

Block 02 · `parseInput` in full

`src/lab/engine.js · lines 63–79 (verbatim)`

```
export function parseInput(text) {
  if (typeof text !== "string" || !text.trim())
    throw new Error("Enter 2–24 integers, separated by commas.");
  const parts = text.split(",");
  if (parts.length < 2 || parts.length > 24)
    throw new Error(
      "Use 2–24 values. Small inputs keep every operation inspectable.",
    );
  if (parts.some((p) => !/^-?\d+$/.test(p.trim())))
    throw new Error(
      "Use whole numbers separated by commas. Empty values are not allowed.",
    );
  const values = parts.map(Number);
  if (values.some((n) => !Number.isSafeInteger(n) || Math.abs(n) > 999))
    throw new Error("Each number must be between −999 and 999.");
  return values;
}
```

- **What it does** — the single definition of a valid dataset. Throws on violation; returns `number[]` otherwise.
- **Line by line** — (63) a pure exported function; (64–65) type and emptiness, catching `null` from a missing query param; (66) split on commas, keeping empty segments so `"1,,2"` produces a length that fails later; (67–70) the 2–24 count rule with a message that explains why (readability); (71–74) the anchored integer regex on each trimmed part; (75–78) `Number.isSafeInteger` plus the ±999 bound; (79) returns the parsed array.
- **Why it matters** — three callers, all through this one function: the form's `apply()`, the saved-experiment click handler, and the JSON import (which joins the array back to a string first). One validator means the file format cannot drift from the UI.
- **The question** — *"Why split before validating instead of using `Number()` and `isNaN`?"* Because `Number("")` is 0, `Number(" ")` is 0, `Number("1e3")` is 1000, `Number("0x10")` is 16 and `Number("Infinity")` is `Infinity`. Coercion silently accepts all of them. Validating the grammar with an anchored regex and only then converting states the accepted form explicitly.
- **The trap** — `Number.isSafeInteger(n)` can never fail here, because the regex has already restricted the string to digits. It is belt-and-braces rather than load-bearing — but it is what documents the intent, and the ±999 bound is the load-bearing half. Know the difference.

Block 03 · The reducer

`src/lab/engine.js · lines 80–105 (verbatim)`

```
export function playbackReducer(state, action) {
  switch (action.type) {
    case "load":
      return { cursor: 0, length: action.length, playing: false };
    case "toggle":
      return {
        ...state,
        cursor: state.cursor === state.length - 1 ? 0 : state.cursor,
        playing: !state.playing,
      };
    case "tick":
      return {
        ...state,
        cursor: Math.min(state.cursor + 1, state.length - 1),
        playing: state.cursor + 1 < state.length - 1,
      };
    case "seek":
      return {
        ...state,
        cursor: Math.max(0, Math.min(action.cursor, state.length - 1)),
        playing: false,
      };
    default:
      return state;
  }
}
```

Summarised in Chapter 21. **The question** to expect: *"Change `seek` so that seeking to the current position does not pause."* `cursor: action.cursor === state.cursor ? state.playing : false` — and then note that this makes the reducer's behaviour depend on a value-comparison, which is a design decision rather than a bug fix.

Block 04 · The seeded dataset generator

`src/lab/engine.js · lines 106–120 (verbatim)`

```
export function makeDataset(kind, seed = 42) {
  let state = seed >>> 0;
  const random = () => {
    state = (Math.imul(1664525, state) + 1013904223) >>> 0;
    return state / 4294967296;
  };
  const values = Array.from(
    { length: 10 },
    () => 5 + Math.floor(random() * 90),
  );
  if (kind === "reverse") return values.sort((a, b) => b - a);
  if (kind === "sorted") return values.sort((a, b) => a - b);
  if (kind === "duplicates") return values.map((n) => (n % 4) * 15 + 10);
  return values;
}
```

- **What it does** — a deterministic pseudo-random dataset generator with four shapes.
- **Line by line** — (107) coerce the seed to an unsigned 32-bit integer; (108–111) the LCG step with `Math.imul` for 32-bit-correct multiplication and `>>> 0` to keep the state unsigned, returning a float in `[0,1)`; (112–115) exactly ten values in the range 5–94; (116–118) shape transforms; (119) the identity case.
- **Why it matters** — the "shuffle" button and the four presets both go through it, and the tests rely on same-seed equality. It is why "random" in this app is reproducible.
- **The question** — *"Why `Math.imul` and not plain multiplication?"* `1664525 * state` exceeds 2^{53} for large states, and JavaScript numbers lose integer precision beyond that — so the sequence would stop being reproducible. `Math.imul` performs the multiply as a 32-bit operation, matching the LCG's arithmetic definition.
- **The trap** — a questioner may point out that a hand-rolled LCG has poor statistical properties versus `crypto.getRandomValues` or a seeded PRNG library. The answer: *"Correct — it is not for simulation, it is for reproducibility. Its only requirement is that the same seed gives the same ten numbers, which is exactly what my test asserts. If I needed good distribution I would use a library or `crypto`, and I would no longer have reproducibility for free."*

Block 05 · The timer effect

`src/lab/SortingLab.jsx · lines 106–110 (verbatim)`

```
  useEffect(() => {
    if (!playback.playing) return;
    const id = setTimeout(() => dispatch({ type: "tick" }), speed);
    return () => clearTimeout(id);
  }, [playback.playing, playback.cursor, speed]);
```

Covered in Chapters 24–26. **The question**: *"What happens if you remove `playback.cursor` from the array?"* The effect schedules one timeout, the tick changes the cursor, and the effect does not re-run — so playback advances exactly one frame and stops. That is the clearest demonstration of why `cursor` is in the array: it is the re-scheduling trigger.

Block 06 · The keyboard effect

`src/lab/SortingLab.jsx · lines 111–136 (verbatim)`

```
  useEffect(() => {
    const key = (e) => {
      if (
        /INPUT|SELECT|TEXTAREA|BUTTON/.test(e.target.tagName) ||
        e.target.isContentEditable ||
        e.ctrlKey ||
        e.metaKey ||
        e.altKey
      )
        return;
      if (e.code === "Space") {
        e.preventDefault();
        dispatch({ type: "toggle" });
      }
      if (e.key === "ArrowRight") {
        e.preventDefault();
        dispatch({ type: "seek", cursor: cursor + 1 });
      }
      if (e.key === "ArrowLeft") {
        e.preventDefault();
        dispatch({ type: "seek", cursor: cursor - 1 });
      }
    };
    window.addEventListener("keydown", key);
    return () => window.removeEventListener("keydown", key);
  }, [cursor]);
```

- **What it does** — a global `keydown` listener providing Space (play/pause) and the arrow keys (step).
- **Line by line** — (112) the handler; (113–120) the ignore-list for form controls, content-editable targets and modifier keys; (121–124) Space with `preventDefault`; (125–132) the arrows, each seeking `cursor ± 1`; (134) registration; (135) cleanup; (136) the `[cursor]` dependency.
- **Why it matters** — it is the second of the project's two effects and the only place a stale closure would bite.
- **The question** — *"Why does the arrow handler read `cursor` instead of using a functional update?"* Because the reducer's action is an absolute `seek`. React state updaters support functional updates, but `dispatch` with a reducer does not — the action is a value. Hence the effect depends on `cursor`. The cleaner design is a relative action.
- **The trap** — *"a handler that reads state and is registered on `window` must be re-registered when that state changes, or it will be stale — and it must be removed, or it will accumulate."* Say both halves; the second half is what most candidates forget.

Block 07 · The `load` function

`src/lab/SortingLab.jsx · lines 137–147 (verbatim)`

```
  function load(values, algorithm = algo) {
    setInput(values);
    setText(values.join(", "));
    setAlgo(algorithm);
    dispatch({
      type: "load",
      length: algorithms[algorithm].run(values).length,
    });
    setError("");
    setNotice("");
  }
```

- **What it does** — the single entry point for committing a new dataset or algorithm.
- **Line by line** — (137) default parameter `algorithm = algo`; (138) commit the array; (139) normalise the draft text; (140) set the algorithm; (141–144) dispatch `load` with the **length of a freshly computed trace**; (145–146) clear both banners.
- **Why it matters** — every route into the lab funnels through here: apply, presets, shuffle, saved experiments, import.
- **The question** — *"Why do you call `run()` again for the length?"* Because the reducer needs `length` and cannot see the frames. It costs a second full generation per commit. The fix is to

derive `length` from the memo in an effect, or to have the reducer take the frames. [PROPOSED]

- **The trap** — being unaware of the double generation. Volunteer it: "That's a deliberate-looking inefficiency I'd remove — it's a second full trace per input change, which is free at this input cap and would matter at scale."

Block 08 · `apply` and the error path

`src/lab/SortingLab.jsx · lines 148–155 (verbatim)`

```
function apply(e) {
  e?.preventDefault();
  try {
    load(parseInput(text));
  } catch (err) {
    setError(err.message);
  }
}
```

- **What it does** — validates and commits, or reports the validation error.
- **Line by line** — (149) `e?.preventDefault()` — optional chaining because `apply` is called both from the form and programmatically with no event; (150–154) the try/catch around `load(parseInput(text))`.
- **Why it matters** — the failed path touches exactly one piece of state (`error`), which is why a bad input never destroys the working visualisation.
- **The question** — "What state changes when validation fails?" Only `error`. Not `input`, not `frames`, not the cursor, not `playing`. That is the design principle: a failed commit must not destroy the last good state.
- **The trap** — an interviewer may ask why the error is not cleared on the first keystroke. Today it persists until the next successful action, which means an old error can sit under new text. A `setError("")` in the `onChange` handler is the UX refinement [PROPOSED].

Block 09 · Import, gate by gate

`src/lab/SortingLab.jsx · lines 156–180 (verbatim)`

```
async function importExperiment(event) {
  const file = event.target.files?.[0];
  event.target.value = "";
  if (!file) return;
  try {
    if (file.size > 16384)
      throw new Error("Experiment files must be smaller than 16 KB.");
    const data = JSON.parse(await file.text());
    if (
      data?.version !== 1 ||
      !Object.hasOwn(algorithms, data.algorithm) ||
      !Array.isArray(data.input)
    )
      throw new Error("Not a supported AlgoViz experiment.");
    const values = parseInput(data.input.join(","));
    load(values, data.algorithm);
    dispatch({
      type: "seek",
      cursor: Number.isInteger(data.cursor) ? data.cursor : 0,
    });
    setNotice("Experiment imported. Ready to inspect.");
  } catch (err) {
    setError(err.message);
  }
}
```

Covered in Chapter 42. **The question**: *"What is `event.target.value = ""` for?"* It clears the file input so selecting the same file twice fires a second `change` event. Without it, the second selection is a no-op because the input's value never changed.

Block 10 · Save

src/lab/SortingLab.jsx · lines 181–197 (verbatim)

```
  function save() {
    const experiment = {
      version: 1,
      algorithm: algo,
      input: [...input],
      cursor,
      createdAt: new Date().toISOString(),
    };
    const next = [experiment, ...saved].slice(0, 6);
    try {
      localStorage.setItem("algoviz.experiments.v1", JSON.stringify(next));
      setSaved(next);
      setNotice("Saved on this device.");
    } catch {
      setNotice("Browser storage is unavailable. Use Export instead.");
    }
  }
```

Covered in Chapter 41. **The question**: *"Why is `setSaved` inside the try, after `setItem`?"* So the in-memory list only advances if the write succeeded. If the write throws and the state had already advanced, the UI would show a saved experiment that does not exist in storage — the list would survive until reload and then silently lose an entry.

Block 11 · The derivation block

src/lab/SortingLab.jsx · lines 198–203 (verbatim)

```
  const meta = algorithms[algo],
    stats = frame.currentStats || {},
    last = frames.at(-1).currentStats;
  const compareCursor = otherFrames.length
    ? Math.round((cursor / (frames.length - 1)) * (otherFrames.length - 1))
    : 0;
```

- **What it does** — computes everything the render needs from state, with no state of its own.
- **Line by line** — (198–200) `meta` (the registry entry), `stats` (the current frame's counters, defaulting to `{}` so a missing field renders as 0 rather than throwing), `last` (the complete-run counters); (201–203) the percentage-aligned secondary cursor.
- **Why it matters** — this one expression is the entire comparison alignment feature.
- **The question** — *"Why is `stats = frame.currentStats || {}`?"* Defensive: if a generator omitted `currentStats`, reading `.comps` would throw. With `{}`, the tile renders 0. It is a cheap guard on an interface that six independent generators implement.
- **The trap** — the `|| {}` masks a schema violation rather than surfacing it. A stricter design would assert the field's presence in a test and let a missing field fail loudly in development.

Block 12 · `ArrayView`

src/lab/SortingLab.jsx · lines 22–48 (verbatim)

```
function ArrayView({ frame, compact = false }) {
  const max = Math.max(1, ...frame.array.map((x) => Math.abs(x.val)));
  return (
    <div
      className={`av-array ${compact ? "compact" : ""}`}
      aria-label="Array execution state"
    >
      {frame.array.map((item, i) => {
        const active = frame.activeIndices?.includes(i),
          sorted = frame.sortedIndices?.includes(i),
          pivot = frame.pivotIndex === i;
        return (
          <div className="av-array-column" key={item.id}>
            <div
              className={`av-bar ${active ? "active" : ""} ${sorted ? "sorted" : ""} ${pivot ? "pivot" : ""}`}
              style={{ height: `${25 + (Math.abs(item.val) / max) * 75}%` }}
            >
              <strong>{item.val}</strong>
              <span>{pivot ? "P" : active ? "●" : sorted ? "✓" : ""}</span>
            </div>
            <small>{i}</small>
          </div>
        );
      })}
    </div>
  );
}
```

- **What it does** — renders one frame as a row of bars.
- **Line by line** — (23) the height normaliser: the largest absolute value in this frame, floored at 1 to avoid division by zero; (29–33) map over the array, deriving three booleans; (34) `key={item.id}` — stable element identity; (36) the class list combining active/sorted/pivot; (37) the height percentage `25 + (|val| / max) * 75`; (39–40) the value label and the state glyph; (42) the index label.
- **Why it matters** — the entire visualisation is this one pure function of a frame. Nothing else in the UI reads the array.
- **The question** — *"Why `key={item.id}` and not the index?" Because identity is what the visualisation is about. With index keys, React would reuse the DOM node positionally, and since the trace shows movements, a swap would look like a value change instead of a move. The ID travels with the element through every swap. Also: with duplicate values* in the array, only the ID distinguishes them.
- **The trap** — this is where the merge-sort duplicate-key bug lives. Know it: "Except in merge sort's buffer write-back frames, where the same id temporarily exists twice — 23 of 107 frames on the default input. I'd fix it by making the key unique per frame, since every frame is a full snapshot."

Block 13 · The status tag ternary chain

`src/lab/SortingLab.jsx · lines 305–313 (verbatim)`

```
                <span className="av-tag">
                  {cursor === frames.length - 1
                    ? "Completed"
                    : playback.playing
                      ? "Running"
                      : cursor
                        ? "Paused"
                        : "Ready"}
                </span>
```

- **What it does** — derives "Completed" / "Running" / "Paused" / "Ready" from two values.
- **The question** — "Why is the order of the conditions important?" Because `cursor === frames.length - 1` must be tested first: a completed trace with `playing` false would otherwise fall through to the `cursor ? "Paused" : "Ready"` branch and report "Paused" for a finished run. Order encodes precedence.

Block 14 · The timeline range input

src/lab/SortingLab.jsx · lines 356–368 (verbatim)

```
              <div className="av-timeline">
                <input
                  aria-label="Execution timeline"
                  type="range"
                  min="0"
                  max={frames.length - 1}
                  value={cursor}
                  onChange={(e) => seek(Number(e.target.value))}
                />
                <span>
                  {cursor} / {frames.length - 1}
                </span>
              </div>
```

- **What it does** — the scrubber.
- **Line by line** — `min="0"`, `max={frames.length - 1}`, `value={cursor}` (the derived cursor), `onChange` seeking `Number(e.target.value)`, and a text label showing `cursor / frames.length - 1`.
- **Why it matters** — it is one of five controls that all reduce to `seek`.
- **The question** — *"Why `Number(...)`?"* Because a DOM input's value is always a string. `Math.min("10", 19)` happens to work by coercion, but `"10" < 19` and arithmetic on a string are the kind of implicit coercion that eventually bites. Converting at the boundary is explicit.
- **The trap** — "what is the maximum value of this input?" `frames.length - 1`. If it were `frames.length` the user could select an index that does not exist, and the reducer would clamp it — a silently ignored interaction.

Block 15 · The transport buttons

src/lab/SortingLab.jsx · lines 371–408 (verbatim)

```
                <button
                  className="av-icon-button"
                  onClick={() => seek(0)}
                  aria-label="Restart"
                >
                  <SkipBack size={17} />
                </button>
                <button
                  className="av-icon-button"
                  disabled={!cursor}
                  onClick={() => seek(cursor - 1)}
                  aria-label="Step backward"
                >
                  <ChevronLeft size={19} />
                </button>
                <button
                  className="av-button primary"
                  onClick={() => dispatch({ type: "toggle" })}
                >
                  {playback.playing ? (
                    <Pause size={17} />
                  ) : (
                    <Play size={17} />
                  )}{" "}
                  {playback.playing
                    ? "Pause"
                    : cursor === frames.length - 1
                      ? "Replay"
                      : "Play"}
                </button>
                <button
                  className="av-icon-button"
                  disabled={cursor === frames.length - 1}
                  onClick={() => seek(cursor + 1)}
                  aria-label="Step forward"
                >
                  <ChevronRight size={19} />
                </button>
```

- **What it does** — restart, step back, play/pause/replay, step forward.
- **The question** — "Explain the disabled conditions." Back: `disabled={!cursor}` — falsy only at 0 (given the clamp, `cursor` is always a non-negative integer). Forward: `disabled={cursor === frames.length - 1}`. The asymmetry is idiomatic rather than meaningful.
- **The trap** — "what does the Play button do at the last frame?" It shows "Replay" and `toggle` resets the cursor to 0. That behaviour lives in the reducer, not the button — the button only changes its label.

Block 16 · Speed select

`src/lab/SortingLab.jsx · lines 410–421 (verbatim)`

```
                <label className="av-speed">
                  Speed
                  <select
                    value={speed}
                    onChange={(e) => setSpeed(Number(e.target.value))}
                  >
                    <option value={600}>0.5×</option>
                    <option value={300}>1×</option>
                    <option value={160}>2×</option>
                    <option value={50}>6×</option>
                  </select>
                </label>
```

- **What it does** — four preset delays.
- **The question** — "Are the labels accurate?" 600/300 = 0.5×, 300/300 = 1× correct; 300/160 = 1.875× but labelled 2×; 300/50 = 6× correct. Three of four are exact; the 2× label is rounded. Say it before you are asked.

Block 17 · The state inspector

`src/lab/SortingLab.jsx · lines 435–462 (verbatim)`

```
              <div className="av-metrics">
                <div>
                  <span>Comparisons</span>
                  <strong>{stats.comps || 0}</strong>
                </div>
                <div>
                  <span>Passes / calls¹</span>
                  <strong>{stats.passes || 0}</strong>
                </div>
                <div>
                  <span>Recorded exchanges²</span>
                  <strong>{stats.swaps || 0}</strong>
                </div>
              </div>
              <div className="av-variable-list">
                <div>
                  <span>active indices</span>
                  <code>{JSON.stringify(frame.activeIndices || [])}</code>
                </div>
                <div>
                  <span>pivot index</span>
                  <code>{frame.pivotIndex ?? "—"}</code>
                </div>
                <div>
                  <span>sorted indices</span>
                  <code>{JSON.stringify(frame.sortedIndices || [])}</code>
                </div>
              </div>
```

- **What it does** — three metric tiles plus three raw-value rows.
- **The question** — "Which frame's counters are these?" The current frame's — `stats = frame.currentStats`. The comparison panel shows complete-run counters. Two different things on one screen, deliberately labelled "LIVE" and "COMPLETE-RUN COMPARISONS".

Block 18 · The comparison panel

`src/lab/SortingLab.jsx · lines 513–539 (verbatim)`

```
              <div className="av-compare-results">
                <span className="av-eyebrow">COMPLETE-RUN COMPARISONS</span>
                {[
                  [meta.name, last.comps],
                  [
                    algorithms[other].name,
                    otherFrames.at(-1).currentStats.comps,
                  ],
                ].map(([name, count], i) => (
                  <div key={i}>
                    <div>
                      <span>{name}</span>
                      <strong>{count}</strong>
                    </div>
                    <div className="av-meter">
                      <i
                        style={{
                          width: `${Math.max(2, (count / Math.max(1, last.comps, otherFrames.at(-1).currentStats.comps)) * 100)}%`,
                        }}
                      />
                    </div>
                  </div>
                ))}
                <p>
                  Actual comparison events from the same input. No wall-clock
                  claims: animation speed is not algorithm performance.
                </p>
```

- **What it does** — two labelled counts and two proportional meters.
- **Line by line** — the array of `[name, count]` pairs; the meter width expression with its `Math.max(2, …)` visibility floor; the explanatory paragraph.
- **The question** — "What does the meter width mean?" Only the ratio between two counts, scaled to the larger. Not a score, not a percentage of anything meaningful.
- **The trap** — "so the wider bar is the faster algorithm" — an invitation to overclaim. Correct answer: "No. It is the algorithm that performed more comparisons on this input. That is a count of decisions, not a measurement of speed, and the panel text says so."

Block 19 · The page footnote

`src/lab/SortingLab.jsx · lines 587–592 (verbatim)`

```
        <p className="av-fine">
          ¹ Passes/calls are implementation-specific and should not be compared
          across algorithms. ² Legacy generators count exchanges differently;
          comparison mode deliberately uses comparisons only. Trace storage is
          bounded by the 24-element input limit.
        </p>
```

- **What it does** — three superscripted disclaimers.
- **Why it matters** — this is the most interview-valuable twenty lines in the file. It documents that pass/call counters are implementation-specific, that exchange counters are counted differently per generator, that comparison mode uses comparisons only, and that trace storage is bounded by the 24-element cap.
- **The question** — "Did you write these because you were worried about criticism?" The best answer: "I wrote them because the alternative was a UI that implies more than the code measures. A comparison panel with two bars invites the reader to rank them; the footnote is what stops that being a claim I can't support."

Block 20 · Bubble sort, the whole generator

`src/modules/core/algorithm/sorting/algorithms/bubbleSort.js (verbatim, 93 lines)`

```
export const generateBubbleSort = (rawArray) => {
    let arr = rawArray.map((val, index) => ({
        id: `block-${index}`,
        val: val
    }));

    const frames = [];
    const n = arr.length;
    const sortedIndices = [];
    let stats = { passes: 0, comps: 0, swaps: 0 };

    const cloneArray = () => arr.map(item => ({ ...item }));

    frames.push({
        array: cloneArray(), activeIndices: [], sortedIndices: [...sortedIndices],
        isSwapping: false, minimumIndex: null, codeLine: 'start',
        currentStats: { ...stats },
        msg: "System Ready. Initializing Bubble Sort."
    });

    for (let i = 0; i < n; i++) {
        stats.passes++;

        frames.push({
            array: cloneArray(), activeIndices: [], sortedIndices: [...sortedIndices],
            isSwapping: false, minimumIndex: null, codeLine: 'outer',
            currentStats: { ...stats },
            msg: `Pass ${stats.passes} of ${n}.`
        });

        for (let j = 0; j < n - i - 1; j++) {
            frames.push({
                array: cloneArray(), activeIndices: [j, j + 1], sortedIndices: [...sortedIndices],
                isSwapping: false, minimumIndex: null, codeLine: 'inner',
                currentStats: { ...stats },
                msg: `Scanning window at index ${j} and ${j + 1}.`
            });

            stats.comps++;
            frames.push({
                array: cloneArray(), activeIndices: [j, j + 1], sortedIndices: [...sortedIndices],
                isSwapping: false, minimumIndex: null, codeLine: 'compare',
                currentStats: { ...stats },
                msg: `Comparing ${arr[j].val} and ${arr[j + 1].val}. Is ${arr[j].val} > ${arr[j + 1].val}?`
            });

            if (arr[j].val > arr[j + 1].val) {
                stats.swaps++;
                frames.push({
                    array: cloneArray(), activeIndices: [j, j + 1], sortedIndices: [...sortedIndices],
                    isSwapping: true, minimumIndex: null, codeLine: 'swap',
                    currentStats: { ...stats },
                    msg: `Yes! ${arr[j].val} > ${arr[j + 1].val}. Swapping blocks...`
                });

                let temp = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = temp;

                frames.push({
                    array: cloneArray(), activeIndices: [j, j + 1], sortedIndices: [...sortedIndices],
                    isSwapping: false, minimumIndex: null, codeLine: 'swap',
                    currentStats: { ...stats },
                    msg: `Swap complete.`
                });
            } else {
                frames.push({
```

… continued (2/2)

```
                    array: cloneArray(), activeIndices: [j, j + 1], sortedIndices: [...sortedIndices],
                    isSwapping: false, minimumIndex: null, codeLine: 'compare',
                    currentStats: { ...stats },
                    msg: `No swap needed. Moving window forward.`
                });
            }
        }

        sortedIndices.push(n - i - 1);
        frames.push({
            array: cloneArray(), activeIndices: [], sortedIndices: [...sortedIndices],
            isSwapping: false, minimumIndex: null, codeLine: 'outer',
            currentStats: { ...stats },
            msg: `Pass complete. Block locked into its final sorted position.`
        });
    }

    frames.push({
        array: cloneArray(), activeIndices: [], sortedIndices: [...Array(n).keys()],
        isSwapping: false, minimumIndex: null, codeLine: 'start',
        currentStats: { ...stats },
        msg: "Bubble Sort Complete! Array is fully sorted."
    });

    return frames;
};
```

Covered in Chapters 31–33. **The question**: "Where exactly is the trace made immutable?" Two places: `cloneArray()` on every push (so the array is copied), and `{...stats}` on every push (so the counters are copied). Everything else in a frame is a primitive, a fresh literal, or a spread of `sortedIndices`. Name all three.

Block 21 · Merge sort's buffer write-back

`src/modules/core/algorithm/sorting/algorithms/mergeSort.js · lines 80–122 (verbatim)`

```
        let tempArr = [];
        for(let i = low; i <= high; i++) tempArr.push({...arr[i]});

        let leftIdx = 0;
        let rightIdx = mid - low + 1;
        let k = low;

        while (leftIdx <= mid - low && rightIdx <= high - low) {
            stats.comps++;
            const actualLeftIdx = low + leftIdx;
            const actualRightIdx = low + rightIdx;

            frames.push({
                array: cloneArray(), activeIndices: [actualLeftIdx, actualRightIdx], sortedIndices: [...sortedIndices],
                isSwapping: false, pivotIndex: null, subArrayRange: [low, high], codeLine: 'compare',
                currentStats: { ...stats }, msg: `Comparing ${tempArr[leftIdx].val} and ${tempArr[rightIdx].val}.`
            });

            if (tempArr[leftIdx].val <= tempArr[rightIdx].val) {
                stats.swaps++;
                arr[k] = tempArr[leftIdx];
                arr[k].yOffset = 0;

                frames.push({
                    array: cloneArray(), activeIndices: [k], sortedIndices: [...sortedIndices],
                    isSwapping: true, pivotIndex: null, subArrayRange: [low, high], codeLine: 'write_left',
                    currentStats: { ...stats }, msg: `${arr[k].val} is smaller. Writing to main array.`
                });
                leftIdx++;
            } else {
                stats.swaps++;
                arr[k] = tempArr[rightIdx];
                arr[k].yOffset = 0;

                frames.push({
                    array: cloneArray(), activeIndices: [k], sortedIndices: [...sortedIndices],
                    isSwapping: true, pivotIndex: null, subArrayRange: [low, high], codeLine: 'write_right',
                    currentStats: { ...stats }, msg: `${arr[k].val} is smaller. Writing to main array.`
                });
                rightIdx++;
            }
            k++;
        }
```

- **What it does** — copies the window into a buffer and merges it back.
- **Line by line** — (81) the shallow-clone copy into `tempArr`; (83–85) window-local and array-global index bookkeeping; (87) the merge loop condition; (92–96) the comparison frame; (98–108) the left branch, writing `tempArr[leftIdx]` into `arr[k]` and pushing a `write_left` frame; (110–120) the right branch; (124–131) the "remaining elements" announcement; (133–157) the two tail loops that copy whatever is left.
- **Why it matters** — it is the source of the duplicate-ID frames, the reason the merged value appears to vanish, and the reason merge sort's "swaps" counter is really a write counter.
- **The question** — "Is this in-place?" No — it allocates an O(n) buffer per merge, which is why the algorithm's declared auxiliary space is O(n). The recursion is O(log n) deep on top of that.
- **The trap** — "why does the array show a duplicate value?" Because the buffered element is a shallow clone that keeps its `id`, and the write-back overwrites the left of the window while the originals are still present. This is the bug; own it.

Block 22 · Quick sort's partition

`src/modules/core/algorithm/sorting/algorithms/quickSort.js · lines 76–122 (verbatim)`

```
    function partition(low, high) {
        const pivot = arr[high].val;
        let i = low - 1;

        frames.push({
            array: cloneArray(), activeIndices: [], sortedIndices: [...sortedIndices],
            isSwapping: false, pivotIndex: high, subArrayRange: [low, high], codeLine: 'partition',
            currentStats: { ...stats }, msg: `Selecting ${pivot} at index ${high} as the pivot.`
        });

        for (let j = low; j < high; j++) {
            stats.comps++;
            frames.push({
                array: cloneArray(), activeIndices: [j], sortedIndices: [...sortedIndices],
                isSwapping: false, pivotIndex: high, subArrayRange: [low, high], codeLine: 'compare',
                currentStats: { ...stats }, msg: `Comparing ${arr[j].val} with pivot ${pivot}.`
            });

            if (arr[j].val <= pivot) {
                i++;

                if (i !== j) {
                    stats.swaps++;
                    frames.push({
                        array: cloneArray(), activeIndices: [i, j], sortedIndices: [...sortedIndices],
                        isSwapping: true, pivotIndex: high, subArrayRange: [low, high], codeLine: 'swap',
                        currentStats: { ...stats }, msg: `${arr[j].val} <= ${pivot}. Swapping elements.`
                    });

                    let temp = arr[i];
                    arr[i] = arr[j];
                    arr[j] = temp;

                    frames.push({
                        array: cloneArray(), activeIndices: [i, j], sortedIndices: [...sortedIndices],
                        isSwapping: false, pivotIndex: high, subArrayRange: [low, high], codeLine: 'swap',
                        currentStats: { ...stats }, msg: `Swap complete.`
                    });
                } else {
                    frames.push({
                        array: cloneArray(), activeIndices: [i, j], sortedIndices: [...sortedIndices],
                        isSwapping: false, pivotIndex: high, subArrayRange: [low, high], codeLine: 'swap',
                        currentStats: { ...stats }, msg: `${arr[j].val} <= ${pivot}, but already in correct position.`
                    });
                }
            }
        }
```

- **What it does** — Lomuto partition around the last element.
- **Line by line** — (77) the pivot **value** is captured, so later swaps cannot change the pivot; (78) `i = low - 1`, the boundary of the ≤ region; (80–84) the pivot-selection frame; (86) the scan, excluding `high`; (87–92) the comparison frame and increment; (94) the `<=` test — this is the stability-breaking and worst-case-driving comparison; (95–120) the conditional swap with both frames, plus the "already in position" branch; (124–147) the pivot placement with its own pair of

frames and an else branch; (149) returns the pivot's final index.

- **Why it matters** — the pivot's index is final the instant the partition returns, which is why `quickSortHelper` pushes it into `sortedIndices` immediately afterwards. That is the clearest invariant in the project.
- **The question** — *"Why capture `pivot = arr[high].val` before the loop instead of comparing against `arr[high]`?"* Because the loop swaps `arr[i]` and `arr[j]`, and if the pivot were read from the array it could be moved by a swap — the comparison would then be against the wrong value. Capturing the value is what makes the partition correct.
- **The trap** — "is it stable?" No. `<=` sends equal elements left, and the long-distance swaps reorder them.

Block 23 · Heap sort's `heapify`

`src/modules/core/algorithm/sorting/algorithms/heapSort.js · lines 20–71 (verbatim)`

```
    function heapify(size, i, phaseMsg) {
        let maxIdx = i;
        const left = 2 * i + 1;
        const right = 2 * i + 2;

        let nodesToHighlight = [i];
        if (left < size) nodesToHighlight.push(left);
        if (right < size) nodesToHighlight.push(right);

        frames.push({
            array: cloneArray(), activeIndices: nodesToHighlight, sortedIndices: [...sortedIndices],
            isSwapping: false, pivotIndex: null, subArrayRange: [], codeLine: 'pointers',
            currentStats: { ...stats }, msg: `${phaseMsg}: Evaluating Parent [${arr[i].val}] at index ${i}.`
        });

        stats.comps++;
        if (left < size && arr[left].val > arr[maxIdx].val) {
            maxIdx = left;
        }

        stats.comps++;
        if (right < size && arr[right].val > arr[maxIdx].val) {
            maxIdx = right;
        }

        frames.push({
            array: cloneArray(), activeIndices: nodesToHighlight, sortedIndices: [...sortedIndices],
            isSwapping: false, pivotIndex: null, subArrayRange: [], codeLine: 'compare_children',
            currentStats: { ...stats }, msg: `Comparing with children. Largest value is ${arr[maxIdx].val}.`
        });

        if (maxIdx !== i) {
            stats.swaps++;
            frames.push({
                array: cloneArray(), activeIndices: [i, maxIdx], sortedIndices: [...sortedIndices],
                isSwapping: true, pivotIndex: null, subArrayRange: [], codeLine: 'swap_heapify',
                currentStats: { ...stats }, msg: `Sinking down: Swapping ${arr[i].val} with ${arr[maxIdx].val}.`
            });

            let temp = arr[i];
            arr[i] = arr[maxIdx];
            arr[maxIdx] = temp;

            frames.push({
                array: cloneArray(), activeIndices: [i, maxIdx], sortedIndices: [...sortedIndices],
                isSwapping: false, pivotIndex: null, subArrayRange: [], codeLine: 'swap_heapify',
                currentStats: { ...stats }, msg: `Recursively heapifying the affected sub-tree.`
            });

            heapify(size, maxIdx, phaseMsg);
        }
    }
```

- **What it does** — sift down, recursively, with two unconditional comparison increments.
- **Line by line** — (21–27) compute the candidate children and build the highlight list; (29–33) the `pointers` frame; (35–38) the left-child comparison with an unconditional increment; (40–43) the right-child comparison, also unconditional; (45–49) the `compare_children` frame; (51–70) the swap, the two frames, and the recursive call on the affected subtree.

- **Why it matters** — the unconditional increments are why heap sort's comparison counter is inflated on leaf nodes, and why the counters must not be compared across algorithms.
- **The question** — "Why is this recursive when a loop would be O(1) space?" Because recursion is what a reader expects from `heapify` and it keeps the code aligned with the pseudocode, while the panel honestly declares `O(log n)` auxiliary space with a footnote. Converting it to a `while (true)` sift loop is the O(1) version and is what I would write on a whiteboard.
- **The trap** — "how many comparisons does heapify perform?" Careful: it increments twice per call regardless of whether the children exist. The number of comparisons evaluated is at most two and often zero.

Block 24 · The `windowTrace` bug flag

`src/lab/engine.js · lines 121–151 (verbatim)`

```
export function windowTrace(values, target, buggy = false) {
  let left = 0,
    sum = 0,
    best = Infinity;
  const trace = [];
  for (let right = 0; right < values.length; right++) {
    sum += values[right];
    trace.push({
      left,
      right,
      sum,
      best,
      action: `Add ${values[right]} to the window.`,
    });
    while (sum >= target) {
      if (!buggy || sum === target) best = Math.min(best, right - left + 1);
      trace.push({
        left,
        right,
        sum,
        best,
        action:
          buggy && sum !== target
            ? `${sum} ≠ ${target}: incorrectly skip a qualifying window.`
            : `${sum} ≥ ${target}: record length ${right - left + 1}.`,
      });
      sum -= values[left++];
    }
  }
  return { trace, result: best === Infinity ? 0 : best };
}
```

- **What it does** — one function producing both the correct and the deliberately buggy sliding-window behaviour.
- **Line by line** — (122–124) window state and `best = Infinity`; (126–134) expand right, push an expand frame; (135) the shrink loop; (136) the **entire bug**, in one line: `if (!buggy || sum === target)`; (137–146) the record/explanation frame with a different message for the buggy branch; (147) shrink; (150) normalise `Infinity` to 0.
- **Why it matters** — this is the counterexample lab's engine. Two behaviours in one function guarantees they share the same framing and differ only in the condition, which is exactly what makes the comparison instructive.
- **The question** — "Isn't having a buggy implementation in production code dangerous?" It is unreachable unless the `buggy` flag is passed, which happens in exactly two places: the counterexample component and the tests. The value is that it makes the detector testable — if `findCounterexample` stopped finding divergences, the test fails. That is a "test the instrument" pattern.
- **The trap** — note that `windowTrace` does **not** guard against non-positive input, while `generateWindow` does. So calling the reference directly on negative numbers gives a wrong answer silently. I measured it: `[-1,3,2]` with target 3 gives 2 instead of 1.

Block 25 · The bounded search

src/lab/engine.js · lines 152–169 (verbatim)

```
export function findCounterexample() {
  // Bounded exhaustive search in increasing length and lexicographic order.
  for (let length = 1; length <= 3; length++) {
    for (let code = 0; code < 4 ** length; code++) {
      const values = Array.from(
        { length },
        (_, i) => 1 + (Math.floor(code / 4 ** (length - i - 1)) % 4),
      );
      for (let target = 1; target <= 8; target++) {
        const correct = windowTrace(values, target),
          buggy = windowTrace(values, target, true);
        if (correct.result !== buggy.result)
          return { values, target, correct, buggy };
      }
    }
  }
  return null;
}
```

Covered in Chapter 46. **The question**: *"Is [2] with target 1 the smallest counterexample?"* The precise answer: it is the first failure in the defined enumeration order — increasing length, then lexicographic by the digit scheme, then increasing target. It is not a minimality proof, and your own UI says exactly that.

Block 26 · The test harness for sorting

tests/engine.test.js · lines 12–36 (verbatim)

```
for (const [id, algorithm] of Object.entries(algorithms)) {
  test(`${id}: correctness, input preservation and reproducible trace`, () => {
    for (const input of [
      [],
      [1],
      [2, 2, 1],
      [-4, 0, 3, -1],
      [5, 4, 3, 2, 1],
      ...Array.from({ length: 20 }, (_, seed) => makeDataset("random", seed)),
    ]) {
      const before = [...input],
        frames = algorithm.run(input);
      assert.deepEqual(
        frames.at(-1).array.map((x) => x.val),
        [...input].sort((a, b) => a - b),
      );
      assert.deepEqual(input, before);
      assert.deepEqual(frames, algorithm.run(input));
      assert.equal(
        new Set(frames.at(-1).array.map((x) => x.id)).size,
        input.length,
      );
    }
  });
}
```

- **What it does** — one data-driven test group per algorithm, asserting four properties over 25 inputs.
- **Line by line** — (12) iterate the registry; (13) the group name including the algorithm key; (14–21) the input list; (22–23) snapshot the input and run the generator; (24–27) sortedness; (28) input preservation; (29) trace reproducibility; (30–33) final-frame ID uniqueness.
- **Why it matters** — it is the entire correctness story for the engine, and it is 24 lines long because the registry is data.
- **The question** — "Which assertion finds the most bugs per line?" The `deepEqual(frames, algorithm.run(input))` on line 29. It compares the whole recording including every message string and counter value, so any non-determinism anywhere in a generator fails it.
- **The trap** — "does it check every frame's IDs?" No — only the final frame. That is precisely why the merge duplicate-key bug survives CI. Saying this unprompted is the single strongest thing

you can do in this section.

Block 27 · The brute-force oracle

tests/engine.test.js · lines 82–101 (verbatim)

```
test("window reference matches brute force within bounded domain", () => {
  for (let code = 0; code < 64; code++) {
    const a = [
      (code % 4) + 1,
      (Math.floor(code / 4) % 4) + 1,
      (Math.floor(code / 16) % 4) + 1,
    ];
    for (let target = 1; target <= 12; target++) {
      let best = Infinity;
      for (let l = 0; l < a.length; l++) {
        let sum = 0;
        for (let r = l; r < a.length; r++) {
          sum += a[r];
          if (sum >= target) best = Math.min(best, r - l + 1);
        }
      }
      assert.equal(windowTrace(a, target).result, best === Infinity ? 0 : best);
    }
  }
});
```

- **What it does** — 768 comparisons against an independently written reference.
- **Line by line** — (83–88) build the array from base-4 digits; (89) targets 1–12; (90–97) the oracle: every start, every end, running sum, record the shortest qualifying length; (98) compare the implementation against it.
- **Why it matters** — it is the only place in the project where correctness is established against something other than my own code.
- **The question** — "Why is the oracle written in the test file rather than imported?" So it cannot share code with the implementation. If both used a shared helper, a bug in the helper would infect both and the test would pass while both were wrong — correlated failure.
- **The trap** — the oracle uses `sum >= target`, the same specification as the implementation. That is required (it is the definition of the problem) but it means the tests verify agreement with the specification, not the specification's correctness.

Block 28 · The routing table

src/pages/Home.jsx · lines 39–100 (verbatim)

```
        <Routes>
          <Route path="/" element={<Overview />} />
          <Route path="/library" element={<Overview library />} />
          <Route path="/lab" element={<SortingLab />} />
          <Route path="/counterexample" element={<Counterexample />} />
          <Route path="/modules/core/simple-sort" element={<SortingLab />} />
          <Route
            path="/modules/core/advanced-sort"
            element={<SortingLab key="advanced" />}
          />
          <Route
            path="/modules/core/binary-search"
            element={<BinarySearch />}
          />
          <Route path="/modules/core/n-queens" element={<NQueens />} />
          <Route
            path="/modules/patterns/two-pointers"
            element={<TwoPointers />}
          />
          <Route
            path="/modules/patterns/fast-slow"
            element={<FastSlowPointers />}
          />
          <Route
            path="/modules/patterns/bs-variants"
            element={<BinaryVariants />}
          />
          <Route path="/modules/patterns/prefix-sum" element={<PrefixSum />} />
          <Route
            path="/modules/patterns/sliding-window"
            element={<SlidingWindow />}
          />
          <Route
            path="/modules/patterns/merge-intervals"
            element={<MergeIntervals />}
          />
          <Route
            path="/modules/core/tree-traversal"
            element={<TreeTraversal />}
          />
          <Route path="/modules/core/pathfinding" element={<Pathfinder />} />
          <Route
            path="*"
            element={
              <Shell>
                <main className="av-page av-loading">
                  <div className="av-eyebrow">
                    404 / OUTSIDE THE SEARCH SPACE
                  </div>
                  <h1>Experiment not found.</h1>
                  <p>
                    That path does not lead to an experiment. Let’s find you a
                    new starting point.
                  </p>
                  <Link className="av-button primary" to="/library">
                    ← Explore the library
                  </Link>
                </main>
              </Shell>
            }
          />
        </Routes>
```

Covered in Chapter 49. **The question**: *"What is `key="advanced"` for?"* To force a distinct component instance so the initial-algorithm computation, which reads `location.pathname` at mount, is re-evaluated when navigating between the two sorting routes.

Block 29 · The ESLint configuration

`eslint.config.js (verbatim, 43 lines)`

```
import js from '@eslint/js'
import globals from 'globals'
import reactHooks from 'eslint-plugin-react-hooks'
import reactRefresh from 'eslint-plugin-react-refresh'
import { defineConfig, globalIgnores } from 'eslint/config'

export default defineConfig([
  globalIgnores(['dist']),
  {
    files: ['**/*.{js,jsx}'],
    extends: [
      js.configs.recommended,
      reactHooks.configs.flat.recommended,
      reactRefresh.configs.vite,
    ],
    languageOptions: {
      ecmaVersion: 2020,
      globals: globals.browser,
      parserOptions: {
        ecmaVersion: 'latest',
        ecmaFeatures: { jsx: true },
        sourceType: 'module',
      },
    },
    rules: {
      'no-unused-vars': ['error', { varsIgnorePattern: '^[A-Z_]' }],
    },
  },
  {
    // Legacy pages intentionally synchronize editable inputs to preview state.
    // New lab code uses explicit reducer transitions and retains the stricter rule.
    files: ['src/modules/**/*.jsx'],
    rules: { 'react-hooks/set-state-in-effect': 'off' },
  },
  {
    files: ['src/context/ModeContext.jsx'],
    rules: { 'react-refresh/only-export-components': ['error', { allowExportNames: ['useMode'] }] },
  },
  {
    files: ['tests/**/*.js'],
    languageOptions: { globals: globals.node },
  },
])
```

- **What it does** — flat ESLint config with a deliberate, documented exception.
- **Line by line** — ignores `dist`; applies `js.configs.recommended`, the React Hooks rules and the React Refresh rules to all JS/JSX; sets 2020+ syntax, browser globals and JSX parsing; overrides `no-unused-vars` to permit `^[A-Z_]` (component names and constants); **disables `react-hooks/set-state-in-effect` for `src/modules/***` with a comment explaining that the legacy pages synchronise editable inputs to preview state in effects; keeps the rule on for the new lab code; allows the `useMode` export from a context file; and gives the test files Node globals.
- **Why it matters** — it is the only place where the project's two generations are explicitly distinguished by policy, and it is documented in code rather than in a wiki.
- **The question** — "Why disable a lint rule instead of fixing the code?" Because the rule flags a pattern that is a genuine design flaw in new code and an intentional, working pattern in legacy code. Fixing the legacy modules means restructuring their input synchronisation, which is a real migration with no tests to catch regressions. Scoping the exception to that directory keeps the guard active everywhere it can be acted on, and the README documents it. The alternative — disabling it globally — would have removed the guard from the code I was actively writing.
- **The trap** — "isn't that technical debt?" Yes, and it is **labelled** debt. The honest framing: "It's debt with an address. The rule stays on for the lab and off for the modules I haven't migrated, and both the config comment and the README say so."

Block 30 · package.json

package.json (verbatim, 40 lines)

```
{
  "name": "algoviz",
  "private": true,
  "version": "0.0.0",
  "type": "module",
  "scripts": {
    "dev": "vite",
    "build": "vite build",
    "lint": "eslint .",
    "preview": "vite preview",
    "test": "node --test tests/*.test.js",
    "lint:lab": "eslint src/lab src/pages/Home.jsx"
  },
  "dependencies": {
    "lucide-react": "^0.556.0",
    "react": "^19.2.0",
    "react-dom": "^19.2.0",
    "react-router-dom": "^7.11.0"
  },
  "devDependencies": {
    "@eslint/js": "^9.39.1",
    "@types/react": "^19.2.5",
    "@types/react-dom": "^19.2.3",
    "@vitejs/plugin-react": "^5.1.1",
    "autoprefixer": "^10.4.22",
    "eslint": "^9.39.1",
    "eslint-plugin-react-hooks": "^7.0.1",
    "eslint-plugin-react-refresh": "^0.4.24",
    "globals": "^16.5.0",
    "postcss": "^8.5.6",
    "tailwindcss": "^3.4.17",
    "vite": "npm:rolldown-vite@7.2.5"
  },
  "overrides": {
    "vite": "npm:rolldown-vite@7.2.5"
  },
  "engines": {
    "node": ">=22.12.0"
  }
}
```

- **What it does** — scripts, dependencies, the engines floor, and the Vite alias.
- **The question** — "Which script would you run before opening a PR?" `npm test && npm run lint && npm run build` — exactly what CI runs, in the same order. The README instructs contributors to do exactly this.
- **The trap** — *"why is there both `lint` and `lint:lab`?"* `lint:lab` targets `src/lab` and `src/pages/Home.jsx` specifically, for a fast inner loop while working on the new code. CI runs the full `lint`, so `lint:lab` is a developer convenience, not the gate.

CHECKLIST
RED-LINE DRILLS FOR PART XXIX

- ■ Pick five blocks at random and explain each without reading the answer.
- ■ Recite why `key={item.id}` and not the index — then immediately say the merge-sort exception.
- ■ Explain the `Math.imul` LCG detail.
- ■ Explain why the oracle lives in the test file.
- ■ Explain the ESLint exception and call it labelled debt.
- ■ Name the weakest assertion in the test suite (final-frame-only ID uniqueness) and its two-line fix.

PART XXX

Whiteboard coding

Eleven exercises drawn from the project, with approach, edge cases, complexity, common mistakes and a model solution.

CHAPTER 56

56 · Whiteboard exercises

You will be asked to write code. These are the exercises that AlgoViz makes natural, and every model solution is one you can defend by pointing at the repository.

56.0 How to perform on a whiteboard

MUST KNOW

THE FIVE-STEP PROTOCOL — SAY IT WHILE YOU DO IT, NOT AFTER

1. **Restate the problem** and confirm the signature out loud: "Input is an array of numbers, output is a sorted copy, and I must not mutate the input."
2. **State the approach and its complexity before writing**: "I'll do a Lomuto partition with a last-element pivot; average O(n log n), worst O(n²) on sorted input." This buys you credit even if the code has a typo.
3. **Write the skeleton first** (loop bounds, base case), then fill in the body. Sketches that start with variable declarations read as hesitation.
4. **Test it on a tiny input by hand** — always `[3,1,2]` or `[2,1]`. Narrate: "i=0, j=1, 1<=2 so i becomes 0, swap..."
5. **State the edge cases you would handle and the tests you would write.** This is what separates a candidate who has shipped code from one who has only written functions.

Exercise 1 · Bubble sort — the interview version

Question. "Write bubble sort. Don't mutate the input. What's its best case?"

Expected approach. Copy the array, nested loops, early-exit flag, swap when out of order. The early exit is the part being tested.

```
function bubbleSort(input) {
  const arr = [...input];                      // never mutate the caller's array
  const n = arr.length;
  for (let i = 0; i < n - 1; i++) {
    let swapped = false;
    for (let j = 0; j < n - i - 1; j++) {      // the sorted tail shrinks
      if (arr[j] > arr[j + 1]) {               // strict > keeps it stable
        [arr[j], arr[j + 1]] = [arr[j + 1], arr[j]];
        swapped = true;
      }
    }
    if (!swapped) break;                       // no swaps means already sorted
  }
  return arr;
}
```

Aspect	Answer
Time	Best Θ(n) **with** the flag; average/worst Θ(n²)
Space	O(1) auxiliary; O(n) for the returned copy
Stable	Yes — the comparison is strict
Edge cases	`[]`, `[1]`, all-equal, already sorted, reverse sorted, negatives
Common mistakes	Omitting the flag and claiming O(n) best case; `j < n - i` (reading into the sorted tail); using `>=` (breaks stability); returning the input array itself
AlgoViz tie-in	**The generator deliberately omits the early exit**, so the app's bubble sort is Θ(n²) on every input — measured: a sorted 24-element array still reports 276 comparisons. Say this; it converts a textbook answer into project knowledge.

Exercise 2 · Selection sort

Question. "Write selection sort and tell me something interesting about its comparison count."

```
function selectionSort(input) {
  const arr = [...input], n = arr.length;
  for (let i = 0; i < n - 1; i++) {
    let minIdx = i;
    for (let j = i + 1; j < n; j++) {
      if (arr[j] < arr[minIdx]) minIdx = j;
    }
    if (minIdx !== i) [arr[i], arr[minIdx]] = [arr[minIdx], arr[i]];
  }
  return arr;
}
```

The interesting fact: the comparison count is **exactly n(n−1)/2 regardless of the input**. It is not adaptive, and it does at most n−1 swaps — the minimum for a comparison sort that only swaps. Verification from your own traces: 28 comparisons for n = 8 and 45 for n = 10, both exact.

Common mistakes	Consequence
Comparing against `arr[i]` instead of `arr[minIdx]`	Finds the minimum of each pair, not the running minimum
Swapping unconditionally	Inflates the swap count; loses the "zero swaps on equal input" property
Starting the inner loop at `i`	An extra self-comparison that inflates the count by n

Exercise 3 · Insertion sort

Question. "Write insertion sort. Why would a real library use it?"

```
function insertionSort(input) {
  const arr = [...input];
  for (let i = 1; i < arr.length; i++) {
    const key = arr[i];
    let j = i - 1;
    while (j >= 0 && arr[j] > key) {   // strict > keeps it stable
      arr[j + 1] = arr[j];             // shift, don't swap
      j--;
    }
    arr[j + 1] = key;                  // drop into place
  }
  return arr;
}
```

Why libraries use it: adaptive (Θ(n) on nearly-sorted input), tiny constant factor, no allocation, no recursion, and online — it can sort a stream as items arrive. Timsort and introsort implementations switch to it below a size threshold, typically 16–32 elements.

Aspect	Detail
Best case	n − 1 comparisons, 0 shifts — Θ(n)
Common mistakes	Swapping instead of shifting (doubles the writes); `arr[j] >= key` (breaks stability); forgetting `j >= 0` (reads `arr[-1]`, which is `undefined` and every comparison against it is false — so it can silently "work" and stop early)
AlgoViz tie-in	The generator swaps instead of shifting, so its exchange counter is not the same measure as a shift-based implementation's — the reason the comparison panel uses comparisons only. This implementation does have the early exit the bubble sort lacked; that asymmetry is visible in the app, where insertion sort's trace on a sorted input is dramatically shorter than bubble sort's.

Exercise 4 · Merge sort

Question. "Write merge sort. Is it stable? In-place? What's the space complexity?"

```
function mergeSort(a) {
  if (a.length <= 1) return [...a];
  const mid = a.length >> 1;
  return merge(mergeSort(a.slice(0, mid)), mergeSort(a.slice(mid)));
}

function merge(L, R) {
  const out = [];
  let i = 0, j = 0;
  while (i < L.length && j < R.length) out.push(L[i] <= R[j] ? L[i++] : R[j++]);
  while (i < L.length) out.push(L[i++]);
  while (j < R.length) out.push(R[j++]);
  return out;
}
```

Aspect	Answer
Time	Θ(n log n) in every case — best, average and worst
Space	O(n) auxiliary (the merged arrays plus the recursion depth O(log n))
Stable	Yes, **because** the comparison is `<=`: ties take from the left run first
In-place	No
Common mistakes	Writing `<` instead of `<=` (breaks stability); forgetting one of the two tail loops (drops the remaining elements); using `mid` as both a split point and an inclusive bound inconsistently; recursion on `a.length === 2` producing an infinite loop if the base case is `a.length === 0` only
AlgoViz tie-in	The generator sorts **in place with a buffer**, not by returning new arrays, because a frame must show one canonical current array. And that buffer write-back is where the duplicate-ID frames come from — 23 of 107 frames on the default input.

Exercise 5 · Quick sort

Question. "Write quick sort with Lomuto partitioning. What's the worst case, and when does it occur?"

```
function quickSort(a, low = 0, high = a.length - 1) {
  if (low >= high) return a;
  const p = partition(a, low, high);
  quickSort(a, low, p - 1);      // pivot excluded
  quickSort(a, p + 1, high);
  return a;
}

function partition(a, low, high) {
  const pivot = a[high];         // capture the VALUE, not the index
  let i = low - 1;
  for (let j = low; j < high; j++) {
    if (a[j] <= pivot) { i++; [a[i], a[j]] = [a[j], a[i]]; }
  }
  [a[i + 1], a[high]] = [a[high], a[i + 1]];
  return i + 1;
}
```

Aspect	Answer
Time	Best/average Θ(n log n); **worst Θ(n²)**
Worst case	**Already-sorted input with a last-element pivot.** Every partition places the pivot at `high` itself, so the recursion becomes a single subproblem one element smaller: n levels of O(n) work
Space	O(log n) average stack, O(n) worst — the recursion is not tail-call optimised
Stable	No (the long-distance swaps reorder equal keys)

Aspect	Answer
Common mistakes	Recursing on `p` instead of `p ± 1` (infinite recursion on a size-1 range); `j <= high` (compares the pivot with itself); reading `a[high]` inside the loop after swaps have moved it
AlgoViz tie-in	The app ships exactly this worst case, and the user can trigger it by pressing the **"sorted" preset** — a live, interactive demonstration of an algorithmic worst case. The fix is median-of-three or a randomised pivot.

Follow-up you should be ready for: "How would you avoid the worst case?" Median-of-three (median of `low`, `mid`, `high` as the pivot), randomising the pivot by swapping `a[high]` with a random index before partitioning, or switching to introsort (fall back to heap sort when the recursion depth exceeds 2 log n).

Exercise 6 · Heap sort — the sixth algorithm

Question. "Write heap sort. Why is building the heap O(n) rather than O(n log n)?"

```
function heapSort(input) {
  const a = [...input], n = a.length;
  for (let i = (n >> 1) - 1; i >= 0; i--) siftDown(a, n, i);   // build: O(n)
  for (let end = n - 1; end > 0; end--) {
    [a[0], a[end]] = [a[end], a[0]];
    siftDown(a, end, 0);
  }
  return a;
}

function siftDown(a, size, i) {
  while (true) {
    let largest = i;
    const l = 2 * i + 1, r = 2 * i + 2;
    if (l < size && a[l] > a[largest]) largest = l;
    if (r < size && a[r] > a[largest]) largest = r;
    if (largest === i) break;
    [a[i], a[largest]] = [a[largest], a[i]];
    i = largest;
  }
}
```

Aspect	Answer
Time	**Θ(n log n) in every case** — no input distribution is bad for it
Build phase	**O(n)**, not O(n log n): most nodes are shallow, so the sum of (nodes at depth d) × (height below d) telescopes to O(n)
Space	O(1) auxiliary with the iterative sift above; O(log n) if `siftDown` recurses
Stable	No
Common mistakes	Starting the build loop at `n-1` (works but does wasted work); `l < size` omitted (reads past the heap); using the full array length in the second phase's sift instead of the shrinking `end`
AlgoViz tie-in	The generator uses a **recursive** `heapify` and increments two comparison counters per call unconditionally — verified: the counter rises even on a leaf with no children. Both facts are on screen: the panel shows `O(log n)*` with a call-stack footnote, and the counters carry a footnote about being implementation-specific.

Exercise 7 · The playback reducer

Question. "Here's a UI with a cursor, a frame count and a playing flag. Design the state machine. What actions do you need? What are the invariants?"

Expected approach. Enumerate the transitions first, then write the reducer, then state the invariants.

```
const initial = (length) => ({ cursor: 0, length, playing: false });

function playbackReducer(state, action) {
  switch (action.type) {
    case "load":
      return { cursor: 0, length: action.length, playing: false };
    case "toggle":
      return {
        ...state,
        cursor: state.cursor === state.length - 1 ? 0 : state.cursor,
        playing: !state.playing,
      };
    case "tick":
      return {
        ...state,
        cursor: Math.min(state.cursor + 1, state.length - 1),
        playing: state.cursor + 1 < state.length - 1,
      };
    case "seek":
      return {
        ...state,
        cursor: Math.max(0, Math.min(action.cursor, state.length - 1)),
        playing: false,
      };
    default:
      return state;
  }
}
```

Aspect	Answer
Invariants	`0 <= cursor <= length - 1`; reaching the last frame always clears `playing`; every `seek` pauses
Four actions, not eight	`restart` is `seek(0)`; `next` is `seek(cursor+1)`; `previous` is `seek(cursor-1)`; `replay` is `toggle` at the end. Fewer actions means fewer transitions to test
Why a reducer	The three fields have cross-field rules; a reducer states them once, purely, and testably
Edge cases to mention	`length === 1`; seek beyond either end; an unknown action type returning the same object so React bails out; **non-integer and `NaN` seeks, which this reducer does not reject**
The improvement you should volunteer	A `step` action with a `delta` would let the keydown effect drop its `cursor` dependency
AlgoViz tie-in	This is the shipped reducer — `src/lab/engine.js` lines 80–105 — and it is asserted by the test group "playback keeps final state, pauses on seek and bounds its cursor"

Exercise 8 · A frame generator

Question. "Write a function that takes an array of numbers and returns a list of snapshots of an insertion sort, where each snapshot shows the array, which indices are being compared, and the running comparison count. Don't mutate the input."

Expected approach. Wrap values into identified objects, clone on every push, push a frame before each decision and after each mutation.

```
function recordInsertionSort(raw) {
  const arr = raw.map((val, i) => ({ id: `block-${i}`, val }));
  const frames = [];
  const sortedIndices = [];
  let comps = 0;

  const snap = (active, code, msg, sorted = []) => ({
    array: arr.map((it) => ({ ...it })),   // fresh element objects
    activeIndices: active,
    sortedIndices: [...sorted],
    codeLine: code,
    currentStats: { comps },               // clone the counters
    msg,
  });

  frames.push(snap([], "start", "Ready."));
  for (let i = 1; i < arr.length; i++) {
    let j = i;
    while (j > 0) {
      comps++;
      frames.push(snap([j, j - 1], "compare",
        `Comparing ${arr[j].val} with ${arr[j - 1].val}.`));
      if (arr[j].val < arr[j - 1].val) {
        [arr[j], arr[j - 1]] = [arr[j - 1], arr[j]];
        frames.push(snap([j, j - 1], "swap", "Shifted."));
        j--;
      } else {
        frames.push(snap([j, j - 1], "break", "In position."));
        break;
      }
    }
    sortedIndices.push(i);
    frames.push(snap([], "drop", "Dropped.", [...sortedIndices]));
  }
  return frames;
}
```

Aspect	Answer
The four requirements	(1) fresh array on every push; (2) fresh counter object on every push; (3) indices as metadata, since intent is not recoverable from data; (4) the input untouched
Missing from this skeleton	The terminal frame with all indices sorted, and `passes`/`swaps` counters. Both are easy additions and mentioning them shows you know the contract
Complexity	O(frames × n) time and space — for an O(n^2) sort with per-step cloning that is conceptually O(n^3) storage
The insight to state	**"The frames must be independent. If I reused one array, every frame would show the final state, because they would all point at the same object."**
AlgoViz tie-in	This mirrors `generateInsertionSort` exactly, including the `break` frame — except the shipped version also carries `isLifting`, and it swaps rather than shifts

Exercise 9 · The input parser

Question. *"Write a function that turns `' 38, 66, 12 '` into `[38, 66, 12]`, rejecting anything that isn't 2 to 24 whole numbers between −999 and 999. Give me the error messages."*

```
function parseInput(text) {
  if (typeof text !== "string" || !text.trim())
    throw new Error("Enter 2–24 integers, separated by commas.");

  const parts = text.split(",");
  if (parts.length < 2 || parts.length > 24)
    throw new Error("Use 2–24 values. Small inputs keep every operation inspectable.");

  if (parts.some((p) => !/^-?\d+$/.test(p.trim())))
    throw new Error("Use whole numbers separated by commas. Empty values are not allowed.");

  const values = parts.map(Number);
  if (values.some((n) => !Number.isSafeInteger(n) || Math.abs(n) > 999))
    throw new Error("Each number must be between −999 and 999.");

  return values;
}
```

Aspect	Answer
Why regex first	`Number("")` is 0, `Number("1e3")` is 1000, `Number("0x10")` is 16, `Number("Infinity")` is `Infinity`. Coercion accepts all of them silently; an anchored regex states the grammar
Why `.trim()` in the test	`" 66 "` must be accepted; the regex is anchored so the whitespace has to be removed first
Why split before counting	So `"1,,2"` counts as three parts and fails the regex — a shorter, clearer failure than a special-case check
Edge cases to mention	`""`, `" "`, `"1"` (too few), `"1,2.5"`, `"1,,2"`, `"1,"`, 25 values, `"1,1000"`, `"1,Infinity"`, `"-0"` (parses to `-0`, which `Number.isSafeInteger` accepts — a genuine curiosity), and `"007"` (accepted, becomes 7)
AlgoViz tie-in	This is `parseInput` verbatim, and the same seven rejection cases are asserted in the test file

Exercise 10 · JSON validation

Question. *"You receive a JSON file from a user. You expect `{version: 1, algorithm: string, input: number[], cursor: number}`. Write the validation. Assume you must not trust anything."*

```
const ALGORITHMS = { bubble: 1, selection: 1, insertion: 1, merge: 1, quick: 1, heap: 1 };
const MAX_BYTES = 16384;

function loadExperiment(rawText) {
  if (rawText.length > MAX_BYTES) throw new Error("File is too large.");

  let data;
  try { data = JSON.parse(rawText); }
  catch { throw new Error("This file is not valid JSON."); }

  if (!data || typeof data !== "object" || Array.isArray(data))
    throw new Error("Not a supported AlgoViz experiment.");

  if (data.version !== 1) throw new Error("Unsupported experiment version.");

  // prototype-safe: Object.hasOwn, never `in` and never a bare lookup
  if (typeof data.algorithm !== "string" || !Object.hasOwn(ALGORITHMS, data.algorithm))
    throw new Error("Unknown algorithm in this experiment.");

  if (!Array.isArray(data.input) || data.input.some((v) => typeof v !== "number"))
    throw new Error("The experiment input must be an array of numbers.");

  const values = parseInput(data.input.join(","));   // reuse the one validator

  const cursor = Number.isInteger(data.cursor) && data.cursor >= 0 ? data.cursor : 0;

  return { version: 1, algorithm: data.algorithm, input: values, cursor };
}
```

Aspect	Answer
Ordering of gates	Cheapest and most specific first: size → parse → shape → version → algorithm → input type → value rules → cursor
The critical security detail	`Object.hasOwn(ALGORITHMS, data.algorithm)`. `'constructor' in ALGORITHMS` is `true` because `in` walks the prototype chain, so a crafted file could reach a prototype function
The design principle	Reuse `parseInput` for the values. One validator means the file format cannot drift from what the UI accepts
What it still does not do	Allowlist the fields (extra keys pass silently); bound nesting depth; validate `cursor` against the trace length (the reducer clamps it); or verify a signature
AlgoViz tie-in	This mirrors `importExperiment`, with one difference: the app's version is slightly more permissive, because it checks only `Array.isArray(data.input)` and then relies on `join(",")` to coerce — so `[[1,2],[3,4]]` imports as `[1,2,3,4]`. Say that if asked "is your validation strict?"

Exercise 11 · The sliding window, and its precondition

Question. "Find the length of the shortest contiguous subarray whose sum is at least a target on positive integers. Then tell me what breaks if I allow negatives."

```
function shortestAtLeast(nums, target) {
  if (target <= 0 || !nums.length || nums.some((n) => n <= 0)) {
    throw new Error("Requires positive values and a positive target.");
  }
  let left = 0, sum = 0, best = Infinity;
  for (let right = 0; right < nums.length; right++) {
    sum += nums[right];                       // expand
    while (sum >= target) {                   // shrink while it still qualifies
      best = Math.min(best, right - left + 1);
      sum -= nums[left++];
    }
  }
  return best === Infinity ? 0 : best;
}
```

Aspect	Answer
Time	**O(n)** — each pointer moves forward at most n times — versus O(n²) brute force
Space	O(1)
Why positivity is required	It guarantees monotonicity: expanding can only increase the sum and shrinking can only decrease it, so the walk is monotone. The `while` loop's exit is then a proof that no shorter window ending at `right` qualifies
With a negative value	The monotonicity is lost. Measured against the project's implementation: `[-1, 3, 2]` with target 3 returns **2** where the true answer is **1**; `[1, -1, 1, 1]` with target 2 returns **4** where the true answer is **2**
The correct fix for negatives	The sliding window does not apply. Either use prefix sums with a monotonic deque, or brute force; the simplest correct approach is O(n²) enumeration, or a different technique entirely
AlgoViz tie-in	The precondition is enforced in `generateWindow`, disclosed on the Counterexample page, and stated in the README — three places for one assumption. The `windowTrace` reference function, by contrast, has no guard, which is a real inconsistency.

MUST KNOW

THE CLOSING LINE THAT TIES EVERY EXERCISE BACK TO THE PROJECT

"This project's exercises are unusual in one way: for most of them I can show you the code in the repository, the trace it produces, and the test that pins it. All six sorts, the reducer, the parser and the JSON validation are shipped and asserted; the generator exercise is the shape that all six of mine follow. So if you want to see any of these running rather than on a whiteboard, I can open the file."

CHECKLIST

RED-LINE DRILLS FOR PART XXX

- ■ Write all six sorts from memory in under twelve minutes total.
- ■ Write the reducer including the `default` branch.
- ■ Write `parseInput` with its four error messages.
- ■ Write a frame-generating function that clones on every push and explain why.
- ■ Explain the sliding window's monotonicity requirement and the negative-input failure with a concrete example.
- ■ State the five-step whiteboard protocol and follow it in a mock session.

PART XXXI

Resume defence

Every claim in your AlgoViz entry taken apart: what it means, where it lives, how you prove it, and the trap behind it.

CHAPTER 57

57 · Defending the resume entry, claim by claim

The interviewer is holding your resume and has decided to start there. This chapter is the script.

57.0 The entry, split into atomic claims

Your AlgoViz bullet contains **fourteen** checkable assertions — eight of them are the grouped audit in Chapter 2, and the list below splits them to the granularity at which each one can be disproved independently. Most candidates cannot even enumerate their own claims; enumerating them first is itself a strong signal.

#	Claim	Verdict	Proof location
1	Recorded-frame playback	[VERIFIED]	Six generators return `frames[]`; `SortingLab` indexes into it
2	For 6 sorting algorithms	[VERIFIED]	`engine.js` 8–62, six entries
3	Using React reducers	[VERIFIED]	`useReducer(playbackReducer, …)`, `SortingLab.jsx` 98
4	Pause / resume	[VERIFIED]	the `toggle` action
5	Single-step navigation	[VERIFIED]	`seek(cursor ± 1)` plus `seek(0)`
6	Speed control	[VERIFIED]	four preset delays 600/300/160/50
7	Timeline seeking	[VERIFIED]	`<input type="range" min=0 max={frames.length-1}>`
8	Same-input algorithm comparison	[VERIFIED]	`otherFrames` memo over the identical `input` reference
9	Percentage-aligned replay	[VERIFIED]	`compareCursor` computation, line 201–203
10	Recorded operation counts	[VERIFIED] with a scope limit	`currentStats {passes, comps, swaps}`; comparison uses `comps` only
11	Browser-local saves + validated JSON import/export	[VERIFIED]	`localStorage` under `algoviz.experiments.v1`; size/version/algorithm/shape/`parseInput` gates
12	150 algorithm-input cases for correctness and reproducibility	[VERIFIED]	`tests/engine.test.js` 12–36: 6 × 25
13	768 sliding-window cases against brute force	[VERIFIED]	same file, 82–101: 64 × 12
14	Tests, lint and build automated through GitHub Actions	[VERIFIED]	`.github/workflows/ci.yml`, four steps

MUST KNOW

THE OPENING LINE THAT SETS THE TONE

"Before we go into any of it — everything in that bullet is traceable to a file, and I'm happy to open any of them. There's also one thing in the bullet I'd phrase more carefully than I did, which is the operation-count claim: it's comparisons that are recorded and comparable, and the pass and exchange counters are implementation-specific. The UI says that on screen."

That opening does three things: it asserts verifiability, it demonstrates that you have audited your own claim, and it hands the interviewer a correction — which is the single most credible thing a candidate can do.

57.1 Claim 1–3 — "Engineered recorded-frame playback for 6 sorting algorithms using React reducers"

RESUME CLAIM
CLAIM: ENGINEERED RECORDED-FRAME PLAYBACK FOR 6 SORTING ALGORITHMS USING REACT REDUCERS

What it means. You designed an execution model in which an algorithm runs to completion and emits a complete, immutable history, and you built a transport layer over that history with a reducer.

Where it exists.

- The six registrations and their metadata: `src/lab/engine.js` lines 8–62
- The six generators: `src/modules/core/algorithm/sorting/algorithms/{bubble,selection,insertion,merge,quick,heap}Sort.js`
- The reducer: `src/lab/engine.js` lines 80–105
- Its single consumer: `src/lab/SortingLab.jsx` line 98
- The frame-consuming render: `ArrayView`, `SortingLab.jsx` lines 22–48

How you prove it. Open `engine.js`, show the sixth entry of the registry, then show `playbackReducer`, then show `const frame = frames[cursor]` and say: "that's the whole playback path — one array index." Then show the test group named after each algorithm, which loops over the registry and asserts four properties over 25 inputs each.

Likely question. "Why record frames rather than animate as you go?"

Follow-up. "What's the cost of that decision?"

Trap question. "So where's the animation library?" — there isn't one, and that is the point: "No animation library. The bars are positioned by inline height percentages and the state changes by re-rendering a different array element. There's no interpolation because the design is step-wise — you're looking at discrete recorded states, not a tween."

The perfect answer. "I split the problem into two halves that don't know about each other. The engine — the six generators — runs the algorithm synchronously and returns a complete array of frames. A frame is a full snapshot plus the metadata that explains it: the array cloned, the indices being compared, the indices already final, the pivot or sub-array range where the algorithm has those concepts, a machine-readable line tag, a sentence for the user, and a snapshot of the counters at that instant. The second half is playback: a `useReducer` holding `{cursor, length, playing}` with four actions — `load`, `toggle`, `tick`, `seek`. It knows nothing about sorting; it's three numbers and a boolean. So rewinding is selecting an earlier frame, not running JavaScript backwards, and the render is a pure function of one integer. The reducer is exported as a plain function, which is why I can unit-test the transport without rendering anything."

Why this phrasing works. It states the architecture in one breath, gives the frame schema, names the four actions, and ends on the testability consequence — which is the part that shows you understood why the split matters, not just that you made it.

57.2 Claims 4–7 — "enabling pause/resume, single-step navigation, speed control, and timeline seeking"

RESUME CLAIM

CLAIM: PAUSE/RESUME, SINGLE-STEP NAVIGATION, SPEED CONTROL, TIMELINE SEEKING

What it means. Four transport features which, in this architecture, are four arguments to one action and one effect.

Where it exists.

Feature	Where	How
Pause / resume	`engine.js` 84–89	`toggle` flips `playing`; resets the cursor to 0 when it is on the last frame (so the same action is Replay)
Step	`SortingLab.jsx` 378–408	`seek(cursor - 1)` and `seek(cursor + 1)`, with disabled states at the ends, plus `seek(0)` for restart
Speed	`SortingLab.jsx` 410–421	A select whose value is the delay in ms; it is a dependency of the timer effect
Seeking	`SortingLab.jsx` 356–367	A range input bound to the derived cursor, dispatching `seek` on change

How you prove it. Click through them live — but the stronger proof is structural: *"All four are the same action. There's no `next`, no `previous`, no `restart` — the reducer has four actions total and navigation is entirely expressed through `seek`. That's why the transport has no duplicated boundary logic: the clamp lives in one place."*

Likely question. "How does Pause actually work? Does the timer stop?"

Follow-up. "How do you prevent duplicate timers?"

Trap question. "You said pause/resume — resume from where, exactly?" The precise answer: *"From the exact frame you paused on. `toggle` leaves the cursor alone unless it's on the last frame. So pause at frame 42, resume at frame 42. The one exception is at the last frame, where the same action restarts from 0 — that's what turns the Play button into Replay."*

The perfect answer. "Pause is a state change and a cleanup, not a message to the timer. `toggle` sets `playing` to false, React re-runs the effect, and the effect's cleanup calls `clearTimeout` on the single pending timer — so while paused there are zero live timers, not one that's pretending to be off. On resume, the effect sees `playing` true and schedules exactly one timeout for the current speed. There's no interval anywhere, which is deliberate: an interval would capture its delay at creation, so changing the speed couldn't take effect until the next frame, and after a main-thread block it would fire several queued callbacks in a burst and the animation would jump. A self-scheduling timeout chain always measures from the last frame that actually rendered."

57.3 Claim 8–9 — "same-input algorithm comparison with percentage-aligned replay"

RESUME CLAIM

CLAIM: SAME-INPUT COMPARISON WITH PERCENTAGE-ALIGNED REPLAY

What it means. Two algorithms run over the identical dataset, and their playbacks are positioned proportionally rather than by index.

Where it exists. `SortingLab.jsx` lines 74–76 (`compare`, `other`), 94–97 (`otherFrames`), 201–203 (`compareCursor`), 486–543 (the panel).

How you prove it. Open the app, enable Compare, select two algorithms. Then open the code and point at the identical `input` reference in both memo calls, plus the two on-screen disclaimers.

Likely question. "What does percentage-aligned replay mean?"

Follow-up. "So at 50% both have done half the work?" — no.

Trap question. "This proves which algorithm is faster, right?" — the whole feature exists to not claim that.

The perfect answer. "Both traces come from the same `input` array — the same reference, not a copy — which is safe because the generators don't mutate their input, and that's asserted for all six in the test suite. Alignment is proportional: `round(cursor / (frames.length - 1) * (otherFrames.length - 1))`. On the default eight-element input the traces are 65 frames for selection and 118 for bubble, so an index-based alignment would run out of secondary frames for two-thirds of the playback; proportional mapping keeps both views near their start and near their end at the same time.

What it does not mean is that both algorithms have done equivalent work at the same percentage. At 50% of a bubble trace you're past roughly half of 28 comparisons; at 50% of a merge trace you're past half of 16. The percentages are positions in two different recordings. That's why the UI caption says 'aligned by percentage of trace—not equivalent operations', and why the numbers I actually show are recorded comparison counts, with the panel text saying there are no wall-clock claims because animation speed isn't algorithm performance."

57.4 Claim 10 — "recorded operation counts"

RESUME CLAIM
CLAIM: RECORDED OPERATION COUNTS

What it means. Every frame carries the counters as they stood at that moment. Three counters exist: comparisons, passes/calls, and exchanges.

Where it exists. `currentStats` in all six generators; displayed in the State Inspector (`SortingLab.jsx` 435–448) and used by the comparison panel (515–521); defined by the footnote at 587–592.

How you prove it. Step forward one frame and read the "Comparisons" tile — it changes because `currentStats` is cloned per frame with `{...stats}`. Then show the footnote.

Likely question. "What exactly is counted?"

Follow-up. "Are those counts equivalent to runtime?"

Trap question. "So merge sort beats selection sort — fewer comparisons?" On the default input merge does 16 comparisons against selection's 28, and merge does **more** writes (24 vs 6). The correct answer: "The counters tell you how many decisions each algorithm made, not how long it took. Merge does fewer comparisons and more data movement here, so the counters don't answer 'which is faster' — and at any realistic n, n log n versus n^2 swamps both numbers."

The perfect answer. "Three counters per frame, cloned at generation time. Comparisons is the one I treat as comparable across algorithms, because every generator increments it exactly at a value comparison. Passes is labelled 'Passes / calls' on screen because a pass means an outer-loop iteration in bubble sort, a recursion node in merge sort and an extraction in heap sort — three different units. Exchanges is labelled 'recorded exchanges' because legacy generators count them differently: merge sort increments on every write out of its buffer, so its exchange count is really a write count, while bubble sort's is a real swap count. There's a footnote on the page saying exactly that, and it's the reason the comparison panel deliberately uses comparisons only. None of them is a runtime measurement — there's no timing code anywhere in the project, and the panel says 'No wall-clock claims.'"

57.5 Claim 11 — "browser-local experiment saves and validated JSON import/export"

RESUME CLAIM

CLAIM: BROWSER-LOCAL SAVES AND VALIDATED JSON IMPORT/EXPORT

What it means. An experiment — algorithm, input, and the frame you were looking at — can be kept in the browser or moved as a file.

Where it exists. Save: `SortingLab.jsx` 181–197 and the lazy read at 77–92. Export: 49–58 helper, 233–240 button. Import: 156–180.

How you prove it. Save at a specific frame, reload the page, click the saved entry and show that it lands on the same frame. Then export, open the JSON in an editor, hand-edit the input, and re-import.

Likely question. "Where are experiments saved, and what if the stored data is malformed?"

Follow-up. "What's in the file?"

Trap question. "So if I open this on my phone, will my experiments be there?" No — `localStorage` is per-origin per-browser per-device. Export is the only portability path.

The perfect answer. "The record is `{version, algorithm, input, cursor, createdAt}` under the key `algoviz.experiments.v1`, capped at the most recent six. What's deliberately not in it is the trace: I store how to re-run it, not the result, so a record is a few hundred bytes instead of a megabyte. The trade-off is real and it's in the README: a stored cursor is a frame index, so if a future release changes frame ordering, an old cursor could mean a different moment.

Storage is untrusted even though I wrote it, so the read path has four defences: a `|| "[]"` for the missing key, a `try/catch` for malformed JSON, an `Array.isArray` shape check, and a filter that drops any record whose algorithm key no longer exists — which matters because saved data outlives code. Then clicking a record re-validates through the same `parseInput` the form uses. Import is the same discipline with a size gate first: 16 KB maximum, then `version === 1`, then `Object.hasOwn(algorithms, ...)` so a crafted `algorithm` can't reach the prototype chain, then `Array.isArray(input)`, then the values go back through `parseInput` by joining them into a string. So there's exactly one definition of a valid dataset in the codebase."

57.6 Claims 12–14 — the verification bundle

RESUME CLAIM

CLAIM: VERIFIED CORRECTNESS AND TRACE REPRODUCIBILITY ACROSS 150 ALGORITHM-INPUT CASES; 768 SLIDING-WINDOW CASES AGAINST BRUTE FORCE; CI OVER TESTS, LINT AND BUILD

What it means. Three separate verification mechanisms: property assertions over generated inputs, exhaustive bounded differential testing, and automated gates on every push.

Where it exists. `tests/engine.test.js` (all of it), `.github/workflows/ci.yml`, and the `test` script in `package.json`.

How you prove it. Run `npm test` and show 14 passing groups; open the test file and point at the loop that produces the 150 and the inner loop that produces the 768; show the workflow file.

Likely question. "Walk me through the 150."

Follow-up. "What's the runner?" — Node's built-in `node:test` with `node:assert/strict`, no test dependency at all.

Trap question. "So 150 cases means your sort is correct?" The nuanced answer in Chapter 44.

The perfect answer. "Two different strategies, chosen per problem. For sorting, 150 is six algorithms times twenty-five inputs: five hand-picked edge cases — empty, single element, duplicates, negatives and zero, reverse-sorted — plus twenty seeded random datasets of ten values. Because the seed is deterministic, all six algorithms see the same twenty-five arrays. Each of the 150 asserts four things: the final frame is correctly sorted, the input array wasn't mutated, a second run produces a deep-equal trace including every message string and counter, and the final frame's element ids are unique. That third assertion is the one that matters most here — it's what makes the recording a testable artefact rather than just the output.

For the sliding window I could be exhaustive instead of sampled, because the domain is small: all 64 arrays of length three with values one to four, crossed with targets one to twelve, is 768 cases — and each one is checked against a brute-force reference written in the test file itself, so it shares no code with the implementation. The interesting part is why brute force: an oracle has to be independent in mechanism. If my oracle used the same sliding logic, a bug in the window arithmetic would infect both sides and the test would pass while both were wrong.

I'd be careful about what that proves. 768 cases establishes agreement with an independent implementation inside a bounded box — length exactly three, values one to four — and it says nothing about length four or about the negative values the technique can't handle at all. And the sorting tests have a known weakness: they check id uniqueness only on the final frame, which is why a duplicate-key bug in merge sort's buffer frames passes CI. The fix is two lines — assert uniqueness on every frame."

57.7 The three sentences that must never leave your mouth

DANGER ZONE — WHERE CANDIDATES DIE

LANGUAGE THAT LOSES THE INTERVIEW

1. **"It's a full-stack MERN app."** There is no backend. Checked: zero backend dependencies, no server code, and the only `fetch` in the repository is inside a component that no file imports, so it never runs.
2. **"The comparison shows which algorithm is faster."** It shows recorded comparison counts on the same input with explicit on-screen disclaimers that it makes no wall-clock claims. Claiming more than the code measures is the fastest way to lose credibility with a senior engineer.
3. **"I wrote every line."** [NOT VERIFIED] The repository cannot establish authorship — there is a single commit in the available history and no co-author metadata. Do not claim sole authorship and do not deny it; describe what you did and, if asked, be straightforward: "The history in this checkout is a single squashed commit, so you can't get authorship from Git here. What I can do is explain any part of it — including the parts I'd change." That answer is honest, stays within the evidence, and immediately pivots to something you can prove.

The safe substitute for #3, if you want a strong closed statement: "I can defend every design decision in it, and I can point at the things I'd do differently. Ask me about the weakest part and I'll tell you before you find it."

CHECKLIST

RED-LINE DRILLS FOR PART XXXI

- ■ Enumerate all fourteen claims from memory.
- ■ Recite the "everything is traceable to a file" opener plus the self-correction.
- ■ Give the pause/resume answer including the last-frame exception.
- ■ Give the percentage-alignment answer including the four disclosures.
- ■ Give the counting answer including the merge-writes-versus-swaps distinction.
- ■ Give the 150/768 answer with both coverage weaknesses.
- ■ Say what you must not claim about MERN, speed and authorship.

PART XXXII

The hostile interviewer

Fifty-five adversarial questions, grouped by the attack, with the answer that holds up.

CHAPTER 58

58 · "You're bluffing" — the hostile round

Some interviewers open with pressure on purpose. The test is not whether you know the answer; it is whether you stay inside the evidence when pushed. Every answer below follows one rule: **concede what is true, correct what is false, and never reach past what the code shows.**

58.0 The rules for this round

MUST KNOW

FOUR RULES FOR SURVIVING A HOSTILE ROUND

1. **Never mirror the aggression.** Stay flat, factual and short.
2. **Concede fast and precisely.** "Yes — that's a real weakness, and here's what it costs." A candidate who concedes accurately is believed on everything else.
3. **Correct false premises explicitly.** "That specific claim isn't in the code — what's in the code is X." Do not accept a premise you did not make.
4. **End with the thing you can prove.** Every answer finishes at a file, a number, or a behaviour you can demonstrate.

Group A · "You didn't build this" (Q1–Q6)

Q1. "This is clearly a tutorial project." "Parts of it are, and I'll point at them. The `/modules/core/*` pages are a deliberate reproduction of the classic visualizer pattern — a module page with a preview and a control panel, styled as lesson content. The distinguishing part is the recorded-frame lab: the generators, the reducer, the alignment formula, the persistence schema and the test suite are specific to this codebase and there's no tutorial that produces that arrangement. If you think a specific file is boilerplate, name it and I'll either agree or show you what's mine."

Q2. "I've seen this exact design on YouTube." "Might well be — visualizers converge on the same shape: an array of frames, a cursor, a timer. What I'd ask you to compare isn't the shape but the details, because that's where the decisions are: the reducer's edge cases, the duplicate-id bug in merge sort's buffer, the disclosed precondition on the sliding window, the fact that the comparison panel refuses to rank the algorithms. Those are the parts that could have gone either way, and I can defend each one."

Q3. "Prove you wrote it." "I can't prove authorship from this checkout: there's a single squashed commit and no co-author metadata, so Git gives you nothing. What I can do is be interrogated on any part of it — including the parts I'd change — and I'd rather be judged on that than on a commit log."

Q4. "If you wrote it, tell me what's in `engine.js` line 137 without looking." "I'd rather look, honestly — I know the file's structure better than its line numbers. Line range 106 to 120 is `makeDataset`, the seeded generator with the LCG. If you want a specific behaviour: line 137 is inside `windowTrace`, and the whole of the counterexample lab's bug is one word there — the condition is `if (!buggy || sum === target)`, so the `buggy` flag swaps an `>=` for an `===`. That's the interesting line."

Q5. "Your README says 'local-first'. That's marketing for 'I have no backend'." "Correct — and the README says the same thing in plainer words: no account, no server, no database, no network calls in the shipped app. 'Local-first' here means the entire application is a static bundle

and experiments live in `localStorage`. I'd rather state the constraint than imply a service that doesn't exist."

Q6. "What did you personally not understand while building this?" "The proportional alignment, the first time. I wrote it index-based, which broke immediately — selection sort's trace is 65 frames on the default input and bubble's is 118, so the secondary panel ran out of frames two-thirds through and then sat frozen. That's what forced the proportional formula, and it's why the first version of the feature is not in the repository. The other thing I got wrong was assuming merge sort's duplicate ids were harmless because they never reach the final frame — they're in 23 of 107 frames, and I only found that by measuring."

Group B · "The architecture is wrong" (Q7–Q18)

Q7. "Recording every frame is absurd. That's O(n³) memory." "It's worse than it needs to be, and I'll give you the number: the worst trace the app can produce is a reverse-sorted 24-element input under bubble sort — 1,154 frames, about 981 KB serialised. That's acceptable inside a 24-element cap and unacceptable outside it, which is exactly why the cap exists. The structural fix is not to record full snapshots: it's to record the operations and rebuild state, or to keyframe every k-th frame and replay forward. Both are real work and neither is in the repository — they're the first item on my own improvement list."

Q8. "Why is `length` in the reducer state at all? You have `frames.length`." "Because a reducer is a pure function of its state and its action, and it can't see `frames`. It needs to know where the end is, so either `length` lives in the state or every action carries it. I chose the state, and I'll concede the cost: the length is now duplicated, and a code path that replaces the frames without dispatching `load` leaves the reducer's length stale. There's a derived clamp in the render that keeps that from being user-visible, but the correct fix is to remove the duplication rather than to guard it."

Q9. "You compute the trace twice on every commit. That's just sloppy." "Yes, it is. `load()` calls the generator once to get the length for the `load` action, and the `useMemo` calls it again to build the frames. At this input size it's microseconds, so it's invisible — but it's a real inefficiency and it's the kind of thing that becomes a bug rather than a cost at scale, because the two calls are two chances to disagree. The fix is to derive the length from the memo, or to let the reducer take the frames."

Q10. "Why is your reducer full of clamping logic? That belongs in the component." "Arguably. My reasoning was that the reducer is the only place that knows both the cursor and the length, so it's the only place that can enforce the invariant without duplicating the bound. The counter-argument is real: clamping hides invalid input. That's measurable here — `seek(2.5)` stores `2.5`, `seek(NaN)` stores `NaN`, and the clamp doesn't catch either, because it checks the range but not the domain. So the clamp gives you a false sense of completeness."

Q11. "Your reducer's `default` branch returns the same state. That's how typo'd actions get swallowed." "That's exactly the risk, and it's a fair criticism. The benefit is that React bails out of the re-render when the reducer returns the identical object, which is what makes an unknown action a no-op rather than a render. The alternative is to throw on an unknown type in development — a `default` that throws when `NODE_ENV !== 'production'` — and I'd take that if I were doing it again."

Q12. "Two different generations of code in one repo — the module pages and the lab — with different execution models. That's incoherent." "It is, and it's visible in the lint config: `react-hooks/set-state-in-effect` is disabled for `src/modules/**` only, with a comment saying why. Those pages run an async loop with a paused ref and a speed ref; the lab records frames and replays them. I migrated the pattern for the new surface rather than rewriting twelve working pages with no tests to catch a regression, and I labelled the boundary in code and in the README. If the question is whether I'd keep both forever: no — the modules should migrate onto the frame

model, and that's a stated piece of planned work."

Q13. "You have dead code in the repo." "I do: nine components under `src/components` — one of them is a zero-byte `Layout.jsx` — plus `SimpleSort`, `EfficientSort`, a context file and some CSS. None of them is imported; the production build confirms it, because the bundle contains one chunk graph from `main.jsx` and those files contribute nothing. They cost repo readability and nothing else, and deleting them is a one-line diff I'd rather do in daylight than have done silently."

Q14. "The only network call in the repo is a hit counter in a component nobody imports. Why is that there?" "It's inside `CardsGrid.jsx`, which has zero importers, so it never executes — which is also why it isn't a privacy issue in the shipped build. It's leftover from an earlier shell. If it were live I'd remove it, because a third-party counter in a local-first app contradicts the README's own description."

Q15. "Why does `Home.jsx` hold all seventeen routes in one file?" "Because routing here is a table, not a hierarchy: seventeen flat paths with no nesting, no nested layouts and no loaders, so a single declarative table is the shortest true description of the app's surface. The cost is that the file is a hundred lines of literal data, and the moment a route needs auth, a layout or a loader, that file becomes the wrong shape and should split."

Q16. "There's no error boundary. So one bad route takes down the app." "Correct, and there's no boundary anywhere in the tree. It's a real gap: a throwing module unmounts the whole app instead of showing an error in one pane. `Suspense` gives you a loading fallback, not an error fallback, and it's a common confusion — I'd add a boundary inside each lazy route so the shell survives. I'd rather name it than have you find it."

Q17. "Your `App.jsx` is twelve lines and renders one component. Why does it exist?" "Because `main.jsx` should be about mounting and `App` should be about composition — the moment the app needs a provider, a boundary or an auth gate, that belongs in one place and not in the mount file. Today it's a pass-through, and I'd say so on the way in rather than pretend it's doing work."

Q18. "No state management library. Is that a choice or a limitation?" "A choice, and the sizing is deliberate: the shared state is a cursor and a boolean, and it's consumed by one component. Redux or Zustand for that would be more machinery than the problem. What I'd flag is that `SortingLab.jsx` is 596 lines with ten `useState` calls plus a reducer, and that's the honest smell — not the absence of a library, but the size of the component that made one unnecessary."

Group C · "Your tests prove nothing" (Q19–Q28)

Q19. "150 test cases is a made-up number. Real test suites are thousands." "The number isn't the claim. The claim is twenty-five inputs per algorithm, chosen to cover four specific failure modes — empty, single, duplicates, and sign — plus twenty seeded random ones, with four assertions each: sortedness, input preservation, trace reproducibility, and final-frame uniqueness. Reproducibility is the unusual one, and it's a product requirement rather than a nicety, because a saved experiment stores a frame index and is meaningless if a re-run reorders frames."

Q20. "You assert the final frame is sorted. Any sort that returns a sorted array passes. That's trivial." "Agreed — that alone is nearly worthless, and I'd never defend it as the suite's value. The three companions are what carry the weight, and specifically trace reproducibility, which deep-compares the entire recording including every message string and every counter snapshot. A single field differing anywhere in 1,154 frames fails it."

Q21. "Your id assertion only checks the last frame. That's why your merge bug exists." "That's the strongest criticism of the suite and I agree with it completely. The assertion is on `frames.at(-1)`, and the duplicate-id frames are in the write-back phase — 23 of 107 on the default input. The fix is two lines: move the uniqueness check inside the per-frame loop. That change turns a browser console warning into a CI failure, and it's the first thing on my list."

Q22. "768 cases on length-3 arrays is a toy. What about length 10?" "Length three with values one to four is what makes exhaustiveness affordable, and the honest reading is that it verifies agreement with an independent oracle on a bounded box, not correctness in general. The property it's actually protecting is the counterexample detector, which is the app's own instrument — if the detector stopped finding real divergences, that test fails. For broader coverage the lever isn't more cases of the same shape, it's length four and five with a wider value range, plus property-based generation."

Q23. "Your oracle could have the same bug as your implementation." "That's precisely why it's written inline in the test file rather than imported: the oracle is $O(n^2)$ triple-nested enumeration, the implementation is a two-pointer walk. Different mechanism, no shared code. The one thing they do share is the specification — `sum >= target`, which is the definition of the problem, so by construction the test can't catch a misreading of the spec. That's why the counterexample lab exists and why it's labelled a plausibility check rather than a proof."

Q24. "You have no component tests at all." "Correct — no React Testing Library, no jsdom, no browser automation. The suite is `node:test` over pure modules: the generators, the reducer, the parser, the dataset generator, the window function, the counterexample search. That means the reducer's transitions are tested directly and rendering is not tested at all. I'd add component tests for the reducer-to-timer wiring, because 'play for one second and stop' is behaviour I cannot verify today — and I'd start there rather than with snapshot tests."

Q25. "No coverage percentage anywhere. So you don't know what's tested." "That's a gap, and I'd frame it as an incomplete metric rather than a missing one: lines covered says nothing about the assertions' strength, and the merge bug is the proof — the generator has near-total line coverage and a real defect in the frames it emits. What I'd add is a mutation check or, more cheaply, the per-frame assertions that make this class of bug detectable."

Q26. "If CI passes, can I deploy?" "No, and I wouldn't say so. CI runs tests, lint and a production build on Node 22. It establishes that the pure modules behave, the style rules hold and the bundle builds. It cannot establish that playback looks right, that the layout survives a narrow viewport, that the timer wiring is correct, or that the alignment formula produces something readable. Zero browser automation exists in this repository."

Q27. "Your test command has no watch mode and no reporting. That's a hobby setup." "It's the Node built-in runner with no test dependency at all, which was the choice — the suite is fast, the assertion library comes from `node:assert/strict`, and the workflow is four steps: `npm ci`, `npm test`, `npm run lint`, `npm run build`. If I needed parallel shards, coverage thresholds or reports I'd move to a runner that provides them; for fourteen test groups over pure functions I'd rather have zero test dependencies."

Q28. "You test the reducer's edge cases you happened to think of. Where's the property test?" "That's a fair hit and the reducer is the cheapest place in this codebase to add one: assert that for any sequence of actions from a fixed set, the cursor stays inside `[0, length-1]`, the cursor stays an integer, and reaching the last frame always leaves `playing` false. It found something the moment I looked at it by hand — `seek(2.5)` stores a non-integer, which means the invariant as I've stated it is false today. So the property test wouldn't have just added coverage; it'd have failed."

Group D · "The product is trivial" (Q29–Q38)

Q29. "Six sorting algorithms a first-year can write. Where's the difficulty?" "The algorithms aren't the difficulty, and I wouldn't claim they are. The difficulty is the instrumentation: making a synchronous algorithm emit a complete, immutable, deterministic, human-explainable history while keeping the array the source of truth for the final result, so the trace and the algorithm can't disagree. That's an inversion of the normal relationship — normally the algorithm's output is the deliverable and you throw the intermediate state away."

Q30. "A visualizer is a toy. Nobody uses it." "I have no usage numbers and I won't invent any — the app has no analytics and no accounts, by design. What I'd say is that the interesting engineering isn't adoption, it's that visualization forces you to expose state you'd otherwise discard, and that constraint produced genuinely non-obvious findings: the duplicate-id bug, a comparison counter that increments on leaves, a precondition asymmetry between the reference function and the one that validates."

Q31. "Why 24 elements maximum? Because it looks like a toy." "Because of what one frame costs — a full clone of the array plus a clone of the counters, per instrumentation point — and because of what the screen can show. The worst trace at that cap is 1,154 frames and about a megabyte serialised. I could raise the cap and freeze the tab; I could lower it and lose the reverse-sorted cases. Twenty-four is a stated bound rather than a discovered one, and raising it is exactly why delta frames are the first item in the improvement plan."

Q32. "You don't handle the case where the array has more than 24 elements." "I don't, and I say so on screen: the validation message is 'Use 2–24 values. Small inputs keep every operation inspectable.' Refusing is the design; silently truncating or accepting-and-freezing would be worse. If someone needs sorting at scale, this is the wrong tool and the README's own description says what it is."

Q33. "Your UI has footnotes. Real products don't footnote their own graphs." "They do when the graph invites an inference the data can't support. Two bars labelled 'comparisons' next to each other read as a ranking; the footnote is what stops the product from claiming something the counters don't measure. I'd rather a slightly denser panel than a misreading — and a senior interviewer has usually been on the receiving end of a product claiming more than its data."

Q34. "Where's the dark mode toggle?" "There isn't one. The styling is a set of custom properties in `lab.css` and Tailwind is used in one place, so they're all in one file — a `data-theme` attribute on the root plus a second block of variables would do it. It's a real gap in polish, not a limitation of the design."

Q35. "No keyboard shortcut help, no onboarding." "Guarded shortcuts exist — Space, the arrow keys — with an ignore-list for form fields, and there's no discoverability UI for them. The two pages carry introductory copy rather than a tour. That's a product gap: the shortcuts are invisible unless you read the code, which is the wrong place for a user to find a feature."

Q36. "What happens on small screens? The lab has six panels." "The layout is media-query driven at four breakpoints, with a reduced-motion query, and at the narrowest the panels stack vertically rather than disappear — so the trace stays visible and the inspector moves below it. It's usable at the narrowest breakpoint but it's not optimised for it; the comparison panel's two arrays side by side compress the bars considerably and I'd stack them at that width rather than shrink them."

Q37. "Your bar heights use a 25% floor, so the visualisation lies about the values." "That's a real distortion and I'll name it precisely: the height is `25 + (|val| / max(|val|)) * 75`. So a zero-valued element renders a bar a quarter of the maximum height, and the mapping is affine rather than proportional to the value. The reason is legibility at small magnitudes and it's not disclosed. The fix is either a true proportional scale with a minimum render height of a few pixels, or a documented note — and I'd want the note if the floor stayed."

Q38. "Why does the shuffle button reload seed 42 the first time?" "Because the shuffle does `setSeed(seed + 1)` and then loads with the `current` seed — which at mount is 42. So the first click shows seed 42's dataset and the sequence effectively starts one step behind. It's a genuine off-by-one in the interaction, not the dataset generator, which is deterministic and asserted."

Group E · "Your numbers are made up" (Q39–Q46)

Q39. "1,154 frames. Did you count that or guess it?" "Counted, by importing `engine.js` into a Node script and running every algorithm over the reverse-sorted 24-element input. Bubble sort produces 1,154 frames; its serialised size is about 981 KB; selection is 492, insertion 876, merge 375, quick 632, heap 348. I can re-run the script and show you the table."

Q40. "Compare bubble and merge — bubble does more comparisons, so bubble is slower. Right?" "Bubble does more comparisons here, but comparisons are a count of decisions, not a measure of time. Merge does fewer comparisons and more writes — 24 writes against bubble's 16 exchanges on the default input — so the counters don't produce a ranking. What genuinely matters is that bubble is quadratic and merge is n log n, and at 24 elements you can't see that difference in any of these numbers."

Q41. "Is the percentage alignment a measurement of execution time?" "No. The percentages are positions in two recorded sequences. The caption says 'aligned by percentage of trace—not equivalent operations', and the panel text says there are no wall-clock claims. If both traces were timed, the alignment would still be positional — the alignment and the timing are unrelated concerns."

Q42. "You said 150 cases. Is it 150 or 300?" "150 is the algorithm-input count: six algorithms times twenty-five inputs. Each of those runs four assertions, so if you prefer counting runs rather than cases you get 600. Both numbers are true; the one I quote is the narrower one, because it's the one that maps to the loop bounds in the file."

Q43. "Does the app measure speed at all?" "No. There's no `performance.now()`, no `Date`-based timing, and nothing in the UI is labelled with a duration. The one thing that has a duration is the playback speed control — 600, 300, 160 and 50 milliseconds per frame — which is a viewing preference, not an algorithm property."

Q44. "Your heap sort counters are inflated. So the comparison is broken." "The counter is inflated relative to a hand-written comparison count, because `heapify` increments twice per call regardless of whether the children exist — verified directly: the counter rises even on a leaf with no children. It's not corruption, it's a definition. The panel carries a footnote that comparison counts are implementation-specific and it uses comparisons only, which is the one counter where the definitions agree."

Q45. "Merge sort's exchange count of 24 is nonsense for eight elements." "It's a write count, not a swap count. Merge sort never swaps; it copies out of a buffer and writes back, and every one of those writes increments the exchange counter. Eight elements across seven merges with the tail copies gives 24 writes. The UI labels it 'recorded exchanges' and the footnote says the exchange counter is counted differently per generator. The honest fix is to rename the counter per algorithm."

Q46. "How do I know your 768 number isn't just printed in the docs?" "Because the loop bounds produce it: 64 arrays times 12 targets. The 64 comes from all three-element arrays over the digits 1 to 4 — four to the third — and the targets run 1 to 12. Run `npm test` and the group executes 768 comparisons against the inline brute-force oracle. If you want to see the arithmetic, it's two nested loops in `tests/engine.test.js`."

Group F · "You're hiding something" (Q47–Q55)

Q47. "What's the worst bug in this codebase?" "The merge sort duplicate-id bug: the buffer write-back stores shallow clones, so the same id exists twice in the main array for the write-back frames, and `ArrayView` keys columns by id. Twenty-three of 107 frames on the default input; 141 of 375 on the reverse-sorted 24. It's a real defect, it produces a React duplicate-key warning, and my own test suite misses it because the id assertion only covers the final frame."

Q48. "What breaks first under load?" "Frame storage. Every frame is a full cloned array plus a cloned counter object, so memory grows with instrumentation density times array length — the worst case in the app is about a megabyte of serialised frames for one trace. Right after that comes the render, because `ArrayView` maps every element on every frame with no virtualization, and then the comparison mode, which doubles both."

Q49. "What's not implemented that a reviewer would assume is?" "Five things worth naming: no error boundary; no component or browser testing; no dark mode; no shareable URL for a full experiment, only `?algorithm` and `?compare`; and no export format other than the single-experiment JSON. Also, four fields the frame schema carries that the app's UI never draws: the swap flag, the sub-array range, the tracked minimum and merge sort's buffer offset. Two unimported legacy files do read them, which is why I'm precise about saying the shipped UI doesn't."

Q50. "You claim a trust boundary. Where's the code that enforces it?" "Three places. The import path, where every field is re-validated and `Object.hasOwn` is used instead of `in` so a crafted algorithm key can't walk the prototype chain; the storage read path, with a try/catch, a shape check and a filter that drops unknown algorithms; and the URL path, where `?algorithm` is checked against the registry before use. What I do not claim is that those make the app secure — there's no authentication, no server and no user data to protect. The boundary is about not executing or rendering something a file told me to."

Q51. "Your saved data includes user input. Is that a privacy issue?" "There is no personal data in a record: an algorithm key, an array of integers, a frame index and a timestamp. It stays in the user's own browser and is never transmitted — there's no code path that sends anything anywhere. The exposure is exactly what `localStorage` is: readable by any script on the same origin, so any XSS would read it. The app renders user text through JSX interpolation rather than raw HTML, which is what makes script injection by-construction unlikely, but I wouldn't describe it as a security guarantee."

Q52. "If I give you a JSON file with a million entries, what happens?" "It's rejected before parsing, by the size gate: 16,384 bytes. That bound exists so a large file can't be parsed into a gigabyte of array before the shape check runs. What it doesn't do is stop a small file that expands — there's no depth limit on the parsed object, though JSON.parse doesn't execute anything and the shape gates reject anything that isn't the expected flat form."

Q53. "What happens if the algorithm is deleted from a future version but the user has saved experiments using it?" "Those records are dropped on read rather than crashing: the filter requires the algorithm key to still exist in the registry. The consequence is silent data loss for that record, which I'd rather surface — a 'three saved experiments could not be loaded' notice. Dropping is the safe half of the trade; telling the user is the missing half."

Q54. "What happens if the team changes frame ordering in a release?" "A saved cursor becomes meaningless — it addresses an index, and the record stores an index, not a semantic position. That trade-off is in the README. Two ways out: store the cursor as a fraction of the trace, which survives reordering but not regression, or store the `codeLine` plus the counters and resolve on load, which is the robust version and needs the frames to be stable enough to search. Neither is implemented."

Q55. "Tell me something about this project that isn't in the README and isn't in the code." "Then I'd be making it up, and I won't. What I can give you is something the README doesn't say: the strongest thing I've learned from building it is that instrumentation is a design decision, not a logging concern. Where you put a snapshot determines what a user can be told, and where you don't put one becomes a feature gap — merge sort's buffer is invisible because there's no frame exposing it, not because nobody thought of the idea. The generator emits the flag for it; the renderer just doesn't read it."

MUST KNOW

THE ONE-SENTENCE LANDING FOR A HOSTILE ROUND

"Everything I've told you is either in a file you can open, a number I can re-measure, or a limitation I named first. Where I don't know, I'll say I don't know — and I'd rather be trusted on the small things than impressive on the big ones."

CHECKLIST

RED-LINE DRILLS FOR PART XXXII

- ■ Answer the six "you didn't build this" questions without being defensive.
- ■ Name the four architectural criticisms you agree with and their fixes.
- ■ Recite the merge duplicate-id bug and the two-line test fix.
- ■ Recite the worst-trace numbers: 1,154 frames, ~981 KB, bubble on reverse-24.
- ■ Explain why the bars' 25% floor is a real distortion.
- ■ Answer Q49 — the five things a reviewer would assume exist.

PART XXXIII

Honest weakness defence

Thirty weaknesses ranked by severity, each with the concession, the cost, the fix and the sentence to say.

CHAPTER 59

59 · Naming the weaknesses first

This chapter is the reason an interviewer trusts the rest of the book. Every project has weaknesses; the difference between a strong and a weak candidate is who says the number first.

59.0 The severity model

Rank by **user impact × cost to fix**, not by how embarrassing it is.

Tier	Meaning	How to introduce it
S1	A real defect a user can hit; produces a visible or console-level wrong behaviour	"There's a bug I found and haven't fixed — let me show you exactly where."
S2	A genuine gap in verification or robustness; nothing visibly broken today	"The tests can't catch this class of bug, and here's the two-line change that would."
S3	An honest limitation of scope, disclosed in the UI or the README	"That's a stated boundary, not an oversight — here's why the boundary is where it is."
S4	Polish, debt or migration work with no user-visible effect	"That's outstanding cleanup I'd rather do openly than have discovered."

MUST KNOW

THE FRAMING SENTENCE FOR THE WHOLE SECTION

"I'll start with what's wrong with it, because that's more useful to you than what works. There's one real bug I can point at precisely, two or three verification gaps that matter more than the bug, a handful of deliberate scope boundaries, and some cleanup. Ask me to go deeper on any of them."

59.1 S1 — Defects you should volunteer

W1 · Merge sort emits frames with duplicate element ids

The concession. "`generateMergeSort` copies the window into a buffer with shallow clones, so each cloned element keeps its original `id`, and then writes them back into the main array while the originals are still present. During those write-back frames the same id appears twice, and `ArrayView` keys its columns by `id`. Measured: 23 of 107 frames on the default input, 4 of 30 on `[3,1,2]`, 141 of 375 on the reverse-sorted 24." **The cost.** "React logs a duplicate-key warning in development, and in production reconciliation with duplicate keys is unspecified — a column can fail to update for one frame. It never affects the final frame, and the final frame's uniqueness is asserted, so nothing persists." **The fix.** "Minimal: `key={item.id + ':' + i}` — safe because every frame is a complete snapshot, so positional keys cannot corrupt anything across frames. Deeper: re-id the buffered clone at write time, which preserves element identity at the cost of changing what an id means for merge sort." **Say this.** "It's a bug in my instrumentation, not in the sorting, and my own test suite can't see it because the id assertion only covers the last frame."

W2 · The reducer accepts non-integer and `NaN` cursors

The concession. "`seek` clamps to `[0, length-1]` but never checks that the value is an integer. `seek(2.5)` stores `2.5`, and `frames[2.5]` is `undefined`; `seek(NaN)` stores `NaN`, because `Math.min(NaN, 19)` is `NaN`." **The cost.** "Nothing today — every caller is a range input, a button computed from an integer, or a clamp-protected import. It's unreachable through the UI." **The fix.** "`Math.trunc` plus an `Number.isFinite` guard in the reducer, and a property test asserting the cursor is always an integer in range." **Say this.** "My stated invariant — that the cursor is always a valid index — is false for two inputs. It happens to be unreachable, but an invariant that's guarded

by 'the UI won't do that' isn't an invariant."

W3 · `tick` starts playback even when it was paused

The concession. "The `tick` case sets `playing: cursor + 1 < length - 1` unconditionally, so a `tick` dispatched while paused produces `playing: true`. It's UI-unreachable, because the only dispatcher is the timer effect, which is only scheduled while playing." **The cost.** "None observable. It matters as a design point: a reducer that relies on its caller for correctness is harder to test in isolation." **The fix.** "Either drop `playing` from `tick` entirely — the transport already has `toggle` for the flag — or return the state unchanged when `playing` is false."

W4 · `windowTrace` has no positivity guard

The concession. "`generateWindow` rejects non-positive input with 'Use positive numbers and a positive target.', and the Counterexample page states the precondition on screen. But `windowTrace` — the reference the detector compares against — has no guard, so calling it directly on negative values gives a wrong answer silently. Measured: `[-1,3,2]` with target 3 returns 2 where the true answer is 1; `[1,-1,1,1]` with target 2 returns 4 where the true answer is 2." **The cost.** "Contained, because the search space only generates values 1–4. It's an inconsistency in where the precondition is enforced: the library rejects, the reference does not." **The fix.** "Either guard in `windowTrace` and have `generateWindow` call it, or document the reference as precondition-dependent in a comment."

W5 · Merge's buffer state is invisible, so a value appears to vanish

The concession. "During the merge phase the element held in the buffer is not in the rendered array for a moment — frame 15 of `[3,1,2]` renders `[1,1,2]`, and the `3` is sitting in `tempArr`. The generator already emits `yOffset = 80` and a `buffer` frame for exactly this visual; `ArrayView` doesn't read `yOffset`, so the flag arrives and nothing draws it." **The cost.** "A confusing half-second during merge playback, and a rendered array that is momentarily not a permutation of the input." **The fix.** "Apply `translateY` when `yOffset` is non-zero, or render a second 'buffer lane' under the array during merges. No engine change is needed — the data is already there." **Say this.** "The instrument emits the signal and the renderer ignores it. That's the cheapest visual fix in the project and I'd do it first."

59.2 S2 — Verification gaps (these matter more than the bugs)

W6 · The id assertion is on the final frame only

The concession. "`tests/engine.test.js` asserts unique ids on `frames.at(-1)`, so a duplicate-id frame anywhere in the middle passes CI. That's why W1 survived to be found by hand." **The fix.** "Move the assertion inside the per-frame loop: for each frame, `new Set(frame.array.map(x => x.id)).size === frame.array.length`." **Say this.** "Two lines. It's the highest-value test change available in this repository, and it converts a browser console warning into a CI failure."

W7 · Nothing renders a component, so the timer is untested

The concession. "The suite is `node:test` over pure modules. There is no jsdom, no React Testing Library, no browser automation. So the reducer is tested directly, and the wiring from the reducer to the timer to the frame is not tested at all." **The fix.** "Component tests with fake timers asserting: play advances exactly one frame per interval; pause leaves zero pending timers; unmount cancels; changing speed while playing reschedules with the new delay. That covers the class of bug that a user would actually report."

W8 · No error boundary anywhere

The concession. "A throw in any route unmounts the entire app rather than degrading one panel. `Suspense` provides a loading fallback, not an error fallback." **The fix.** "A small boundary component mounted inside each lazy route."

W9 · Nothing verifies the alignment formula beyond its arithmetic

The concession. "`compareCursor` is one `Math.round` expression and it's correct as arithmetic, but there's no test asserting it stays inside `[0, otherFrames.length - 1]` for every primary cursor, and no test of the degenerate case where a trace has one frame — which would divide by zero and produce `NaN`." **The fix.** "A loop over every cursor in every algorithm pair, asserting the result is an integer in range. Practically unreachable, because every generator emits at least two frames, but that's an argument for the assertion, not against it."

W10 · Import has nine defined behaviour cases and zero automated ones

The concession. "Chapter 42 lists the nine files a reviewer would try — invalid JSON, wrong version, unknown algorithm, non-array input, out-of-range values, oversized, valid, empty object, and a prototype-pollution attempt. None is covered by a test, because the handler lives in a component." **The fix.** "Extract the gates into a pure `validateExperiment(text)` in the engine, test it directly, and have the component call it. That is a refactor worth doing for its own sake — it also makes the schema a single source of truth instead of two."

W11 · No test would catch a regression in the derived clamp

The concession. "The render clamps the cursor to the frames length. Nothing asserts that no render ever sees `frames[undefined]`, so the safety net itself is unverified."

W12 · `catch {}` on the storage write discards the reason

The concession. "The write path catches and reports 'Browser storage is unavailable' without logging the cause, so a quota error and a security error are indistinguishable to the user and to me." **The fix.** "Log the exception and branch the message. One line, and it converts an unexplained failure into a diagnosable one."

59.3 S3 — Deliberate scope boundaries (disclosed, therefore defensible)

W13 · A 24-element cap

"A frame is a full clone of the array plus a clone of the counters, so cost is instrumentation density times length. Twenty-four elements with the densest instrumentation — bubble sort on reverse-sorted input — produces 1,154 frames and about 981 KB. The cap is what keeps that from becoming a frozen tab, and the validation message says why: 'Small inputs keep every operation inspectable.'"

W14 · Recording frames is memory-hungry by construction

"Full snapshots are the honest representation and they make rewind free, at roughly a kilobyte per frame. [PROPOSED] The fix is delta frames plus periodic keyframes, which trades reconstruction cost for memory. It's why the cap exists, not a mistake that hid the cap."

W15 · `length` duplicated in the reducer state

"The reducer can't see `frames`, so the end position has to be in its state or in every action. Duplication means a code path that swaps the trace without dispatching `load` leaves a stale bound. A derived clamp keeps it invisible, and I'd rather remove the duplication than keep the guard."

W16 · The trace is computed twice per commit

"`load()` calls the generator for the length, and the memo calls it again for the frames. Microseconds at this size, two chances to disagree at any other size."

W17 · Counter units differ per algorithm

"'Passes / calls' means an outer-loop iteration in bubble sort, a recursion node in merge, an extraction in heap. 'Exchanges' is a real swap count in bubble and a buffer-write count in merge — 24 writes for eight elements. Hence the on-screen footnote and hence the comparison panel using comparisons only."

W18 · Heap sort's comparison counter increments unconditionally

"Two increments per `heapify` call whether or not the children exist. Verified on a leaf: the counter rises with no children. It's a definition, not corruption, and the footnote says comparison counts are implementation-specific."

W19 · Bubble sort has no early exit

"So a sorted 24-element input still costs 276 comparisons. The shipped algorithm is the textbook worst case, which is arguably a feature of a teaching tool — you can see the quadratic behaviour — but it is not the code you'd ship in a library."

W20 · No backend, no accounts, no sync

"By design and stated in the README. The consequence a reviewer should know: `localStorage` is per-origin, per-browser and per-device, so experiments don't follow the user. Export is the portability path."

W21 · `localStorage` is not secure storage

"It's readable by any script on the same origin and it is not encrypted. There's no sensitive data in a record — an algorithm key, an array of integers, a frame index, a timestamp — and the app never transmits anything anywhere."

W22 · No URL-encoded full experiment

"`?algorithm` and `?compare` can be shared; a dataset, a frame and a saved experiment cannot. Sharing the whole experiment needs either a compact encoding in the query string or an export file, and the export file already exists — so a URL encoding is the missing half."

59.4 S4 — Debt and polish

W23 · Nine dead components plus two dead modules

"`src/components/*` including a zero-byte `Layout.jsx`, plus `SimpleSort`, `EfficientSort`, a context file and some CSS. Zero importers; the production bundle confirms it. It's repo readability, nothing else."

W24 · A `fetch` to a third-party counter sits in unimported code

"`CardsGrid.jsx` calls countapi. It has no importers so it never executes — which is also why it isn't a live privacy issue. It should be deleted, because it contradicts the app's own description."

W25 · A lint rule is disabled for a directory

"`react-hooks/set-state-in-effect` is off for `src/modules/**` with a comment explaining that those pages are the legacy execution model. It's labelled debt with an address, and I'd rather keep the rule on for the code I'm actively writing than disable it globally."

W26 · `SortingLab.jsx` is 596 lines

"Ten `useState` calls, a reducer, two effects, the derivations and the whole render in one file. The smell isn't that it uses `useState` — it's the size. The lines that would leave first are the persistence paths, which are pure enough to be lifted out and tested."

W27 · The keyboard effect re-registers on every cursor change

"Its dependency array is `[cursor]`, so one `removeEventListener` and one `addEventListener` per frame. At six frames a second that's twelve DOM calls a second — harmless, and the fix is a relative `step` action that removes the dependency."

W28 · The bar height has a 25% floor

"`25 + (|val|/max) * 75` means a zero renders a quarter-height bar and the mapping is affine rather than proportional. Legible at small magnitudes, and an undisclosed distortion — which is the part I'd change."

W29 · No dark mode, no shortcut discovery, no onboarding

"All three are product polish. The shortcuts exist and are documented in the code rather than in the interface, which is the wrong place for a user to find a feature."

W30 · Shuffle's first click reloads seed 42

"`setSeed(seed + 1)` then load with the pre-increment value, so the first click shows seed 42 and the sequence effectively starts behind. An off-by-one in the interaction, not in the generator, which is deterministic and asserted."

59.5 The three-sentence version

SAY THIS OUT LOUD
THE NINETY-SECOND WEAKNESS ANSWER

"I'd start with the one real bug: merge sort's buffer write-back produces frames where the same element id appears twice — 23 of 107 frames on the default input — and my columns are keyed by id, so React warns about duplicate keys. It's my instrumentation, not the algorithm, the final frame is always clean, and the reason CI misses it is that my id assertion only covers the last frame; moving that assertion inside the per-frame loop is a two-line fix that would have caught it.

Then, ranked by what I'd fix next: the reducer accepts non-integer and NaN cursors, so my stated cursor invariant isn't actually true; nothing in the suite renders a component, so the timer wiring is untested; there's no error boundary; and the import gates are well designed and completely unautomated, which is a refactor I'd do — pull the validation out of the component into a pure function and test the nine cases directly.

And the boundaries that are deliberate rather than accidental: a 24-element cap because a frame is a full snapshot and the worst trace is 1,154 frames and about a megabyte; counters whose units differ per algorithm, which is why the comparison panel uses comparisons only and carries a footnote; no backend and no accounts by design; and localStorage, which I'd never describe as secure storage. The README lists most of this, and where it doesn't, it's because I found it after writing the README — which is the honest reason a doc goes stale."

CHECKLIST
RED-LINE DRILLS FOR PART XXXIII

- ■ Name the S1 defects in severity order without notes.
- ■ Give W1 in under 45 seconds, including the measured numbers and the two-line fix.
- ■ Say why W6 matters more than W1.
- ■ Name three deliberate boundaries and why each is where it is.
- ■ Deliver the ninety-second answer without hedging language.
- ■ Never say "it's basically fine" about any of them.

PART XXXIV

Cross-domain questioning

Fifty-four questions that cut across two subjects at once — the format senior interviewers actually use.

CHAPTER 60

60 · Fifty-four cross-domain questions

Single-topic questions can be answered from notes. These cannot: each one requires you to hold two domains at once and decide which one governs. Every answer below is short on purpose — say the mechanism, then the consequence in this codebase.

60.1 Algorithms × React rendering

1. "You clone the array on every frame. Where does that memory actually live, and when does React care about it?" The frames are plain objects in a JavaScript array held by a `useMemo` value — ordinary heap memory, not React state and not DOM. React only cares about the small part that reaches the render: one frame object, which `ArrayView` maps into ~24 elements. So the kilobyte-per-frame cost is GC pressure, not render cost; the frames are retained for as long as the memo is, and the memo survives until `input` or `algo` changes.

2. "Does React re-render when the cursor changes? Does it re-render the algorithm?" It re-renders the component, yes — a cursor change is state, so the render runs. It does **not** re-run the algorithm: the frames are memoised on `[input, algo]`. That split is the entire performance story of the app: quadratic work happens once per commit, linear work happens per frame, and per-frame work is a `map` over at most 24 elements.

3. "You said rendering is a pure function of a frame. Is that literally true?" Almost — `ArrayView` is pure in its frame, but the parent render also reads `cursor`, `compare`, `speed`, the saved list and the banners. The stage itself is pure; the page isn't. That distinction matters because it's what would let you extract the stage into its own memoised component today, and it's the first thing I'd do before profiling anything.

4. "Where would a virtualised list help this app, and where would it hurt?" It would help `frames.length` — the timeline and any frame browser — and it would help the comparison panel at large n, because it maps two arrays per render. It would **hurt** the main stage: virtualization exists to avoid mounting thousands of DOM nodes, and the stage mounts at most 24 bars, of which only the active two change class. Virtualizing 24 elements single-file would add measurement and scroll-sync code to remove nothing.

5. "What's the algorithmic complexity of a single React render here?" O(n) in the array length for the `map`, plus O(n) for the `max` reduction inside `ArrayView` — so O(n) with n ≤ 24, and the constant is dominated by React's reconciliation of ~24 elements with ~6 class strings each. Comparing to the generation cost, a render is negligible relative to one memo recomputation, which is O(frames × n).

6. "If you memoised `ArrayView` with `React.memo`, what would break?" Nothing would break, and nothing would be gained: its only prop is the current frame, which changes on every render of the parent, so the shallow comparison never short-circuits. `React.memo` on a component whose props change every frame is a pure cost. The version that would help is memoising the bar rows keyed by element id, because only two of them change class per frame — and that's only worth doing if profiling says so.

60.2 Algorithms × the browser event loop

7. "You run a quadratic algorithm in the middle of a React useMemo. Does that block the UI?" It blocks the thread, and it is synchronous. At n = 24 the worst case is bubble sort with 1,154 frame pushes and about 1,154 array clones of 24 elements each — tens of thousands of operations, sub-millisecond to a few milliseconds. It's imperceptible, which is exactly why the 24

cap exists. At n = 500 the same code would block noticeably, and the fix is a worker, not a `setTimeout` split.

8. "Where would a Web Worker help, and what would it cost you?" Generation is pure and takes plain data, so it's the ideal worker payload: post the array, get back frames, keep the thread responsive at any n. The costs are real — structured-clone serialisation of the whole trace across the boundary, which for a megabyte trace is worse than computing it; a loading state the UI doesn't have; and a second copy of `engine.js` in the bundle unless it's shared. [PROPOSED] The right first step is a heavier cap plus incremental chunking, not a worker.

9. "Your timer is 50 ms at the fastest speed. What's the actual frame interval?" At least 50 ms, and often more, for two reasons: the browser clamps nested timeouts to a minimum (4 ms after nesting depth 5), and timers are macrotasks, so a long render or a GC pause delays the next one. So the animation is approximately 20 fps at best, and the label "6×" describes the ratio between delay values, not a guaranteed frame rate.

10. "What happens to playback if the user switches tabs for ten seconds?" Throttled: background tabs get clamped timers, typically to one per second or less. Because the design is a self-scheduling chain rather than an interval, no burst fires on return — the outstanding single timeout is simply late. So the animation effectively pauses and resumes without skipping, which is precisely the failure mode intervals have.

11. "Is `Math.min` in the reducer in any sense asynchronous?" No. The reducer is a synchronous pure function; the entire transport is synchronous. The only asynchronous things in the lab are `setTimeout` for scheduling, the `File.text()` promise in import, and React's own batching — which is why `setInput` and `dispatch({type:'load'})` in `load()` land in the same commit rather than causing an intermediate render with a mismatched pair.

12. "If I click Step 20 times fast, how many frames render?" Up to 20 commits, one per `seek`, because each click is a discrete event and React batches synchronously within the handler. Automatic batching in React 19 collapses updates within a single event, not across events, so 20 clicks are 20 renders. Each render is cheap by design — one array index and a `map` of 24 — which is why this isn't a problem worth solving.

60.3 State management × testing

13. "You test the reducer but not the component. Which bugs does that leave invisible?" Three classes: timer wiring (a missing cleanup, a missing dependency, duplicate schedules), render-time derivation (a stale frame index, an off-by-one in a disabled condition), and persistence round-trips, because `localStorage` and `File` are browser APIs. Everything about the state machine is covered; everything about the edges between the machine and the browser is not.

14. "How would you test the timer without a browser?" Fake timers — Node's `mock.timers` in the built-in runner, or a library's fake clock — plus a component test that renders the lab, clicks Play, advances the clock one interval, and asserts the cursor advanced exactly one. That requires a DOM, which means adding jsdom or the react-test-renderer. The honest sequencing: extract the timer wiring into a small custom hook, test the hook, leave the component thin.

15. "What's the cheapest test that would have caught your merge bug?" Two lines inside the existing loop: for every frame, assert that the number of distinct ids equals the frame's array length. It needs no new dependency, no DOM, no new file — it's an assertion that already exists, moved from the last frame into the loop.

16. "Property test versus example test — which suits each module here?" Property tests suit `parseInput` (for any accepted string, the parsed values round-trip to the same string form), the reducer (the cursor is always an integer in range; the last frame always clears `playing`), and `compareCursor` (the result is always in range for any pair of lengths). Example tests suit the

generators' specific shapes — the seed frame, the `break` frame, the duplicate-id frames — because those are decisions, not properties.

17. "You have no coverage number. What would it buy you, and what would it not?" It would buy two things: an automated check that a new module is referenced by some test at all, and a diff-coverage gate on PRs. It would not measure assertion strength, and the merge bug proves it — the generator has essentially full line coverage and a real defect in the frames it emits. Coverage is a floor detector, not a quality signal.

18. "Is the reducer's `default` branch testable? Should it be?" It is: dispatch an unknown action and assert reference equality with the input state, which is the same assertion that guarantees React bails out of the re-render. It should be tested precisely because the branch exists for a runtime reason and not merely for exhaustiveness. The stronger version is to throw in development, and then the test asserts the throw.

60.4 Persistence × security

19. "Name every way something the app trusts can be wrong." Three sources: `localStorage`, which the app wrote but users and other scripts can edit; the URL query string, which is entirely user-controlled; and an imported file, which is entirely user-controlled. All three end up as `{algorithm, input, cursor}` and all three are re-validated — the registry check for the algorithm, `parseInput` for the values, integer-and-clamp for the cursor.

20. "You use `Object.hasOwn`. What does it prevent, concretely?" It prevents a lookup from reaching inherited properties. `'constructor' in algorithms` is `true` because `in` walks the prototype chain, so an import file with `"algorithm":"constructor"` would pass an `in` check and then `algorithms['constructor']` would be `Object` — a function — and calling `.run()` on it would throw inside the component. `Object.hasOwn` requires the key to be an own property of the registry object.

21. "Could a crafted import file execute code?" Not through this path. The file's text goes to `JSON.parse`, which produces data and executes nothing; the values then go through a regex-anchored validator; the algorithm key goes through an own-property check; nothing is passed to `eval`, `new Function`, `setTimeout` with a string, or `innerHTML`. React renders text through JSX interpolation, which escapes by default. What I don't claim is that the app is secure — there's no server, no authentication and no user data, so there's no meaningful attack surface beyond the user's own browser.

22. "What's the risk of storing input in `localStorage`?" Any script that runs on the same origin can read it, and it is not encrypted. The data is an array of integers, so the privacy exposure is minimal; the availability exposure is that clearing site data destroys the user's saved experiments with no warning and no recovery path. Export is the only backup, and there's no prompt to use it.

23. "Is your storage read path a security boundary or a robustness boundary?" Robustness, deliberately. It exists so a corrupted store produces an empty list rather than a crash on mount. The security-relevant part is the prototype-safe algorithm check on the import path, because that data arrives from outside. I'd describe the storage path as defensive, not as a security control.

24. "What would you have to add before this could safely sync to a server?" Almost everything that doesn't exist: an identity model, transport encryption, authorisation on the record, server-side validation of exactly the same schema, a conflict-resolution policy for two devices editing one record, and a migration story for the `version` field. The one thing I'd keep as-is is the record's shape — it's a few hundred bytes of plain data with an explicit version number, which is the right starting point for that work.

60.5 Build, bundling and module systems

25. "You use `rolldown-vite` through an npm alias. What does that mean for a reviewer?" It means `package.json` says `"vite": "npm:rolldown-vite@7.2.5"`, so the package named `vite` is actually Rolldown-based Vite, and `overrides.vite` makes the transitive dependencies resolve to the same one. A reviewer cloning the repo gets exactly that build, and the practical consequence is that it's a fast, native-Rust bundler that is not the current default — so "why is your vite different from everyone's" has an answer in the manifest, plus a chapter in the README.

26. "Your bundle is 251 KB for an app this small. What's in it?" React and React DOM dominate; `react-router-dom` and `lucide-react` add the rest. The measured build prints 1,736 modules, one main chunk at 251.78 kB (80.90 kB gzip), plus a 29.85 kB SortingLab chunk and an 18.65 kB engine chunk, with 72.37 kB of CSS. The size is unremarkable for React 19 and there's no obvious dead weight to cut; the lever if I needed one is the icon package, which imports per-icon but is the kind of dependency that grows silently.

27. "Why is there no code splitting beyond the routes?" Because the routes are the natural seams and they're already split: fourteen of the seventeen routes render a lazily-imported component — twelve distinct `React.lazy` components, with `SortingLab` used by three routes — so the module pages and the lab load on demand. Splitting further — the algorithm generators out of the lab, for example — would create a chunk boundary inside a feature that's always loaded together, adding a request to save nothing.

28. "How does the build ensure the test files aren't shipped?" Two ways. Vite only follows the import graph from the HTML entry, so `tests/` is never reachable — it's not in `src/` and nothing imports it. And the test command is `node --test tests/*.test.js`, which is a separate execution path entirely. The guarantee is structural: the entry graph is the bundle.

29. "What would break if someone upgraded Node?" The floor is `engines.node >= 22.12.0` with a matching `.nvmrc`, so a downgrade is a warning today and an install-time failure under `engine-strict`. An upgrade is the interesting direction: Node 22's test runner is a stable feature, `node --test` with glob patterns behaves consistently, and the code uses no Node APIs beyond `node:test` and `node:assert/strict` — so the risk is in the toolchain, not in my source.

30. "Why does CI run the build at all, if tests are the gate?" Because a test suite over pure modules can pass while the app fails to build — a bad import path, a syntax error in a JSX file no test imports, a Tailwind or PostCSS config problem, a missing asset. The build step is the only gate that exercises the whole module graph. In this repository it's genuinely non-redundant: none of the fourteen lazily-loaded route components is imported by any test.

60.6 URL, routing and HTTP semantics

31. "A URL is a user input. How do you treat it?" Like an untrusted one. `?algorithm` is validated against the registry before use and falls back to a default; `?compare` is tested with `params.has`, so `?compare=0` still enables comparison — a deliberate "presence means yes" reading rather than an accidental one. Nothing from the URL is rendered as HTML, and nothing from it is used as a key without an own-property check.

32. "Why does a static SPA need a rewrite rule?" Because the server sees `/modules/core/advanced-sort` as a path, not a route. Without a rewrite, a direct hit or a refresh returns 404 since no such file exists. `vercel.json` rewrites everything to `index.html`, and the client router resolves the path. That's why the rule exists in the deployment config rather than in the app.

33. "What's the difference between the dev server's host config and the deployment's?" `vite.config.js` allows specific hosts and binds to all interfaces so the dev server is reachable from outside the machine — a preview requirement. `vercel.json` is about path resolution in production. Neither affects the app's routing logic, which is entirely client-side in

both cases.

34. "If you added a `/experiment/:id` route, what would have to change?" The route table, the lazy import, and the data source — there's no server to fetch an id from, so `:id` would have to address something local: an index into the saved list, or a compact encoding of the experiment in the URL itself. The second is the version worth building for shareability, and it's [PROPOSED]; today only `?algorithm` and `?compare` travel in a link.

35. "Does deep-linking a lab state survive a refresh?" Partly. The algorithm and the compare flag are in the URL, so they survive; the dataset, the cursor and the speed do not, because they're component state. A saved experiment survives a refresh but is not in the URL. So the honest answer: the route is linkable, the state is not.

60.7 CSS, layout and rendering

36. "Your bars change class per frame. Is that more or less expensive than animating height?" Less, in this design. A class change invalidates style for that element and typically only triggers paint, and there are four states changing a background and a border. Animating `height` would invalidate layout for the whole row every frame — and with flexbox, every sibling. If the design wanted smooth growth, the correct property to animate is `transform: scaleY`, which is compositor-friendly.

37. "Why `prefers-reduced-motion` matters in a visualizer." Because the motion is the content: continuous bar movement and a self-scheduling timer at up to 20 fps is exactly the pattern that triggers motion sensitivity. The media query in `lab.css` is the accessibility affordance, and the honest limitation is that it doesn't automatically stop autoplay — a user still has to press Pause, and ideally playback would start paused when the query matches.

38. "What's the layout cost of the comparison panel?" Two flex rows of up to 24 bars each, plus two meters — so roughly 50 additional DOM elements and a second `ArrayView` invocation per frame while comparison is on. It's the most expensive per-frame render path in the app, and the cheapest mitigation is to memoise the secondary view on `compareCursor`: while the primary advances, the secondary also advances, so that doesn't help; the real fix would be a static preview rather than a second live trace.

39. "Your bars use percentage heights inside a fixed-height stage. What breaks?" A fixed-height parent with percentage children is the standard way to get proportional sizing without measurement, and it's stable — but the child's height is a percentage of the parent, not of the data, so the 25% floor and the max-normalisation both live in the formula rather than the layout. What breaks first is very large `|val|` spread: one 999 and a row of 1s makes every other bar 25% of the stage, because the scale is relative to the frame's maximum.

40. "Is your UI accessible?" Partially, and I'd say so plainly. There are real flaws: the bars carry no `aria-label`, the range input has no associated label, the metric tiles are visual only, and the keyboard shortcuts are undiscoverable. There are also things done right by construction: every actionable control is a real `<button>` or `<input>`, focus order follows DOM order, and the icon-only transport buttons have `title` attributes. What's missing is the general affordance — nothing announces that the visualisation changed, which is exactly the part a screen-reader user would need.

60.8 Numerics and JavaScript semantics

41. "Your dataset generator uses `Math.imul`. Why does that matter for reproducibility?" Because `1664525 * state` overflows 2^{53} for large 32-bit states, and once a JavaScript number loses integer precision the low bits are gone — so the recurrence stops being the mathematical LCG the seed implies, and could differ across engines that handle the rounding differently.

`Math.imul` performs the multiply as a 32-bit operation with wraparound, which is the definition the algorithm needs, and `>>> 0` keeps the state unsigned.

42. "Integers between −999 and 999. Why not floats?" Because the visualisation, the counters and the trace all assume exact equality: a comparison count is only meaningful if `a === b` is well-defined, and duplicate detection matters for the id assertion. Floating point introduces values where `0.1 + 0.2 !== 0.3`, and any algorithm using `<=` or `>=` would then have input-order-dependent behaviour. Integer-only is a stated precondition that removes an entire class of subtlety from the tool.

43. "Is `-0` a problem anywhere?" It's accepted by the parser and by `Number.isSafeInteger`. It renders as `0`, sorts fine, and compares equal to `0` — so it's invisible. The one place it would matter is a strict `Object.is`-style comparison in a test; nothing here uses one.

44. "Why is `Array.prototype.sort`'s comparator written as `(a, b) => a - b`?" Because the default sort converts to strings, which puts 100 before 20, and because the `a - b` form returns a number rather than a boolean. Also worth knowing: `Array.prototype.sort` is guaranteed stable since ES2019, and it mutates — which is why the test's expected value is computed from a copy, not the input the generator receives.

45. "Where could a floating-point value reach the UI?" Only through a cursor: `compareCursor` is `Math.round` of a ratio, so it's always an integer. The percentage displays are ratios rendered as text. Nothing else does division, which is why the app has no rounding drift anywhere in its displayed numbers.

60.9 Complexity, scale and cost

46. "Give me the full cost model of one commit." Generation is O(frames) time and O(frames × n) space, and for a quadratic algorithm with per-step instrumentation that's conceptually $O(n^3)$ storage. At n = 24 the worst measured case is 1,154 frames and about 981 KB serialised. On top of that, `load()` runs the generator a second time for the length, and the memo runs the comparison trace if comparison is on — so a commit with comparison enabled is four generator runs.

47. "What's the cost of one playback frame?" One `seek`/`tick` through the reducer — O(1) — one component render, one array index, one `map` over ≤ 24 elements, and up to two class recomputations. Plus, if comparison is on, a second `map` over its own trace. So per-frame cost is O(n) with n bounded at 24, independent of the algorithm.

48. "Which grows first: time, memory or DOM?" Memory, by a wide margin. Time is bounded by the instrumented operation count, which at this cap is a few tens of thousands of operations; DOM is capped at 24 bars plus the comparison's second row. Memory grows as frames × n and is the only quantity that has ever made me reduce a bound.

49. "If you raised the cap to 200, what's the first measured thing that breaks?" Generation time and memory together: bubble sort on a reverse-sorted 200-element input is 19,900 comparisons at roughly 4 frames per comparison — on the order of 80,000 frames, each cloning a 200-element array, which is millions of object allocations and hundreds of megabytes. That's a freeze, not a slowdown. The lever is the instrumentation density — push a frame per pass rather than per comparison — and the structural fix is delta frames. [PROPOSED]

50. "How would you decide the frame budget — measured, or by feel?" Measured, with a stated target: interact at 60 fps, never let a commit exceed about 100 ms, and never retain more than a few megabytes. Then choose the instrumentation density to meet it. The current cap was chosen before I took those measurements, which is the honest order of operations — measurement first, bound second.

60.10 Process, tooling and the human layer

51. "You have one commit. How would you review this repository as a stranger?" Start at `package.json` for the scripts and the Node floor, then `README.md` for scope and non-claims, then `src/main.jsx` → `src/App.jsx` → `src/pages/Home.jsx` for the route table, then `src/lab/engine.js` because it holds the algorithms, the parser and the reducer, then `SortingLab.jsx` as the only consumer, then `tests/engine.test.js`, then the workflow. That's about twenty minutes and it covers the whole surface. The one thing a stranger cannot get from the repository is history, because there isn't any.

52. "What would you put in the README that isn't there?" Three things: a screenshot or a short recording, because the value is visual and text can't carry it; a 'known issues' section listing the merge duplicate-key bug and the non-integer cursor, so the weaknesses are documented rather than discovered; and a one-paragraph 'how to review this codebase in 20 minutes' section, because the route I just described took me months to learn.

53. "How do you know your README is accurate?" Because it's written to be falsifiable. It names files and behaviours, it lists non-claims explicitly — no backend, no timing, no coverage claim, no deployment — and every claim in it maps to something I can point at. What I can't guarantee is that it's current: the duplicate-key bug was found after the README was written. That's the honest failure mode of documentation, and the fix is the same known-issues section.

54. "What's the one thing you'd redo from the start?" The instrumentation boundary. I started by pushing a frame wherever something interesting happened, which produced the densest possible trace and a memory bound I then had to justify. If I started again I'd decide the frame budget first — how many frames per element, and what the user is actually supposed to notice — and let that decision determine where snapshots go. That ordering changes the design: coarser instrumentation means fewer frames, which means the 24-element cap could be much higher, which means the tool could be used on the inputs a person actually has.

MUST KNOW
THE CROSS-DOMAIN ANSWER FORMULA

1. **Name the governing mechanism** ("that's the event loop, not the reducer").
2. **Give the consequence in this codebase** ("so a background tab pauses instead of bursting").
3. **Name the boundary** ("at 24 elements this is imperceptible; at 200 it's a freeze").
4. **Name the fix and label it** ("[PROPOSED] delta frames, or a worker with the serialisation cost").

CHECKLIST
RED-LINE DRILLS FOR PART XXXIV

- ■ Answer five questions at random, out loud, using the four-step formula.
- ■ Explain why `React.memo` on `ArrayView` would gain nothing.
- ■ Explain why the timer chain means a background tab pauses rather than bursts.
- ■ Give the full cost model of one commit, including the double generation.
- ■ Name the accessibility gaps honestly.
- ■ Give the "one thing I'd redo" answer — the instrumentation boundary.

PART XXXV

Two hundred rapid-fire questions

One line to ask, two lines to answer. Drill these out loud until they're reflex.

CHAPTER 61

61 · 200 rapid-fire questions

Use this as a drill sheet: cover the answers, read the question, answer in two sentences maximum, then check. If you can't answer in two sentences, you don't know it yet — go to the chapter in brackets.

61.1 Engine and frames (1–18)

1. What does a sorting generator return? An array of frame objects — the complete recording of the run. [Ch12]

2. Name every field in a frame. `array`, `activeIndices`, `sortedIndices`, `isSwapping`, `codeLine`, `currentStats`, `msg`, plus per-algorithm extras. [Ch11]

3. What are the per-algorithm extras? `minimumIndex` (bubble, selection, insertion), `pivotIndex` and `subArrayRange` (merge, quick, heap), `isLifting` (insertion), `yOffset` (merge). [Ch11]

4. What shape is `array`? An array of `{id, val}` objects, id being `block-<original index>`. [Ch11]

5. Why not store plain numbers? Identity: the visualisation is about movement, and values can be duplicated, so positions can't identify an element. [Ch11]

6. What are the two mandatory clones? `cloneArray()` for the array and `{...stats}` for the counters. [Ch11]

7. What breaks if you drop the array clone? Every frame shows the final state, because they all reference one live array. [Ch14]

8. What breaks if you drop the stats clone? Every frame shows the final counters. [Ch14]

9. Is `sortedIndices` cloned? Yes — a spread on every push, so it's never shared across frames. [Ch12]

10. What is `codeLine` for? A machine-readable tag naming the step, so the UI can explain it and tests can group frames by type. [Ch11]

11. How many distinct message templates per algorithm? Bubble 9, selection 8, insertion 8, merge 15, quick 17, heap 10. [Ch11]

12. Trace length for the default 8 values? Bubble 118, selection 65, insertion 76, merge 107, quick 75, heap 84. [Ch12]

13. Worst trace the app can produce? 1,154 frames — bubble sort on the reverse-sorted 24-element input, about 981 KB. [Ch14]

14. Minimum trace length for an empty input? Two frames for bubble, selection and insertion; four for merge, quick and heap. [Ch14]

15. Does a generator mutate its input? No — asserted for all six algorithms over 25 inputs each. [Ch44]

16. Why does the generator need the array and not just the values? It builds its own `{id, val}` array from the values, so the caller's array is never touched. [Ch12]

17. Where do the algorithms live? `src/lab/engine.js` from line 8, and the generator bodies in `src/modules/core/algorithm/sorting/algorithms/`. [Ch7]

18. Is there a seventh algorithm? No — six entries in the registry, and every list in the app iterates it. [Ch30]

61.2 The reducer and transport (19–34)

19. What is the reducer's state shape? `{cursor, length, playing}`. [Ch21]

20. How many actions does it have? Four: `load`, `toggle`, `tick`, `seek`. [Ch21]

21. What is the initial state? `{cursor: 0, length: <frame count>, playing: false}`. [Ch21]

22. What does `load` do? Resets the cursor to 0, sets the new length and clears `playing`. [Ch21]

23. What does `toggle` do to the cursor? Leaves it alone — unless it's on the last frame, in which case it resets to 0, which is what makes the same action a replay. [Ch21]

24. What does `tick` do at the end of the trace? Sets the cursor to the last index and `playing` to false. [Ch21]

25. What does `seek` do to `playing`? Always sets it to false — every seek pauses. [Ch21]

26. What is the exact `tick` completion expression? `playing: state.cursor + 1 < state.length - 1`. [Ch21]

27. What is the exact `toggle` expression? `cursor: state.cursor === state.length - 1 ? 0 : state.cursor`, `playing: !state.playing`. [Ch21]

28. What does an unknown action type return? The same state object, so React bails out of the re-render. [Ch21]

29. Where does the clamp live? In `seek`, against `[0, length - 1]`. [Ch21]

30. What's wrong with the clamp? It checks the range but not the domain: `seek(2.5)` stores 2.5 and `seek(NaN)` stores `NaN`. [Ch21]

31. `length = 20`, `cursor = 5`, click Play, then three ticks? Cursors 6, 7, 8; `playing` stays true, because 9 < 19. [Ch22]

32. `length = 20`, `cursor = 19`, click Play? `cursor` becomes 0 and `playing` becomes true — that's Replay. [Ch22]

33. Is the reducer exported? Yes — as a plain function, which is why it's unit-testable without rendering. [Ch21]

34. How many consumers does the reducer have? Exactly one: `SortingLab`. [Ch21]

61.3 Playback, timers and input (35–52)

35. What schedules the animation? A `useEffect` that calls `setTimeout` for `speed` milliseconds. [Ch24]

36. Why not `setInterval`? An interval captures its delay at creation, so a speed change can't take effect until the next tick, and a stalled thread produces a burst of queued callbacks. [Ch25]

37. What are the effect's dependencies? `playing`, `cursor` and `speed`. [Ch24]

38. Why is `cursor` a dependency? It's the re-scheduling trigger: the tick changes the cursor, which re-runs the effect and schedules the next one. [Ch24]

39. What happens if you remove the cleanup? Timers accumulate and playback advances multiple frames per interval. [Ch25]

40. What happens if you remove `playing` from the deps? Pausing won't stop the animation. [Ch25]

41. What stops the animation on unmount? The cleanup returned by the effect. [Ch25]

42. Four speed values? 600, 300, 160 and 50 ms, labelled 0.5×, 1×, 2× and 6×. [Ch26]

43. Which label is inaccurate? 2× — 300/160 is 1.875×. Three of four labels are exact. [Ch26]

44. Two keyboard shortcuts? Space toggles playback; the arrow keys seek one frame. [Ch26]

45. What guards the keyboard handler? An ignore-list for `INPUT`, `SELECT`, `TEXTAREA`, `BUTTON`, content-editable targets and modifier keys. [Ch26]

46. What's the keyboard effect's weakness? Its dependency is `[cursor]`, so the window listener is re-registered on every frame change. [Ch26]

47. What validates the dataset? `parseInput`, in `engine.js`. [Ch10]

48. How many values are allowed? Two to twenty-four. [Ch10]

49. What's the accepted number format? An anchored integer regex — optional minus sign, then digits. No decimals, no exponents, no hex. [Ch10]

50. What's the numeric bound? Absolute value at most 999, and `Number.isSafeInteger`. [Ch10]

51. Why the 24-value cap? A frame is a full snapshot, so cost is density times length; the cap keeps the worst trace around a megabyte. [Ch14]

52. What happens to the visualisation when validation fails? Nothing — only the error banner changes, so the last good trace stays on screen. [Ch10]

61.4 React state and effects (53–70)

53. How does state enter the lab? Ten `useState` calls, one `useReducer`, one `useRef` and two effects. [Ch15]

54. Name the nine core state variables. `text`, `input`, `algo`, `speed`, `error`, `notice`, `compare`, `other`, `seed`. [Ch15]

55. What is `text` versus `input`? `text` is the editable draft string; `input` is the validated, committed array. [Ch9]

56. Why keep both? So the text box stays editable and the trace only changes on a valid commit. [Ch9]

57. What does `useMemo` hold? The primary frames, the secondary frames, and now the saved-experiment list's derived values. [Ch16]

58. Why derive instead of store? A stored copy can go stale; a derived value cannot. [Ch16]

59. What's the most important derived value? `const cursor = Math.min(playback.cursor, frames.length - 1)`. [Ch16]

60. Why is that clamp there? It keeps the render safe if the reducer's `length` is stale — a second line of defence. [Ch16]

61. What is `frame`? `frames[cursor]` — one object, the only thing the stage needs. [Ch16]

62. When does the primary memo recompute? When `input` or `algo` changes. [Ch16]

63. When does the secondary memo recompute? When `input`, `other` or `compare` changes. [Ch16]

64. How many effects does the lab have? Two: the timer and the keyboard. [Ch24]

65. What's the naming convention for the event handler state? `saved` for the list and `savedError`/`notice` for the messages. [Ch15]

66. Is there a `useRef` in the lab? No — the two refs in the codebase are in the legacy modules. [Ch15]

67. Why is there no `useReducer` for the form? Because there are no cross-field invariants in the form — no rule couples the text to the algorithm. [Ch21]

68. What's the rule for choosing `useState` or `useReducer`? Does a rule span two fields? Then it's a reducer. [Ch21]

69. Where does the saved list come from? A lazy `useState` initialiser that reads and filters `localStorage`. [Ch41]

70. Why lazy? So the read happens once on mount rather than on every render. [Ch41]

61.5 The six algorithms (71–90)

71. Which are stable? Bubble, insertion and merge. [Ch30]

72. Which are in place? Bubble, selection, insertion and heap — merge needs O(n) auxiliary, quick needs O(log n) stack. [Ch30]

73. Which has an early exit in the app? Insertion sort only — bubble's is deliberately absent. [Ch31]

74. Bubble sort's comparison count on a sorted 24-element array? 276 — no early exit. [Ch31]

75. Selection sort's comparison count? Exactly n(n−1)/2, independent of the input. [Ch32]

76. Selection sort's swap count? At most n−1. [Ch32]

77. What does insertion sort's `break` frame cost? One extra comparison, because the comparison happens before the break. [Ch33]

78. How does insertion sort implement shifts in the app? As swaps, which over-reports exchanges. [Ch33]

79. Merge sort's auxiliary space? O(n) for the buffer, plus O(log n) recursion. [Ch34]

80. Merge sort's exchange counter? It's a buffer-write count, not a swap count — 24 for eight elements. [Ch34]

81. Which algorithm's `sortedIndices` stays empty throughout? Merge sort's, which is a real gap in its instrumentation. [Ch34]

82. Quick sort's pivot choice? The last element — which makes an already-sorted input the worst case. [Ch35]

83. Quick sort's base case cost? A single-element range still pushes two frames. [Ch35]

84. Can quick sort's `sortedIndices` be out of order? Yes — e.g. `[1, 0]`, because pivots are finalised in partition order, not index order. [Ch35]

85. Heap sort's heapify implementation? Recursive, incrementing the comparison counter twice per call unconditionally. [Ch36]

86. Heap sort's declared auxiliary space? O(log n), footnoted as call-stack space. [Ch36]

87. Which algorithm is quadratic in every case? Selection and bubble. [Ch30]

88. Which is n log n in every case? Merge and heap. [Ch30]

89. Which has the least data movement? Selection — at most n−1 swaps. [Ch32]

90. Which is the only one where a value can be absent from the rendered array? Merge, while it's held in the buffer. [Ch34]

61.6 Comparison and counts (91–106)

91. What does `compareCursor` compute? `Math.round((cursor / (frames.length - 1)) * (otherFrames.length - 1))`. [Ch39]

92. Why percentage alignment rather than index? The traces have different lengths — 65 versus 118 on the default input — so index alignment runs out of frames. [Ch39]

93. Does 50% mean the same amount of work in both? No. The percentages are positions in two different recordings. [Ch39]

94. What does the panel disclaim on screen? That alignment is by percentage of trace, not equivalent operations, and that there are no wall-clock claims. [Ch39]

95. How many disclosures surround the comparison feature? Four: same input, percentage alignment, comparisons only, and no timing. [Ch38]

96. Do both traces use the same array reference? Yes, and that's safe because generators don't mutate their input. [Ch38]

97. Which counter is comparable across algorithms? Comparisons only. [Ch40]

98. What does "passes / calls" mean? Unit-dependent: an outer-loop pass, a recursion node, or a heap extraction. [Ch40]

99. What does "recorded exchanges" mean? A real swap count in bubble sort and a buffer-write count in merge sort. [Ch40]

100. Where is that disclosed? In the footnote at the bottom of the lab page. [Ch40]

101. Default input comparison counts? Bubble 28, selection 28, insertion 22, merge 16, quick 16, heap 44. [Ch40]

102. Which algorithm has the most comparisons on the default input? Heap sort, at 44 — because of the unconditional increments. [Ch40]

103. Are the counters guaranteed monotone along a trace? They only increase, because they're cloned from a mutable object that is only incremented. [Ch40]

104. Is any counter a timing measurement? None. There is no `performance.now()` anywhere in the project. [Ch40]

105. What does the meter width mean? Only the ratio between two counts, scaled to the larger. [Ch40]

106. Does the bigger bar mean the faster algorithm? No — more comparisons means more decisions, not more time. [Ch40]

61.7 Testing (107–124)

107. What test runner? Node's built-in `node:test`, with `node:assert/strict`. [Ch43]

108. How many test groups? Fourteen. [Ch43]

109. How many algorithms × inputs in the sorting suite? Six × twenty-five = 150. [Ch44]

110. Name the five hand-picked inputs. Empty, `[1]`, `[2,2,1]`, `[-4,0,3,-1]`, `[5,4,3,2,1]`. [Ch44]

111. And the other twenty? Seeded random datasets of ten values in `[0,19]`. [Ch44]

112. Name the four assertions per case. Final frame is sorted; the input is unmutated; a re-run deep-equals the trace; the final frame's ids are unique. [Ch44]

113. Which assertion is unusual, and why? Trace reproducibility — because a saved experiment stores a frame index, so a reordered re-run would break it. [Ch45]

114. What does the reproducibility assert compare? The whole recording: array contents, metadata, every message string, every counter snapshot. [Ch45]

115. How many sliding-window cases? 768 = 64 arrays × 12 targets. [Ch46]

116. What are those 64 arrays? All three-element arrays over values 1–4, enumerated by base-4 decomposition. [Ch46]

117. What is the oracle? A brute-force triple-nested enumeration written inline in the test file. [Ch46]

118. Why inline rather than imported? So it shares no code with the implementation — correlated failure would make the test useless. [Ch46]

119. Does the oracle use the same predicate? Yes, `sum >= target`, which is the problem's definition — so the tests can't catch a misread spec. [Ch46]

120. What does the sliding window require of its input? Positive values and a positive target. [Ch46]

121. Where is that precondition enforced? In `generateWindow`; `windowTrace` has no guard, which is an inconsistency. [Ch46]

122. What breaks with a negative value? Monotonicity. Measured: `[-1,3,2]` with target 3 gives 2 instead of 1. [Ch46]

123. What is the suite's biggest verification gap? The id assertion covers only the last frame, which is why the merge duplicate-id bug passes CI. [Ch44]

124. What is the highest-value test change available? Move that assertion inside the per-frame loop — two lines. [Ch44]

61.8 Persistence and import (125–142)

125. What's the storage key? `algoviz.experiments.v1`. [Ch41]

126. What's the record shape? `{version: 1, algorithm, input, cursor, createdAt}`. [Ch41]

127. How many records are kept? Six, the most recent, by slicing the front of the list. [Ch41]

128. How is the list read? A lazy `useState` initialiser with four defence layers. [Ch41]

129. Name the four defences. `|| "[]"` for a missing key; `try/catch` for malformed JSON; an `Array.isArray` shape check; a filter dropping unknown algorithms. [Ch41]

130. Why is the algorithm filter necessary? Saved data outlives code — a later release may delete an algorithm, and the record must not crash the page. [Ch41]

131. Where does clicking a saved experiment go? Through `parseInput` again, into `load()`. [Ch41]

132. Is `localStorage` secure? No — it's readable by any script on the origin and is not encrypted. [Ch53]

133. Does anything leave the browser? No. There is no fetch in the shipped app. [Ch53]

134. What does the export payload contain? Four fields: version, algorithm, input, cursor — `createdAt` is omitted. [Ch42]

135. What is the import size limit? 16,384 bytes. [Ch42]

136. Name the import gates in order. Size; JSON parse; object shape; `version === 1`; `Object.hasOwn` registry check; `Array.isArray(input)`; then `parseInput` on the joined values. [Ch42]

137. Why `Object.hasOwn` and not `in`? `in` walks the prototype chain, so `'constructor' in algorithms` is true. [Ch42]

138. What does the import do with the cursor? Keeps it if it's a non-negative integer, otherwise 0 — and the reducer clamps it anyway. [Ch42]

139. What's the biggest gap on the import path? All nine behaviour cases are unautomated, because the gates live in a component. [Ch42]

140. What's the fix for that? Extract the gates into a pure function in the engine and test it directly — nine cases. [Ch42]

141. How does the user get an experiment off one machine? Export the JSON and import it on the other. [Ch41]

142. What happens if a storage write fails? The catch sets a notice and the in-memory list does not advance, because `setSaved` is inside the try after `setItem`. [Ch41]

61.9 Build, CI and tooling (143–160)

143. What's the dev command? `npm run dev`, which runs Vite. [Ch48]

144. What's unusual about this project's Vite? `package.json` aliases it to `rolldown-vite@7.2.5` through `npm:`, with an `overrides` entry. [Ch48]

145. What's the Node floor? `>= 22.12.0`, with a matching `.nvmrc`. [Ch48]

146. What does `npm run test` execute? `node --test tests/*.test.js`. [Ch43]

147. Difference between `lint` and `lint:lab`? `lint:lab` targets `src/lab` and the home page for a fast inner loop; `lint` is the full gate. [Ch47]

148. Name the four CI steps. `npm ci`, `npm test`, `npm run lint`, `npm run build`. [Ch47]

149. What triggers CI? Push and pull request, on `ubuntu-latest`. [Ch47]

150. What Node version does CI use? 22, with npm dependency caching. [Ch47]

151. Is there a deploy job? No — no deploy, no matrix, no artifacts. [Ch47]

152. What does CI not prove? Anything about rendering, timing, layout or browser behaviour — there's no browser automation. [Ch47]

153. Measured build output? 1,736 modules; main chunk 251.78 kB (80.90 kB gzip); SortingLab 29.85 kB; engine 18.65 kB. [Ch48]

154. How many routes are lazy? Fourteen of seventeen render a lazily-imported component — twelve distinct `React.lazy` components, with `SortingLab` used by three routes. [Ch49]

155. What's the deployment rewrite for? Serving `index.html` for unknown paths so client routes survive a refresh. [Ch49]

156. Which Tailwind utilities does the app actually use? `flex` and `min-h-screen` in one component; everything else is custom CSS in `lab.css`. [Ch50]

157. What's in `src/index.css`? Twelve lines — just the three `@tailwind` directives. [Ch50]

158. Which lint rule is disabled, and where? `react-hooks/set-state-in-effect`, for `src/modules/**` only, because those pages are the legacy execution model. [Ch47]

159. Why does CI run the build? To exercise the full module graph, which no test imports. [Ch47]

160. Which script would you run before opening a PR? `npm test && npm run lint && npm run build` — the CI order. [Ch47]

61.10 JavaScript and React fundamentals (161–178)

161. `??` versus `||`? `??` only falls back on `null` and `undefined`; `||` also falls back on `0`, `""` and `false`. [Ch51]

162. Where does that distinction matter here? Reading a stored cursor, where `0` is a valid value — `??` keeps it, `||` would replace it. [Ch51]

163. `useMemo` versus `useCallback`? `useMemo` caches a value; `useCallback` caches a function — which is just `useMemo` returning a function. Used once in this repo: `resetBoard` in `Nqueens.jsx`, so an effect can depend on a stable function identity. [Ch18]

164. What does a lazy state initialiser look like? `useState(() => read())` — the function runs once, on mount. [Ch41]

165. Why does `key={item.id}` matter? It gives React element identity, so a swap looks like a move rather than a value change. [Ch37]

166. What's the duplicate-key problem? Two children with the same key; React's reconciliation with duplicates is unspecified. [Ch34]

167. What does `Array.prototype.sort` do without a comparator? Sorts by string conversion, which puts 100 before 20. [Ch51]

168. Is `Array.prototype.sort` stable? Yes, guaranteed since ES2019 — and it mutates the array. [Ch51]

169. What does `Number.isSafeInteger` guarantee? That the value is an integer within $\pm 2^{53}-1$, so it round-trips exactly. [Ch51]

170. What does `Object.hasOwn` do? Tests for an own property without consulting the prototype chain. [Ch51]

171. Why `structuredClone`-safe data matter here? A frame must be a plain, deep-independent value so it can be compared, stored and rendered without shared references. [Ch11]

172. What does spread on an object do? Shallow-copies own enumerable properties — which is enough for `{...stats}`, whose values are numbers. [Ch11]

173. Does shallow copying suffice for an array of objects? No — the objects are still shared, so the array needs a per-element clone. [Ch11]

174. What is `Math.imul` for? 32-bit integer multiplication with wraparound, used in the seeded LCG. [Ch10]

175. Why `>>> 0`? To coerce to an unsigned 32-bit integer, keeping the generator's state in range. [Ch10]

176. What's a stale closure? A function that captured a value from an earlier render and still reads that old value. [Ch26]

177. What's the standard fix? Add the captured value to the effect's dependency array, or restructure so the value isn't read. [Ch26]

178. What does `Suspense` do here? Shows a shell-wrapped loader while a lazy route's chunk downloads. [Ch49]

61.11 Performance and security (179–200)

179. What's the worst case for memory in this app? 1,154 frames at about 981 KB, from bubble sort on the reverse-sorted 24-element input. [Ch52]

180. What's the cost of one commit? One or two generator runs, depending on whether comparison is on — and two runs even without it, because `load()` needs the length. [Ch52]

181. What's the cost of one playback frame? One reducer call, one render, one `map` over at most 24 elements. [Ch52]

182. What breaks first if you raise the cap? Generation time and memory, together — the frame count is quadratic and every frame clones. [Ch52]

183. Name an existing optimisation. Memoising the traces so the algorithm doesn't re-run per frame. [Ch52]

184. Name an existing inefficiency. The double generation per commit. [Ch52]

185. What's the `React.memo` trap here? Wrapping `ArrayView` gains nothing, because its only prop changes every frame. [Ch52]

186. What's the proposed fix for memory? Delta frames with periodic keyframes. [Ch52]

187. Where is that proposal recorded? In this handbook and the README's improvement list — not in the code. [Ch52]

188. How many trust boundaries does the app have? Three: the URL query string, the storage read path and the imported file. [Ch53]

189. Which one is a security boundary? The imported file — the other two are robustness boundaries. [Ch53]

190. What stops a crafted algorithm key? The `Object.hasOwn` registry check. [Ch53]

191. Is there any `eval` or `innerHTML`? No, and no `dangerouslySetInnerHTML` either. [Ch53]

192. How is user text rendered? Through JSX interpolation, which escapes by default. [Ch53]

193. What proof is there that nothing leaves the browser? The only `fetch` in the repository is in a component with zero importers, so it never runs. [Ch53]

194. What would you add before storing anything sensitive? Don't — this is the wrong place. Sensitive data belongs behind a server with authentication. [Ch53]

195. What is the app's threat model, honestly? A user experiment could have personal data pasted in; everything else is a local, single-user tool with no accounts and no server. [Ch53]

196. Is the app accessible? Partially: real buttons and inputs, a reduced-motion query, but no `aria-label`s on the bars and no live-region announcements. [Ch60]

197. What's the biggest accessibility improvement available? Announcing the state change, either via the message or a live region, and labelling the bars. [Ch60]

198. What's the honest one-line description of this project? A client-side algorithm laboratory that records complete execution traces and provides a controllable, comparable, savable way to inspect them. [Ch03]

199. And the one-line non-claim? It is not a benchmark, not a backend application, and not a proof of correctness — it is an instrument for looking at algorithms. [Ch03]

200. What's the single strongest answer in this whole handbook? The merge-sort duplicate-id bug: found by measuring, explained with numbers, fixed in two lines, and it exposes a testing weakness rather than hiding one. [Ch59]

> **MUST KNOW**
> **HOW TO DRILL THIS CHAPTER**
> Cover the answers. Go through 24 questions in four minutes. Mark every question you cannot answer in two sentences and read only those chapters. Repeat daily until the marked set is empty — that is what "ready" actually means.

CHECKLIST

RED-LINE DRILLS FOR PART XXXV

- Full pass in under 45 minutes with no more than 10 misses.
- Second pass in under 30 minutes with no more than 3 misses.
- Answer the twelve questions marked in brackets as the hard chapters — 33, 39, 44, 46, 52 — without hesitation.
- Do one pass standing up, speaking, with a timer.

PART XXXVI

One hundred flashcards

Physical flashcards for spaced repetition — front, back, and the chapter to revisit when the back doesn't come.

CHAPTER 62

62 · 100 flashcards

Print this table, cut it into cards, and put the odd-numbered ones in one pile and the even-numbered in another. Review odd on day one, even on day two, both on day five. Anything you miss goes back a pile.

62.1 Engine, frames and algorithms (1–30)

#	Front	Back
1	The two mandatory clones in a frame	`cloneArray()` for the array, `{...stats}` for the counters
2	Why clone the array per frame	Otherwise every frame references one live array and they all show the final state
3	Frame field that identifies an element	`id`, shaped `block-<original index>`
4	Three metadata fields unique to merge/quick/heap	`pivotIndex`, `subArrayRange`, `yOffset`
5	`minimumIndex` is emitted by which generators	Bubble, selection, insertion
6	`isLifting` belongs to	Insertion sort
7	Why `codeLine` exists	Machine-readable step tag for explanation and test grouping
8	Worst trace in the app	1,154 frames — bubble sort, reverse-sorted 24 elements, ~981 KB
9	Trace length: default 8 values, bubble	118
10	Trace length: default 8 values, selection	65
11	Trace length: `[3,1,2]`, merge	30
12	Minimum frames for an empty input	2 (bubble/selection/insertion), 4 (merge/quick/heap)
13	Which sorts are stable	Bubble, insertion, merge
14	Which sorts are truly in place	Selection, heap (and bubble/insertion in practice)
15	Selection sort's comparison count	Exactly n(n−1)/2, input-independent
16	Bubble sort's early exit	Absent — 276 comparisons even on a sorted 24-element array
17	Insertion sort's `break` frame cost	One extra comparison before the break
18	Merge sort's exchange counter really counts	Buffer writes, not swaps — 24 for eight elements
19	Merge sort's `sortedIndices`	Stays empty for the whole trace
20	Merge's buffer `yOffset`	Set to 80 in frames; read by no live component (only by the unimported `EfficientSort.jsx`)
21	Quick sort's pivot	The last element — sorted input is the worst case
22	Quick sort's `sortedIndices` quirk	Can be out of order, e.g. `[1, 0]`
23	Heap sort's `heapify`	Recursive, with two unconditional comparison increments per call
24	Heap sort's declared auxiliary space	O(log n), footnoted as call-stack space
25	Merge sort's duplicate-id frames, default input	23 of 107
26	Merge sort's duplicate-id frames, reverse-24	141 of 375
27	Why the duplicate ids occur	Shallow clones keep their `id` during the buffer write-back
28	The two-line fix for the duplicate-key warning	`key={item.id + ':' + i}`, safe because frames are full snapshots

#	Front	Back
29	What a rendered frame is a function of	One integer — the cursor
30	Which generator line range holds the LCG	`makeDataset`, engine.js 106–120

62.2 Reducer and transport (31–52)

#	Front	Back
31	Reducer state shape	`{cursor, length, playing}`
32	The four action types	`load`, `toggle`, `tick`, `seek`
33	`load` produces	`{cursor: 0, length, playing: false}`
34	`toggle` at the last frame	Resets the cursor to 0 — that's Replay
35	`tick` completion expression	`state.cursor + 1 < state.length - 1`
36	`seek` and `playing`	Every seek pauses
37	Unknown action type	Returns the same state object — React bails out
38	`seek(2.5)` stores	`2.5`, and `frames[2.5]` is undefined
39	`seek(NaN)` stores	`NaN` — `Math.min` propagates it
40	`seek(-5)` at length 20	`0`
41	`seek(99)` at length 20	`19`
42	`length = 1`, `toggle` then `tick`	`{0,1,true}` then `{0,1,false}`
43	`tick` while paused	Sets `playing` to true — UI-unreachable weakness
44	Where `length` lives	Reducer state — duplicated from `frames.length`
45	The render-time clamp	`Math.min(playback.cursor, frames.length - 1)`
46	Number of reducer consumers	Exactly one: `SortingLab`
47	Why the reducer is testable	It's an exported pure function
48	Missing action that would remove a stale-closure risk	A relative `step` action with a delta
49	What schedules the animation	A `useEffect` calling `setTimeout(speed)`
50	The timer effect's dependencies	`playing`, `cursor`, `speed`
51	Why not `setInterval`	It captures its delay and bursts after a stall
52	What happens without the effect cleanup	Duplicate timers and multi-frame jumps

62.3 State, input and comparison (53–72)

#	Front	Back
53	`text` versus `input`	Editable draft string versus validated committed array
54	What a failed commit changes	Only the error message — the last good trace survives
55	`parseInput` accepts	2–24 whole numbers, `^-?\d+$`, ≤ 999 in absolute value
56	Value that coercion would wrongly accept	`Number("")` is 0; `Number("1e3")` is 1000; `Number("0x10")` is 16
57	Four speed delays	600, 300, 160, 50 ms
58	Which speed label is rounded	2× — 300/160 is 1.875×
59	Keyboard shortcuts	Space toggles, arrows seek
60	Keyboard effect's dependency	`[cursor]` — re-registers the listener every frame
61	`compareCursor` formula	`round(cursor / (frames.length-1) * (otherFrames.length-1))`

#	Front	Back
62	Why percentage alignment	Traces differ in length — 65 versus 118 on the default input
63	Does the alignment imply equal work	No — the percentages are positions in different recordings
64	Comparable counter across algorithms	Comparisons only
65	What "passes / calls" means	Unit-dependent: pass, recursion node, or extraction
66	Four on-screen comparison disclosures	Same input; percentage alignment; comparisons only; no timing
67	Default-input comparison counts	Bubble 28, selection 28, insertion 22, merge 16, quick 16, heap 44
68	Where the counting caveats are stated	The footnote at the foot of the lab page
69	The bar height formula	`25 + (|val| / max) * 75`
70	What the 25% floor distorts	Zero renders a quarter-height bar; the scale is affine, not proportional
71	Why the same-input guarantee holds	Both memos take the identical `input` reference, and generators don't mutate
72	The meter's `Math.max(2, …)`	A visibility floor so a non-zero count draws a visible bar

62.4 Persistence, import, build and CI (73–90)

#	Front	Back
73	Storage key	`algoviz.experiments.v1`
74	Record shape	`{version: 1, algorithm, input, cursor, createdAt}`
75	Records kept	Six most recent
76	Four storage-read defences	`|| "[]"`, `try/catch`, `Array.isArray`, algorithm-key filter
77	Why the algorithm filter exists	Saved data outlives code — a deleted algorithm must not crash the page
78	Where saved data goes through validation again	Clicking a record re-runs `parseInput`
79	Is `localStorage` secure	No — same-origin readable, unencrypted
80	Export payload fields	Four: version, algorithm, input, cursor
81	Which field export omits	`createdAt`
82	Import size limit	16,384 bytes
83	Import gate that prevents prototype access	`Object.hasOwn(algorithms, data.algorithm)`
84	The import gap	Nine behaviour cases, zero automated
85	The fix for that gap	Extract the gates into a pure function and test it directly
86	Vite alias	`"vite": "npm:rolldown-vite@7.2.5"` with an `overrides` entry
87	Node floor	`>= 22.12.0`, matching `.nvmrc`
88	The four CI steps	`npm ci`, `npm test`, `npm run lint`, `npm run build`
89	What CI cannot prove	Rendering, timing, layout, browser behaviour — no browser automation
90	Route counts	17 total; 14 render a lazy component (12 distinct `React.lazy` components)

62.5 Testing (91–100)

#	Front	Back
91	Test runner and assertion library	`node:test` and `node:assert/strict` — no test dependency
92	150 =	6 algorithms × 25 inputs
93	The five hand-picked inputs	Empty, `[1]`, `[2,2,1]`, `[-4,0,3,-1]`, `[5,4,3,2,1]`
94	The four assertions per sorting case	Sorted final frame; input unmutated; deep-equal re-run; unique final ids
95	Why trace reproducibility is a product requirement	A saved experiment stores a frame index, meaningless if frames reorder
96	768 =	64 three-element arrays over values 1–4 × 12 targets
97	Why the oracle is inline	So it shares no code with the implementation — no correlated failure
98	The oracle's shared assumption	`sum >= target`, the problem's definition — so a misread spec can't be caught
99	The sliding window's precondition	Positive values and a positive target — enforced in `generateWindow`, not `windowTrace`
100	The single highest-value test change	Move the id-uniqueness assertion from the final frame into the per-frame loop

MUST KNOW

THE THREE-CARD RULE

If you remember only three cards, remember **25** (the measured duplicate-id frames), **62** (the alignment formula) and **100** (the two-line test fix). Those three are the ones that turn a rehearsed candidate into a credible engineer.

CHECKLIST

RED-LINE DRILLS FOR PART XXXVI

- ■ One full pass in under 25 minutes.
- ■ Second pass with no more than 5 misses.
- ■ Write cards 61, 62 and 100 from memory on paper.
- ■ Explain card 27, 28 and 100 as one connected story.

PART XXXVII · 1–5

Ten mock interviews, first half

Five interviewer personalities, five full question sets, and the answers placed after the questions — never immediately.

CHAPTER 63

63 · Mock interviews 1–5

How to use this chapter. Do not read a question and then its answer. Read the whole question list for one interview, answer all of them out loud (record yourself), and only then read the answer block. The delay is the entire point: real interviews don't give you the answer after each question.

Interview 1 · Dr. Arundhati Rao — the algorithms professor

MOCK INTERVIEW
INTERVIEWER 1 · DR. ARUNDHATI RAO · PROFESSOR OF ALGORITHMS

Style. Slow, precise, unimpressed by frameworks. She will interrupt a hand-wave with "define that". She cares about invariants, preconditions, and whether you know the difference between a proof and a test.

What she is testing. Whether you can reason about correctness rather than describe implementations.

Ground rules. Every claim must be stated in terms of inputs, invariants and outputs. Nothing is "just faster".

Questions

1. Define the problem your sorting engine solves, precisely.
2. State the loop invariant of your bubble sort.
3. Your bubble sort has no early exit. What is the resulting complexity, and is that a defect?
4. Selection sort's comparison count — prove the count is input-independent.
5. Prove insertion sort's worst case.
6. Which of your six algorithms are stable, and give the exact comparison that decides stability in each.
7. Merge sort: what is its recurrence, and what is the space cost of your implementation specifically — not the textbook one?
8. Quicksort: when exactly is your implementation quadratic? Show the input.
9. Is your partition correct? Convince me, not by testing.
10. Heap sort: is the build phase O(n) or O(n log n) in your implementation? Justify.
11. Your sliding window has a precondition. What is it, and what is the mathematical reason?
12. You test 768 cases of length three. Is that a proof of correctness?
13. What is an oracle, and is your brute-force reference a valid one?
14. Define "trace reproducibility" as a property, not a feature.
15. What would it take to prove your sorting correct rather than test it?

Model answers

1. The input is an array of 2–24 safe integers; the output is a complete, ordered sequence of snapshots such that the first frame's values equal the input, the last frame's values are the input sorted non-decreasing, and every frame is an independent, deeply-equal-on-rerun value. The sort itself is the secondary output — the primary deliverable is the recording.

2. Before iteration i of the outer loop, the last i elements are final and in their sorted positions, and every element before them is ≤ every element after them. Note that this invariant is not enforced in the code, because the early-exit flag is absent — so the shrinking inner bound encodes the

invariant but the algorithm doesn't verify it.

3. Θ(n²) in every case, since the inner loop runs the full n − i − 1 regardless. It is a defect relative to the best-known bubble sort and a deliberate property of a teaching visualiser: on a sorted 24-element input it still costs 276 comparisons, and that is precisely the quadratic behaviour the visualiser exists to show. I'd state it as a design choice with a cost, not as an oversight.

4. The inner loop executes for j from i+1 to n−1, exactly n − 1 − i iterations, independent of the values. Summing over i from 0 to n−2 gives Σ(n−1−i) = n(n−1)/2, which contains no reference to the input at all. Measured: 28 for n = 8, 45 for n = 10.

5. The worst case is a reverse-sorted input: for each i, the inner loop shifts i elements, giving Σi = n(n−1)/2 comparisons and the same number of shifts. The best case is a sorted input, where the first comparison fails and the loop breaks: n − 1 comparisons. So insertion sort is adaptive where my bubble sort isn't — a visible asymmetry in the traces.

6. Bubble and insertion are stable because their comparison is strict (>), so an element never moves past an equal one. Merge is stable because the merge comparison is <=, taking from the left run on a tie. Selection is unstable because a long-distance swap can move an element past an equal one. Quick is unstable for two reasons: <= sends equals left, and the pivot swap is long-distance. Heap is unstable because the swap of root and last element reorders equal keys arbitrarily.

7. T(n) = 2T(n/2) + Θ(n), giving Θ(n log n) by the master theorem. My implementation is in-place with a buffer rather than allocating per level, so its auxiliary cost is one O(n) buffer reused across the recursion plus O(log n) stack. That is the same asymptotic O(n) but with a much better constant than allocating a new array per merge.

8. Any input where every partition chooses the maximum remaining element as the pivot — which is exactly what a **last-element pivot on already-sorted input** does. Each partition places the pivot at the end of its range, so the recursion has one subproblem of size n−1: T(n) = T(n−1) + Θ(n) = Θ(n²). The app ships that behaviour and the user can trigger it with the "sorted" preset.

9. The invariant is that at the start of each iteration, elements in `[low, i]` are ≤ the pivot value and elements in `(i, j)` are > it, where both intervals grow monotonically. Each iteration either does nothing, preserving the invariant because `arr[j] > pivot`, or swaps `arr[i+1]` with `arr[j]`, restoring it by moving a small element into the small region and a large element out. Termination: `j` goes from `low` to `high − 1`. At exit, `i + 1` is the boundary, and swapping the pivot into it places every small element to its left and every large one to its right — which is the postcondition.

10. O(n) for the build. My `heapify` is recursive, so the sift cost for a node at depth d is bounded by its height h. The sum over all nodes of their heights is at most 2n, because the number of nodes at height h is at most ⌈n/2^(h+1)⌉ and Σ h·n/2^h converges. Summing the per-call work gives O(n) for the build and O(n log n) for the extraction phase.

11. Positivity: all values must be > 0 and the target > 0. The reason is monotonicity: with positive values, extending the right pointer strictly increases the sum and advancing the left pointer strictly decreases it, so the two-pointer walk is monotone and a `while` loop that exits has proven no shorter window ends at the current right pointer. With a negative value both directions lose that property and the walk can terminate early — measured in this codebase: `[-1,3,2]` with target 3 returns 2 where the true answer is 1.

12. No. It is an exhaustive check of a bounded domain: 64 arrays of length exactly three over values 1–4, crossed with 12 targets, compared against an independent oracle. It establishes agreement inside that box. It says nothing about length four, about larger value ranges, or about negative values — where the algorithm is known to be wrong.

13. An oracle is an independent mechanism that produces the correct answer for a case, so agreement between implementation and oracle is evidence of correctness. Mine is a brute-force triple-nested enumeration written inline in the test file, so it shares no code with the two-pointer implementation. It's a valid oracle for the specification as I stated it — it uses the same `sum >=`

`target` predicate, which means it cannot catch a misreading of the problem statement, only a bug in the mechanism.

14. For a pure function `f`, reproducibility means: for any input `x`, `f(x)` deep-equals `f(x)` on every invocation and across processes. In this codebase it also means across algorithms and dataset generation, because the seeded LCG must produce identical arrays from identical seeds — and it is a product property, not just a test property, because saved experiments store a frame index.

15. Either a machine-checked proof — a refinement proof in a proof assistant connecting the implementation to a sortedness specification — or a proof-carrying approach: instrument the algorithm to emit a certificate (a permutation and the comparison log) that a small verified checker can validate. Practically: the certificate approach is the engineering answer, because the checker is small enough to review, and it also gives you the trace you need for the visualiser.

Interview 2 · Kevin Osei — the senior frontend engineer

> **MOCK INTERVIEW**
> **INTERVIEWER 2 · KEVIN OSEI · SENIOR FRONTEND ENGINEER**
>
> **Style.** Fast, concrete, React-fluent. He asks "why" twice per answer and will ask you to write code on a shared editor. He dislikes abstractions that don't pay for themselves.
>
> **What he is testing.** Whether you actually understand React's execution model or have memorised patterns.
>
> **Ground rules.** Cite the effect's dependency array, the render's inputs, and the reconciliation behaviour. No hand-waving about "performance".

Questions

1. Walk me through one frame of playback, from the timer firing to pixels.
2. Why is `cursor` in the timer effect's dependency array? What breaks without it?
3. Where do the frames live — state, memo, ref? Why?
4. Which component reads the array, and what is its `key`?
5. What is the key's bug, and what are the two fixes?
6. Why `useReducer` here and not `useState` with three setters?
7. Show me how you'd add a "step backward 5" control.
8. If I put `React.memo` on `ArrayView`, what happens?
9. Your keyboard listener is registered on `window`. What are the three bugs this invites?
10. What happens if the user presses Play, then immediately changes the algorithm?
11. Is there any state that could be derived instead of stored? Name them all.
12. How would you test the timer without a browser?
13. What's the accessibility problem with a bar chart of divs?
14. You have no error boundary. Where would you add one and why there?
15. What's the single biggest structural refactor you'd do to `SortingLab.jsx`?

Model answers

1. `setTimeout(speed)` fires → the effect body dispatches `{type:'tick'}` → the reducer returns a new state with `cursor + 1`, or clamped at the end with `playing: false` → React re-renders `SortingLab` → the derived `cursor` clamps the reducer value against `frames.length − 1` → `frame = frames[cursor]` → `ArrayView` maps over `frame.array`, computing a class string and an inline height per element → React's reconciler diffs the ~24 elements, changing class attributes on at most two of them and the height on all of them → the browser recalculates styles and paints. Then the effect re-runs, because `cursor` changed, and schedules the next timeout — unless `playing` is

false.

2. It's the re-scheduling trigger. The effect's purpose is "schedule one timeout, and when that timeout fires, re-run me so I can schedule the next one". `tick` changes the cursor, and a changed dependency is what causes React to re-run the effect. Remove it and the sequence is: one timeout scheduled, one tick, one frame advance — and then the effect never re-runs, so playback advances exactly one frame and stops. It's the clearest demonstration of why the dependency array isn't bookkeeping.

3. A memo — `useMemo` on `[input, algo]`. Not state, because the frame array is a pure function of the input and the algorithm key: storing it would require an effect to keep it in sync, and every sync point is a chance to disagree. Not a ref, because it participates in the render and must invalidate the render when it changes. The memo gives invalidation from the dependency array and recomputation once per input change, with no synchronisation code at all.

4. `ArrayView`, a local function component in `SortingLab.jsx`, rendering one column per array element with `key={item.id}` — the element's identity, `block-<original index>`, which travels with the element through swaps.

5. In merge sort's buffer write-back the clone keeps its id, so the same id appears twice in frames near the merge — 23 of 107 on the default input, 141 of 375 on the reverse-24 input. React logs a duplicate-key warning and reconciliation with duplicates is unspecified. Fix one, minimal: `key={item.id + ':' + i}`, which is safe because every frame is a complete snapshot, so a positional key can't carry state across frames — the cost is that bars become position-identified, which would matter if I added movement animation. Fix two: re-id the clone at write time, preserving identity semantics at the cost of changing what an id means for merge sort.

6. Because the three fields have cross-field rules. `tick` must set the cursor and clear `playing` when it lands on the last frame; `toggle` must reset the cursor to 0 when it's at the end; `seek` must clamp and pause. With three `useState`s those rules become three interdependent setters and each transition is a set of calls that can partially apply. With a reducer, each transition is one line, it's a pure exported function, and I can test every edge case in Node without rendering. Also the reducer's identity-preserving `default` gives React a bail-out for free.

7. No new reducer case is needed: `dispatch({type:'seek', cursor: cursor - 5})` — the reducer's clamp handles the boundaries. I'd add a button with `disabled={cursor === 0}` and reuse the existing styling. That's the payoff of expressing navigation as `seek` rather than as `step`, `back`, `forward` and `restart` actions.

8. Nothing happens, functionally — and nothing is gained. Its only prop is the current frame object, which is a different object on every cursor change, so the shallow prop comparison never short-circuits and you pay one extra comparison per render. `React.memo` pays off when props are stable across parent renders; here the prop is the thing that changed. The version that would help is memoising individual bar rows keyed by element id, because only two rows change class per frame — but that's a measurable optimization to make after profiling, not before.

9. (a) Stale closure — if `cursor` isn't in the dependency array, the handler keeps seeking the cursor value it captured at registration, so the arrows appear to work once and then do nothing. (b) Accumulation — if the effect returns no cleanup, every re-registration adds another live listener, so one press advances 2, then 3, then 4 frames. (c) Hijacking — a global `keydown` listener sees keystrokes aimed at form controls, so Space must be ignored when the target is an input, and modifier combinations must be ignored so `Ctrl+←` and `Cmd+←` still belong to the browser.

10. The input change commits a new array (a new reference), so the frames memo recomputes; `load()` dispatches `load`, which resets the cursor to 0 and sets `playing` false. So the animation stops and starts at the beginning of the new trace. That's deliberate: the alternative — continuing to play into a different recording — would make the frame index meaningless. And because `setInput` and `dispatch` are batched in the same event, there's no intermediate render where the cursor points past the end of the new trace.

11. Three. `meta` is `algorithms[algo]`, and `last` is `frames.at(-1).currentStats`. The interesting one is the render-time cursor: `playback.cursor` is genuine state, but it's clamped against `frames.length − 1` on every render, which is why the displayed position and the reducer's position can differ for one frame. Everything else in the component is either genuine input state or a memo.

12. Extract the timer into a small custom hook — `usePlaybackTimer(playing, cursor, speed, dispatch)` — so the wiring exists as a named, testable unit. Then with fake timers and a DOM you assert four things: play advances one frame per interval; pause leaves zero pending timers, not one; unmount cancels the pending timer; changing speed while playing reschedules with the new delay. Today none of that is tested, because the suite is `node:test` over pure modules and no component is ever rendered.

13. A bar chart built from divs conveys everything visually and nothing programmatically. Specifically: the bars have no `aria-label`, so a screen reader announces a list of unlabelled boxes; there's no live region, so the fact that the visualisation changed is never announced; the range input has no label; and the keyboard shortcuts are undiscoverable. The parts that are right by construction: every control is a real `<button>` or `<input>`, focus order follows the DOM, and the active bars are distinguished by more than colour — there's a glyph as well as a background change, which covers the common colour-only failure.

14. Inside each lazy route, wrapping the lazy component. That placement means a module that throws shows an error in the content area while the shell, the header and the navigation survive, and the error is scoped to the thing that failed. If I put one boundary at the root I'd get a full-page error for a bug in one panel, which is strictly worse than a partial failure.

15. Move the persistence out. Between the `localStorage` read, the filter, the save path, the export helper and the import gates, there's a coherent module — call it `experiments.js` — that owns the schema, the version constant and all the validation, with no React in it. That would cut the component substantially, make the nine import cases unit-testable in the existing runner, and give the schema one home instead of two.

Interview 3 · Marisol Vega — the staff engineer

> **MOCK INTERVIEW**
> **INTERVIEWER 3 · MARISOL VEGA · STAFF ENGINEER**
>
> **Style.** Conversational and strategic. She asks about boundaries, ownership and the next six months. She is less interested in syntax than in whether you know what you own and what you'd refuse to own.
>
> **What she is testing.** Judgement, scope control, and whether you can talk about a system over time rather than at a moment.
>
> **Ground rules.** Every answer should name what's in scope, what's out, and what it costs.

Questions

1. Draw the system boundary for me in one sentence.
2. What is this application not?
3. Name the three external dependencies of the app at runtime.
4. If you had six more months, what are the first three things you'd build, in order?
5. What would you explicitly refuse to build, and why?
6. Where does the app's data live, and what is the migration story for that data?
7. Your frame model doesn't scale. At what n does it fail, and what's the structural fix?
8. If a teammate joined tomorrow, what would you have them do first, and what would you not let them touch?

9. What's the most expensive thing in the codebase to change, and why?
10. What single decision would you reverse if you could?
11. How would you introduce a backend, if you had to? What changes first?
12. Where is the project's biggest maintenance risk?
13. What would make you delete this project and start over?
14. How do you decide when a feature is done?
15. What does "production-ready" mean for something with no users?

Model answers

1. A single static client-side application: a pure engine (parsers, generators, a reducer) plus a React presentation layer, with no server, whose only persistence is the browser's own `localStorage`, and whose only compatibility contract is the JSON file format at version 1.

2. It is not a benchmark — there's no timing anywhere. It is not a backend application — no server, no database, no accounts, no network calls in the shipped code. It is not a proof of correctness — the tests establish behaviour over bounded inputs. And it is not a library — the generators are not published, versioned or documented as an API for anyone else.

3. React (with `react-dom`), `react-router-dom`, and the browser platform — `localStorage`, `File`, `Blob`, `setTimeout`. `lucide-react` is a fourth if you count an icon set, and the build toolchain is a build-time dependency rather than a runtime one.

4. First, delta frames with keyframes, because the frame model is the constraint that caps input size and therefore the tool's usefulness — that's an architectural change, so it goes first. Second, component tests for the timer and the playback wiring, because that's the largest untested behaviour a user would actually notice. Third, the visualisation of merge sort's buffer, because the generator already emits `yOffset` and `buffer` frames and the renderer ignores them, so it's the cheapest user-visible improvement in the project.

5. A backend. As soon as there's an account model there's authentication, session management, authorisation, a data-retention policy, a privacy surface and an operational burden — for an app whose entire value proposition is that your experiments are yours, offline, on your machine. I'd refuse multi-user collaboration for the same reason, and I'd refuse a "shareable link" feature unless the state encoded in the URL was designed for it from the start.

6. A single `localStorage` key, `algoviz.experiments.v1`, holding an array of at most six records with an explicit `version: 1` field. The migration story today is: unknown versions are rejected on import and unknown algorithms are filtered on read. That's not a migration story, it's a defensive read — the real one requires a per-version upgrade function and a decision about whether to rewrite records in place or read them lazily. The version field is the hook I left myself.

7. It fails when frames × n becomes large — at n = 200 with quadratic instrumentation the frame count is roughly 80,000 and each frame clones the whole array, so hundreds of megabytes and a frozen tab. The structural fix is to stop recording state and start recording transitions: store the operations and a keyframe every `k` frames, then reconstruct any frame by replaying from the nearest keyframe. Memory becomes O(operations + keyframes × n) instead of O(frames × n), and the trade is reconstruction cost on seek.

8. I'd have them start by reading `engine.js` and writing new test assertions against it — specifically the per-frame id-uniqueness assertion and a property test for the reducer. It's real work with an immediate payoff, it forces them to understand the frame contract, and the test runner has zero setup. What I wouldn't let them touch in the first week is the reducer's boundary semantics or `SortingLab.jsx`, because those are the two places where a plausible-looking change breaks an invariant that isn't asserted anywhere.

9. The reducer's boundary semantics, because two separate screens depend on them and neither has a test. Changing `seek` to not pause, or `toggle` to not reset at the end, looks like a one-line

improvement and silently changes the behaviour of four controls. After that, the frame schema — adding a field is easy, but every generator has to emit it consistently and the reproducibility assertion compares the whole frame, so a field missing from one generator is a test failure that reads like an unrelated diff.

10. The instrumentation density. I recorded a frame at every interesting point, which produced the deepest possible trace and then forced a 24-element cap to keep memory sane. If I'd decided the frame budget first — how many frames per element, and what the user is meant to notice — the cap could be much higher and the tool would work on inputs people actually have.

11. The record schema. Everything else can stay: the engine is pure, the reducer is pure, the UI doesn't know where data comes from. What has to change is that the storage read and write become asynchronous, which means the saved list gains a loading and an error state, and the "save" button becomes a request. Then the version field becomes a real migration concern, and authorisation appears. So: schema and the read/write interface first, then the privacy and identity questions, and none of the algorithm or playback code touches any of it.

12. Documentation drift, and it's already happened: the README describes the app accurately except for the merge duplicate-key bug, which I found after writing it. The structural mitigation is to keep the things that can drift closest to the code — the ESLint config already explains its own exception in a comment, and the schema constant should live next to the gates that use it. Prose in a separate file will always drift.

13. If the goal changed from "an instrument for understanding algorithms" to "a fast way to sort things", the entire recording layer becomes dead weight — the frames exist to be inspected, not to produce a result. That would be a different product, and I'd start over rather than strip this one, because the value of this codebase is precisely the parts that make it slow.

14. When the behaviour is asserted at the lowest layer that can express it, and the assertion would fail if the feature were reverted. Not when the UI looks right — the merge duplicate-key bug looked right on screen and was wrong in the frames. Practically, for anything touching the engine: a test in the existing runner, and a decision recorded about what the feature deliberately does not do, because a feature with undisclosed limits is a future bug report.

15. For this project it means four things: the build is reproducible from a clean checkout; the gates (tests, lint, build) pass in a clean environment; the failure modes are known and documented rather than surprising; and the data format has a version. It does not mean high availability, monitoring, on-call or a rollback plan — there's nothing to run and no users to page. Saying which half of the phrase applies is the actual answer.

Interview 4 · Tim Brennan — the skeptical hiring manager

> **MOCK INTERVIEW**
> **INTERVIEWER 4 · TIM BRENNAN · ENGINEERING MANAGER**
>
> **Style.** Blunt, resume-first. He opens by reading your bullet out loud and asking you to defend it line by line. He is not hostile, but he is not convinced by enthusiasm either.
>
> **What he is testing.** Ownership, honesty about scope, and whether you can state what you did without inflation.
>
> **Ground rules.** Every claim needs a location, a measurement, or an admission.

Questions

1. Read your bullet. Which part is the weakest claim?
2. You say "validated JSON import". Prove it's validated.
3. What percentage of this did you write?
4. How long did it take, and what took the longest?

5. What did you get wrong the first time?
6. Did anyone review this code?
7. Why React? Justify the choice rather than describing it.
8. Why not Next.js?
9. Why not plain JavaScript modules with no framework?
10. What's the hardest bug you fixed, and how did you find it?
11. What part of this project would you not defend in a code review?
12. Tell me about a decision you reversed.
13. How do you know the app works if you don't have users?
14. What would you do differently if a teammate had to maintain this?
15. Why should I believe any of this over the next candidate who claims the same thing?

Model answers

1. "Verified correctness and reproducibility across 150 algorithm-input cases." The number is exact and the mechanism is real, but "verified correctness" is doing too much work — 150 cases over a bounded input space establish behaviour, not correctness, and there's a known gap in those assertions, which is that id uniqueness is only checked on the final frame. I'd rewrite the bullet as "150 algorithm-input cases asserting sortedness, input preservation, trace reproducibility and final-frame integrity" — a longer bullet and a more accurate one.

2. Four gates in the import path: a 16,384-byte size limit before parsing, `JSON.parse` inside a try/catch, `version === 1`, then `Object.hasOwn(algorithms, data.algorithm)` — deliberately not `in`, because `in` walks the prototype chain and `'constructor' in algorithms` is `true`. Input is checked with `Array.isArray`, and then the values are joined into a string and pushed through the same `parseInput` the form uses, so there's one definition of a valid dataset in the codebase. What it does not do: reject extra fields, or bound nesting depth.

3. I can't prove authorship from this checkout — the history is a single squashed commit with no co-author metadata, so Git gives you nothing. What I'll say is that I can defend every design decision in it and name the ones I'd change, and you can test that claim by asking about any part of it. That's a weaker claim than "I wrote all of it" and it's the one I can actually support.

4. The work split roughly three ways: the engine and the six generators; the playback layer and the lab's UI; and the verification and documentation. By far the longest single item was calibration — deciding what a frame should contain. Every field I added to a frame was a decision about what the UI could explain, and the frame schema was rewritten more than once. The second longest was the documentation, because writing the non-claims honestly took longer than writing the claims.

5. Three things. I built the comparison with index-based alignment first, which broke immediately — selection's trace is 65 frames and bubble's is 118 on the default input, so the secondary panel ran out of frames two-thirds through. I assumed merge sort's duplicate ids were harmless because they never reach the final frame; they're in 23 of 107 frames. And I initially reported the merge exchange counter as swaps; it's a buffer-write count, which is why the footnote exists.

6. No. There's one commit and no review metadata. In practice the review I got came from re-reading the README's own non-claims against the code — which is how I found that the README over-described the "correctness" of the test suite before I softened it.

7. Because the state model is the feature. The UI is a pure function of one integer — the cursor — plus a frame array, and the two things I needed most were fine-grained control over what re-renders when that integer changes, and the ability to test the transport as a plain function. React gives me both: a memoised derivation with an explicit dependency array, and a reducer I can call from Node. A framework that re-runs a render function server-side would add a runtime I don't use.

8. Next.js exists for server rendering, routing conventions, data fetching and image optimisation. This app has no server, no fetching and no SEO requirement — it's a local instrument behind a client route. Next.js would add a build toolchain, a server runtime and a mental model I'd never exercise. The honest answer is that it would be strictly more machinery for zero benefit, and the one thing I'd take from that ecosystem — file-based routing — I don't need for seventeen flat routes.

9. I'd have to write the routing, the component lifecycle, the reconciliation and the memoisation. Those are the parts I want to be correct rather than mine, and the frame/derive/effect model is what makes the playback code as small as it is. The counter-argument is bundle size: React and React DOM dominate a 251 KB build for an app with maybe 600 lines of my own UI logic. That's a real cost and I'd accept it because the alternative is hand-written DOM diffing in the exact place where a diffing bug would look like an algorithm bug.

10. The one I'd call the hardest wasn't a crash — it was the merge-sort mid-frame state. The rendered array was momentarily not a permutation of the input: frame 15 of `[3,1,2]` renders `[1,1,2]`, and the `3` was in `tempArr`, which no frame exposes. There was no error and no failing test; the only way to find it was to compare a frame's values against the input's as multisets and notice they disagreed. That's what led to finding the duplicate ids, because the same clone semantics cause both. I found it by writing a script that imports the generator directly and prints every frame's id list, which is also how I measured the 23-of-107 number.

11. The duplication of `length`. It's in the reducer's state and it's derivable from `frames.length`, so a code path that swaps the trace without dispatching `load` leaves a stale bound — and the only thing preventing a visible bug is a clamp in the render. In review I'd be told, correctly, to remove one of them rather than guard the mismatch. I'd also flag `SortingLab.jsx` at 596 lines as a file that's too big to review comfortably.

12. The comparison alignment, from index-based to proportional. That was forced — the first version visibly broke. The more interesting reversal is the counter display: I originally showed a single combined "operations" number, and had to split it into comparisons, passes/calls and exchanges once I realised the three aren't comparable across algorithms. Adding the footnote was admitting that the panel needed to say less than it appeared to.

13. Three ways. The engine's contract is asserted in Node — sortedness, input preservation, deep-equal reproducibility over 150 cases. The build runs from a clean checkout and CI enforces the gates on every push. And the behaviour I can't verify, I state: rendering, timing, layout and browser behaviour are tested by me opening the app, not by any automation, and I'd say that in an interview before someone else discovered it.

14. I'd add three things: a `known issues` section, because the duplicate-key bug currently lives only in my head and in a chapter of the handbook I wrote; a twenty-minute guide to reviewing the codebase, because the route in — `package.json` → `README` → `main.jsx` → `Home.jsx` → `engine.js` → `SortingLab.jsx` → tests — took me months to see as a path; and I'd split the persistence code out of the lab component so the schema has one home.

15. You shouldn't, on the claims alone — you should ask any of us to open the file. I'll offer three specific, checkable things: the merge duplicate-key bug with its measured frame counts, the two-line test change that would have caught it, and the fact that the reducer accepts a non-integer cursor, which means the invariant I'd state in a design doc is false. Those are the kinds of things you only know about code you've actually worked on.

Interview 5 · Priya Nair — the QA lead

MOCK INTERVIEW
INTERVIEWER 5 · PRIYA NAIR · QA & TEST ENGINEERING LEAD

Style. Methodical and slightly adversarial about evidence. She will ask what a test would have caught, and she treats an untested claim as an unverified one.

What she is testing. Test strategy — not tooling, judgement: what to test, at which layer, and what you accept as unverified.

Ground rules. Name the failure mode, the layer that should catch it, and the cost of the test.

Questions

1. Describe your test pyramid, honestly.
2. You have 14 groups. Which three carry the most weight?
3. What's the difference between a test that's slow and a test that's useless?
4. Your reproducibility test compares whole frames. What's the failure mode it can't catch?
5. How do you test something non-deterministic? Or do you not allow it?
6. What's a flaky test, and would any of yours ever be flaky?
7. You have 768 differential cases. What's the boundary of that coverage, in one sentence?
8. If I deleted your entire test suite, which bug would ship first?
9. What would you add with one more day? With one more week?
10. How do you test a bug fix?
11. Is there any code in the project that's untestable as written?
12. Do you use coverage? Why not?
13. What's the most dangerous thing about a passing test suite?
14. Which of your assertions is weakest, and what's the stronger form?
15. When would you not write a test?

Model answers

1. It's not a pyramid, it's a base and nothing else: a wide base of unit tests over pure modules — generators, the reducer, the parser, the dataset generator, the window function, the counterexample search — and zero component, integration, or end-to-end tests. So the shape is right at the bottom and missing in the middle. The honest description is "a thorough unit base plus a large unverified surface where React and the browser meet".

2. The sorting group, because it's 150 cases asserting four independent properties and it's the only thing standing between a generator change and silently wrong frames. The sliding-window differential group, because it's exhaustive within its domain and compared against an independently written oracle. And the playback group, because the reducer is the one place where boundary semantics are load-bearing for four separate controls.

3. A slow test is a test whose cost is execution time; you fix it with parallelism, caching or fake clocks. A useless test asserts something that cannot fail, or that fails only when something else already failed. The clearest example is asserting that `run(input)` returns an array — the generator can't return anything else. Useless tests are worse than slow tests because they consume the attention that should go to real ones.

4. Two things. First, a consistently wrong trace passes: if a generator is deterministic but wrong — merge's `sortedIndices` stays empty for the entire trace — the deep comparison is stable and passes. Second, it can't catch a specification error: if the frame schema itself is wrong, both runs are equally wrong, and equality holds. Reproducibility is a property about non-determinism, and it's easy to mistake it for a property about correctness.

5. You test the seam, not the randomness. There are two non-deterministic-looking things in this project and both are actually deterministic: the "random" dataset comes from a seeded LCG, and the trace comes from pure functions. So the strategy is to eliminate non-determinism rather than tolerate it — and then test that it was eliminated, which is what a deep-equality rerun assertion does. If genuine randomness existed in a generator, I'd inject the source of randomness and assert against a fixed sequence.

6. A flaky test passes and fails on the same code, so it destroys the signal from a suite. None of mine should ever flake, because there's no timing, no network, no dates and no shared mutable state between tests, and the random data is seeded. If I ever added a test with a timer, that's where flakiness would enter — fake clocks or nothing.

7. It verifies agreement with an independent implementation on all length-three arrays over values 1–4 for targets 1–12, and nothing outside that box.

8. The merge duplicate-key bug would be the first visible one — a React warning and a potentially mis-reconciled column — but the more interesting answer is that the entire timer and playback wiring could break and nothing would fail, because no test renders a component. Deleted tests would ship a broken Play button before they shipped a wrong sort, because the sorts are what the suite covers.

9. With a day: move the id-uniqueness assertion inside the per-frame loop and add a property test asserting the reducer's cursor is always an integer inside `[0, length−1]` — which would fail immediately, since `seek(2.5)` currently stores 2.5. With a week: extract the import gates and the storage read into pure functions in the engine, and write the nine import cases and the five corrupt-storage cases, still in the `node:test` runner with no new dependency.

10. Three steps, in order. First an assertion that fails on the old behaviour — if I can't write that, I don't understand the bug. Then the smallest fix at the layer that owns the invariant. Then the regression test at the lowest layer that expresses it, which for the duplicate-key bug is the generator's frames, not the component — because a component test would pass or fail depending on React's internal warning, which isn't a contract.

11. The import and export paths, as written, because they're component methods tangled with `File`, `Blob` and `URL.createObjectURL`. The logic is pure — gate ordering and schema checks — so the fix is extraction rather than mocking. That's the general rule I'd apply: if something is untestable, it's usually two things, one of which is pure.

12. No, and I'd frame it as a deliberate non-claim rather than a missing metric. Line coverage measures execution, not assertion strength, and this codebase is the perfect counter-example: the merge generators have essentially complete line coverage and a real defect in the frames they emit. If I added a number it would be for a different purpose — a diff-coverage gate, to stop a new module arriving with no tests at all.

13. That it becomes the evidence instead of the question. A green suite is a reason to stop looking, and the merge bug lived inside a green suite for as long as it did because the assertion people would have written — "ids are unique" — already existed, in a weaker form. The discipline is to keep asking what the suite would not catch, which is what a weakness register is for.

14. The id assertion, twice over: it checks only the final frame, and even then it's the least interesting frame, because every trace converges to a unique-id final state. The stronger form is per-frame uniqueness for all frames, which is a two-line change and would have caught the merge bug — plus, if I wanted to go further, a multiset check that every frame's values are a permutation of the input, which would catch any future instrumentation that loses an element.

15. When the behaviour is deliberately, visibly provisional — an experiment in a spike branch — or when the cost of the test exceeds the cost of the failure, which for a throwaway script is normal. What I wouldn't skip is a test for a decision: any place where the code chooses one behaviour over a plausible alternative. Decisions are precisely what regresses silently.

PART XXXVII · 6–10

Ten mock interviews, second half

Five more personalities: platform, security, startup, performance, and the behavioural round.

CHAPTER 64

64 · Mock interviews 6–10

Same rule as the previous chapter: read the whole question list, answer out loud, then read the answer block.

Interview 6 · Dmytro Kovalenko — the platform engineer

MOCK INTERVIEW
INTERVIEWER 6 · DMYTRO KOVALENKO · DEVOPS / PLATFORM ENGINEER

Style. Procedural and literal. He wants the commands, the versions and the failure modes. He will ask what happens on a clean machine and expects an exact sequence.

What he is testing. Whether the build is reproducible and whether you understand what your pipeline actually guarantees.

Ground rules. Name the command, the version and the failure you're guarding against.

Questions

1. Give me the exact sequence from a clean machine to a passing pipeline.
2. Why `npm ci` and not `npm install`?
3. What pins your Node version, and what happens on Node 20?
4. What does your CI do on a pull request from a fork?
5. Could your build pass while the app is broken? Give an example.
6. Is the build reproducible byte for byte? Why not?
7. What's in the production bundle, roughly, and how would you check?
8. What's your rollback plan?
9. There's an npm alias in your manifest. Explain the risk.
10. How would you add a preview deployment? What would it cost?
11. What would you monitor if this had real users, and what would you not bother with?
12. What happens if the CI runner's Node version changes?
13. Where are secrets, and what would you do if you needed one?
14. How long does your pipeline take end to end, and what dominates it?
15. What's the one thing in your pipeline you'd fix first?

Model answers

1. Clone, then `nvm use` (which reads `.nvmrc` and gives Node 22), then `npm ci` — which requires the lockfile and installs exactly it — then `npm test`, `npm run lint`, `npm run build`. That's the same four gates CI runs, in the same order, which is the point: if it passes locally in that order, there are no surprises in the pipeline. The failure mode people hit is skipping `npm ci` and then running `npm run build`, which fails with "vite: not found" because there's no `node_modules`.

2. `npm ci` installs strictly from `package-lock.json` and deletes `node_modules` first, so it cannot silently update a transitive dependency. `npm install` may resolve a newer version within a semver range and rewrite the lockfile, which means the pipeline could install something no developer has ever run. In CI, the lockfile is the environment, so `npm ci` is the only command that reproduces it.

3. `.nvmrc` contains 22 and `package.json` declares `engines.node: ">= 22.12.0"`. On Node 20, `npm ci` prints an `EBADENGINE` warning by default — and would fail only under `engine-strict=true`, which isn't set. So today the floor is advisory. That's a deliberate trade: advisory failures keep the repo usable for someone on an older runtime, at the cost of letting them run it in an unsupported environment.

4. It runs the same four steps, read-only: the default token has no write permission on a fork's PR except to report status, no secrets are exposed, and there's no deploy step to be gated. That's the safe default. The consequence is that a maintainer merging a fork's PR is trusting that CI's four steps are meaningful — which is another argument for the build step being in there.

5. Easily — and it has. The merge duplicate-key bug is exactly that case: every gate green, and a real defect in the frames. More generally, any bug that lives in the seam between React and the browser is invisible to a suite that never renders. Concretely: a broken timer cleanup, a stale closure in the keyboard handler, a wrong `disabled` condition, a layout that collapses at 400 px — all of these can pass tests, lint and build.

6. No, and it isn't worth pretending otherwise. The bundle includes a hash of file content, so identical inputs give identical output of my code — but the dependency tree comes from the registry and the toolchain is Rust-based (Rolldown), so byte-for-byte determinism across machines and platforms isn't guaranteed by anything in this project. What is guaranteed is a reproducible dependency graph, via the lockfile plus `npm ci`.

7. Roughly: React and React DOM dominate a 251.78 kB main chunk (80.90 kB gzip), plus a 29.85 kB SortingLab chunk, an 18.65 kB engine chunk and 72.37 kB of CSS, from 1,736 modules. To check, I'd run the build and read the output — Vite prints the chunk table — and if I wanted more, a bundle visualiser to see which dependency owns the space. The number I actually watch is whether a new dependency silently doubles the main chunk.

8. For the deployed artefact, there isn't one in the repository: the config is a rewrite rule for a static host, and no deploy job exists in CI. [PROPOSED] The practical version is that a static SPA is rolled back by redeploying the previous build, and the reason that's safe is that there's no server state and no schema migration — the one thing that isn't rolled back is the user's `localStorage`, which is exactly why the record has a `version` field and the import path rejects unknown versions.

9. The risk is that `"vite": "npm:rolldown-vite@7.2.5"` means the name `vite` and the actual package differ, so anyone reasoning about "what Vite does" from the package name can be looking at different docs or behaviour. The `overrides.vite` entry is what stops a transitive dependency from pulling a different Vite, which would put two bundlers in the tree. The mitigation is the lockfile plus a README note — and the honest caveat is that this is a non-default, fast-moving bundler, so an upgrade is a deliberate act rather than a `npm update`.

10. A preview deployment needs two things: a host that builds on push (Vercel is already referenced via `vercel.json`) and a per-branch URL. The cost is the build minutes plus the fact that a preview URL is a public URL for an app whose data is local — so nothing sensitive, but it does mean anyone with the link can use the app, and `allowedHosts` in the Vite config becomes relevant for the dev server rather than the preview. I'd add it, because a clickable preview link in a PR is the cheapest possible review improvement.

11. Monitoring for a static SPA with no users: I'd want error reporting for uncaught exceptions, because without an error boundary an exception is a blank page and I'd otherwise never know. I'd want basic availability on the static host, which is the provider's job. What I wouldn't bother with: performance monitoring, because there's no server and the client work is bounded by design; and any product analytics, because the app's promise is that it doesn't phone home.

12. Nothing, in the sense that CI pins Node 22 via `actions/setup-node@v4` with `node-version: 22`, so the runner's default is irrelevant. But 22 is a major version, so a patch or minor release will change the runtime under the pipeline without any change to the repository. That's the accepted norm for GitHub Actions, and the more careful version is pinning the exact minor and bumping deliberately — which is also what a supply-chain-conscious team does with the action versions

themselves, currently `@v4` tags rather than commit SHAs.

13. There are no secrets, which is a property rather than an accident: nothing is deployed, nothing is authenticated, and the app makes no network calls at runtime. If I needed one — a preview-deployment token, say — it goes in repository secrets and is referenced by `${{ secrets.NAME }}` inside the workflow, never in the repository, and I'd scope it to the environment rather than the whole repo. The relevant local rule is that anything in the client bundle is public, so a key that must be secret cannot be a `VITE_` variable.

14. Minutes, and what dominates it is the `npm ci` step — installing React, Tailwind, PostCSS and ESLint from the registry, with dependency caching configured in the workflow. The test suite itself is a few hundred milliseconds; lint and build are seconds. That dominance is why the cache exists and why a lockfile change is the slowest kind of push.

15. The build's position in the sequence. Right now it's last, after lint, so if the build fails you've paid for install, test and lint first. Reordering to `test → build → lint` would surface the most expensive-to-fix failure earlier. It's a two-line change and it's exactly the kind of thing that's only worth doing once you've watched the pipeline run fifty times.

Interview 7 · Sofia Lindqvist — the security engineer

> **MOCK INTERVIEW**
> **INTERVIEWER 7 · SOFIA LINDQVIST · APPLICATION SECURITY ENGINEER**
>
> **Style.** Calm and systematic. She will not accept "it's just a local app" as an answer to a design question, but she will accept it as an answer to a scoping question. She asks for the threat model before the controls.
>
> **What she is testing.** Whether you can reason about trust boundaries instead of listing best practices.
>
> **Ground rules.** Name the asset, the actor, the boundary, and what the control actually prevents.

Questions

1. Give me your threat model in three sentences.
2. What are the assets here, and are any of them sensitive?
3. Enumerate the trust boundaries.
4. You parse JSON from a file. Walk me through what an attacker could put in it.
5. What does `Object.hasOwn` prevent, precisely?
6. What does React protect you from for free, and what does it not?
7. Could a stored XSS happen in this app? What would have to be true?
8. Is `localStorage` a security problem here?
9. What's the riskiest dependency decision you made?
10. If you added a backend, what are the first three things you'd have to get right?
11. Where would you put rate limiting, and why haven't you?
12. What would you do if a user pasted a password into the dataset field and saved it?
13. What's the difference between validation and sanitisation, in your code specifically?
14. How would you handle a security report against this repository?
15. Name one thing in this codebase you'd flag in a security review, even though nothing is exploitable today.

Model answers

1. The system is a single-user, client-side application with no server, no accounts and no network egress; the only assets are the user's own experiments, which live in their browser. The only untrusted inputs are the user's own data — a URL query string, a `localStorage` value that any script on the origin could have written, and an imported JSON file that could have come from anyone. So the realistic risks are self-inflicted: crashing the page with malformed data, or a rendering path that turns data into executable content. I'm explicitly not defending against a hostile server, a hostile co-tenant on the origin, or a compromised dependency — because the first two don't exist and the third is unfixable at this scale.

2. The saved experiments — an algorithm key, an array of integers, a frame index and a timestamp. They're not intrinsically sensitive, but "not sensitive" is an assumption: the input field is free text and a user could paste anything into it, so the asset is really anything the user chose to type into a number field. There's no credential, no token, no PII by design, and nothing is transmitted anywhere.

3. Three. First, the URL query string — `?algorithm` and `?compare` — which is attacker-controllable when a link is crafted. Second, the `localStorage` read path, which is attacker-controllable if anything else ever runs on the origin. Third, the imported file, which is attacker-controllable by definition. Everything else — the frames, the reducer, the rendering — is downstream of those three and takes the validated output.

4. Before parsing: a file large enough to make `JSON.parse` expensive — bounded by the 16 KB limit. After parsing: a payload whose `algorithm` is `'constructor'` or `'__proto__'`, intending to reach the prototype chain — blocked by the own-property check. A payload where `input` is not an array — blocked. An array of non-numbers, or of numbers outside the integer grammar — caught by re-running `parseInput` on the joined values. A `cursor` that isn't a number or is negative or enormous — normalised to 0 or clamped by the reducer. What's left: extra fields pass silently, nesting depth isn't bounded, and a carefully-shaped object could still be large enough to be slow, though the size limit makes that uninteresting.

5. It requires an own property of the object rather than any property reachable through the prototype chain. Concretely, `'constructor' in algorithms` evaluates to `true` because `Object.prototype.constructor` exists, so an `in` check would pass and `algorithms['constructor']` would be the `Object` function — a truthy value that then fails confusingly later, in the component. Own-property checking makes the registry a closed allowlist.

6. React protects against HTML injection through JSX interpolation — text is escaped, so a string containing `<script>` is rendered as text. It does not protect you from `dangerouslySetInnerHTML`, from `href` values with `javascript:` schemes, from writing untrusted data into a `style` attribute's expression position, or from anything outside the render path. In this codebase there's no `dangerouslySetInnerHTML`, no user-controlled `href`, and the only dynamic style assignment is a numeric height percentage computed from a validated integer — so the by-construction argument is real, and it's an argument about absence.

7. It would require a script from a different origin running in the page's context, or a dependency that turns data into a sink. Neither exists: the app loads no third-party scripts, there's no CDN, no analytics, no service worker and no `postMessage` handler. If one were added — an analytics snippet, say — the stored experiments become readable by it and the argument changes entirely. That's the honest version: the app is safe from stored XSS because it has no untrusted script, not because it has a sanitizer.

8. Not for confidentiality. It's not encrypted, it's origin-scoped, and any script on the origin can read it — but the data is integers the user typed, and the app never transmits it. The genuine problems with `localStorage` here are availability and integrity: clearing site data destroys the user's work irreversibly, and any script on the origin could silently rewrite it. Both are handled defensively rather than cryptographically — the read path treats the contents as untrusted — and I'd never describe it as secure storage.

9. Depending on the network at build time at all, with a semver-range dependency tree and no integrity verification in the repository. That's the universality of npm rather than a decision I made, but it is the largest supply-chain surface here, and the standard mitigations — a committed lockfile, `npm ci` in CI, and Dependabot-style review of bumps — are partially present. The stronger version, pinning `actions/*` to commit SHAs rather than `@v4` tags, isn't.

10. Authentication that doesn't put a bearer token anywhere readable by JavaScript; an authorisation check on every read and write, because an object identifier in a URL is not an authorisation; and server-side validation of the exact same schema, because client-side validation is a UX feature and nothing more. And a fourth that people forget: a data-retention policy, because "the user's experiments" becomes "my stored data" the moment it's on a server.

11. At the server, which doesn't exist — so there's nowhere to put it and nothing to protect. In the client, the only unbounded work is the generation path, and that's bounded by the 24-element cap rather than by a rate limit, which is the right way to bound it: a limit would let the work happen and then punish it, whereas a cap makes the work impossible. [PROPOSED] If a worker or a server ever did the generation, the bound would need to move with it.

12. Nothing happens today — it sits in their own browser and is never transmitted. But it's a good example of why "the data isn't sensitive" is an assumption rather than a guarantee: the field accepts a string and the only validation is that it parses as integers, so a user can type prose into it and save it. The mitigation I'd want is a hint in the UI that the dataset field is for numbers and that saves are stored unencrypted in the browser — a disclosure, not a control, because there's no control available in a local app.

13. Validation is rejecting input that doesn't satisfy a schema — `parseInput` rejects anything that isn't 2–24 integers, and the import gates reject anything that isn't version 1 with a known algorithm and an array. Sanitisation would be transforming input to make it safe, and I deliberately don't do it, because silently coercing `"2.5"` to `2` would hide the user's mistake. The one place transformation happens is benign and explicit: joining the imported array into a comma-string so it can go through the same validator as the form.

14. Reproduce it first, in a scratch script that imports the engine directly, so the report becomes a failing assertion rather than a story. Then fix at the layer that owns the property, add the regression test, and write the finding into the known-issues list even if it isn't exploitable — because "we decided this wasn't a risk" is exactly the decision that gets lost. If it were in a dependency rather than my code, the response would be different: assess exposure, then pin or replace, and check whether the lockfile means the vulnerable path is even reachable.

15. The `catch {}` on the storage write, which discards the exception. It's not exploitable, but it's a blind spot by construction: quota errors, disabled-storage errors and serialisation errors all become the same message, so a real failure is indistinguishable from a non-event. The second is the `default` branch of the reducer silently accepting any action, which means a typo in an action name is a no-op rather than an error — in a security context that pattern is how an authorisation check quietly stops being called.

Interview 8 · Jake Torres — the startup CTO

> **MOCK INTERVIEW**
> **INTERVIEWER 8 · JAKE TORRES · CTO OF A TWENTY-PERSON STARTUP**
>
> **Style.** Fast, impatient, product-oriented. He interrupts long answers and asks "so what". He is deciding whether you'd be useful in a week, not whether you're rigorous.
>
> **What he is testing.** Prioritisation, pragmatism, and whether you can ship.

Questions

1. What does this do that a whiteboard doesn't?

2. Would you ship it today? Yes or no.
3. You've got two days before a demo. What do you cut?
4. What's the first thing a user will complain about?
5. If this had to make money, what's the product?
6. What's the smallest change with the biggest perceived improvement?
7. You have no analytics. How do you know anything?
8. Why are there two generations of UI in the repo? Which one wins?
9. You spent time on a 30-page testing chapter. Was that the right call?
10. What would you cut from the app entirely?
11. What's the fastest way to make this feel more finished?
12. What would you build next, in one sprint?
13. Someone says "make it handle 10,000 elements". What do you say?
14. What's your one metric for success?
15. Why would anyone use this instead of watching a YouTube video?

Model answers

1. It's self-paced, reversible and precise. A video plays at one speed and you can't inspect a single comparison interval; here you can pause at the exact frame where a swap happens, step backwards, read the comparison count at that instant, and see two algorithms on the same input side by side. The specific thing it does that a whiteboard can't is preserve the history — a whiteboard gets wiped, whereas a frame is a permanent, inspectable record.

2. Yes, for the audience it has: a browser, no accounts, no server, saved work in the browser, and an honest README. What holds it back from a wider audience isn't a defect, it's the 24-element cap, and that's disclosed on the input itself rather than discovered.

3. Everything except six features and one page: load an input, pick an algorithm, play, step, compare, and show the counts. I'd hide the JSON import/export, the saved-experiment list and the module pages behind a flag or simply out of the nav, and I'd keep the comparison panel because it's the demo moment. Then I'd check the demo path at 800×600, because that's the resolution the projector will actually be.

4. That it doesn't accept their input — 30 values, or decimals, or a spreadsheet paste with a trailing comma. The rejection is correct and the message explains why, but it will still be the first complaint, which is why the message exists. The second complaint is that shuffle's first click reloads the default dataset, which looks like a bug because it is one.

5. Teaching. Not the app as a product but the material around it: a guided path with a fixed set of exercises, a classroom mode where the input is constrained to the concept being taught, and paid licences to institutions. The core insight is that a visualiser's value is instructional, so the customer is a course, not an individual. The alternative — a "sorting API" — makes no sense, because nobody needs a visualization service.

6. Render merge sort's buffer. The generator already emits `yOffset` and a `buffer` frame, and the renderer ignores both, so the change is a `translateY` or a second lane during merges. It fixes the most visible oddity in the app — a value appearing to vanish mid-merge — for a fraction of the effort of anything else on the list.

7. By using it and watching. That's the honest answer for a project with no instrumentation, and it's why the weakness register exists in the handbook rather than in a dashboard: the bugs I know about were found by running the app and by measuring the engine in a script, not by a metric. If it had users I'd want uncaught-exception reporting first, because without an error boundary a crash is a blank page that nobody reports.

8. Because the module pages came first and the lab is where the design ended up; both work, but the lab has the better execution model and the better tests. The lab wins, and the modules' job in the medium term is to migrate onto the frame model one page at a time — starting with the ones that have the most logic in them, since they have the most to gain from recorded frames and tests.

9. Yes, with one caveat. The testing content isn't decoration — it's the part an interviewer can verify, and almost none of it is generic: the 150 and the 768 are specific to this codebase, the reproducibility assertion is a product requirement, and the known gaps are the interesting half. The caveat is that I wrote it after the code rather than alongside it, so the handbook found the merge duplicate-key bug instead of the test suite finding it. Writing the assertions first would have been better engineering and a worse handbook.

10. The module pages' lesson scaffolding, or the dead components. The dead code is free to remove — nine components with no importers — and the lesson content is duplicated context with the generator, which is a maintenance cost. What I wouldn't cut is the comparison panel, even though it's the most complex surface, because it's the feature that makes the tool more than an animation.

11. Three visual things, in order: render the buffer in merge sort so nothing appears to vanish; stack the comparison panel at narrow widths instead of shrinking both arrays; and add a one-line hint under the dataset field saying that the keyboard shortcuts exist. All three are cosmetic, and all three are the difference between "a project" and "an app".

12. Delta frames with keyframes, plus a raised input cap. It's the one piece of work that changes what the tool can be used for rather than how it looks, and it's the item that everything else gets easier after: with a higher cap, the comparison feature becomes genuinely useful and the saved experiments become worth keeping.

13. I'd ask what they'd do with the output. Visualization of a 10,000-element sort at 60 fps is not a visualization — the bars are sub-pixel, and no human can perceive a single comparison. The right answers are aggregate views: a chart of comparisons per pass, a heat map of accesses, or a coarser trace. That's a different product surface, and I'd rather build that than pretend the current one scales.

14. For a learning tool, the metric is whether someone can answer a question about an algorithm using the tool that they couldn't answer before. That's not instrumentable in a local app, which is why the honest proxy is a teaching pilot: give it to a class and see whether the comparison panel gets used voluntarily. What I would not measure is session length or frames viewed, because those reward making the app slower and longer.

15. Because a video can't answer a question you have — it can only play the answer someone else picked. Here you choose the input, choose the algorithm, and stop wherever the confusion is; you can put the interviewer's favourite adversary input in — already sorted, all duplicates, one element — and see what the algorithm does with it in three seconds. That's not a better explanation; it's a different activity.

Interview 9 · Hana Sato — the performance engineer

> **MOCK INTERVIEW**
> **INTERVIEWER 9 · HANA SATO · PERFORMANCE / DATA ENGINEER**
>
> **Style.** Measurement-obsessed. She asks for numbers, units and the method behind them, and she treats an unmeasured claim as a hypothesis, not a fact.
>
> **What she is testing.** Whether you can reason quantitatively about your own system.

Questions

1. What is the cost model of one commit, in units?
2. Give me the measured worst case, with the input that produces it.
3. Where does the memory actually go?
4. What's the per-frame cost during playback?
5. How did you measure any of this? Show me the method.
6. What would you measure first if I said the app felt slow?
7. Is there anything in the code that's O(n^2) that shouldn't be?
8. What's the constant factor hiding behind your complexity claims?
9. If you raised the cap to 100, predict the numbers before measuring.
10. What's your frame budget, and how was it chosen?
11. What would a profiler show you that your own counters don't?
12. Where's the risk of a memory leak?
13. How would you make seek O(1) if frames were deltas?
14. What's the most expensive single line of code in the project?
15. What's your confidence in your own numbers, and where's the weakest one?

Model answers

1. Units are operations and objects. Generation is O(F) time where F is the frame count, and O(F × n) objects allocated, because every frame clones the array and the counters. The frame count itself is O(n^2) for bubble, selection and insertion, O(n log n) for merge, quick and heap — so the worst-case storage is O(n^3) in operations for the quadratic sorts with per-comparison instrumentation. Then `load()` runs the generator twice, and comparison mode runs it a third and fourth time.

2. 1,154 frames, about 981 KB serialised, from bubble sort on the reverse-sorted 24-element input. For comparison, the same input with selection sort is 492 frames, insertion 876, merge 375, quick 632, heap 348 — merge's 375 frames are 429,684 bytes and heap's 348 are 310,978 bytes. So bubble is roughly three times heap's frame count and roughly three times its bytes.

3. Into two places per frame: the cloned array — n `{id, val}` objects plus the container — and the cloned `currentStats` object, plus the metadata arrays (`activeIndices`, `sortedIndices`) and the message string. The strings are interned-ish by the engine but each is a distinct template result, so they're real allocations. For n = 24 that's roughly a few hundred bytes of objects per frame, and the 24-element case with 1,154 frames lands around a megabyte with serialisation overhead included.

4. One reducer call, one React render of the component, one array lookup, one `map` over ≤ 24 elements with a class-string computation and a `max` reduction for the height normaliser, plus React reconciling ~24 host elements. So O(n) with n ≤ 24 and a small constant — hundreds of microseconds at worst, dominated by reconciliation rather than by my arithmetic. With comparison enabled, add a second `map` over a second trace.

5. By importing the engine's modules directly into a standalone Node script — `import("<abs path>/src/lab/engine.js")` — and printing aggregate tables: for every algorithm and a fixed set of inputs, the frame count, the serialised size via `JSON.stringify(frames).length`, the final counters, and the id-duplication count per frame. That method matters because it measures the engine, not the UI: no rendering, no timer, no React, so the numbers are about the data model rather than about the browser. I re-run the script rather than quoting remembered values.

6. Input latency first: click a preset and time to the first painted frame, because that's the path that includes two full generations plus a React commit. If that's fine, I'd look at playback smoothness — the interval between rendered frames versus the configured delay — because a long render would show up as jitter rather than as a slow commit. Only then would I look at

memory, because memory here fails loudly rather than gradually.

7. Not in the algorithmic sense — the six generators are what they are, and O(n²) is the point for three of them. But there is a repeated-work pattern: `load()` computes the length by running the generator and the memo runs it again, so every commit does two identical O(F) computations. That's not quadratic, it's a factor of two, and it's the kind of thing that becomes a real bug rather than a cost at scale, because the two runs are two chances to disagree.

8. The constant factor is the clone. Each frame allocates n objects plus containers, so a "single comparison" in bubble sort costs roughly a 24-element deep copy — which means the instrumentation is asymptotically larger than the work it's instrumenting. That's the honest framing: at this scale the trace costs more than the sort, by a wide margin, and that's an accepted trade for inspectability.

9. Frame counts scale roughly with the operation count, so 100/24 ≈ 4.2× the elements gives roughly 17× the comparisons for a quadratic sort — about 20,000 comparisons, and if the density holds, on the order of 80,000 frames. Each frame clones a 100-element array, so it's 8 million object allocations and, at roughly 100 bytes per element plus metadata, on the order of hundreds of megabytes — and the strings for the messages add more. Prediction: a freeze on generation, and a browser tab that may be killed. Then I'd measure, and I'd expect to be within a factor of two.

10. No explicit animated-frame budget — and that's a weakness in the design process, not just the code. The measurements I have are: worst commit under about 100 ms, worst retained trace under about 1 MB, and playback interval equal to the configured delay with no visible jitter. If I set them deliberately I'd state 60 fps for interaction, 16 ms per frame for playback, and 4 MB retained, and then choose the instrumentation density to meet them — which would probably mean frame-per-pass rather than frame-per-comparison for the quadratic sorts.

11. A profiler would separate my cost from React's: my counters measure the engine, and the profiler would show the commit phase, layout, paint, and how much time React spends reconciling versus how much is spent in `ArrayView`'s arithmetic. It would also catch something my counters can't see at all — the cost of the effect churn from the keyboard listener being re-registered every frame, which is invisible to anything measuring the engine.

12. Three candidates. The `URL.createObjectURL` used by export is the classic one, though it's revoked in the same handler. The event listeners are the real one: the keyboard effect registers on `window` and does remove its listener, so it's balanced, but a future early-return added above the cleanup call would leak a listener per frame change. And in the legacy module pages, `iscPausedRef`/`speedRef` loops are cleared on unmount — that's the place where an async loop that outlives the component is a realistic leak.

13. Seek with deltas is either replay-from-keyframe or a precomputed cumulative index. The standard design is: store a full keyframe every k frames, store per-frame deltas, and carry a map from frame index to byte/array offset. Seek is then O(k) reconstruction worst case — bounded by the keyframe interval, not by the trace length — which is O(1) with respect to n if k is a constant. The trade is memory for reconstruction time, and the visible consequence is that seeking becomes slightly slower while memory drops by roughly the keyframe interval factor.

14. By allocation count, the frame-construction line in each generator — the one that builds `{array: cloneArray(), ..., currentStats: {...stats}}` — because it runs F times and allocates n+2 objects each time. By wall-clock in the browser, the generator call inside `load()`, which runs twice per commit. By frequency in the UI, the class-string computation inside `ArrayView`'s `map`, which runs 24 times per frame and 1,154 times in a long trace, though each is trivial.

15. High confidence in the frame counts and the serialised sizes — those come from re-runnable scripts over the real engine and I can reproduce any of them on request. Medium confidence in the predictions, like what happens at n = 100, because they extrapolate from a single density measurement. Lowest confidence in the browser-side numbers — commit latency and playback jitter — because I have no profiling data for those, only the absence of visible problems on my

machine, which is a sample size of one.

Interview 10 · Youssef Al-Amin — the director's round

MOCK INTERVIEW
INTERVIEWER 10 · YOUSSEF AL-AMIN · DIRECTOR OF ENGINEERING

Style. Warm, unhurried, and listening for structure and self-awareness rather than detail. He asks about people and decisions, and he will ask the same question twice in different words to see whether the answer changes.

What he is testing. Communication, ownership, judgement and whether you'd raise the level of a team.

Ground rules. No technical detail unless asked. Say what you did, why, and what you learned.

Questions

1. Walk me through this project in ninety seconds, for someone non-technical.
2. Why this project? What problem were you solving for yourself?
3. What's the most important decision you made, and why was it yours to make?
4. Tell me about something you were wrong about.
5. What did you do when the tests passed but the app was still wrong?
6. How did you decide what not to build?
7. What would you do differently about the order you worked in?
8. What are you proud of that isn't in the code?
9. What was the hardest thing to explain to someone else?
10. If a teammate disagreed with your frame design, what would happen?
11. How do you know when to stop improving something?
12. What's the next thing you'd learn, and why that?
13. What would you want to be true about your work in five years?
14. What's your relationship with criticism of this project?
15. And the same question again in different words: why should I care about this?

Model answers

1. "It's a tool for watching an algorithm think. You give it a handful of numbers, pick a sorting method, and instead of just seeing the result, you get the entire story of how it got there — every comparison, every swap, each one as a picture you can pause on, step through, and rewind. You can run two methods on the same numbers side by side and see exactly where they diverge in effort. It runs entirely in the browser, it never sends your data anywhere, and you can save an experiment or hand it to someone as a file. The point isn't sorting — nobody needs help sorting eight numbers. The point is that watching the process is how the process becomes obvious."

2. I kept getting the result of an algorithm when what I wanted was the behaviour. Even a good explanation of merge sort tells you what it does and skips over the moment where the comparison goes one way rather than another, and that moment is the whole algorithm. So the project started as an attempt to make the intermediate state the product rather than the by-product — which, once I framed it that way, meant the recording wasn't an implementation detail any more, it was the deliverable.

3. Deciding that a frame is a complete snapshot rather than a diff. That was mine to make because it's a trade-off, not a fact: snapshots are simpler, they make rewinding free, and they're trivially comparable in tests — at roughly a kilobyte per frame for the kind of arrays this tool handles. The alternative is smaller and faster and much harder to reason about. I chose the

version I could defend line by line, and I've been explicit ever since that it's the constraint that caps the input size.

4. I assumed that because the final frame is always correct, the intermediate frames must be too. That's not a logical step at all — it's an assumption that the end being right says something about the middle — and it was wrong in a specific, measurable way: merge sort's buffer write-back produces frames where the same element identifier appears twice, 23 of 107 frames on my default input. I found it by writing a script that printed every frame's identifiers, which was a method I only reached for because I'd stopped trusting my own reasoning.

5. I asked what the passing tests actually asserted. That's the whole technique: a green suite is not evidence, it's a set of specific claims, and if I write the claims down, the gap becomes obvious. In this case the assertion said "the final frame's identifiers are unique" — so of course a bug in the middle frames passes. The fix was two lines, and the more valuable output was the habit: for every green test, ask which bug it would not catch.

6. By asking who it was for. Three candidates lost that argument: a backend and accounts, because the value of the tool is that your work is local and private, and an account would add infrastructure I couldn't test; real-time animation easing, because the design is about discrete inspectable states and interpolation would blur exactly the thing being shown; and a general-purpose algorithm framework, because abstractions I'm not using are abstractions I can't defend. What I kept was the boring, defensible core.

7. I'd have written the assertions before the last three generators. Writing tests first against three algorithms would have made the frame contract explicit at the moment it was still negotiable — instead, I discovered what the contract was by writing six implementations and then trying to describe the common shape. The duplicate-identifier bug is the direct consequence: the contract would have said "identifiers are unique within a frame" if I'd had to state it up front.

8. The weakness register — the list of things that are wrong, ranked by how much they matter, with what each one would cost to fix. It took real work to make it honest, and it changed how I talk about the project: I now open with what's broken rather than what works, because that's the only way anyone can calibrate the rest. The other thing is the non-claims in the README. It's easy to write what a thing does; it takes discipline to write down what it deliberately doesn't.

9. Why the alignment is proportional rather than index-based. The code is one line, but the reasoning is about two recordings of different lengths — 65 frames versus 118 on my default input — and why index alignment fails visibly rather than subtly. What made it hard was that the correct version isn't obviously better until you see the two traces diverge, so I had to explain it with a concrete pair of numbers rather than with a principle. That's a lesson I've kept: when someone doesn't understand a design, they don't need the principle, they need the failing example.

10. I'd want them to attack it with a specific case rather than a preference, and if their case held I'd change it. The frame design is a trade-off, so the useful disagreement is about a consequence — memory, seek cost, test comparability — not about aesthetics. And I'd rather find out early, because this is the decision that everything else is built on; every generator, the reducer, the tests and the comparison feature all assume snapshots.

11. When the remaining improvement costs more than the next feature's entire value, or when it no longer changes what a user can do. The merge buffer visualization is the interesting case: the data is already there, and rendering it would fix a visible oddity, so it passes both tests — which is why it's item one on the list. Adding a seventh sorting algorithm fails both: it makes nothing clearer and costs a generator, a tutorial entry and tests.

12. Performance measurement — specifically profiling in the browser, because that's the one area where my numbers are weakest. I can measure the engine exactly, and I have essentially no data on commit latency, render cost or where the browser spends its time. Learning to read a flame graph properly would make me able to say what's slow instead of what should be slow, which is the difference between reasoning and knowing.

13. That the things I build are trusted because they're honest about their limits, not because they're impressive. Specifically, I'd like a reputation where a claim I make about a system can be checked — so the first thing someone does when reading my work is not to look for what I've oversold. That's a smaller ambition than "be a staff engineer" and it's the one I actually control.

14. Criticism is the most useful thing anyone can give me about it, and I've built the project so that criticism has somewhere to land: the weakness register, the non-claims, the footnotes on the counters. The failure mode I try to avoid is defending a design because I made it. When someone points at the duplicate-key bug, the useful part isn't the two-line fix — it's that my testing had a blind spot that the fix explains, and that's true regardless of who found it.

15. "Because the way you handle the parts that don't work will tell you more about me than the parts that do. There's a real bug in this project that I can point at, measure, and explain, and there's a testing gap that let it through — and I'd rather show you that than show you a demo that only runs on the inputs I chose. Nearly everyone can build something that works on the happy path; the signal is whether someone knows where theirs stops working and can say why."

PART XXXVIII–XXXIX

Five-day plan and priority tiers

A day-by-day plan with exit criteria, then a full RED/ORANGE/YELLOW/GREEN tiering of everything worth knowing.

CHAPTER 65

65 · The five-day master plan

One rule governs this plan: **each day ends with an exit criterion you either meet or you don't.** "I read the chapter" is not an exit criterion. "I explained the reducer's four transitions out loud in ninety seconds without notes" is.

65.1 How the five days are shaped

```
DAY 1  READ       the repository, not the handbook          -> 90 min
DAY 2  UNDERSTAND the engine, the reducer, the frame model  -> 4 h
DAY 3  SPEAK      the pitch, the defence, the attacks       -> 4 h
DAY 4  CODE       six sorts, the reducer, the parser        -> 4 h
DAY 5  PRACTICE   two full mock interviews, then weakness   -> 3 h
```

Each day below has the same four-part structure: what to read, what to understand, what to say, and the exit criterion. Do not proceed past a failed exit criterion — the days are cumulative and the exit criteria are the dependencies.

65.2 Day 1 — READ the source

Target. 90 minutes. By the end you should be able to name every file that matters and say what it does in one sentence.

Time	Task
0:00–0:10	Read `package.json` and `.github/workflows/ci.yml`. Write down the four CI steps and the Node floor from memory.
0:10–0:25	Read `README.md` end to end. Mark every claim that names a file or a number — those are the claims you will be asked to prove.
0:25–0:35	Read `index.html`, `src/main.jsx`, `src/App.jsx`, `src/pages/Home.jsx`. Draw the boot path from `<div id="root">` to a rendered route, on paper.
0:35–0:55	Read `src/lab/engine.js` in full, twice. First pass for structure, second pass for the four things it holds: the registry, the parser, the reducer, and the dataset/window functions.
0:55–1:20	Read `src/lab/SortingLab.jsx` in three passes: state declarations, effects, render. Do not try to read it linearly.
1:20–1:30	Read `tests/engine.test.js` and count the groups. Note which assertion you think is weakest.

CHECKLIST
DAY 1 EXIT CRITERIA

- ■ I can name the purpose of these nine files from memory: `main.jsx`, `App.jsx`, `Home.jsx`, `constants.js`, `engine.js`, `SortingLab.jsx`, `Shell.jsx`, `engine.test.js`, `ci.yml`.
- ■ I can draw the boot path on paper without looking.
- ■ I can state the four CI steps in order.
- ■ I can name the file that holds the six algorithms' registry entries and the file that holds their bodies.
- ■ I identified one assertion I believe is too weak — before reading any chapter that tells me which.

65.3 Day 2 — UNDERSTAND the engine

Target. Four hours, in three sittings. This is the day the interview is actually decided on.

Sitting 1 — the frame contract (75 min).

- Read Chapters 11–14 (frames, execution, memory, the schema).
- Open `bubbleSort.js` and read it line by line against the chapter.
- Then open `mergeSort.js` and find the three places where the buffer write-back happens. Write down, in your own words, why the identifiers are duplicated.
- Re-derive from memory: the two mandatory clones, and what breaks without each.

Sitting 2 — the reducer and the transport (75 min).

- Read Chapters 21–26 (the reducer, worked simulations, play/pause, timers, seeking, speed).
- Write the reducer's four transitions on a blank page: state before, action, state after, for `load`, `toggle`, `tick`, `seek`. Include the exact expressions.
- Write down the three state pairs from the worked simulations, then check them against Chapter 22.
- State out loud, in one breath, why `cursor` is in the timer effect's dependency array.

Sitting 3 — counts, comparison and the numbers (90 min).

- Read Chapters 38–40 (comparison, alignment, counts).
- Write down the alignment formula from memory and compute the secondary cursor for three pairs by hand.
- Learn the number set: 1,154 frames / ~981 KB worst case; 118 / 65 / 76 / 107 / 75 / 84 frames on the default input; comparison counts 28 / 28 / 22 / 16 / 16 / 44; the 150 and the 768.
- Say out loud, twice: why a count of comparisons is not a measurement of speed.

CHECKLIST
DAY 2 EXIT CRITERIA

- ■ I wrote all four reducer transitions from memory with the correct expressions.
- ■ I explained the merge duplicate-identifier bug in under 45 seconds, with the 23-of-107 figure.
- ■ I stated the alignment formula and computed an example by hand.
- ■ I recited six frame counts and six comparison counts without looking.
- ■ I stated why the two mandatory clones exist and what fails without each.

65.4 Day 3 — SPEAK it

Target. Four hours, mostly out loud, standing up, with a timer.

Block	Exercise	Timer
1	The 30-second pitch, five times, each one different	10 min
2	The 2-minute pitch, three times, then the 5-minute version once	30 min
3	Resume defence: all fourteen claims, each with its proof location	45 min
4	The weakness answer — the ninety-second version, three times	20 min
5	The hostile round: pick 20 questions from Chapter 58 at random, answer aloud	60 min
6	Cross-domain: 15 questions from Chapter 60, using the four-step formula	40 min
7	Record the 2-minute pitch and listen to it once, critically	15 min

Rules for this day. Speak in complete sentences. Never say "basically", "kind of", or "I think" before a claim you can verify. When you don't know, say "I don't know, and here's how I'd find

out". Time yourself and stop mid-sentence when the timer ends — the discipline transfers to interviews.

CHECKLIST
DAY 3 EXIT CRITERIA

- ■ I delivered the 2-minute pitch in under 2:30 with no notes and no filler words.
- ■ I delivered the ninety-second weakness answer without hedging.
- ■ I answered 20 hostile questions without getting defensive or overclaiming.
- ■ I used the four-step cross-domain formula (mechanism → consequence → boundary → fix) without thinking about it.
- ■ I listened to my own recording and named two things to fix.

65.5 Day 4 — CODE it

Target. Four hours at a keyboard or whiteboard. Nothing read except what you need to check.

Block A — the six sorts (60 min). Write all six from memory, in this order: bubble, insertion, selection, merge, quick, heap. For each, write the complexity table underneath: best, average, worst, space, stable. Then check against Chapter 56 and mark which ones you got wrong — the comparison detail (`<=` versus `<`) is what usually differs.

Block B — the reducer (30 min). Write it from memory including the `default` branch. Then write four assertions you'd add: last-frame completion, seek clamp low, seek clamp high, and the property that the cursor is an integer in range. Note out loud that the last one fails today.

Block C — the parser and the JSON gates (40 min). Write `parseInput` from memory with its four error messages. Then write the import gate sequence as a list, in order, and say why the size check is first and why `Object.hasOwn` replaces `in`.

Block D — a frame generator (40 min). Write a recording insertion sort that clones on every push and carries `codeLine`, `currentMsg`, and a cloned counter object. Then answer out loud: what are the four things that must be fresh per frame?

Block E — the two-line test fix (20 min). Write the per-frame identifier uniqueness assertion as it would appear inside the existing loop. Then write the multiset property ("every frame's values are a permutation of the input") and say what extra class of bug it catches.

Block F — install and run (30 min). `npm ci`, `npm test`, `npm run lint`, `npm run build`. Note the frame counts and the timings you observe; they're your evidence for the "I ran it in front of you" moment.

CHECKLIST
DAY 4 EXIT CRITERIA

- ■ Six sorts written from memory, with at most two comparison-operator mistakes.
- ■ The reducer written from memory including the `default` branch returning the same state.
- ■ The four error messages of `parseInput` reproduced accurately.
- ■ The import gate sequence in the right order, with reasons.
- ■ The two-line assertion written correctly.
- ■ The full gate sequence run locally: tests, lint, build all green.

65.6 Day 5 — PRACTICE under pressure

Target. Three hours. Simulate the real thing, then close the gaps.

Block	Simulation	Duration
1	Interview 2 (Kevin — React deep dive) from Chapter 63	35 min
2	Interview 4 (Tim — skeptical manager) from Chapter 64	30 min
3	Interview 7 (Sofia — security) from Chapter 64	25 min
4	Rapid-fire: 100 questions from Chapter 61 against a clock	30 min
5	The weakness answer, timed, then the "what would you redo" answer	15 min
6	Open the app and walk through it as if sharing a screen	15 min
7	Write your three questions for the interviewer	10 min

How to run a simulation. Set a timer. Read a question, answer aloud as if the person were in the room, then read the model answer and score yourself as **strong** / **adequate** / **weak**. Keep a tally. Anything scored weak goes on a single sheet of paper — that sheet, not this book, is what you revise on the morning of the interview.

The three questions to ask them. Pick from these, and make them real: "What does the first month look like for someone in this role?" · "How does the team decide when something is tested enough?" · "What's the piece of this codebase that's hardest to work in, and why?" The third one signals that you've worked in a codebase someone else owns.

CHECKLIST
DAY 5 EXIT CRITERIA

- ■ Three simulations completed with a written tally, not a feeling.
- ■ At most five items on the weak sheet.
- ■ 100 rapid-fire questions in under 30 minutes with fewer than 10 misses.
- ■ A 15-minute app walkthrough with no dead air: input → algorithm → play → step → seek → compare → counts → save → export.
- ■ Three questions written for the interviewer.

65.7 If you only have one day

```
HOUR 1   Chapter 22 (worked reducer simulations) + Chapter 21
HOUR 2   Chapter 33 (weaknesses) + Chapter 58 Group A and F
HOUR 3   Chapter 63 interview 2 + Chapter 64 interview 4, aloud
HOUR 4   Six sorts from memory, then the reducer
HOUR 5   Chapters 61.1–61.3 (rapid fire, first 52) with a clock
HOUR 6   Chapter 68 (night-before sheet) read twice, then recite
FINAL    The ninety-second weakness answer, three times, recorded
```

CHAPTER 66

66 · Priority tiers: what to know versus what to recognise

Tiering is how you allocate scarce attention. **RED** means you will be destroyed if you don't know it. **ORANGE** means you'll be asked and it should be fluent. **YELLOW** means you must recognise it and know where to look. **GREEN** means you should be able to say one correct sentence if it comes up, and otherwise stay quiet.

66.1 Files

Tier	Files	Why this tier
RED	`src/lab/engine.js`, `src/lab/SortingLab.jsx`, `tests/engine.test.js`	The registry, parser and reducer; the only reducer consumer and the whole UI; the entire verification story
ORANGE	`src/modules/core/algorithm/sorting/algorithms/*.js`, `package.json`, `.github/workflows/ci.yml`, `src/pages/Home.jsx`, `README.md`	The six generators, the toolchain contract, the gates, the routing table, the scope document
YELLOW	`src/lab/lab.css`, `eslint.config.js`, `vite.config.js`, `vercel.json`, `src/constants.js`, `src/lab/Shell.jsx`, `src/lab/Counterexample.jsx`, `src/lab/Overview.jsx`, `src/lab/ModuleFrame.jsx`	You need to know they exist and what they own — not their contents
GREEN	`src/components/*`, `SimpleSort.jsx`, `EfficientSort.jsx`, `ModeContext.jsx`, `CardsGrid.jsx`, `src/modules/**` (page bodies), `src/lab/lesson.css`, `src/lab/modules.css`	Dead code and the legacy generation — one sentence each, don't volunteer detail

66.2 Hooks and React concepts

Tier	Item	The one thing to know
RED	`useReducer`, `useMemo`, `useEffect` (cleanup), `useState` (derived vs stored)	The reducer is pure and exported; the memo keys on `[input, algo]`; the effect's cleanup cancels the timeout; derived values are computed during render
RED	Dependency arrays, stale closures	`cursor` in the timer effect is the re-scheduling trigger; the keyboard effect reads `cursor`, so it depends on it
ORANGE	Lazy state initialiser, batching, `useCallback`, `key` semantics, `Suspense`, `React.memo`	The saved list reads storage once via a lazy initialiser; `setInput` and `dispatch` batch together; `key={item.id}` gives element identity; `memo` on `ArrayView` would gain nothing; `useCallback` appears exactly once — `resetBoard` in `Nqueens.jsx` — so an effect can depend on a stable function
YELLOW	`useRef`, `createContext`, StrictMode double-invoke, error boundaries	The refs are in the legacy modules (`Nqueens.jsx`); there's no context in the lab; StrictMode is on in `main.jsx`; there's no error boundary — mention that as a gap
GREEN	Portals, transitions, `useId`, `useSyncExternalStore`	Not used anywhere; say so if asked

66.3 Algorithms

Tier	Item	The one thing to know
RED	Bubble, insertion, selection	Quadratic, stable except selection, insertion is adaptive and bubble sort here is not

Tier	Item	The one thing to know
RED	Merge	Θ(n log n) always, stable, O(n) buffer, and **this** implementation duplicates identifiers in 23 of 107 frames
RED	Quick	Lomuto, last-element pivot, sorted input is the worst case, unstable, pivot index is final
ORANGE	Heap	n log n in every case, build is O(n), and this `heapify` increments twice per call unconditionally
ORANGE	Sliding window	O(n), requires positive values, fails on negatives — measured
YELLOW	Binary search, BFS/DFS, DP	Not in this codebase; know them as general knowledge, don't connect them to AlgoViz

66.4 Tests and verification

Tier	Item	The one thing to know
RED	150 = 6 × 25, the four assertions	Sortedness, input preservation, deep-equal re-run, final-frame id uniqueness
RED	768 = 64 × 12, the oracle	Brute force, inline, independent mechanism, bounded to length 3 and values 1–4
RED	The weakest assertion	Id uniqueness is final-frame-only — the two-line fix
ORANGE	`node:test` + `node:assert/strict`	Zero test dependencies; `npm test` is `node --test tests/*.test.js`
ORANGE	What is untested	Rendering, timers, `localStorage` round-trips, all nine import cases
YELLOW	Coverage, mutation testing, property tests	None are used; the arguments for each are in Chapter 43 — don't claim them
GREEN	Jest, Vitest, Playwright, Cypress	Not installed, never referenced — do not claim them

66.5 Config and tooling

Tier	Item	The one thing to know
RED	The four CI steps	`npm ci` → `npm test` → `npm run lint` → `npm run build`, Node 22, GitHub-hosted
RED	Vite alias	`"vite": "npm:rolldown-vite@7.2.5"` plus an `overrides` entry
ORANGE	The ESLint exception	`react-hooks/set-state-in-effect` disabled for `src/modules/**` only
ORANGE	Bundle numbers	251.78 kB main / 80.90 kB gzip, 29.85 kB lab chunk, 18.65 kB engine
YELLOW	`vercel.json`, `.nvmrc`, `tailwind.config.js`	SPA rewrite, Node 22 floor, three-file Tailwind surface
GREEN	Docker, Kubernetes, Terraform, AWS	Nothing of the kind exists here

66.6 Concepts, in one ranked list

Tier	Concept	Why it's there
RED	Immutability of recorded state	The two clones; without them the whole model collapses
RED	Execution versus playback	The separation is the architecture
RED	Derived versus stored state	The cursor clamp, `meta`, `last`, `frame`
RED	Why counts are not timings	The single most likely place to overclaim
ORANGE	Trust boundaries and prototype safety	`Object.hasOwn` on three inputs

Tier	Concept	Why it's there
ORANGE	Memory bounded by design	The 24-element cap and the ~1 MB worst trace
ORANGE	Determinism as a product requirement	Saved cursors depend on stable frames
YELLOW	Accessibility, i18n, theming, error handling	Real gaps; describe honestly, don't defend
GREEN	Micro-frontends, SSR, GraphQL, WebSockets	Not applicable — don't bring them up

MUST KNOW

THE ONE-SCREEN SUMMARY OF THIS CHAPTER

RED: `engine.js`, `SortingLab.jsx`, `engine.test.js`, the frame contract's two clones, the reducer's four transitions, the 150 and the 768, the duplicate-identifier bug with its number, the two-line test fix, the four CI steps, the Vite alias, and "counts are not timings". If you know only that list, you can hold a 45-minute interview.

If asked anything from ORANGE or YELLOW, answer in one sentence and point at the file. Knowing where things live is a legitimate engineering skill; pretending to know their contents is not.

If asked anything from GREEN, say what it is and stop. The worst version of this interview is a candidate who volunteers detail about the legacy modules they can't defend.

CHECKLIST

RED-LINE DRILLS FOR PARTS XXXVIII–XXXIX

- ■ Five-day plan runs in order, with every exit criterion met — no skipping.
- ■ The one-day crash plan executed once end to end.
- ■ Every RED item answered without hesitation.
- ■ Every GREEN item answered with exactly one sentence and no elaboration.

PART XL–XLII

Final checklist, night before, and the cheat sheet

One hundred and ten verification items, a five-page revision sheet, and a single page for the last fifteen minutes.

CHAPTER 67

67 · The final checklist

Run this the day before. Tick only what you can do **out loud, without notes**. An unticked box is not a failure — it's tomorrow morning's revision list.

67.1 Project identity (1–12)

- ■ 1. Give the project's one-sentence definition.
- ■ 2. Give the one-sentence non-claim (not a benchmark, not a backend, not a proof).
- ■ 3. Deliver the 30-second pitch.
- ■ 4. Deliver the 60-second pitch.
- ■ 5. Deliver the 2-minute pitch.
- ■ 6. Name all six algorithms with their complexity in one breath.
- ■ 7. State the tech stack and the one unusual item in it (the Vite alias).
- ■ 8. State the Node floor and where it's declared.
- ■ 9. Name the three runtime dependencies.
- ■ 10. State what the app does not have: backend, database, accounts, network calls, analytics.
- ■ 11. Explain "local-first" without overclaiming.
- ■ 12. Name the repository's single commit and why it limits what Git can prove.

67.2 Engine and frames (13–32)

- ■ 13. Name every field in a frame, including the per-algorithm extras.
- ■ 14. State the two mandatory clones and what breaks without each.
- ■ 15. Explain why the array holds `{id, val}` rather than numbers.
- ■ 16. Explain what `codeLine` is for and who consumes it.
- ■ 17. Give the frame counts for the default input for all six algorithms.
- ■ 18. Give the worst trace in the app: 1,154 frames, bubble, reverse-24, ~981 KB.
- ■ 19. Explain why frames are full snapshots rather than deltas.
- ■ 20. State the memory trade-off of that decision with the measured number.
- ■ 21. Explain the duplicate-identifier bug and the frames it affects (23/107, 141/375).
- ■ 22. Give both fixes for it and their trade-offs.
- ■ 23. Explain why merge sort's "exchanges" is a write count.
- ■ 24. Explain why merge sort's `sortedIndices` stays empty.
- ■ 25. Explain what `yOffset` is for, that the lab never draws it, and that the only reader is an unimported file.
- ■ 26. Explain the heap counter's unconditional double increment.
- ■ 27. Explain bubble sort's missing early exit and its cost (276 comparisons on sorted 24).
- ■ 28. Explain why selection sort's comparison count is input-independent.
- ■ 29. Explain insertion sort's `break` frame costing one comparison.
- ■ 30. Explain why quick sort's pivot index is final immediately.
- ■ 31. State which sorts are stable and which comparison decides it.
- ■ 32. Name the three algorithms whose traces contain `minimumIndex`.

67.3 Reducer and playback (33–50)

- 33. State the reducer's state shape and initial value.
- 34. Name all four action types and what each produces, exactly.
- 35. Recite the `tick` completion expression.
- 36. Recite the `toggle` cursor expression and explain the replay behaviour.
- 37. Explain why every `seek` pauses.
- 38. Explain what an unknown action returns and why that matters in React.
- 39. Work the `length = 20, cursor = 5` simulation aloud.
- 40. Work the `length = 20, cursor = 19` simulation aloud.
- 41. Work the `length = 1` simulation aloud.
- 42. State the two reducer weaknesses: non-integer cursor, `NaN` cursor.
- 43. Explain why `length` is duplicated state and what guards the mismatch.
- 44. Explain why the reducer is in `engine.js` and not in the component.
- 45. Explain the timer effect's dependency array, item by item.
- 46. Argue why `setInterval` is the wrong choice here.
- 47. Explain what happens with no cleanup and with a missing dependency.
- 48. Explain what happens on unmount mid-playback.
- 49. Explain the keyboard effect's ignore-list and its `[cursor]` weakness.
- 50. Name the relative-`step` action as the fix for that weakness.

67.4 State, input and comparison (51–68)

- 51. Distinguish `text` from `input` and explain why both exist.
- 52. Explain what a failed validation changes (only `error`).
- 53. Recite `parseInput`'s rules: 2–24, `^-?\d+$`, safe integer, ≤ 999.
- 54. Give two characters coercion would wrongly accept.
- 55. Explain why the same-input guarantee holds (same reference, no mutation).
- 56. Recite the alignment formula.
- 57. Compute a secondary cursor by hand for two given lengths.
- 58. Explain why index alignment fails visibly (65 versus 118 frames).
- 59. State what "50%" does not mean.
- 60. Name the four on-screen comparison disclosures.
- 61. Name the three counters and which one is comparable.
- 62. Explain why "passes / calls" is unit-dependent.
- 63. Give the comparison counts for the default input for all six.
- 64. Explain the meter's `Math.max(2, …)` floor.
- 65. Explain the bar height formula and the 25% floor's distortion.
- 66. Explain why there are four speed labels but only three exact ratios.
- 67. State that there is no timing code anywhere in the project.
- 68. Name the label that distinguishes live counters from complete-run counters.

67.5 Persistence, import, testing, tooling (69–90)

- 69. Recite the storage key and the record schema.
- 70. State the record cap and where the slice happens.
- 71. Name the four read-path defences.
- 72. Explain why a record whose algorithm no longer exists is dropped.
- 73. Explain what `setSaved`'s position inside the `try` protects.

- 74. Explain why `localStorage` is not secure storage.
- 75. Recite the export payload's four fields and the omitted one.
- 76. Recite the import gates in order.
- 77. Explain the 16 KB size gate's purpose.
- 78. Explain `Object.hasOwn` versus `in` concretely.
- 79. Explain why imported values are re-run through `parseInput`.
- 80. State the import path's biggest gap (nine cases, zero automated).
- 81. Recite the four sorting assertions.
- 82. Compute the 150 breakdown aloud.
- 83. Explain why reproducibility is a product requirement.
- 84. Compute the 768 breakdown aloud.
- 85. Explain why the oracle is inline and independent.
- 86. State what the 768 do not prove.
- 87. State the sliding window's precondition and where it's enforced.
- 88. Recite the four CI steps in order.
- 89. State what CI cannot prove.
- 90. Recite the Vite alias and explain what an alias does.

67.6 Weaknesses, defence and meta (91–110)

- 91. Deliver the ninety-second weakness answer.
- 92. Rank the S1 defects and give each in two sentences.
- 93. Explain why the test gap matters more than the bug.
- 94. Name three deliberate scope boundaries and their justifications.
- 95. Name four pieces of dead code.
- 96. Explain the ESLint exception and call it labelled debt.
- 97. Give the "one thing I'd redo" answer.
- 98. Give the "what breaks first" answer with a number.
- 99. Give the "what would you refuse to build" answer.
- 100. Defend all fourteen resume claims with their file locations.
- 101. Say the three sentences you must never say (MERN, faster, sole author).
- 102. Answer "prove you wrote it" without overclaiming.
- 103. Name the untested behaviour a user would notice first.
- 104. Describe the app in 15 seconds to a non-engineer.
- 105. Walk the app live in 15 minutes with no dead air.
- 106. Answer five random hostile questions with composure.
- 107. Use the four-step cross-domain formula without thinking.
- 108. Name your three questions for the interviewer.
- 109. State the first thing you'd build with six more months.
- 110. State your metric for success and why it isn't usage.

CHAPTER 68

68 · Night-before revision sheet

Three pages in this layout, five at most. Nothing new. If a fact isn't on this sheet, it isn't for tonight — it's for the morning, and it's covered by the cheat sheet in Chapter 69.

68.1 The numbers

Number	What it is
150	6 algorithms × 25 inputs (5 hand-picked + 20 seeded of 10 values in [0,19])
768	64 three-element arrays over values 1–4 × 12 targets, versus a brute-force oracle
1,154 / ~981 KB	Worst trace and size — bubble sort, reverse-sorted 24 elements
118 / 65 / 76 / 107 / 75 / 84	Frames on the default input: bubble, selection, insertion, merge, quick, heap
28 / 28 / 22 / 16 / 16 / 44	Comparison counts on the default input, same order
23 / 107	Frames with a duplicate id — merge sort, default input (141 / 375 on reverse-24)
24	The dataset cap. Why: every frame is a full snapshot
4	Reducer actions: `load`, `toggle`, `tick`, `seek`
2	Mandatory clones: the array, and the counters
600 / 300 / 160 / 50 ms	Playback delays, labelled 0.5× / 1× / 2× / 6× (the 2× label is rounded; the real ratio is 1.875×)
16,384	Import size limit in bytes
6	Saved experiments retained
1,736 / 251.78 kB / 80.90 kB	Build: modules, main chunk, gzipped main chunk
22	Node version, in `.nvmrc`, in `engines`, and in CI
14	`node:test` groups
11 of 16	Lazy routes

68.2 The five expressions

```
cursor        = Math.min(playback.cursor, frames.length - 1)
frame         = frames[cursor]
tick playing  = state.cursor + 1 < state.length - 1
toggle cursor = state.cursor === state.length - 1 ? 0 : state.cursor
compareCursor = Math.round((cursor / (frames.length - 1)) * (otherFrames.length - 1))
```

68.3 The five sentences

1. **The architecture, in one line.** "The engine records complete immutable frames; the reducer is a four-action state machine over an index; the render is a pure function of one frame."
2. **The defensible comparison claim.** "Same input, aligned by percentage of trace — not equivalent operations — with recorded comparison counts only, and no wall-clock claims anywhere."
3. **The bug, in one line.** "Merge sort's buffer write-back stores shallow clones, so ids duplicate in 23 of 107 frames, and my columns are keyed by id."

4. **The testing gap, in one line.** "The id assertion only covers the final frame, which is why that bug passes CI; moving it inside the loop is a two-line fix."
5. **The honesty line.** "Everything I've said is either in a file you can open, a number I can re-measure, or a limitation I named first."

68.4 The three never-says

Never	Say instead
"It's a full-stack MERN app"	"A client-side application — no backend, no database, no accounts"
"The comparison shows which algorithm is faster"	"It shows recorded comparison counts on the same input; timing isn't measured anywhere"
"I wrote every line"	"Git here is one squashed commit, so it can't establish authorship — but I can defend any decision in it"

68.5 The ten attacks and their first sentence

Attack	Your first sentence
"You didn't build this"	"I can't prove authorship from one commit — I can be interrogated on any part of it."
"The architecture is wrong"	"Every frame is a full clone; that's the constraint that caps input at 24, and here's the fix."
"Your tests prove nothing"	"The id assertion is final-frame-only, which is a real gap — here's the two-line fix."
"This is a toy"	"The algorithms are textbook; the instrumented recording is not."
"Your numbers are made up"	"Let me show you the script — I re-run it rather than quote it."
"Where's the backend?"	"There isn't one, by design — the README says so in plainer words."
"No coverage, no component tests"	"Correct. Line coverage wouldn't have caught the bug my per-frame assertion would."
"The reducer is sloppy"	"Clamping in the reducer hides invalid input — `seek(2.5)` stores 2.5, so the invariant is false."
"Dead code everywhere"	"Nine components with zero importers plus two dead modules; free to delete."
"What's your worst bug?"	"Merge sort's duplicate ids, 23 of 107 frames, and my tests can't see it."

68.6 Tonight's routine

```
1. Read 68.1 – 68.5 once, slowly. Do not add to them.
2. Say the five sentences out loud, twice.
3. Open the app. Load the default input. Play bubble sort for 10 seconds.
   Step forward three times. Seek to the end. Enable Compare on merge.
   Save the experiment. Export it. Do not fix anything.
4. Confirm the repo state: `npm test` green, `npm run lint` clean.
5. Put the cheat sheet (Chapter 69) on the desk for the morning.
6. Sleep. The frame counts will still be there.
```

> **NOTE**
> **WHAT NOT TO DO TONIGHT**
> Do not read a new chapter. Do not chase a fact you can't find in five minutes. Do not rewrite the reducer in your head. Do not practise the answer you haven't rehearsed yet — the rehearsed ones are the ones that work. Tonight is consolidation, and the most valuable thing you can do is close the book while the numbers are still quiet in your head.

CHAPTER 69

69 · The fifteen-minute cheat sheet

Two pages. Read them in the last fifteen minutes before the call. Nothing here is new — it's the whole interview compressed into what you'd want if you could only keep one sheet.

69.1 The pitch — 20 seconds

"AlgoViz records the complete execution of six sorting algorithms as inspectable frames, then lets you replay them. You pick 2–24 integers, pick an algorithm, and you get every comparison and swap as a step you can pause, rewind, and interrogate. I run two algorithms on the same input to compare their recorded comparison counts, and you can save an experiment locally or export it as JSON. It's entirely client-side — no backend, no accounts, nothing leaves the browser."

69.2 The three files

File	Holds
`src/lab/engine.js`	The algorithm registry, `parseInput`, the playback reducer, `makeDataset`, `windowTrace`, `findCounterexample`
`src/lab/SortingLab.jsx`	Every piece of UI state, both effects, the derivations, `ArrayView`, persistence, comparison
`tests/engine.test.js`	14 `node:test` groups — the 150 sorting cases and the 768 window cases

69.3 The architecture in four lines

```
ENGINE       input -> 6 generators -> frames[] : [{array, activeIndices, sortedIndices,
             codeLine, currentStats, msg, ...extras}]   immutability = 2 clones
TRANSPORT    useReducer({cursor, length, playing}) with load | toggle | tick | seek
DERIVATION   cursor = min(playback.cursor, frames.length-1);  frame = frames[cursor]
RENDER       ArrayView(frame) — pure; key={item.id}; height = 25 + |val|/max * 75 %
```

69.4 The reducer, complete

```
load   -> { cursor: 0,        length: action.length, playing: false }
toggle -> { cursor: cursor === length - 1 ? 0 : cursor, playing: !playing }
tick   -> { cursor: min(cursor + 1, length - 1), playing: cursor + 1 < length - 1 }
seek   -> { cursor: clamp(action.cursor, 0, length - 1), playing: false }
default->  state   // same reference, React bails out
```

69.5 Numbers to have ready

150 cases · **768** cases · **1,154** frames / **~981 KB** worst trace · **24** element cap · **23/107** duplicate-id frames · **16,384** byte import limit · **6** saved experiments · **22** Node · **14** test groups · **14/17** routes backed by lazy imports (12 distinct components) · **251.78 kB** main bundle / **80.90 kB** gzip.

69.6 The four disclosures around comparison

Same input · alignment by percentage of trace, **not equivalent operations** · comparisons only, since pass and exchange units differ per algorithm · **no wall-clock claims** anywhere in the project.

69.7 Top weaknesses, ranked

1. Merge sort's duplicate ids in 23/107 frames — real, measured, fixable in one line of key logic.
2. The test's id assertion covers only the final frame — the two-line fix that would have caught it.
3. The reducer accepts non-integer and `NaN` cursors — my stated invariant is false.
4. No component or browser tests — the timer wiring is untested.
5. No error boundary. 6. Import's nine cases are unautomated. 7. `windowTrace` lacks `generateWindow`'s positivity guard. 8. `SortingLab.jsx` is 596 lines.

69.8 Answers for the six most likely questions

Question	The first line of your answer
"What does this do?"	"It records an algorithm's whole execution as inspectable frames and lets you replay it."
"Where's the complexity?"	"Not the algorithms — the instrumentation, and keeping the trace and the result from disagreeing."
"What's your worst bug?"	"Merge sort's buffer write-back duplicates element ids in 23 of 107 frames."
"How do you know it works?"	"150 property cases, 768 differential cases against an independent oracle, and four CI gates."
"What's not tested?"	"Anything that renders — no component, browser or timer test exists."
"What would you do next?"	"Delta frames with keyframes, to lift the 24-element cap."

69.9 Your three questions for them

1. "What does the first month look like for someone in this role?"
2. "How does the team decide when something is tested enough?"
3. "What's the hardest part of this codebase to work in, and why?"

69.10 The last thing

MUST KNOW
BEFORE YOU CLOSE THIS PAGE

"Everything I've said is either in a file you can open, a number I can re-measure, or a limitation I named first. If you want to see any of it running, I can open the app — and if you want to see what's wrong with it, ask me about merge sort."